

Neural Network Architectures for Sequences of Elements: TCN & LSTM

Table of Contents

1D CNN Architecture for Variable-Length Sequential Data in RL	2
An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling	8
Temporal Convolutional Networks and Forecasting	20
Temporal Convolutional Networks: A Unified Approach to Action Segmentation	35
Convolutional Sequence to Sequence Learning	39
WaveNet: A Generative Model for Raw Audio	52
LSTM-Based Architecture for Sequential Quantum Circuit Inputs	64
Language Understanding for Text-based Games using Deep Reinforcement Learning	70
Counting to Explore and Generalize in Text-based Games	79
Sentiment Classification using Bi-LSTM with Attention	90
Deep learning coupled model based on TCN-LSTM for particulate matter concentration prediction	107
END	116



1D CNN Architecture for Variable-Length Sequential Data in RL

Overview: Recent research has shown that **1D convolutional networks** can model sequential data as effectively as (or even better than) recurrent networks in many domains ¹. This makes them a promising choice for reinforcement learning (RL) policies that must handle **variable-length sequences** of instructions. Below we describe a **proven Conv1D architecture** (inspired by the Temporal Convolutional Network, TCN) and how to adapt it to an actor-critic (A2C/PPO-style) model for your quantum circuit input. This architecture is *fully convolutional* along the sequence dimension (no RNNs), and is designed to handle arbitrary sequence lengths end-to-end ².

Temporal Convolutional Network (TCN) – Robust 1D CNN for Sequences

A **Temporal Convolutional Network (TCN)** is a deep 1D CNN architecture that has been empirically shown to outperform RNNs on various sequence modeling tasks ¹. It's well-suited to sequences of *variable length with repeating patterns* (like a series of quantum gate instructions). Key features of a TCN (or similar Conv1D sequence models) include:

- **Stacked 1D Convolutions with Dilation:** The network stacks multiple Conv1D layers that slide over the sequence (time) dimension. Using *dilated* convolutions allows the receptive field to grow exponentially with depth ³. For example, each layer can have a small kernel (e.g. 3) but with dilation factors doubling at each layer (1, 2, 4, 8, ...). This way, a relatively **deep network (e.g. 8-12 layers)** can capture long-range dependencies (hundreds of time-steps) without needing an impractically large kernel ⁴. In other words, dilation lets the model **cover very long sequences** while keeping kernels small and efficient.
- **Residual Blocks for Stability:** Rather than plain sequential conv layers, TCNs use **residual blocks** (skip connections) to ease training of deep networks ⁴. Each residual block typically contains **two Conv1D layers** (with dilation) plus non-linearities, and the block's input is added to its output (with a 1×1 conv on the input if needed to match dimensions) ⁵. This design lets layers **learn modifications to the identity mapping** instead of entire transformations, which greatly stabilizes training for deep CNNs ⁶ ⁷. In fact, experiments found that adding residual connections yields faster convergence and better final performance on sequence tasks ⁷. *Recommendation:* Wrap your Conv1d layers in residual blocks if you stack many of them, especially if you increase channel counts in deeper layers.
- **Normalization and Dropout:** Successful Conv1D architectures often include normalization and regularization in each block. The TCN reference model applied **weight normalization** to conv filters and **spatial dropout** (dropout entire channels) after conv layers ⁸. In practice, using **LayerNorm + dropout** (or BatchNorm) after conv layers is also effective to stabilize training – you already use

LayerNorm and PReLU, which is consistent with these best practices. These help deal with the high-dimensional input (e.g. 150 features per instruction) and prevent overfitting as the network grows deep.

- **Causal vs. Non-causal:** In some domains (like forecasting), conv layers are made *causal* (no future leakage). For your use-case (analyzing a circuit as a whole to output an action), **causality isn't required** – you can use “same” padding so the conv layers consider the full neighborhood around each instruction. The important part is that **input and output lengths remain the same** through the conv stack (achieved via padding) ⁹, so that every instruction's effects can propagate to the final representation.

Why this architecture fits: A TCN-style ConvNet naturally handles **arbitrary-length sequences** by sliding the filters over the instruction sequence ², just as you described (each “cell” of the kernel covers one instruction's features). It can learn repeating local patterns (via small kernel convolutions) and also long-term patterns (via stacked dilations). In fact, one major advantage of Conv1D over RNNs is the **flexible receptive field** – you can increase it by adding layers or larger dilation factors as needed ¹⁰. This means the network's “memory” of past instructions can be tuned to the task's needs more easily than an RNN's fixed state size. And unlike RNNs, a Conv1D processes the whole sequence in parallel, avoiding sequential bottlenecks and vanishing gradients through time ¹¹ ¹².

Proven performance: Conv1D architectures with these elements (dilations, residuals, gating or ReLU) have matched or surpassed recurrent nets in **audio generation, language modeling, and translation** tasks ¹. For example, DeepMind's WaveNet (audio) and Facebook's ConvS2S (translation) are fully convolutional sequence models that achieved state-of-art results using ~10-20 layers of 1D conv with residual/gated activations. The TCN formulation (Bai et al. 2018) distilled many of these best practices into a simple architecture that worked across diverse tasks ¹³ ¹⁴. All this gives confidence that a well-designed 1D CNN can **learn complex sequence patterns** (even something as unusual as quantum gate sequences) given sufficient depth and training.

Designing the Conv1D Network: Depth, Kernel Size, Channels

When building your 1D CNN policy network, consider the following design choices informed by the above architectures:

- **Kernel Size vs. Depth:** Instead of using an extremely large kernel (e.g. 130) in one layer, it's usually better to use a **small kernel (3-5)** and achieve a large receptive field by stacking layers (with dilation). Research shows that **small kernels focus on local patterns** (which is often important for “language-like” sequences), while stacking/dilating layers captures global context ¹⁵. Larger kernels can help very long-memory tasks converge a bit faster, but they didn't always improve final accuracy in language modeling ¹⁵. A reasonable approach is to start with kernel size 3 or 5 for all conv layers. With dilation doubling each layer, the receptive field grows exponentially – for example, 6 layers with kernel=3 gives a field of 127 time-steps, covering most circuits in your case. This way each layer learns **n-gram patterns of instructions**, and deeper layers combine them into longer motifs. (If you truly expect patterns spanning >100 instructions, you can increase depth or kernel slightly. But avoid one giant filter covering the whole sequence, as it may over-smooth or ignore the sequential structure.)

- **Number of Layers:** Use enough layers (with dilation) such that the network's receptive field $\geq$ the maximum sequence length you need to consider. Bai et al. note that **up to ~12 layers** might be needed if the model should potentially look across an entire long sequence ⁴. In practice, you might start with ~6–8 layers and adjust. Each residual block effectively counts as **one layer** in terms of receptive field growth (since both convs in the block typically use the same dilation before it increases in the next block). Ensure dilation resets or increases properly per layer. For example: layers 1–6 with dilation 1,2,4,8,16,32 (then you have receptive field $\sim 2^6$). If the sequence can be extremely long, you could either add more layers or use a second pass of dilation (some implementations cycle dilation after a certain point).
- **Channels (Filters) per Layer:** It's common to start with a moderate number of filters and **increase the channel count in deeper layers** (to capture higher-level features). For example, one successful design used 32 filters in the first conv layer, 64 in the second ¹⁶, and you could continue to 128, etc., as depth increases. This is similar to how image CNNs increase channels after each block. More channels = more feature maps to represent different instruction patterns. That said, increasing channels will increase model size, so find a balance. You might try a progression like $64 \rightarrow 64 \rightarrow 128 \rightarrow 128$ (depending on your compute budget). The **residual 1×1 conv** in each block will ensure the skip connection matches the new width ⁵ when you increase channels.
- **Activation Functions:** ReLU is standard (TCN used ReLU ⁸), but you are using **PReLU**, which is fine (it can help with negative values and maybe learns a slight bias in activation). The exact choice (ReLU, LeakyReLU, PReLU) is not critical as long as it's applied after each conv. Just ensure it's **within the residual block** (i.e., Conv $\rightarrow$ Activation $\rightarrow$ Norm $\rightarrow$ Conv $\rightarrow$ Activation $\rightarrow$ Norm, then add skip). Gated activations (e.g. GLU or gated tanh as in WaveNet/GLU) are another option. Some NLP conv models used **GLUs** which multiply a tanh and sigmoid branch for each conv output ¹⁷. Gating can improve modeling of certain data (TCN authors found GLU helped on word language modeling but not on other tasks) ¹⁸. If you're feeling experimental, you could try replacing ReLU with a gated unit in the conv blocks. However, **starting with a simple ReLU/PReLU is reasonable** – the **TCN without gating performed robustly across many tasks** ¹⁸.
- **Normalization & Regularization:** Continue using **Layer Normalization** after each conv (or each block) as you have – it helps with training stability, especially given the very different scales (gate indicators vs. continuous parameters) in your 150 features. Dropout is also important for RL to prevent overfitting to early trajectories. TCN used **Spatial Dropout** (drop entire channels) ⁵, which is effective for conv nets on sequences. In PyTorch, you can use `nn.Dropout1d` on the conv outputs to drop whole feature maps. This forces the network to not rely on any single feature channel too much. A dropout rate around 0.1–0.3 is a typical starting point. *Tip:* Place dropout **within the residual block** (e.g. after the first or second conv before adding the residual back). This was done in the original TCN design ⁸.
- **Pooling/Downsampling:** Many sequence CNNs (like TCN) do **not** downsample the time dimension at all – they keep output length equal to input length by padding ("same" convolution) ⁹. This is useful if you need a prediction at every time-step. In your case, you ultimately need a single output (policy & value) for the whole sequence, not at every step. There are two ways to handle this:
- **Global Pooling:** After the final conv layer, apply a **Global Average Pooling (1D)** across the sequence length, converting an output of shape $(seq_len, channels)$ into a single $1 \times channels$ vector. This yields a

fixed-length representation regardless of sequence length ¹⁹ ²⁰. Global Max Pooling is another option (taking the strongest feature activation over time). Global pooling is simple and makes the network truly length-agnostic – it’s often used in sequence classification CNNs to aggregate features ¹⁹.

- **Flatten with Masking/Padding:** Alternatively, you could pad all circuits to a max length and flatten the conv output, then use a fully-connected layer. However, this is less elegant and can introduce a dependence on the max length. Given you prefer an end-to-end conv solution, **Global Average Pooling is recommended** to summarize the sequence features ¹⁹. (It effectively acts like an adaptive stride that reduces the sequence dimension to 1.)

Both approaches end up with a **fixed-size feature vector** that represents the entire circuit. The rest of the network (policy/value heads) can then be standard fully-connected layers.

Integration into an Actor-Critic (A2C/PPO) Agent

To use this Conv1D architecture in an RL agent, you will treat it as a **feature extractor** that feeds into policy and value outputs:

- **Shared Conv Trunk:** In on-policy algorithms like A2C/PPO, it’s common to **share the feature extractor** between the policy and value networks ²¹. That is, you apply the Conv1D stack (with all the layers discussed above) to the input sequence *once*, get the final pooled feature vector, and then have **two output heads** on top of that: one for the action probability distribution $\pi(a|s)$, one for the value $V(s)$. Sharing the conv backbone saves computation and tends to work well unless the value function needs a very different representation. (Off-policy methods like SAC sometimes use separate encoders ²¹, but for simplicity you can start with a shared encoder.)
- **Policy Head:** Take the pooled feature vector and pass it through a fully-connected (dense) layer to produce a logit for each action. For example, a single `Linear(feats_dim, N_actions)` can output unnormalized scores for each of the N possible actions, which then go through a softmax to yield the policy distribution π . If you want a bit more capacity, you could insert one hidden layer (e.g. `Linear to 128 → ReLU → Linear to N_actions`), but often a direct projection is sufficient when the conv already did heavy lifting.
- **Value Head:** Similarly, take the same feature vector and feed it to a linear layer (or small MLP) to output a single scalar value V . For instance `Linear(feats_dim, 1)` gives the state-value. In an Actor-Critic, this value head shares the conv features but has its own weights for the linear output.
- **End-to-End Training:** The entire network (conv + heads) can be trained end-to-end with the RL algorithm’s loss functions (policy gradient loss + value loss, etc.). The conv layers will learn to extract those features from the instruction sequence that are relevant for predicting good actions and state value. Gradient backpropagation works through the conv blocks just like any neural net. Notably, because we use **global pooling**, the conv will receive signal about *the whole sequence’s outcome*. (If using “same” conv without pooling, you might choose the *last time-step output* to connect to the value/policy, but with circuits of varying length it’s safer to pool or average over all time-steps.)

Performance considerations: A Conv1D feature extractor can be heavier than a simple MLP, so monitor the inference speed if your sequences are long. However, convolution can be parallelized across the

sequence, so with a reasonable GPU this architecture should still run efficiently ¹¹. Also, by sharing the conv trunk between actor & critic, you roughly halve the computations compared to separate networks. Memory-wise, 1D conv layers are quite lean compared to sequence length – they don't need to unroll steps like RNNs, so memory usage scales with network depth rather than sequence length ²² (aside from the need to hold the whole sequence in the forward pass, which you're already doing).

Summary of the proposed architecture:

- **Input:** Quantum circuit as a sequence of instructions, each encoded as a 150-dim vector (features = gate type one-hot ± 1 , 3 parameters, 130 qubit role flags). So input tensor shape is `(batch, seq_len, 150)` if using batch-first, or `(batch, 150, seq_len)` for channels-first convention. (Many frameworks use batch $\times$ channels $\times$ length for Conv1D.)
- **Conv1D Stack:** e.g. 6–8 residual blocks, each with two Conv1D layers. Use kernel size 3–5, stride 1, **padding="same"**. Dilation grows each block (1,2,4,8,...). Start with 64 channels, increase to 128 in deeper layers. Each conv is followed by PReLU (or ReLU) and LayerNorm; each block has a skip connection. Apply dropout in blocks (e.g. 10–20% rate). This yields an output of shape `(batch, seq_len, C_final)` (or `(batch, C_final, seq_len)` in channels-first).
- **Global Pooling:** Average (or max) pool over the sequence dimension to get a `(batch, C_final)` vector. This ensures the network output doesn't depend on sequence length (works for any seq_len). ¹⁹
- **Policy Head:** Dense layer from the pooled vector to N_{action} outputs, then softmax to get π .
- **Value Head:** Dense layer from the pooled vector to 1 output (linear).

This design gives you a **fully convolutional pipeline** from input sequence to output, with no manual feature engineering. It leverages **known-effective CNN patterns** (dilated conv + residual + normalization) to handle sequential patterns. You can confidently initialize with this architecture since it has been used in many settings (from text to time-series) with good results ¹³ ¹⁴. Adjust the depth or width as needed based on performance (e.g. if the agent struggles to use very long-term information, you might add more layers or slightly enlarge kernels). But overall, this TCN-based design should provide a **strong starting point** for an RL policy network dealing with variable-length quantum circuit inputs.

Sources:

- Bai et al., “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling” – introduced the TCN architecture and showed Conv1D outperforms RNNs on many sequence tasks ¹ ¹¹. Describes using dilations, residual blocks, etc., for long sequences ⁴ ⁸. Residual connections improved training stability and accuracy ⁷.
- MATLAB Documentation – example of a 1D CNN for sequence classification (uses two Conv1D+ReLU+LayerNorm layers with small kernel=5, followed by global average pooling and dense output) ¹⁶ ¹⁹. Demonstrates a simple but effective Conv1D architecture for variable-length data.
- Stable Baselines3 – RL library noting that for policy networks, CNN feature extractors can be shared between actor and critic in on-policy methods like A2C/PPO ²¹ (recommending a single shared conv backbone with separate output layers for efficiency). This aligns with the proposed architecture for integrating the Conv1D into your RL agent.

1 2 3 4 5 6 7 8 10 11 12 15 17 18 22 [1803.01271] An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

<https://ar5iv.labs.arxiv.org/html/1803.01271>

9 13 14 Temporal Convolutional Networks and Forecasting - Unit8

<https://unit8.com/resources/temporal-convolutional-networks-and-forecasting/>

16 19 20 Sequence Classification Using 1-D Convolutions - MATLAB & Simulink

<https://www.mathworks.com/help/deeplearning/ug/sequence-classification-using-1-d-convolutions.html>

21 Policy Networks — Stable Baselines3 2.7.1a3 documentation

https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Shaojie Bai¹ J. Zico Kolter² Vladlen Koltun³

Abstract

For most deep learning practitioners, sequence modeling is synonymous with recurrent networks. Yet recent results indicate that convolutional architectures can outperform recurrent networks on tasks such as audio synthesis and machine translation. Given a new sequence modeling task or dataset, which architecture should one use? We conduct a systematic evaluation of generic convolutional and recurrent architectures for sequence modeling. The models are evaluated across a broad range of standard tasks that are commonly used to benchmark recurrent networks. Our results indicate that a simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks and datasets, while demonstrating longer effective memory. We conclude that the common association between sequence modeling and recurrent networks should be reconsidered, and convolutional networks should be regarded as a natural starting point for sequence modeling tasks. To assist related work, we have made code available at <http://github.com/locuslab/TCN>.

1. Introduction

Deep learning practitioners commonly regard recurrent architectures as the default starting point for sequence modeling tasks. The sequence modeling chapter in the canonical textbook on deep learning is titled “Sequence Modeling: Recurrent and Recursive Nets” (Goodfellow et al., 2016), capturing the common association of sequence modeling and recurrent architectures. A well-regarded recent online course on “Sequence Models” focuses exclusively on recurrent architectures (Ng, 2018).

¹Machine Learning Department, Carnegie Mellon University, Pittsburgh, PA, USA ²Computer Science Department, Carnegie Mellon University, Pittsburgh, PA, USA ³Intel Labs, Santa Clara, CA, USA. Correspondence to: Shaojie Bai <shaojie@cs.cmu.edu>, J. Zico Kolter <zkolter@cs.cmu.edu>, Vladlen Koltun <vkoltun@gmail.edu>.

On the other hand, recent research indicates that certain convolutional architectures can reach state-of-the-art accuracy in audio synthesis, word-level language modeling, and machine translation (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b). This raises the question of whether these successes of convolutional sequence modeling are confined to specific application domains or whether a broader reconsideration of the association between sequence processing and recurrent networks is in order.

We address this question by conducting a systematic empirical evaluation of convolutional and recurrent architectures on a broad range of sequence modeling tasks. We specifically target a comprehensive set of tasks that have been repeatedly used to compare the effectiveness of different recurrent network architectures. These tasks include polyphonic music modeling, word- and character-level language modeling, as well as synthetic stress tests that had been deliberately designed and frequently used to benchmark RNNs. Our evaluation is thus set up to compare convolutional and recurrent approaches to sequence modeling on the recurrent networks’ “home turf”.

To represent convolutional networks, we describe a generic temporal convolutional network (TCN) architecture that is applied across all tasks. This architecture is informed by recent research, but is deliberately kept simple, combining some of the best practices of modern convolutional architectures. It is compared to canonical recurrent architectures such as LSTMs and GRUs.

The results suggest that TCNs convincingly outperform baseline recurrent architectures across a broad range of sequence modeling tasks. This is particularly notable because the tasks include diverse benchmarks that have commonly been used to evaluate recurrent network designs (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Zhang et al., 2016). This indicates that the recent successes of convolutional architectures in applications such as audio processing are not confined to these domains.

To further understand these results, we analyze more deeply the memory retention characteristics of recurrent networks. We show that despite the theoretical ability of recurrent architectures to capture infinitely long history, TCNs exhibit

substantially longer memory, and are thus more suitable for domains where a long history is required.

To our knowledge, the presented study is the most extensive systematic comparison of convolutional and recurrent architectures on sequence modeling tasks. The results suggest that the common association between sequence modeling and recurrent networks should be reconsidered. The TCN architecture appears not only more accurate than canonical recurrent networks such as LSTMs and GRUs, but also simpler and clearer. It may therefore be a more appropriate starting point in the application of deep networks to sequences.

2. Background

Convolutional networks (LeCun et al., 1989) have been applied to sequences for decades (Sejnowski & Rosenberg, 1987; Hinton, 1989). They were used prominently for speech recognition in the 80s and 90s (Waibel et al., 1989; Bottou et al., 1990). ConvNets were subsequently applied to NLP tasks such as part-of-speech tagging and semantic role labelling (Collobert & Weston, 2008; Collobert et al., 2011; dos Santos & Zadrozny, 2014). More recently, convolutional networks were applied to sentence classification (Kalchbrenner et al., 2014; Kim, 2014) and document classification (Zhang et al., 2015; Conneau et al., 2017; Johnson & Zhang, 2015; 2017). Particularly inspiring for our work are the recent applications of convolutional architectures to machine translation (Kalchbrenner et al., 2016; Gehring et al., 2017a;b), audio synthesis (van den Oord et al., 2016), and language modeling (Dauphin et al., 2017).

Recurrent networks are dedicated sequence models that maintain a vector of hidden activations that are propagated through time (Elman, 1990; Werbos, 1990; Graves, 2012). This family of architectures has gained tremendous popularity due to prominent applications to language modeling (Sutskever et al., 2011; Graves, 2013; Hermans & Schrauwen, 2013) and machine translation (Sutskever et al., 2014; Bahdanau et al., 2015). The intuitive appeal of recurrent modeling is that the hidden state can act as a representation of everything that has been seen so far in the sequence. Basic RNN architectures are notoriously difficult to train (Bengio et al., 1994; Pascanu et al., 2013) and more elaborate architectures are commonly used instead, such as the LSTM (Hochreiter & Schmidhuber, 1997) and the GRU (Cho et al., 2014). Many other architectural innovations and training techniques for recurrent networks have been introduced and continue to be actively explored (El Hichi & Bengio, 1995; Schuster & Paliwal, 1997; Gers et al., 2002; Koutnik et al., 2014; Le et al., 2015; Ba et al., 2016; Wu et al., 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018).

Multiple empirical studies have been conducted to evaluate the effectiveness of different recurrent architectures. These studies have been motivated in part by the many degrees of freedom in the design of such architectures. Chung et al. (2014) compared different types of recurrent units (LSTM vs. GRU) on the task of polyphonic music modeling. Pascanu et al. (2014) explored different ways to construct deep RNNs and evaluated the performance of different architectures on polyphonic music modeling, character-level language modeling, and word-level language modeling. Jozefowicz et al. (2015) searched through more than ten thousand different RNN architectures and evaluated their performance on various tasks. They concluded that if there were “architectures much better than the LSTM”, then they were “not trivial to find”. Greff et al. (2017) benchmarked the performance of eight LSTM variants on speech recognition, handwriting recognition, and polyphonic music modeling. They also found that “none of the variants can improve upon the standard LSTM architecture significantly”. Zhang et al. (2016) systematically analyzed the connecting architectures of RNNs and evaluated different architectures on character-level language modeling and on synthetic stress tests. Melis et al. (2018) benchmarked LSTM-based architectures on word-level and character-level language modeling, and concluded that “LSTMs outperform the more recent models”.

Other recent works have aimed to combine aspects of RNN and CNN architectures. This includes the Convolutional LSTM (Shi et al., 2015), which replaces the fully-connected layers in an LSTM with convolutional layers to allow for additional structure in the recurrent layers; the Quasi-RNN model (Bradbury et al., 2017) that interleaves convolutional layers with simple recurrent layers; and the dilated RNN (Chang et al., 2017), which adds dilations to recurrent architectures. While these combinations show promise in combining the desirable aspects of both types of architectures, our study here focuses on a comparison of generic convolutional and recurrent architectures.

While there have been multiple thorough evaluations of RNN architectures on representative sequence modeling tasks, we are not aware of a similarly thorough comparison of convolutional and recurrent approaches to sequence modeling. (Yin et al. (2017) have reported a comparison of convolutional and recurrent networks for sentence-level and document-level classification tasks. In contrast, sequence modeling calls for architectures that can synthesize whole sequences, element by element.) Such comparison is particularly intriguing in light of the aforementioned recent success of convolutional architectures in this domain. Our work aims to compare generic convolutional and recurrent architectures on typical sequence modeling tasks that are commonly used to benchmark RNN variants themselves (Hermans & Schrauwen, 2013; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016).

3. Temporal Convolutional Networks

We begin by describing a generic architecture for convolutional sequence prediction. Our aim is to distill the best practices in convolutional network design into a simple architecture that can serve as a convenient but powerful starting point. We refer to the presented architecture as a temporal convolutional network (TCN), emphasizing that we adopt this term not as a label for a truly new architecture, but as a simple descriptive term for a family of architectures. (Note that the term has been used before (Lea et al., 2017).) The distinguishing characteristics of TCNs are: 1) the convolutions in the architecture are causal, meaning that there is no information “leakage” from future to past; 2) the architecture can take a sequence of any length and map it to an output sequence of the same length, just as with an RNN. Beyond this, we emphasize how to build very long effective history sizes (i.e., the ability for the networks to look very far into the past to make a prediction) using a combination of very deep networks (augmented with residual layers) and dilated convolutions.

Our architecture is informed by recent convolutional architectures for sequential data (van den Oord et al., 2016; Kalchbrenner et al., 2016; Dauphin et al., 2017; Gehring et al., 2017a;b), but is distinct from all of them and was designed from first principles to combine simplicity, autoregressive prediction, and very long memory. For example, the TCN is much simpler than WaveNet (van den Oord et al., 2016) (no skip connections across layers, conditioning, context stacking, or gated activations).

Compared to the language modeling architecture of Dauphin et al. (2017), TCNs do not use gating mechanisms and have much longer memory.

3.1. Sequence Modeling

Before defining the network structure, we highlight the nature of the sequence modeling task. Suppose that we are given an input sequence $x_0, \dots, x_T$, and wish to predict some corresponding outputs $y_0, \dots, y_T$ at each time. The key constraint is that to predict the output y_t for some time t , we are constrained to only use those inputs that have been previously observed: $x_0, \dots, x_t$. Formally, a sequence modeling network is any function $f : \mathcal{X}^{T+1} \rightarrow \mathcal{Y}^{T+1}$ that produces the mapping

$$\hat{y}_0, \dots, \hat{y}_T = f(x_0, \dots, x_T) \quad (1)$$

if it satisfies the causal constraint that y_t depends only on $x_0, \dots, x_t$ and not on any “future” inputs $x_{t+1}, \dots, x_T$. The goal of learning in the sequence modeling setting is to find a network f that minimizes some expected loss between the actual outputs and the predictions, $L(y_0, \dots, y_T, f(x_0, \dots, x_T))$, where the sequences and outputs are drawn according to some distribution.

This formalism encompasses many settings such as autoregressive prediction (where we try to predict some signal given its past) by setting the target output to be simply the input shifted by one time step. It does not, however, directly capture domains such as machine translation, or sequence-to-sequence prediction in general, since in these cases the entire input sequence (including “future” states) can be used to predict each output (though the techniques can naturally be extended to work in such settings).

3.2. Causal Convolutions

As mentioned above, the TCN is based upon two principles: the fact that the network produces an output of the same length as the input, and the fact that there can be no leakage from the future into the past. To accomplish the first point, the TCN uses a 1D fully-convolutional network (FCN) architecture (Long et al., 2015), where each hidden layer is the same length as the input layer, and zero padding of length (kernel size $- 1$) is added to keep subsequent layers the same length as previous ones. To achieve the second point, the TCN uses *causal convolutions*, convolutions where an output at time t is convolved only with elements from time t and earlier in the previous layer.

To put it simply: TCN = 1D FCN + causal convolutions.

Note that this is essentially the same architecture as the time delay neural network proposed nearly 30 years ago by Waibel et al. (1989), with the sole tweak of zero padding to ensure equal sizes of all layers.

A major disadvantage of this basic design is that in order to achieve a long effective history size, we need an extremely deep network or very large filters, neither of which were particularly feasible when the methods were first introduced. Thus, in the following sections, we describe how techniques from modern convolutional architectures can be integrated into a TCN to allow for both very deep networks and very long effective history.

3.3. Dilated Convolutions

A simple causal convolution is only able to look back at a history with size linear in the depth of the network. This makes it challenging to apply the aforementioned causal convolution on sequence tasks, especially those requiring longer history. Our solution here, following the work of van den Oord et al. (2016), is to employ dilated convolutions that enable an exponentially large receptive field (Yu & Koltun, 2016). More formally, for a 1-D sequence input $\mathbf{x} \in \mathbb{R}^n$ and a filter $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$, the dilated convolution operation F on element s of the sequence is defined as

$$F(s) = (\mathbf{x} *_{\text{d}} f)(s) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}_{s-d \cdot i} \quad (2)$$

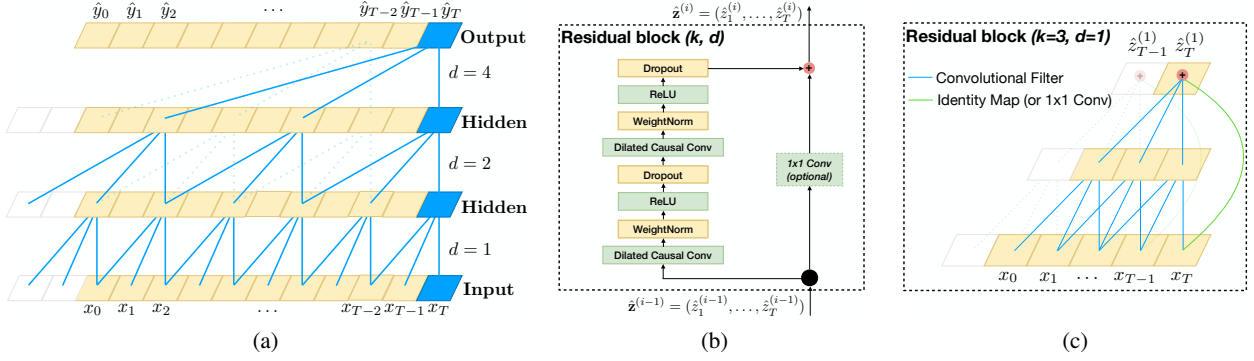


Figure 1. Architectural elements in a TCN. (a) A dilated causal convolution with dilation factors $d = 1, 2, 4$ and filter size $k = 3$. The receptive field is able to cover all values from the input sequence. (b) TCN residual block. An 1x1 convolution is added when residual input and output have different dimensions. (c) An example of residual connection in a TCN. The blue lines are filters in the residual function, and the green lines are identity mappings.

where d is the dilation factor, k is the filter size, and $s - d \cdot i$ accounts for the direction of the past. Dilation is thus equivalent to introducing a fixed step between every two adjacent filter taps. When $d = 1$, a dilated convolution reduces to a regular convolution. Using larger dilation enables an output at the top level to represent a wider range of inputs, thus effectively expanding the receptive field of a ConvNet.

This gives us two ways to increase the receptive field of the TCN: choosing larger filter sizes k and increasing the dilation factor d , where the effective history of one such layer is $(k - 1)d$. As is common when using dilated convolutions, we increase d exponentially with the depth of the network (i.e., $d = O(2^i)$ at level i of the network). This ensures that there is some filter that hits each input within the effective history, while also allowing for an extremely large effective history using deep networks. We provide an illustration in Figure 1(a).

3.4. Residual Connections

A residual block (He et al., 2016) contains a branch leading out to a series of transformations $\mathcal{F}$, whose outputs are added to the input $\mathbf{x}$ of the block:

$$o = \text{Activation}(\mathbf{x} + \mathcal{F}(\mathbf{x})) \quad (3)$$

This effectively allows layers to learn modifications to the identity mapping rather than the entire transformation, which has repeatedly been shown to benefit very deep networks.

Since a TCN’s receptive field depends on the network depth n as well as filter size k and dilation factor d , stabilization of deeper and larger TCNs becomes important. For example, in a case where the prediction could depend on a history of size 2^{12} and a high-dimensional input sequence, a network of up to 12 layers could be needed. Each layer, more specifically, consists of multiple filters for feature extraction. In our design of the generic TCN model, we therefore employ a generic residual module in place of a convolutional layer.

The residual block for our baseline TCN is shown in Figure 1(b). Within a residual block, the TCN has two layers of dilated causal convolution and non-linearity, for which we used the rectified linear unit (ReLU) (Nair & Hinton, 2010). For normalization, we applied weight normalization (Salimans & Kingma, 2016) to the convolutional filters. In addition, a spatial dropout (Srivastava et al., 2014) was added after each dilated convolution for regularization: at each training step, a whole channel is zeroed out.

However, whereas in standard ResNet the input is added directly to the output of the residual function, in TCN (and ConvNets in general) the input and output could have different widths. To account for discrepant input-output widths, we use an additional 1x1 convolution to ensure that element-wise addition $\oplus$ receives tensors of the same shape (see Figure 1(b,c)).

3.5. Discussion

We conclude this section by listing several advantages and disadvantages of using TCNs for sequence modeling.

- **Parallelism.** Unlike in RNNs where the predictions for later timesteps must wait for their predecessors to complete, convolutions can be done in parallel since the same filter is used in each layer. Therefore, in both training and evaluation, a long input sequence can be processed as a whole in TCN, instead of sequentially as in RNN.
- **Flexible receptive field size.** A TCN can change its receptive field size in multiple ways. For instance, stacking more dilated (causal) convolutional layers, using larger dilation factors, or increasing the filter size are all viable options (with possibly different interpretations). TCNs thus afford better control of the model’s memory size, and are easy to adapt to different domains.
- **Stable gradients.** Unlike recurrent architectures, TCN has a backpropagation path different from the temporal direction of the sequence. TCN thus avoids the problem

of exploding/vanishing gradients, which is a major issue for RNNs (and which led to the development of LSTM, GRU, HF-RNN (Martens & Sutskever, 2011), etc.).

- **Low memory requirement for training.** Especially in the case of a long input sequence, LSTMs and GRUs can easily use up a lot of memory to store the partial results for their multiple cell gates. However, in a TCN the filters are shared across a layer, with the backpropagation path depending only on network depth. Therefore in practice, we found gated RNNs likely to use up to a multiplicative factor more memory than TCNs.
- **Variable length inputs.** Just like RNNs, which model inputs with variable lengths in a recurrent way, TCNs can also take in inputs of arbitrary lengths by sliding the 1D convolutional kernels. This means that TCNs can be adopted as drop-in replacements for RNNs for sequential data of arbitrary length.

There are also two notable disadvantages to using TCNs.

- **Data storage during evaluation.** In evaluation/testing, RNNs only need to maintain a hidden state and take in a current input x_t in order to generate a prediction. In other words, a “summary” of the entire history is provided by the fixed-length set of vectors h_t , and the actual observed sequence can be discarded. In contrast, TCNs need to take in the raw sequence up to the effective history length, thus possibly requiring more memory during evaluation.
- **Potential parameter change for a transfer of domain.** Different domains can have different requirements on the amount of history the model needs in order to predict. Therefore, when transferring a model from a domain where only little memory is needed (i.e., small k and d) to a domain where much longer memory is required (i.e., much larger k and d), TCN may perform poorly for not having a sufficiently large receptive field.

4. Sequence Modeling Tasks

We evaluate TCNs and RNNs on tasks that have been commonly used to benchmark the performance of different RNN sequence modeling architectures (Hermans & Schrauwen, 2013; Chung et al., 2014; Pascanu et al., 2014; Le et al., 2015; Jozefowicz et al., 2015; Zhang et al., 2016). The intention is to conduct the evaluation on the “home turf” of RNN sequence models. We use a comprehensive set of synthetic stress tests along with real-world datasets from multiple domains.

The adding problem. In this task, each input consists of a length- n sequence of depth 2, with all values randomly chosen in $[0, 1]$, and the second dimension being all zeros except for two elements that are marked by 1. The objective is to sum the two random values whose second dimensions

are marked by 1. Simply predicting the sum to be 1 should give an MSE of about 0.1767. First introduced by Hochreiter & Schmidhuber (1997), the adding problem has been used repeatedly as a stress test for sequence models (Martens & Sutskever, 2011; Pascanu et al., 2013; Le et al., 2015; Arjovsky et al., 2016; Zhang et al., 2016).

Sequential MNIST and P-MNIST. Sequential MNIST is frequently used to test a recurrent network’s ability to retain information from the distant past (Le et al., 2015; Zhang et al., 2016; Wisdom et al., 2016; Cooijmans et al., 2016; Krueger et al., 2017; Jing et al., 2017). In this task, MNIST images (LeCun et al., 1998) are presented to the model as a 784×1 sequence for digit classification. In the more challenging P-MNIST setting, the order of the sequence is permuted at random (Le et al., 2015; Arjovsky et al., 2016; Wisdom et al., 2016; Krueger et al., 2017).

Copy memory. In this task, each input sequence has length $T + 20$. The first 10 values are chosen randomly among the digits $1, \dots, 8$, with the rest being all zeros, except for the last 11 entries that are filled with the digit ‘9’ (the first ‘9’ is a delimiter). The goal is to generate an output of the same length that is zero everywhere except the last 10 values after the delimiter, where the model is expected to repeat the 10 values it encountered at the start of the input. This task was used in prior works such as Zhang et al. (2016); Arjovsky et al. (2016); Wisdom et al. (2016); Jing et al. (2017).

JSB Chorales and Nottingham. JSB Chorales (Allan & Williams, 2005) is a polyphonic music dataset consisting of the entire corpus of 382 four-part harmonized chorales by J. S. Bach. Each input is a sequence of elements. Each element is an 88-bit binary code that corresponds to the 88 keys on a piano, with 1 indicating a key that is pressed at a given time. Nottingham is a polyphonic music dataset based on a collection of 1,200 British and American folk tunes, and is much larger than JSB Chorales. JSB Chorales and Nottingham have been used in numerous empirical investigations of recurrent sequence modeling (Chung et al., 2014; Pascanu et al., 2014; Jozefowicz et al., 2015; Greff et al., 2017). The performance on both tasks is measured in terms of negative log-likelihood (NLL).

PennTreebank. We used the PennTreebank (PTB) (Marcus et al., 1993) for both character-level and word-level language modeling. When used as a character-level language corpus, PTB contains 5,059K characters for training, 396K for validation, and 446K for testing, with an alphabet size of 50. When used as a word-level language corpus, PTB contains 888K words for training, 70K for validation, and 79K for testing, with a vocabulary size of 10K. This is a highly studied but relatively small language modeling dataset (Miyamoto & Cho, 2016; Krueger et al., 2017; Merity et al., 2017).

Wikitext-103. Wikitext-103 (Merity et al., 2016) is almost

Table 1. Evaluation of TCNs and recurrent architectures on synthetic stress tests, polyphonic music modeling, character-level language modeling, and word-level language modeling. The generic TCN architecture outperforms canonical recurrent networks across a comprehensive suite of tasks and datasets. Current state-of-the-art results are listed in the supplement. ^h means that higher is better. ^ℓ means that lower is better.

Sequence Modeling Task	Model Size ($\approx$)	Models			
		LSTM	GRU	RNN	TCN
Seq. MNIST (accuracy ^h)	70K	87.2	96.2	21.5	99.0
Permuted MNIST (accuracy)	70K	85.7	87.3	25.3	97.2
Adding problem $T=600$ (loss ^ℓ)	70K	0.164	5.3e-5	0.177	5.8e-5
Copy memory $T=1000$ (loss)	16K	0.0204	0.0197	0.0202	3.5e-5
Music JSB Chorales (loss)	300K	8.45	8.43	8.91	8.10
Music Nottingham (loss)	1M	3.29	3.46	4.05	3.07
Word-level PTB (perplexity ^ℓ)	13M	78.93	92.48	114.50	88.68
Word-level Wiki-103 (perplexity)	-	48.4	-	-	45.19
Word-level LAMBADA (perplexity)	-	4186	-	14725	1279
Char-level PTB (bpc ^ℓ)	3M	1.36	1.37	1.48	1.31
Char-level text8 (bpc)	5M	1.50	1.53	1.69	1.45

110 times as large as PTB, featuring a vocabulary size of about 268K. The dataset contains 28K Wikipedia articles (about 103 million words) for training, 60 articles (about 218K words) for validation, and 60 articles (246K words) for testing. This is a more representative and realistic dataset than PTB, with a much larger vocabulary that includes many rare words, and has been used in Merity et al. (2016); Grave et al. (2017); Dauphin et al. (2017).

LAMBADA. Introduced by Paperno et al. (2016), LAMBADA is a dataset comprising 10K passages extracted from novels, with an average of 4.6 sentences as context, and 1 target sentence the last word of which is to be predicted. This dataset was built so that a person can easily guess the missing word when given the context sentences, but not when given only the target sentence without the context sentences. Most of the existing models fail on LAMBADA (Paperno et al., 2016; Grave et al., 2017). In general, better results on LAMBADA indicate that a model is better at capturing information from longer and broader context. The training data for LAMBADA is the full text of 2,662 novels with more than 200M words. The vocabulary size is about 93K.

text8. We also used the text8 dataset for character-level language modeling (Mikolov et al., 2012). text8 is about 20 times larger than PTB, with about 100M characters from Wikipedia (90M for training, 5M for validation, and 5M for testing). The corpus contains 27 unique alphabets.

5. Experiments

We compare the generic TCN architecture described in Section 3 to canonical recurrent architectures, namely LSTM, GRU, and vanilla RNN, with standard regularizations. All

experiments reported in this section used exactly the same TCN architecture, just varying the depth of the network n and occasionally the kernel size k so that the receptive field covers enough context for predictions. We use an exponential dilation $d = 2^i$ for layer i in the network, and the Adam optimizer (Kingma & Ba, 2015) with learning rate 0.002 for TCN, unless otherwise noted. We also empirically find that gradient clipping helped convergence, and we pick the maximum norm for clipping from $[0.3, 1]$. When training recurrent models, we use grid search to find a good set of hyperparameters (in particular, optimizer, recurrent drop $p \in [0.05, 0.5]$, learning rate, gradient clipping, and initial forget-gate bias), while keeping the network around the same size as TCN. No other architectural elaborations, such as gating mechanisms or skip connections, were added to either TCNs or RNNs. Additional details and controlled experiments are provided in the supplementary material.

5.1. Synopsis of Results

A synopsis of the results is shown in Table 1. Note that on several of these tasks, the generic, canonical recurrent architectures we study (e.g., LSTM, GRU) are not the state-of-the-art. (See the supplement for more details.) With this caveat, the results strongly suggest that the generic TCN architecture *with minimal tuning* outperforms canonical recurrent architectures across a broad variety of sequence modeling tasks that are commonly used to benchmark the performance of recurrent architectures themselves. We now analyze these results in more detail.

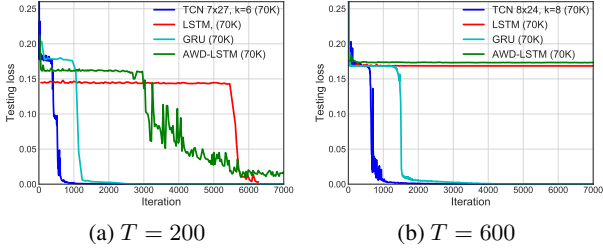


Figure 2. Results on the adding problem for different sequence lengths T . TCNs outperform recurrent architectures.

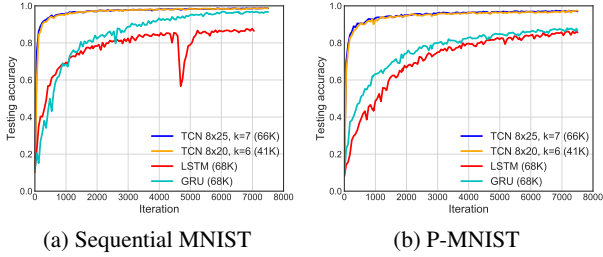


Figure 3. Results on Sequential MNIST and P-MNIST. TCNs outperform recurrent architectures.

5.2. Synthetic Stress Tests

The adding problem. Convergence results for the adding problem, for problem sizes $T = 200$ and 600 , are shown in Figure 2. All models were chosen to have roughly 70K parameters. TCNs quickly converged to a virtually perfect solution (i.e., MSE near 0). GRUs also performed quite well, albeit slower to converge than TCNs. LSTMs and vanilla RNNs performed significantly worse.

Sequential MNIST and P-MNIST. Convergence results on sequential and permuted MNIST, run over 10 epochs, are shown in Figure 3. All models were configured to have roughly 70K parameters. For both problems, TCNs substantially outperform the recurrent architectures, both in terms of convergence and in final accuracy on the task. For P-MNIST, TCNs outperform state-of-the-art results (95.9%) based on recurrent networks with Zoneout and Recurrent BatchNorm (Cooijmans et al., 2016; Krueger et al., 2017).

Copy memory. Convergence results on the copy memory task are shown in Figure 4. TCNs quickly converge to correct answers, while LSTMs and GRUs simply converge to the same loss as predicting all zeros. In this case we also compare to the recently-proposed EURNN (Jing et al., 2017), which was highlighted to perform well on this task. While both TCN and EURNN perform well for sequence length $T = 500$, the TCN has a clear advantage for $T = 1000$ and longer (in terms of both loss and rate of convergence).

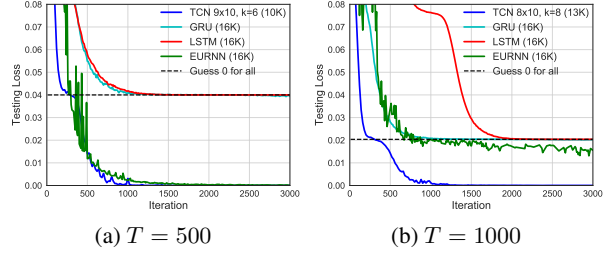


Figure 4. Result on the copy memory task for different sequence lengths T . TCNs outperform recurrent architectures.

5.3. Polyphonic Music and Language Modeling

We now discuss the results on polyphonic music modeling, character-level language modeling, and word-level language modeling. These domains are dominated by recurrent architectures, with many specialized designs developed for these tasks (Zhang et al., 2016; Ha et al., 2017; Krueger et al., 2017; Grave et al., 2017; Greff et al., 2017; Merity et al., 2017). We mention some of these specialized architectures when useful, but our primary goal is to compare the generic TCN model to similarly generic recurrent architectures, before domain-specific tuning. The results are summarized in Table 1.

Polyphonic music. On Nottingham and JSB Chorales, the TCN with virtually no tuning outperforms the recurrent models by a considerable margin, and even outperforms some enhanced recurrent architectures for this task such as HF-RNN (Boulanger-Lewandowski et al., 2012) and Diagonal RNN (Subakan & Smaragdis, 2017). Note however that other models such as the Deep Belief Net LSTM perform better still (Vohra et al., 2015); we believe this is likely due to the fact that the datasets are relatively small, and thus the right regularization method or generative modeling procedure can improve performance significantly. This is largely orthogonal to the RNN/TCN distinction, as a similar variant of TCN may well be possible.

Word-level language modeling. Language modeling remains one of the primary applications of recurrent networks and many recent works have focused on optimizing LSTMs for this task (Krueger et al., 2017; Merity et al., 2017). Our implementation follows standard practice that ties the weights of encoder and decoder layers for both TCN and RNNs (Press & Wolf, 2016), which significantly reduces the number of parameters in the model. For training, we use SGD and anneal the learning rate by a factor of 0.5 for both TCN and RNNs when validation accuracy plateaus.

On the smaller PTB corpus, an optimized LSTM architecture (with recurrent and embedding dropout, etc.) outperforms the TCN, while the TCN outperforms both GRU and vanilla RNN. However, on the much larger Wikitext-103 corpus and the LAMBADA dataset (Paperno et al., 2016), without any hyperparameter search, the TCN outperforms

the LSTM results of Grave et al. (2017), achieving much lower perplexities.

Character-level language modeling. On character-level language modeling (PTB and text8, accuracy measured in bits per character), the generic TCN outperforms regularized LSTMs and GRUs as well as methods such as Normalized LSTMs (Krueger & Memisevic, 2015). (Specialized architectures exist that outperform all of these, see the supplement.)

5.4. Memory Size of TCN and RNNs

One of the theoretical advantages of recurrent architectures is their unlimited memory: the theoretical ability to retain information through sequences of unlimited length. We now examine specifically how long the different architectures can retain information in practice. We focus on 1) the copy memory task, which is a stress test designed to evaluate long-term, distant information propagation in recurrent networks, and 2) the LAMBADA task, which tests both local and non-local textual understanding.

The copy memory task is perfectly set up to examine a model’s ability to retain information for different lengths of time. The requisite retention time can be controlled by varying the sequence length T . In contrast to Section 5.2, we now focus on the accuracy on the last 10 elements of the output sequence (which are the nontrivial elements that must be recalled). We used models of size 10K for both TCN and RNNs.

The results of this focused study are shown in Figure 5. TCNs consistently converge to 100% accuracy for all sequence lengths, whereas LSTMs and GRUs of the same size quickly degenerate to random guessing as the sequence length T grows. The accuracy of the LSTM falls below 20% for $T < 50$, while the GRU falls below 20% for $T < 200$. These results indicate that TCNs are able to maintain a much longer effective history than their recurrent counterparts.

This observation is backed up on real data by experiments on the large-scale LAMBADA dataset, which is specifically designed to test a model’s ability to utilize broad context (Paperno et al., 2016). As shown in Table 1, TCN outperforms LSTMs and vanilla RNNs by a significant margin in perplexity on LAMBADA, with a substantially smaller network and virtually no tuning. (State-of-the-art results on this dataset are even better, but only with the help of additional memory mechanisms (Grave et al., 2017).)

6. Conclusion

We have presented an empirical evaluation of generic convolutional and recurrent architectures across a comprehensive suite of sequence modeling tasks. To this end, we have described a simple temporal convolutional network (TCN)

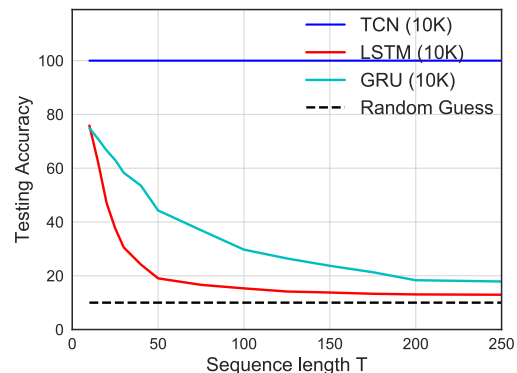


Figure 5. Accuracy on the copy memory task for sequences of different lengths T . While TCN exhibits 100% accuracy for all sequence lengths, the LSTM and GRU degenerate to random guessing as T grows.

that combines best practices such as dilations and residual connections with the causal convolutions needed for autoregressive prediction. The experimental results indicate that TCN models substantially outperform generic recurrent architectures such as LSTMs and GRUs. We further studied long-range information propagation in convolutional and recurrent networks, and showed that the “infinite memory” advantage of RNNs is largely absent in practice. TCNs exhibit longer memory than recurrent architectures with the same capacity.

Numerous advanced schemes for regularizing and optimizing LSTMs have been proposed (Press & Wolf, 2016; Krueger et al., 2017; Merity et al., 2017; Campos et al., 2018). These schemes have significantly advanced the accuracy achieved by LSTM-based architectures on some datasets. The TCN has not yet benefited from this concerted community-wide investment into architectural and algorithmic elaborations. We see such investment as desirable and expect it to yield advances in TCN performance that are commensurate with the advances seen in recent years in LSTM performance. We will release the code for our project to encourage this exploration.

The preeminence enjoyed by recurrent networks in sequence modeling may be largely a vestige of history. Until recently, before the introduction of architectural elements such as dilated convolutions and residual connections, convolutional architectures were indeed weaker. Our results indicate that with these elements, a simple convolutional architecture is more effective across diverse sequence modeling tasks than recurrent architectures such as LSTMs. Due to the comparable clarity and simplicity of TCNs, we conclude that convolutional networks should be regarded as a natural starting point and a powerful toolkit for sequence modeling.

An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling

Supplementary Material

A. Hyperparameters Settings

A.1. Hyperparameters for TCN

Table 2 lists the hyperparameters we used when applying the generic TCN model on various tasks and datasets. The most important factor for picking parameters is to make sure that the TCN has a sufficiently large receptive field by choosing k and d that can cover the amount of context needed for the task.

As discussed in Section 5, the number of hidden units was chosen so that the model size is approximately at the same level as the recurrent models with which we are comparing. In Table 2, a gradient clip of N/A means no gradient clipping was applied. In larger tasks (e.g., language modeling), we empirically found that gradient clipping (we randomly picked a threshold from $[0.3, 1]$) helps with regularizing TCN and accelerating convergence.

All weights were initialized from a Gaussian distribution $\mathcal{N}(0, 0.01)$. In general, we found TCN to be relatively insensitive to hyperparameter changes, as long as the effective history (i.e., receptive field) size is sufficient.

A.2. Hyperparameters for LSTM/GRU

Table 3 reports hyperparameter settings that were used for the LSTM. These values are picked from hyperparameter search for LSTMs that have up to 3 layers, and the optimizers are chosen from {SGD, Adam, RMSprop, Adagrad}. For certain larger datasets, we adopted the settings used in prior work (e.g., [Grave et al. \(2017\)](#) on Wikitext-103). GRU hyperparameters were chosen in a similar fashion, but typically with more hidden units than in LSTM to keep the total network size approximately the same (since a GRU cell is more compact).

B. State-of-the-Art Results

As previously noted, the generic TCN and LSTM/GRU models we used can be outperformed by more specialized architectures on some tasks. State-of-the-art results are summarized in Table 4. The same TCN architecture is used across all tasks. Note that the size of the state-of-the-art model may be different from the size of the TCN.

C. Effect of Filter Size and Residual Block

In this section we briefly study the effects of different components of a TCN layer. Overall, we believe dilation is required for modeling long-term dependencies, and so we mainly focus on two other factors here: the filter size k used by each layer, and the effect of residual blocks.

We perform a series of controlled experiments, with the results of the ablative analysis shown in Figure 6. As before, we kept the model size and depth exactly the same for different models, so that the dilation factor is strictly controlled. The experiments were conducted on three different tasks: copy memory, permuted MNIST (P-MNIST), and Penn Treebank word-level language modeling. These experiments confirm that both factors (filter size and residual connections) contribute to sequence modeling performance.

Filter size k . In both the copy memory and the P-MNIST tasks, we observed faster convergence and better accuracy for larger filter sizes. In particular, looking at Figure 6a, a TCN with filter size ≤ 3 only converges to the same level as random guessing. In contrast, on word-level language modeling, a smaller kernel with filter size of $k = 3$ works best. We believe this is because a smaller kernel (along with fixed dilation) tends to focus more on the local context, which is especially important for PTB language modeling (in fact, the very success of n -gram models suggests that only a relatively short memory is needed for modeling language).

Residual block. In all three scenarios that we compared here, we observed that the residual function stabilized training and brought faster convergence with better final results. Especially in language modeling, we found that residual connections contribute substantially to performance (See Figure 6f).

D. Gating Mechanisms

One component that had been used in prior work on convolutional architectures for language modeling is the gated activation ([van den Oord et al., 2016](#); [Dauphin et al., 2017](#)). We have chosen not to use gating in the generic TCN model. We now examine this choice more closely.

Table 2. TCN parameter settings for experiments in Section 5.

TCN SETTINGS							
Dataset/Task	Subtask	k	n	Hidden	Dropout	Grad Clip	Note
The Adding Problem	$T = 200$	6	7	27	0.0	N/A	
	$T = 400$	7	7	27			
	$T = 600$	8	8	24			
Seq. MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Permuted MNIST	-	7	8	25	0.0	N/A	
		6	8	20			
Copy Memory Task	$T = 500$	6	9	10	0.05	1.0	RMSprop 5e-4
	$T = 1000$	8	8	10			
	$T = 2000$	8	9	10			
Music JSB Chorales	-	3	2	150	0.5	0.4	
Music Nottingham	-	6	4	150	0.2	0.4	
Word-level LM	PTB	3	4	600	0.5	0.4	Embed. size 600
	Wiki-103	3	5	1000	0.4		Embed. size 400
	LAMBADA	4	5	500			Embed. size 500
Char-level LM	PTB	3	3	450	0.1	0.15	Embed. size 100
	text8	2	5	520			

Dauphin et al. (2017) compared the effects of gated linear units (GLU) and gated tanh units (GTU), and adopted GLU in their non-dilated gated ConvNet. Following the same choice, we now compare TCNs using ReLU and TCNs with gating (GLU), represented by an elementwise product between two convolutional layers, with one of them also passing through a sigmoid function $\sigma(x)$. Note that the gates architecture uses approximately twice as many convolutional layers as the ReLU-TCN.

The results are shown in Table 5, where we kept the number of model parameters at about the same size. The GLU does further improve TCN accuracy on certain language modeling datasets like PTB, which agrees with prior work. However, we do not observe comparable benefits on other tasks, such as polyphonic music modeling or synthetic stress tests that require longer information retention. On the copy memory task with $T = 1000$, we found that TCN with gating converged to a worse result than TCN with ReLU (though still better than recurrent models).

Table 3. LSTM parameter settings for experiments in Section 5.

LSTM SETTINGS (KEY PARAMETERS)							
Dataset/Task	Subtask	n	Hidden	Dropout	Grad Clip	Bias	Note
The Adding Problem	$T = 200$	2	77	0.0	50	5.0	SGD 1e-3
	$T = 400$	2	77		50	10.0	Adam 2e-3
	$T = 600$	1	130		5	1.0	-
Seq. MNIST	-	1	130	0.0	1	1.0	RMSprop 1e-3
Permuted MNIST	-	1	130	0.0	1	10.0	RMSprop 1e-3
Copy Memory Task	$T = 500$	1	50	0.05	0.25	-	RMSprop/Adam
	$T = 1000$	1	50		1		
	$T = 2000$	3	28		1		
Music JSB Chorales	-	2	200	0.2	1	10.0	SGD/Adam
Music Nottingham	-	3	280	0.1	0.5	-	Adam 4e-3
		1	500		1	-	
Word-level LM	PTB	3	700	0.4	0.3	1.0	SGD 30, Emb. 700, etc.
	Wiki-103	-	-	-	-	-	Grave et al. (2017)
	LAMBADA	-	-	-	-	-	Grave et al. (2017)
Char-level LM	PTB	2	600	0.1	0.5	-	Emb. size 120
	text8	1	1024	0.15	0.5	-	Adam 1e-2

Table 4. State-of-the-art (SoTA) results for tasks in Section 5.

TCN vs. SoTA RESULTS					
Task	TCN Result	Size	SoTA	Size	Model
Seq. MNIST (acc.)	99.0	21K	99.0	21K	Dilated GRU (Chang et al., 2017)
P-MNIST (acc.)	97.2	42K	95.9	42K	Zoneout (Krueger et al., 2017)
Adding Prob. 600 (loss)	5.8e-5	70K	5.3e-5	70K	Regularized GRU
Copy Memory 1000 (loss)	3.5e-5	70K	0.011	70K	EURNN (Jing et al., 2017)
JSB Chorales (loss)	8.10	300K	3.47	-	DBN+LSTM (Vohra et al., 2015)
Nottingham (loss)	3.07	1M	1.32	-	DBN+LSTM (Vohra et al., 2015)
Word PTB (ppl)	88.68	13M	47.7	22M	AWD-LSTM-MoS + Dynamic Eval. (Yang et al., 2018)
Word Wiki-103 (ppl)	45.19	148M	40.4	>300M	Neural Cache Model (Large) (Grave et al., 2017)
Word LAMBADA (ppl)	1279	56M	138	>100M	Neural Cache Model (Large) (Grave et al., 2017)
Char PTB (bpc)	1.31	3M	1.22	14M	2-LayerNorm HyperLSTM (Ha et al., 2017)
Char text8 (bpc)	1.45	4.6M	1.29	>12M	HM-LSTM (Chung et al., 2016)

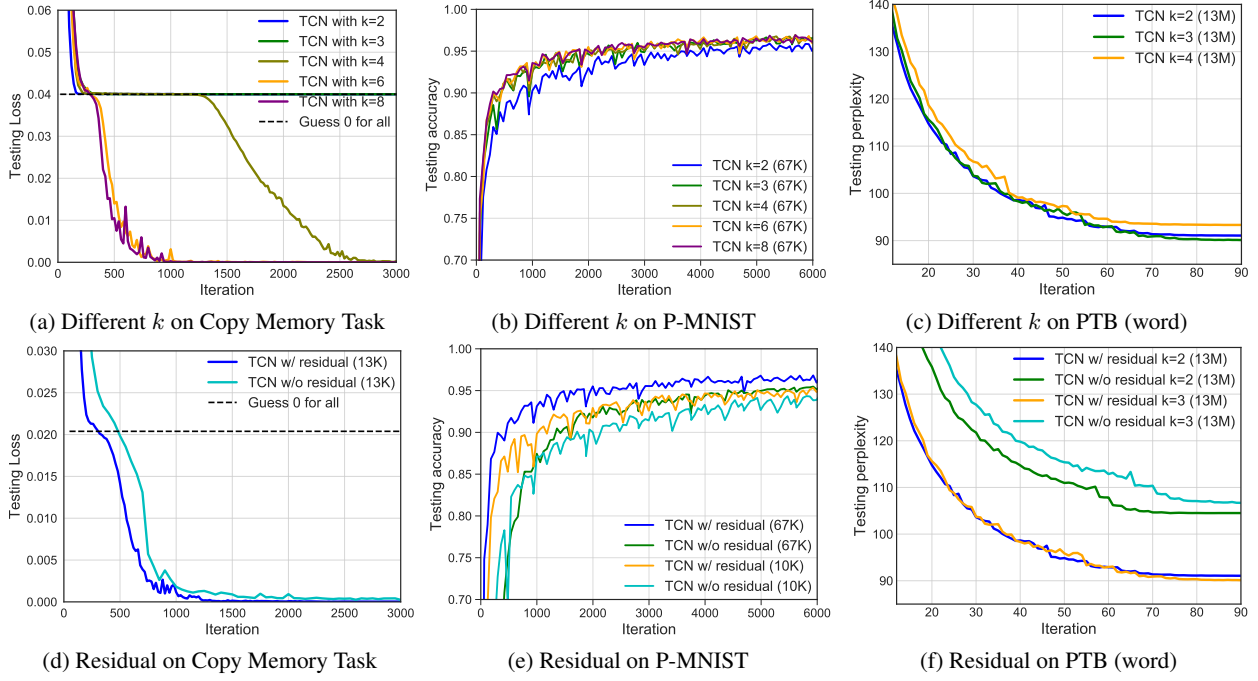


Figure 6. Controlled experiments that study the effect of different components of the TCN model.

Table 5. An evaluation of gating in TCN. A plain TCN is compared to a TCN that uses gated activations.

Task	TCN	TCN + Gating
Sequential MNIST (acc.)	99.0	99.0
Permuted MNIST (acc.)	97.2	96.9
Adding Problem $T = 600$ (loss)	5.8e-5	5.6e-5
Copy Memory $T = 1000$ (loss)	3.5e-5	0.00508
JSB Chorales (loss)	8.10	8.13
Nottingham (loss)	3.07	3.12
Word-level PTB (ppl)	88.68	87.94
Char-level PTB (bpc)	1.31	1.306
Char text8 (bpc)	1.45	1.485



Temporal Convolutional Networks and Forecasting



Share This Article



📅 Jul 6, 2021

🕒 16 min read

Written By

Francesco Lässig

How a convolutional network with some simple adaptations can become a powerful tool for sequence modeling and forecasting.

Although commonly associated with image classification tasks, convolutional neural networks (CNNs) have proven to be valuable tools for sequence modeling and forecasting, given the right modifications. In this article we explore in detail the basic building blocks that a Temporal Convolutional Network (TCN) consists of, and how they all fit together to create a powerful forecasting model. Using our open-source *Darts* TCN implementation, we show that accurate predictions can be achieved on a real-world dataset with only a few lines of code.

The following description of a Temporal Convolutional Network is based on the following <https://arxiv.org/pdf/1803.01271.pdf>. References to this paper will be denoted by (*).

We use cookies to ensure that we give you the best experience on our website. [Accept all](#) [Reject all](#)

protected by reCAPTCHA
[Privacy](#) · [Terms](#)

Motivation

Up until recently, the topic of sequence modeling in the context of deep learning has been largely associated with recurrent neural network architectures such as LSTM and GRU. S. Bai et al. (*) suggest that this way of thinking is antiquated, and that convolutional networks should be taken into consideration as one of the primary candidates when modeling sequential data. They were able to show that convolutional networks can achieve better performance than RNNs in many tasks while avoiding common drawbacks of recurrent models, such as the exploding/vanishing gradient problem or lacking memory retention. Furthermore, using a convolutional network instead of a recurrent one can lead to performance improvements as it allows for parallel computation of outputs. The architecture they propose is called Temporal Convolutional Network (TCN) and will be explained in the following sections. To facilitate understanding the TCN architecture in conjunction with its *Darts* implementation, this article will use the same model parameter names as seen in the library wherever possible (indicated in **bold**).

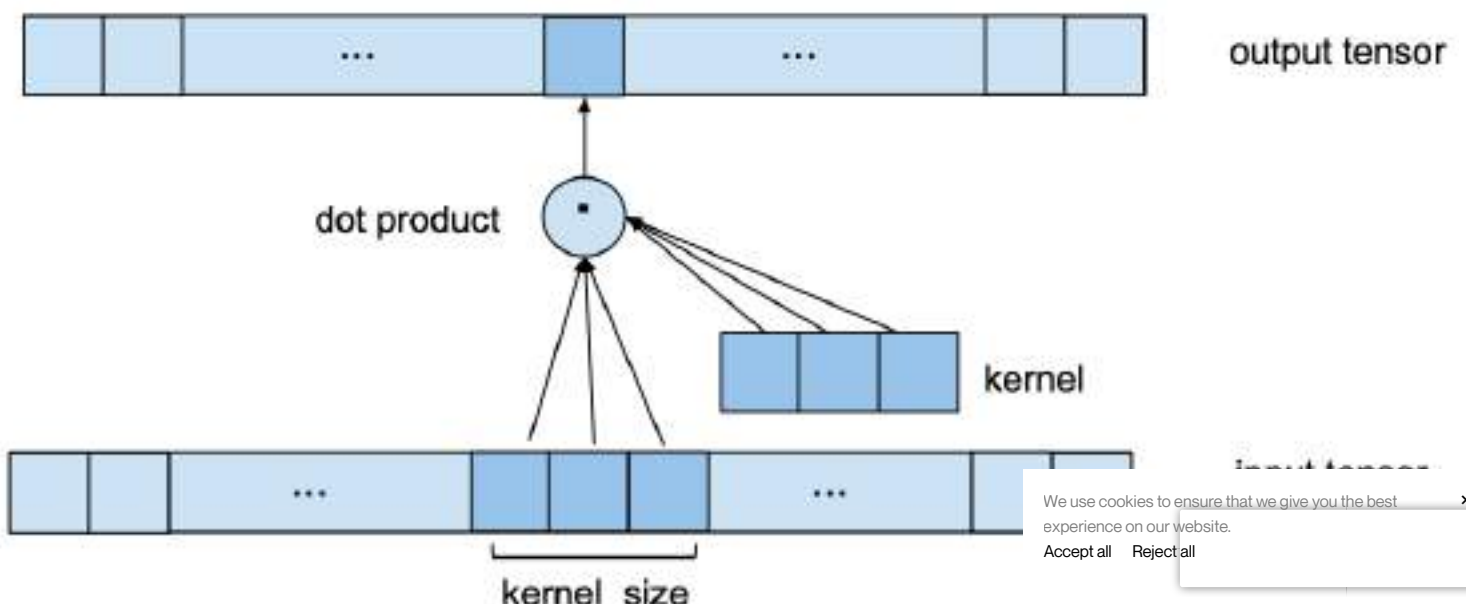
Overview

A TCN, short for Temporal Convolutional Network, consists of dilated, causal 1D convolutional layers with the same input and output lengths. The following sections go into detail about what these terms actually mean.

1D Convolutional Network

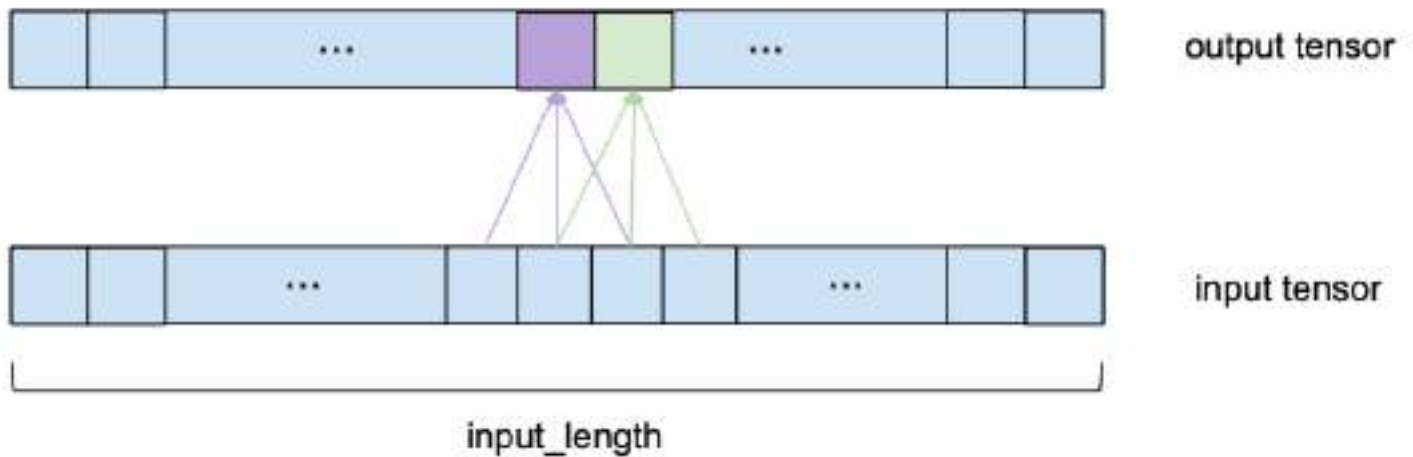
A 1D convolutional network takes as input a 3-dimensional tensor and also outputs a 3-dimensional tensor. The input tensor of our TCN implementation has the shape (batch_size, input_length, input_size) and the output tensor has the shape (batch_size, input_length, output_size). Since every layer in a TCN has the same input and output length, only the third dimension of the input and output tensors differs. In the univariate case, input_size and output_size will both be equal to one. In the more general multivariate case, input_size and output_size might differ since we might not want to forecast every component of the input sequence.

One single 1D convolutional layer receives an input tensor of shape (batch_size, input_length, nr_input_channels) and outputs a tensor of shape (batch_size, input_length, nr_output_channels). To see how one single layer converts its input into the output, let's look at one element of the batch (the same process takes place for every element in the batch). Let's start with the simplest case where nr_input_channels and nr_output_channels both equal 1. In this case we are looking at 1D input and output tensors. The following image shows how one element of the output tensor is computed.



input_length

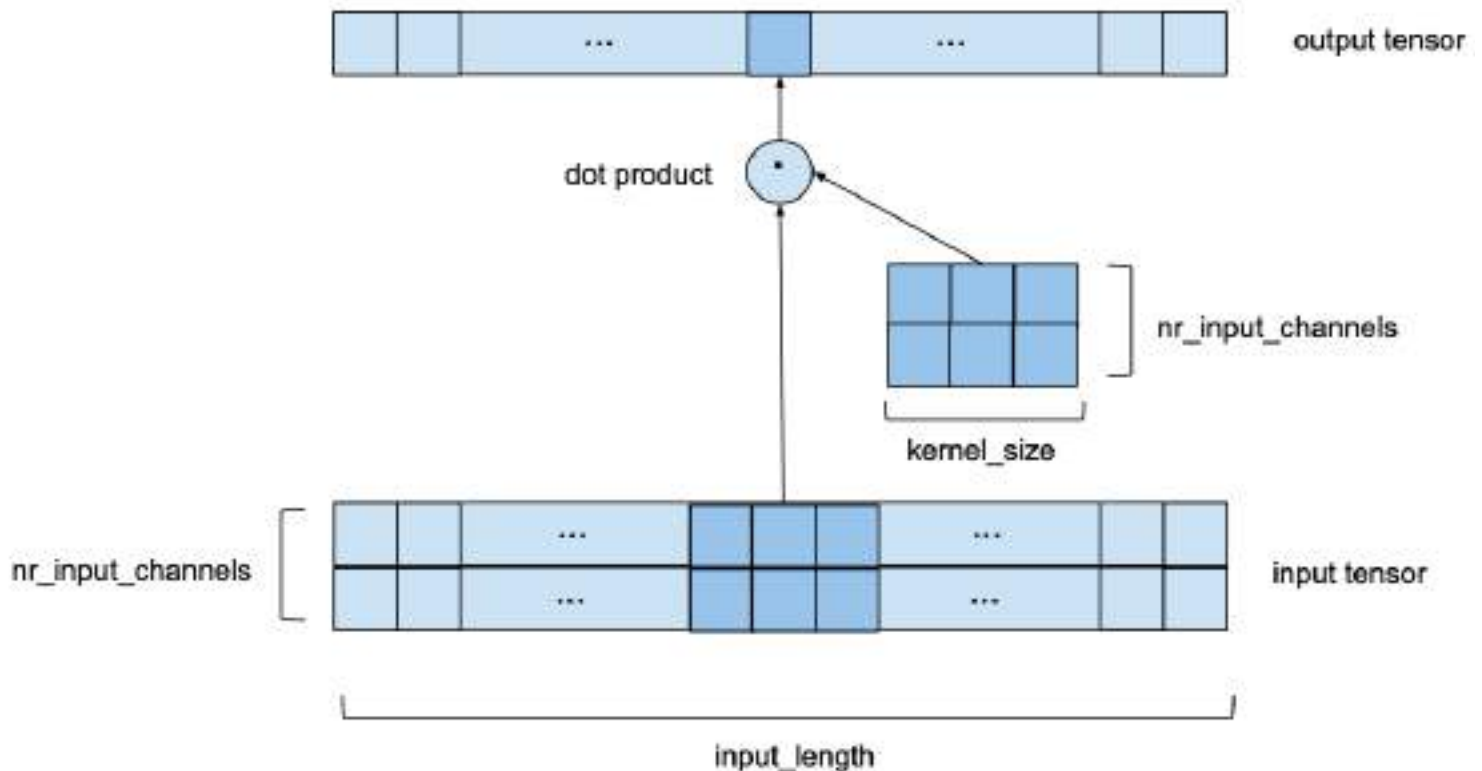
We can see that to compute one element of the output, we look at a series of consecutive elements of length `kernel_size` of the input. In the above example we chose a `kernel_size` of 3. To obtain the output, we take the dot product of the subsequence of the input and a kernel vector of learned weights of the same length. To get the next element of the output, the same procedure is applied, but the `kernel_size`-sized window of the input sequence is shifted to the right by one element (for the purposes of this forecasting model, the stride is always set to 1). Please note that the same set of kernel weights will be used to compute every output of one convolutional layer. The following image shows two consecutive output elements and their respective input subsequences.



To make the visualization simpler, the dot product with the kernel vector is not shown anymore, but takes place for every output element with the same kernel weights.

To make sure that the output sequence has the same length as the input sequence, some zero-padding is applied. This means that additional zero-valued entries are added to either the beginning or the end of the input tensor to ensure that the output has the desired length. How this is done exactly will be explained in later sections.

Now let's look at the case where we have multiple input channels, i.e. `nr_input_channels` is larger than 1. In this case, the process described above is repeated for every single input channel, but each time with a different kernel. This leads to `nr_input_channels` intermediate output vectors and a number of kernel weights of `kernel_size * nr_input_channels`. Then, all the intermediate output vectors are summed up to obtain the final output vector. In some sense this is equivalent to a 2D convolution with an input tensor of shape `(input_size, nr_input_channels)` and a kernel of shape `(kernel_size, nr_input_channels)` as shown in the following image. It's still 1D in the sense that the window only moves along a single axis, but we do have a 2D convolution at every step in the sense that we are using a 2-dimensional kernel matrix.



For this example we chose $nr_input_channels$ to be equal to 2. Now, instead of a kernel vector sliding over a 1-dimensional input sequence we have a $nr_input_channels$ by $kernel_size$ kernel matrix sliding along a $nr_input_channels$ wide series of length $input_length$.

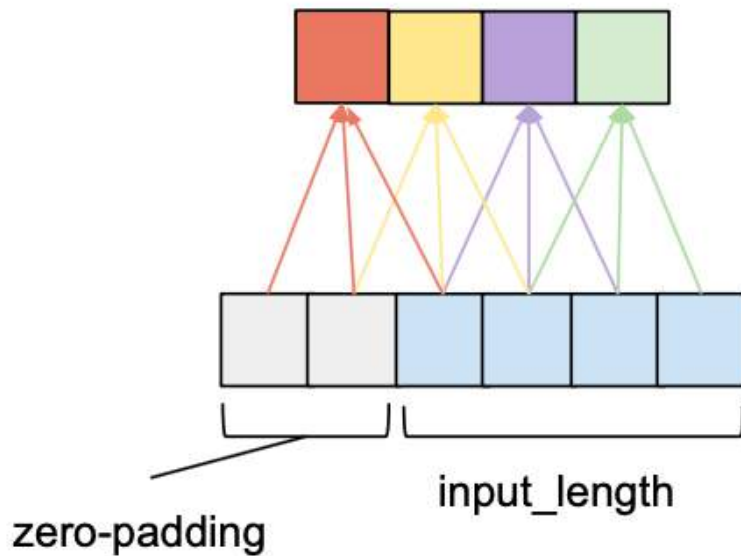
If both $nr_input_channels$ and $nr_output_channels$ are larger than 1, the above process is simply repeated for every output channel with a different kernel matrix. The output vectors are then stacked on top of each other resulting in an output tensor of shape $(input_length, nr_output_channels)$. The number of kernel weights in this case is equal to $kernel_size * nr_input_channels * nr_output_channels$.

The two variables $nr_input_channels$ and $nr_output_channels$ depend on the position of the layer within the network. The first layer will have $nr_input_channels = input_size$, and the last layer will have $nr_output_channels = output_size$. All other layers will use the intermediate channel number given by $num_filters$.

Causal Convolution

For a convolutional layer to be causal, for every i in $\{0, \dots, input_length - 1\}$ the i 'th element of the output sequence may only depend on the elements of the input sequence with indices $\{0, \dots, i\}$. In other words, an element in the output sequence can only depend on elements that come before it in the input sequence. As mentioned before, to ensure an output tensor has the same length as the input tensor, we need to apply zero padding. If we only apply zero-padding on the left side of the input tensor, then causal convolution will be ensured. To understand this, consider the rightmost output element. Given that there is no padding on the right side of the input sequence, the last element it depends on is the last element of the input. Now consider the second to last output element of the output sequence. Its kernel window is shifted to the left by one compared to the last output element, that means its rightmost dependency in the input sequence is the second to last element of the input sequence. It follows by induction that for every element in the output sequence, its latest dependency in the input sequence has the same index as itself. The following figure shows an exam

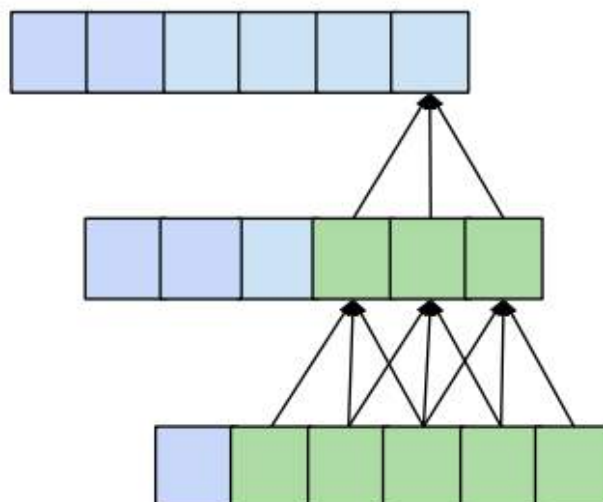
input_length of 4 and a kernel_size of 3.



We can see that with a left zero-padding of 2 entries we can achieve the same output length while obeying the causality rule. In fact, without dilation, the number of zero-padding entries required for maintaining the input length is always equal to $\text{kernel_size} - 1$.

Dilation

One desirable quality of a forecasting model is that the value of a specific entry in the output depends on all previous entries in the input, i.e. all entries that have an index smaller or equal to itself. This is achieved when the receptive field, meaning the set of entries of the original input that affect a specific entry of the output, has size input_length . We also call this 'full history coverage'. As we have seen before, one conventional convolutional layer makes an entry in the output dependent on the kernel_size entries of the input that have an index smaller or equal to itself. For instance, if we have a kernel_size of 3, the 5th element in the output will be dependent on elements 3, 4 and 5 of the input. This reach is expanded when we stack multiple layers on top of each other. In the following figure we can see that by stacking two layers with kernel_size 3 we get a receptive field size of 5.



More generally, a 1D convolutional network with n layers and a kernel_size k has a receptive field r of size

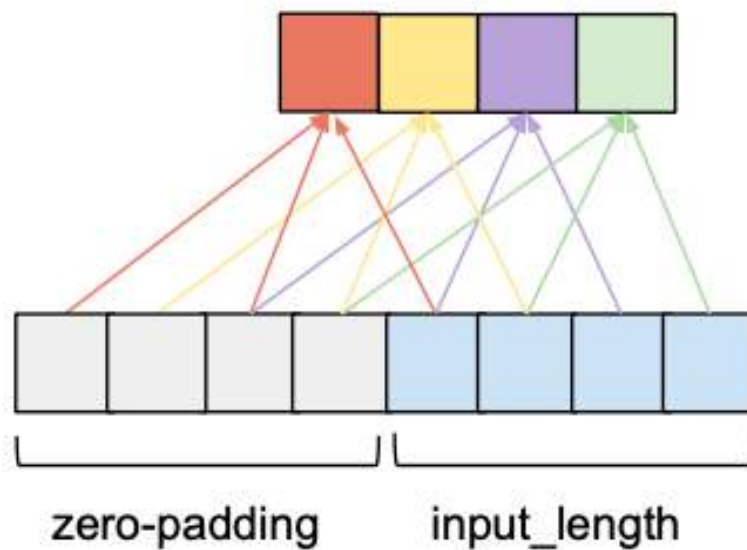
$$r = 1 + n * (k - 1)$$

To know how many layers are needed for full coverage, we can set the receptive field size to input_length l and solve for the number of layers n (we need to round up in case of non-integer values):

$$n = \lceil (l - 1) / (k - 1) \rceil$$

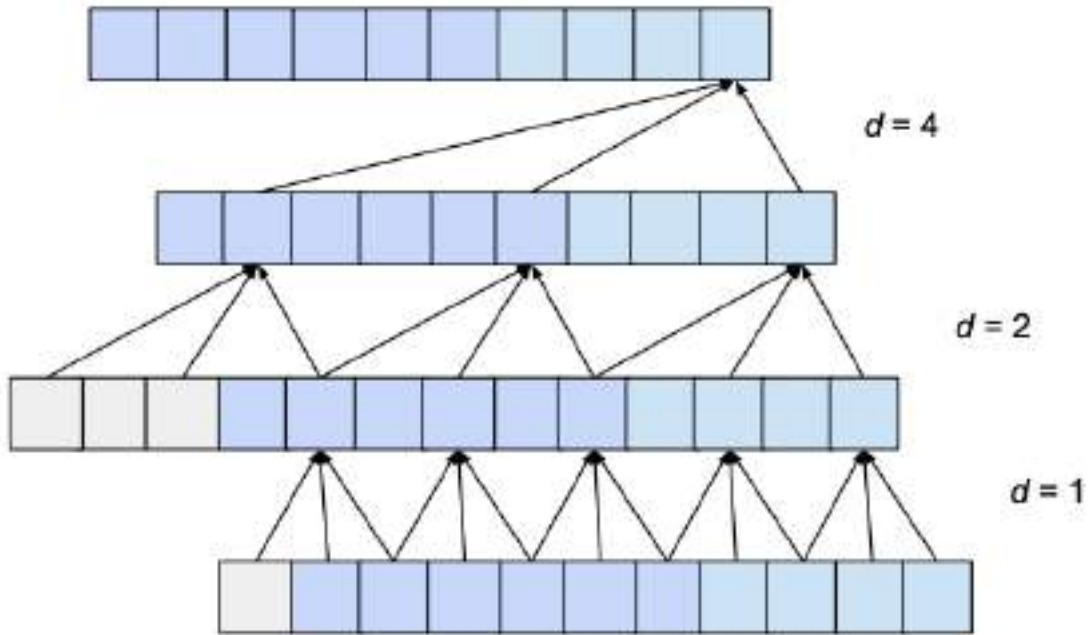
This means that, given a fixed kernel_size, the number of layers required for full history coverage is linear in the length of the input tensor, which will result in networks that become very deep very fast, leading to models with a very large number of parameters that take longer to train. Furthermore, a high number of layers has been shown to lead to degradation problems related to the gradient of the loss function. One way to increase the receptive field size while still keeping the number of layers relatively small is to introduce dilation to the convolutional network.

Dilation in the context of a convolutional layer refers to the distance between the elements of the input sequence that are used to compute one entry of the output sequence. So a conventional convolutional layer could be seen as a 1-dilated layer, since the input elements for 1 output value are adjacent. The following figure shows an example of a 2-dilated layer with an input_length of 4 and a kernel_size of 3.



Compared to the 1-dilated case, this layer has a receptive field that spreads across a length of 5 rather than 3. More generally, a d -dilated layer with a kernel size k has a receptive field spreading across a length of $1+d*(k-1)$. If d is fixed, this will still require a number linear in the length of the input tensor to achieve full receptive field coverage (we just decreased the constant).

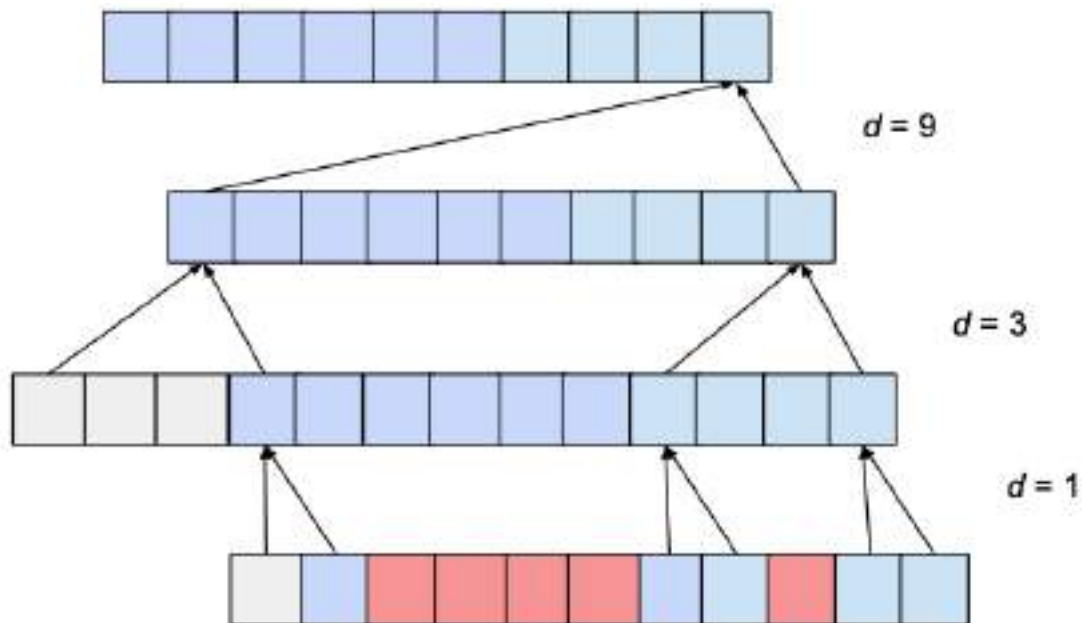
This problem can be addressed by increasing the value of d exponentially as we move up through the layers. For this, we choose a constant dilation_base integer b which will let us compute the dilation d of a specific layer as a function of the number of layers below it, i , as $d = b^{**i}$. The following figure shows a network with an input_length of 10, a kernel_size of 3 and a dilation_base of 2 which results in 3 dilated convolutional layers for full coverage.



Here we only show the influence of inputs that affect the last value of the output. Likewise, only zero-padding entries necessary for the last output value are shown. Clearly, the last output value is dependent on the whole input coverage. Actually, given the hyperparameters, an input_length of up to 15 could be used while maintaining full receptive field coverage. Generally speaking, every additional layer adds a value of $d*(k-1)$ to the current receptive field width, where d is computed as $d=b^{**i}$, with i representing the number of layers below our new layer. Consequently, the width of the receptive field w of a TCN with exponential dilation of base b , kernel size k and number of layers n is given by

$$w = 1 + \sum_{i=0}^{n-1} (k-1) \cdot b^i = 1 + (k-1) \cdot \frac{b^n - 1}{b - 1}$$

However, depending on the values b and k , this receptive field can have 'holes'. Consider the following network with a dilation_base of 3 and a kernel size of 2:



The receptive field does cover a range that is larger than the input size (namely 15). However, the receptive field has holes in it; that is there are entries in the input sequence that the output value is not dependent on (shown above in red). To solve this problem, we need to either increase the kernel size to 3, or decrease the dilation base to 2. Generally speaking, for a receptive field with no holes, the kernel size k has to be at least as big as the dilation base b .

Considering these observations, we can compute how many layers our network needs for full history coverage. Given a kernel size k , dilation base b , where $k \geq b$, and input length l , the following inequality must hold for full history coverage:

$$1 + (k - 1) \cdot \frac{b^n - 1}{b - 1} \geq l$$

We can solve for n and get the minimum number of required layers as

$$n = \left\lceil \log_b \left(\frac{(l - 1) \cdot (b - 1)}{(k - 1)} + 1 \right) \right\rceil$$

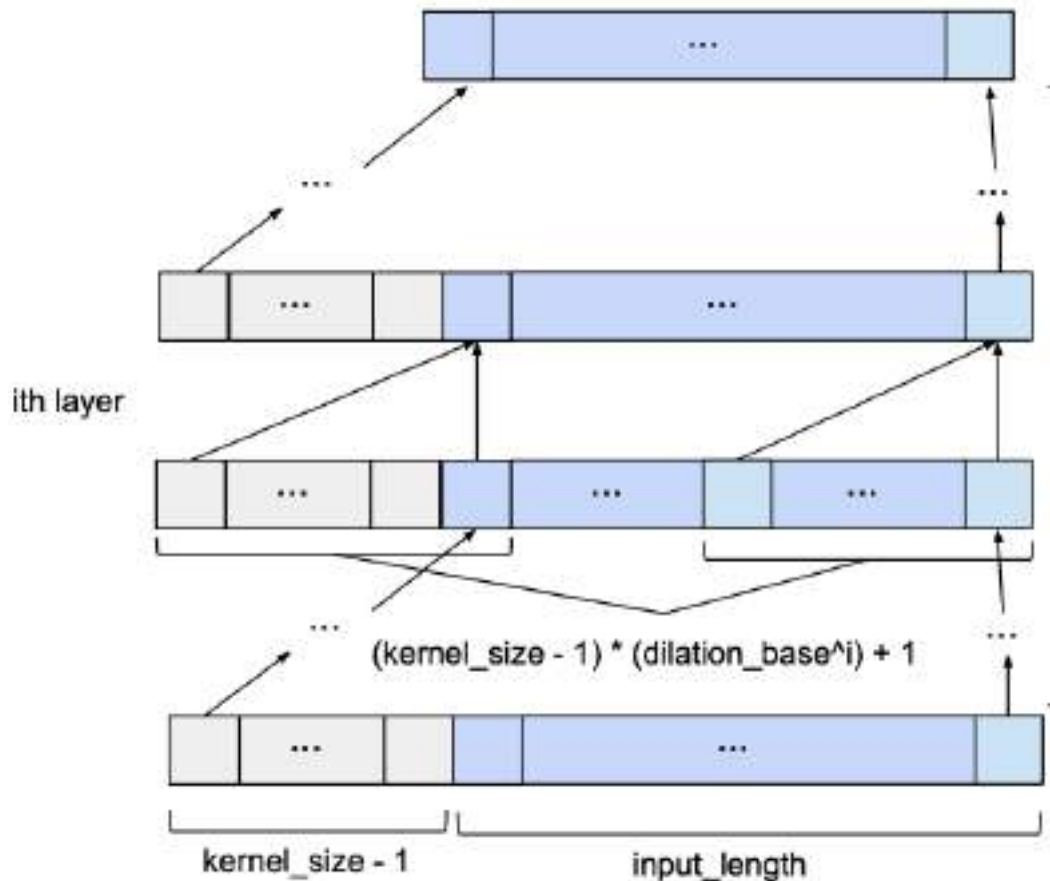
We can see that the number of layers is now logarithmic rather than linear in the length of the input. This constitutes a significant improvement, which can be achieved without sacrificing receptive field coverage.

Now the only thing left to specify is the number of zero-padding entries required at each layer. Given a dilation base b , a kernel size k and a number of i layers below our current layer, then the number of zero-padding entries p needed for the current layer are computed as follows:

$$p = b^j \cdot (k - 1)$$

Basic TCN Overview

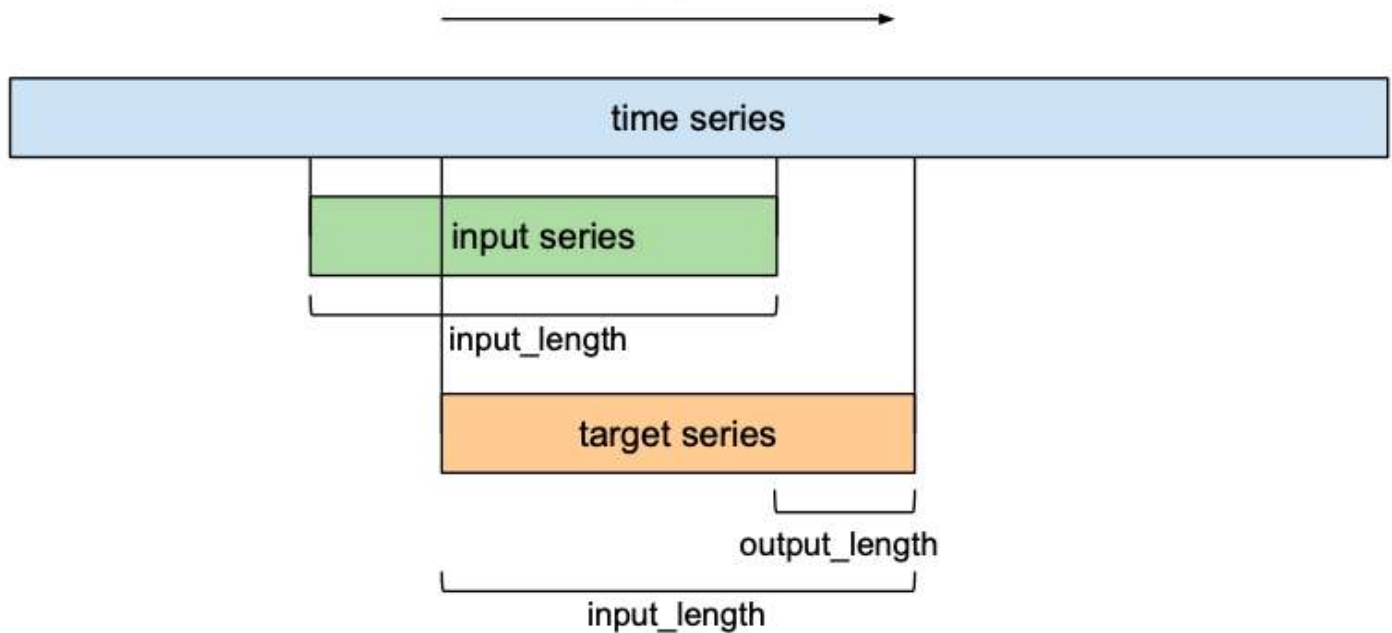
Given input_length, kernel_size, dilation_base and the minimum number of layers required for full history coverage, the basic TCN network would look something like this:



Forecasting

So far we have only talked about the 'input sequence' and the 'output sequence' without getting into how they relate to each other. In the context of forecasting, we want to predict the next entries of a time series into the future. To train our TCN network to do forecasting, the training set will consist of (input sequence, target sequence)-pairs of equally-sized subsequences of the given time series. A target series will be a series that is shifted forward in relation to its respective input series by a certain number output_length. This means that a target series of length input_length contains the last (input_length - output_length) elements of its respective input sequence as first elements, and the output_length elements that come after the last entry of the input series as its final elements. In the context of forecasting, this means that the maximum forecasting horizon that can be predicted with such a model is equal to output_length. Using a sliding window approach, many overlapping pairs of input and target sequences can be created out of one time series.

time



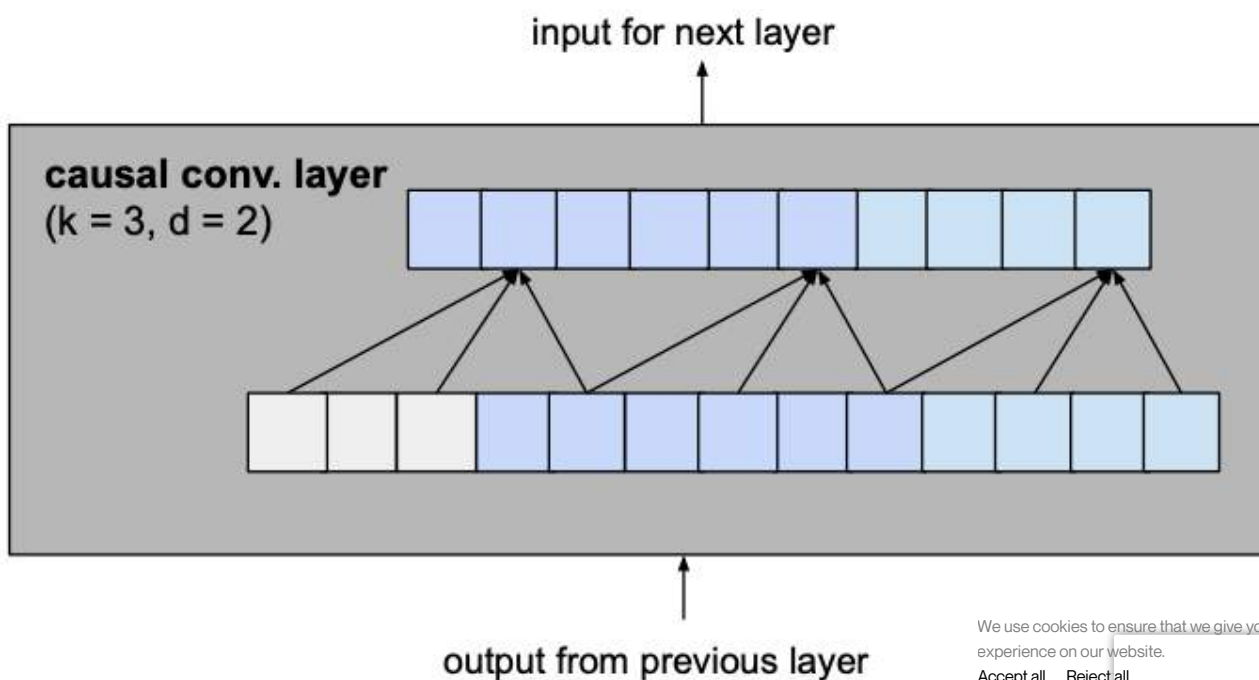
Improvements to the Model

S. Bai et al. (*) suggest a few additions to the basic TCN architecture for improved performance which will be discussed in this section, namely residual connections, regularization and activation functions.

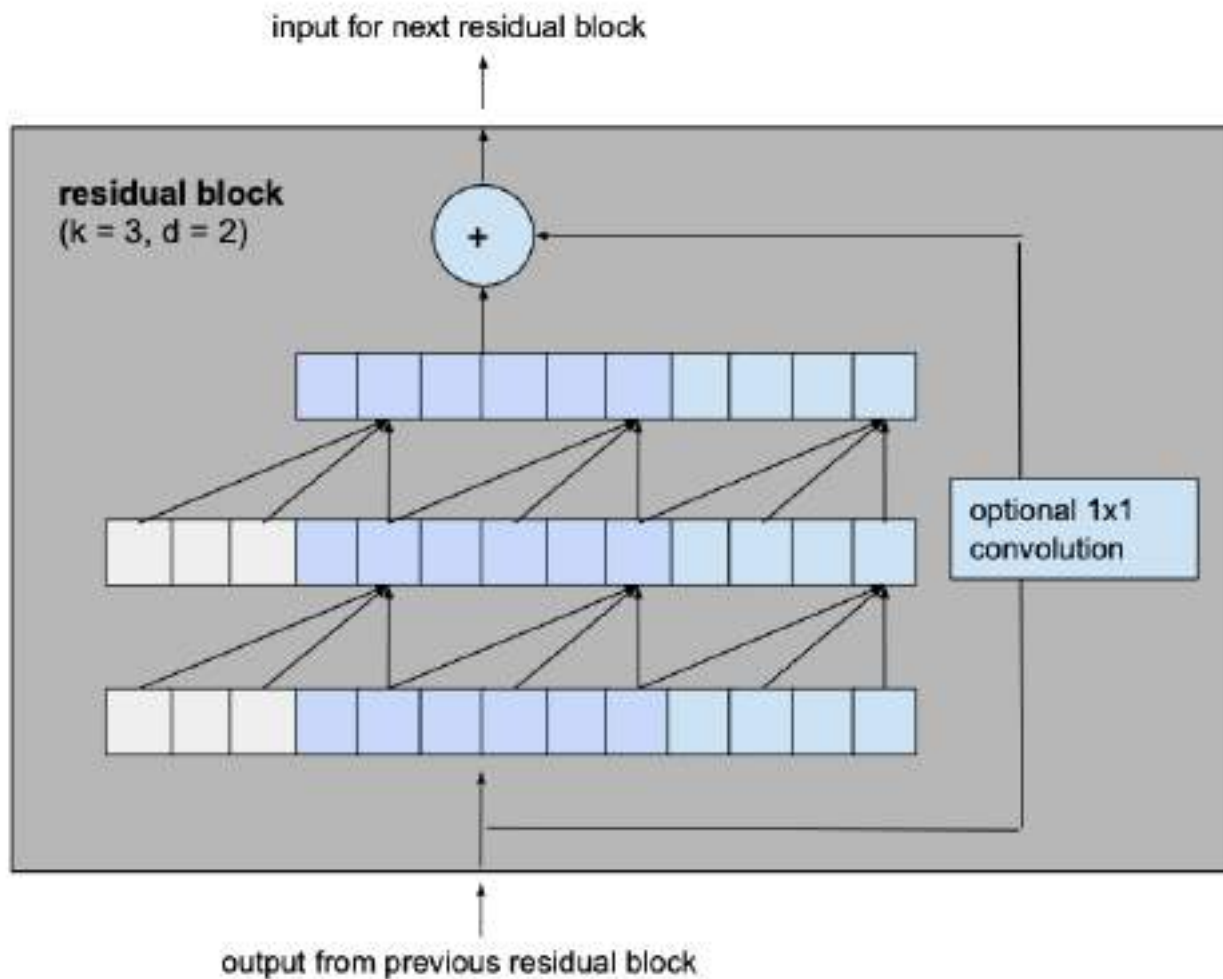
Residual Blocks

The biggest modification we make to the previously introduced basic model is to change the fundamental building block of the model from a simple 1D causal convolutional layer to a residual block which consists of 2 layers with the same dilation factor and a residual connection.

Let's consider a layer with a dilation factor d of 2 and kernel size k of 3 from the basic model to see how this translates into a residual block of the improved model.



becomes



The output of the two convolutional layers will be added to the input of the residual block to produce the input for the next block. For all inner blocks of the network, i.e. all but the first and the last one, the input and output channel width is the same, namely `num_filters`. Since the first convolutional layer of the first residual block and the second convolutional layer of the last residual block may have different input and output channel widths, the width of the residual tensor might have to be adjusted, which is done using a 1x1 convolution.

This change affects the calculus for the minimum number of required layers for full coverage. Now we have to think about how many residual blocks are necessary to achieve a full receptive field coverage. Adding a residual block to a TCN adds twice as much receptive field width than when adding a basic causal layer, since it includes 2 such layers. So the total size of the receptive field r of a TCN with dilation base b , kernel size k with $k \geq b$ and number of residual blocks n can be computed as

$$r = 1 + \sum_{i=0}^{n-1} 2 \cdot (k-1) \cdot b^i = 1 + 2 \cdot (k-1) \cdot \frac{b^n - 1}{b - 1}$$

which leads to a minimum number of residual blocks n for full history coverage of input

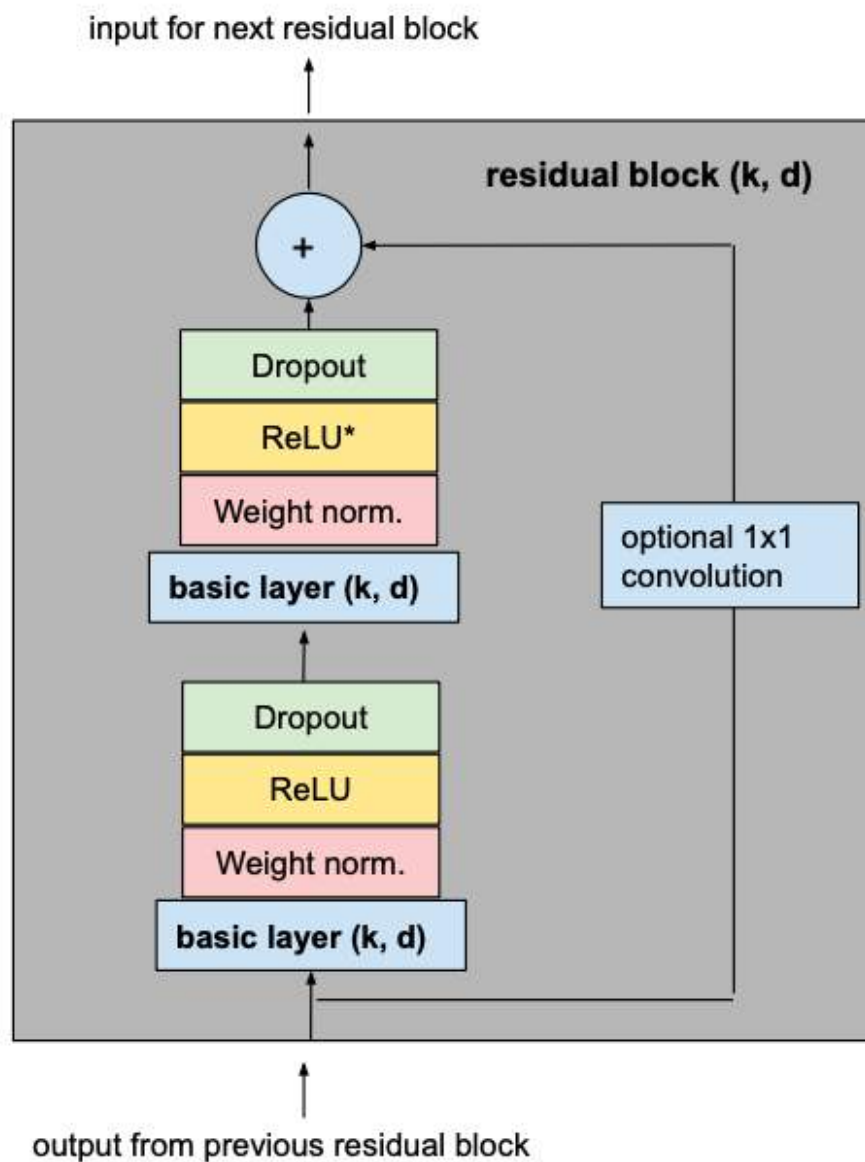
$$n = \left\lceil \log_b \left(\frac{(t-1) \cdot (b-1)}{(k-1) \cdot 2} + 1 \right) \right\rceil$$

Activation, Normalization, Regularization

To make our TCN more than just an overly complex linear regression model, activation functions need to be added on top of the convolutional layers to introduce non-linearities. ReLU activations are added to the residual blocks after both convolutional layers.

To normalize the input of hidden layers (which counteracts the exploding gradient problem among other things), weight normalization is applied to every convolutional layer.

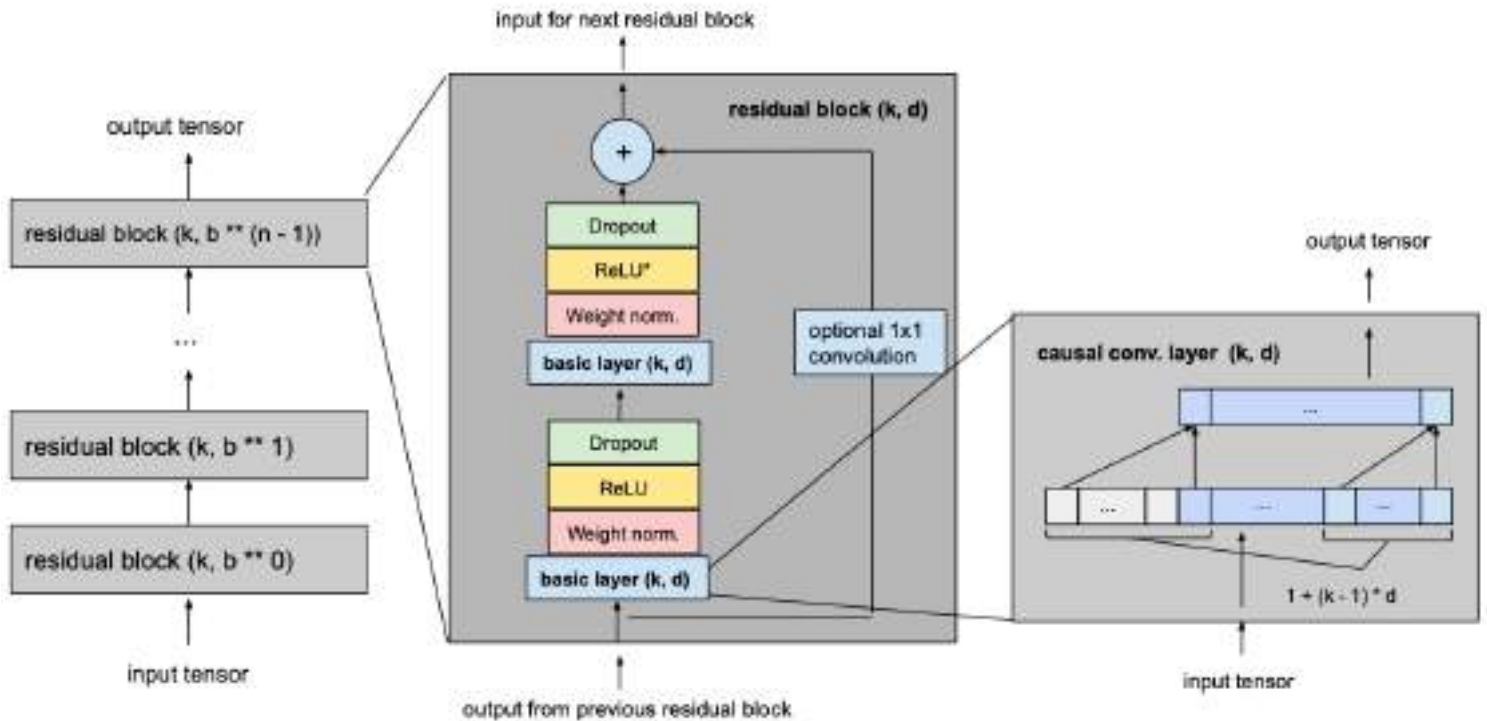
In order to prevent overfitting, regularization is introduced via dropout after every convolutional layer in every residual block. The following figure shows the final residual block.



The asterisk in the second ReLU unit indicates that it is present in every layer but the last. We want our final output to be able to take on negative values as well (this differs from the standard ReLU outlined in the paper).

Final Model

The following picture shows our final TCN model with l equal to input_length, k equal to kernel_size, b equal to dilation_base, $k \geq b$ and with a minimum number of residual blocks for full history coverage n , where n can be computed from the other values as explained above.



Example

Let's take a look at an example of how we can use the TCN architecture to forecast a time series using the [Darts](#) library.

First, we need a time series to train and evaluate our model on. For this, we use a [Kaggle dataset](#) containing hourly energy production data from Spain. More specifically, we choose to predict the 'run-of-river hydroelectricity' production. Furthermore, to make the problem less computation-intensive, we average the energy production over each day to get a daily time series.

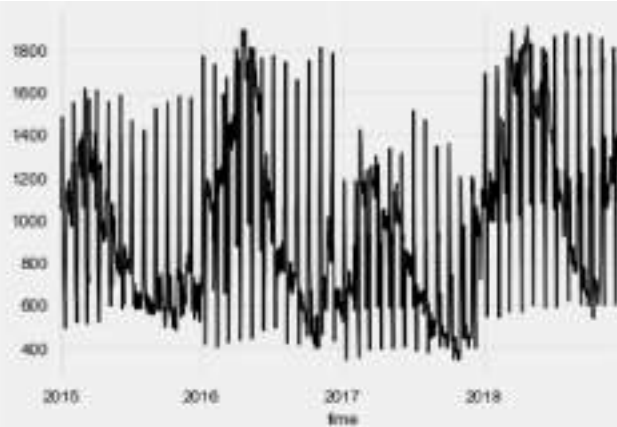
```
from darts import TimeSeries
from darts.dataprocessing.transformers import
MissingValuesFiller
import pandas as pd

df = pd.read_csv('energy_dataset.csv', delimiter=",")
df['time'] = pd.to_datetime(df['time'], utc=True)
df['time'] = df['time'].dt.tz_localize(None)

df_day_avg =
df.groupby(df['time'].astype(str).str.split(" ").str[0]).mean().reset_index()

value_filler = MissingValuesFiller()
series =
value_filler.transform(TimeSeries.from_dataframe(df_day_avg, 'time', ['generation hydro run-of-river']))

series.plot()
```



We can see that, in addition to a yearly seasonality, there are regular 'spikes' in energy production that come in monthly intervals. Since the TCN model supports multiple input channels, we can add additional time series components to the current time series that encode the current day of the month. This can help our TCN model to converge faster.

```
series = series.add_datetime_attribute('day', one_hot=True)
```

Now we split our data into train and validation components and perform standardization.

```
from darts.dataprocessing.transformers import Scaler

train, val =
series.split_after(pd.Timestamp('20170901'))

scaler = Scaler()
train_transformed = scaler.fit_transform(train)
val_transformed = scaler.transform(val)
series_transformed = scaler.transform(series)
```

Now it's time to create and train our TCN model. Note that all bold variable names that have appeared in the above description of the architecture can be used as arguments for the constructor of the *Darts* TCN implementation. The `output_length` parameter is set to 7 since we want to perform weekly forecasts. When training the model, we specify as `target_series` only the first component of our training series, since we do not want to predict the helper time series we added earlier. We tried out a few different hyperparameter combinations, but most of the values were chosen quite arbitrarily.

```
from darts.models import TCNModel

model = TCNModel(
    input_size=train.width,
    n_epochs=20,
    input_length=365,
    output_length=7,
    dropout=0,
    dilation_base=2,
    weight_norm=True,
    kernel_size=7,
    num_filters=4,
    random_state=0
)

model.fit(
    training_series=train_transformed,
    target_series=train_transformed['0'],
    val_training_series=val_transformed,
    val_target_series=val_transformed['0'],
    verbose=True
)
```

To evaluate our model, we want to test its performance using a 7-day forecasting horizon

different points in time in the validation set. For this, we use the historic backtesting functionality from *Darts*. Note that the model is provided new input data for every prediction, but it is never retrained. To save time, we set the stride to 5.

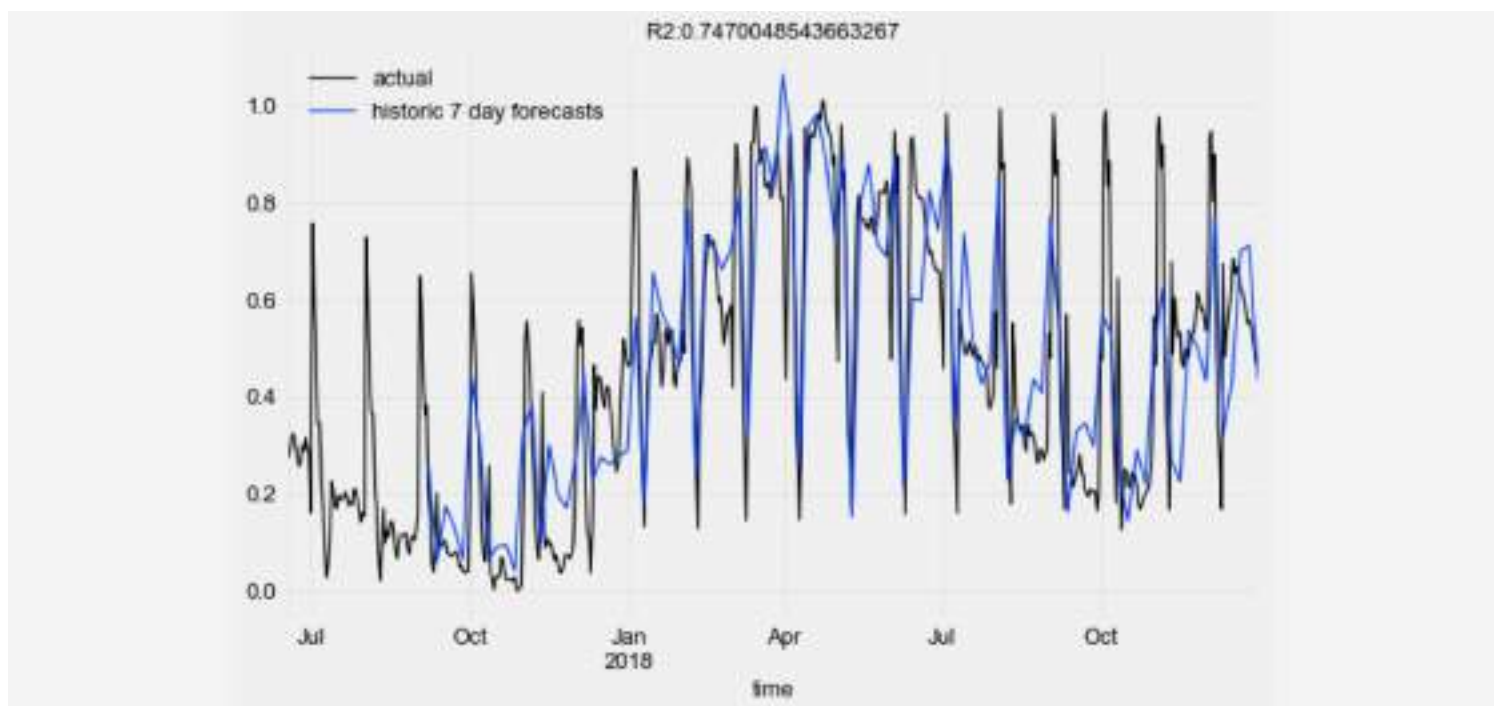
```
pred_series = model.backtest(
    series_transformed,
    target_series=series_transformed['0'],
    start=pd.Timestamp('20170901'),
    forecast_horizon=7,
    stride=5,
    retrain=False,
    verbose=True,
    use_full_output_length=True
)
```

Let's visualize the historic forecast predictions of our TCN model against the ground truth data points and compute the R2 score.

```
from darts.metrics import r2_score
import matplotlib.pyplot as plt

series_transformed[900:]['0'].plot(label='actual')
pred_series.plot(label='historic 7 day forecasts')
r2_score_value = r2_score(series_transformed['0'], pred_series)

plt.title('R2: ' + str(r2_score_value))
plt.legend()
```



For more details and additional examples, please check out our [TCN examples notebook](#) on GitHub.

Conclusion

Deep learning in sequence modeling is, for the most part, still widely associated with recurrent network architectures. [But research has shown](#) that these types of models can be outperformed by a TCN, both in terms of predictive performance and efficiency. In this article we explored how this promising model can be understood in terms of simple building blocks.

Temporal Convolutional Networks: A Unified Approach to Action Segmentation

Colin Lea René Vidal Austin Reiter Gregory D. Hager
Johns Hopkins University
{clea1@, rvidal@cis., areiter@cs., hager@cs.}jhu.edu

Abstract

The dominant paradigm for video-based action segmentation is composed of two steps: first, for each frame, compute low-level features using Dense Trajectories or a Convolutional Neural Network that encode spatiotemporal information locally, and second, input these features into a classifier that captures high-level temporal relationships, such as a Recurrent Neural Network (RNN). While often effective, this decoupling requires specifying two separate models, each with their own complexities, and prevents capturing more nuanced long-range spatiotemporal relationships. We propose a unified approach, as demonstrated by our Temporal Convolutional Network (TCN), that hierarchically captures relationships at low-, intermediate-, and high-level time-scales. Our model achieves superior or competitive performance using video or sensor data on three public action segmentation datasets and can be trained in a fraction of the time it takes to train an RNN.

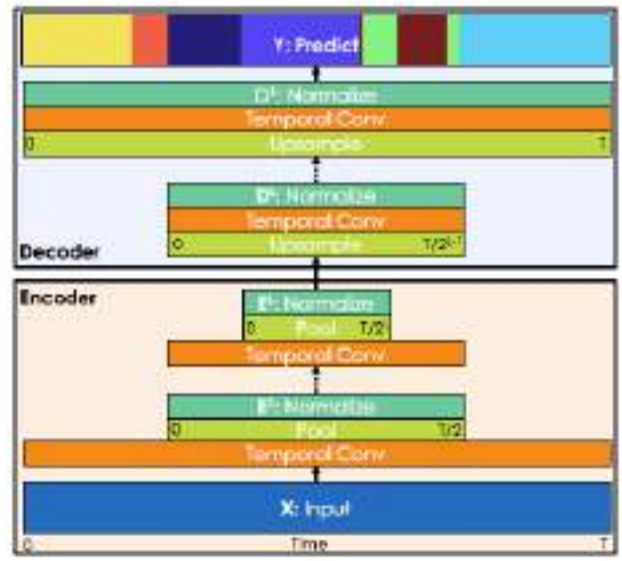


Figure 1. Our temporal encoder-decoder network hierarchically models actions from video or other time-series data.

1. Introduction

Action segmentation is crucial for numerous applications ranging from collaborative robotics to modeling activities of daily living. Given a video, the goal is to simultaneously segment every action in time and classify each constituent segment. While recent work has shown strong improvements on this task, models tend to decouple low-level feature representations from high-level temporal models. Within video analysis, these low-level features may be computed by pooling handcrafted features (e.g. Improved Dense Trajectories (IDT) [21]) or concatenating learned features (e.g. Spatiotemporal Convolutional Neural Networks (ST-CNN) [8, 12]) over a short period of time. High-level temporal classifiers capture a local history of these low-level features. In a Conditional Random Field (CRF), the action prediction at one time step is often a function of the prediction at the previous time step, and in a Recurrent Neural Network (RNN), the predictions are a function

of a set of latent states at each time step, where the latent states are connected across time. This two-step paradigm has been around for decades (e.g., [6]) and typically goes unquestioned. However, we posit that valuable information is lost between steps.

In this work, we introduce a unified approach to action segmentation that uses a single set of computational mechanisms – 1D convolutions, pooling, and channel-wise normalization – to hierarchically capture low-, intermediate-, and high-level temporal information. For each layer, 1D convolutions capture how features at lower levels change over time, pooling enables efficient computation of long-range temporal patterns, and normalization improves robustness towards various environmental conditions. In contrast with RNN-based models, which compute a set of latent activations that are updated sequentially per-frame, we compute a set of latent activations that are updated hierarchically per-layer. As a byproduct, our model takes much

less time to train. Our model can be viewed as a generalization of the recent ST-CNN [8] and is more similar to recent models for semantic segmentation than it is to models for video-analysis. We show this approach is broadly applicable to video and other types of robot sensors.

Prior Work: Due to space limitations, here we will only briefly describe models for time-series and semantic segmentation. See [8] for related work on action segmentation or [20] for a broader overview on action recognition.

RNNs and CRFs are popular high-level temporal classifiers. RNN variations, including Long Short Term Memory (LSTM) and Gated Recurrent Units (GRU), model hidden temporal states via internal gating mechanisms. However, they are hard to introspect and difficult to correctly train [13]. It has been shown that in practice LSTM only keeps a memory of about 4 seconds on some video-based action segmentation datasets [15]. CRFs typically model pairwise transitions between the labels or latent states (e.g., [8]), which are easy to interpret, but over-simplify the temporal dynamics of complex actions. Both of these models suffer from the same fundamental issue: intermediate activations are typically a function of the low-level features at the current time step and the state at the previous time step. Our temporal convolutional filters are a function of raw data across a much longer period of time.

Until recently, the dominant paradigm for semantic segmentation was similar to that of action segmentation. Approaches typically combined low-level texture features (e.g., TextonBoost) with high-level spatial models (e.g., grid-based CRFs) that model the relationships between different regions of an image [7]. This is similar to action segmentation where low-level spatiotemporal features are used in tandem with high-level temporal models. Recently, with the introduction of Fully Convolutional Networks (FCNs), the dominant semantic segmentation paradigm has started to change. Long *et al.* [11] introduced the first FCN, which leverages typical classification CNNs like AlexNet, to compute per-pixel object labels. This is done by intelligently upsampling the intermediate activations in each region of an image. Our model is more similar to the recent encoder-decoder network by Badrinarayanan *et al.* [1]. Their encoder step uses the first half of a VGG-like network to capture patterns in different regions of an image and their decoder step takes the activations from the encoder, which are of a reduced image resolution, and uses convolutional filters to upsample back to the original image size. In subsequent sections we describe our temporal variation in detail.

2. Temporal Convolutional Networks (TCN)

The input to our Temporal Convolutional Network can be a sensor signal (e.g. accelerometers) or latent encoding of a spatial CNN applied to each frame. Let $X_t \in \mathbb{R}^{F_0}$ be the input feature vector of length F_0 for time step t for $0 <$

$t \leq T$. Note that the time T may vary for each sequence, and we denote the number of time steps in each layer as T_l . The true action label for each frame is given by $y_t \in \{1, \dots, C\}$, where C is the number of classes.

Our encoder-decoder framework, as depicted in Figure 1, is composed of temporal convolutions, 1D pooling/upsampling, and channel-wise normalization layers.

For each of the L convolutional layers in the encoder, we apply a set of 1D filters that capture how the input signals evolve over the course of an action. The filters for each layer are parameterized by tensor $W^{(l)} \in \mathbb{R}^{F_l \times d \times F_{l-1}}$ and biases $b^{(l)} \in \mathbb{R}^{F_l}$, where $l \in \{1, \dots, L\}$ is the layer index and d is the filter duration. For the l -th layer of the encoder, the i -th component of the (unnormalized) activation $\hat{E}_t^{(l)} \in \mathbb{R}^{F_l}$ is a function of the incoming (normalized) activation matrix $E^{(l-1)} \in \mathbb{R}^{F_{l-1} \times T_{l-1}}$ from the previous layer

$$\hat{E}_{i,t}^{(l)} = f(b_i^{(l)} + \sum_{t'=1}^d \langle W_{i,t',\cdot}^{(l)}, E_{\cdot, t+d-t'}^{(l-1)} \rangle) \quad (1)$$

for each time t where $f(\cdot)$ is a Leaky Rectified Linear Unit. The normalization process is described below.

Max pooling is applied with width 2 across time (in 1D) such that $T_l = \frac{1}{2}T_{l-1}$.¹ Pooling enables us to efficiently compute activations over a long period of time.

We apply channel-wise normalization after each pooling step in the encoder. This has been effective in recent CNN methods including Trajectory-Pooled Deep-Convolutional Descriptors (TDD) [10]. We normalize the pooled activation vector $\hat{E}_t^{(l)}$ by the highest response at that time step, $m = \max_i \hat{E}_{i,t}^{(l)}$, with some small $\epsilon = 1\text{E-}5$ such that

$$E_t^{(l)} = \frac{1}{m + \epsilon} \hat{E}_t^{(l)}. \quad (2)$$

Our decoder is similar to the encoder, except that upsampling is used instead of pooling, and the order of the operations is now upsample, convolve, then normalize. Upsampling is performed by simply repeating each entry twice.

The probability that frame t corresponds to one of the C action classes is predicted by vector $\hat{Y}_t \in [0, 1]^C$ using weight matrix $U \in \mathbb{R}^{C \times F_0}$ and bias $c \in \mathbb{R}^C$

$$\hat{Y}_t = \text{softmax}(UD_t^{(1)} + c). \quad (3)$$

We explored many other mechanisms, such as adding skip connections between layers, using different patterns of convolutional layers, and other normalization schemes. These helped at times and hurt in others. The aforementioned solution was superior in aggregate.

¹In theory, this implies T must be divisible by 2^L . In practice, we pad each sequence to be of an appropriate length, given the pooling operations, such that the input length of the whole sequence, T , and the length of the output predictions are the same.

Implementation details: Each of the $L = 3$ layers has $F_l = \{32, 64, 96\}$ filters. Filter duration, d , is set as the mean segment duration for the shortest class from the training set. For example, $d = 10$ seconds for 50 Salads. Parameters of our model were learned using the cross entropy loss with Stochastic Gradient Descent and ADAM step updates. All models were implemented using Keras and TensorFlow.

For each frame in our video experiments, the input, X_t , is the first fully connected layer computed in a spatial CNN trained solely on each dataset. We trained the model of [8], except instead of using Motion History Images (MHI) as input to the CNN, we concatenate the following for image I_t at frame t : $[I_t, I_{t-d} - I_t, I_{t+d} - I_t, I_{t-2d} - I_t, I_{t+2d} - I_t]$ for $d = 0.5$ seconds. In our experiments, these difference images – which are a simple type of attention mechanism – tend to perform better than MHI or optical flow across these datasets. Furthermore, for each time step, we perform channel-wise normalization before feeding it into the TCN. This helps with large environmental fluctuations, such as changes in lighting.

3. Evaluation

We evaluate on three public datasets that contain action segmentation labels, video, and in two cases sensor data.

University of Dundee 50 Salads [18] contains 50 sequences of users making a salad. Each video is 5-10 minutes in duration and contains around 30 action instances such as cutting a tomato or peeling a cucumber. This dataset includes video and synchronized accelerometers attached to ten objects in the scene, such as the bowl, knife, and plate. We performed cross validation with 5 splits on the “eval” action granularity which includes 10 action classes. Our sensor results used the features from [9] which are the absolute values of accelerometer values. Previous results (e.g., [9, 14]) were evaluated using different setups. For example, [9] smoothed out short interstitial background segments. We reran all results to be consistent with [14]. We also included an LSTM baseline for comparison which uses 64 hidden states.

JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS) [5] was introduced to improve quantitative evaluation of robotic surgery training tasks. We used Leave One User Out cross validation on the suturing activity, which consists of 39 sequences performed by 8 users about 5 times each. The dataset includes video and synchronized robot kinematics (position, velocity, and gripper angle) for each robot end effector as well as corresponding action labels with 10 action classes. Sequences are a few minutes long and typically contain around 20 action instances.

Georgia Tech Egocentric Activities (GTEA) [4] contains 28 videos of 7 kitchen activities including making a sandwich and making coffee. For each of the four subjects, there is one instance of each activity. The camera is

mounted on the head of the user and is pointing at the area in front them. On average there are about 30 actions per video and videos are around a minute long. We used the 11 action classes defined in [3] and evaluated using leave one user out. We show results for user 2 to be consistent with [3] and [16].

Metrics: We evaluated using accuracy, which is simply the percent of correctly labeled frames, and segmental edit distance [9], which measures the correctness of the predicted temporal ordering of actions. This edit score is computed by applying the Levenshtein distance to the segmented predictions (e.g. $AAABBA \rightarrow ABA$). This is normalized to be in the range 0 to 100 such that higher is better.

4. Experiments and Discussion

Table 1 includes results for all datasets and corresponding sensing modalities. We include results from the spatial CNN which is input into the TCN, the Spatiotemporal CNN of Lea *et al.* [8] applied to the spatial features, and our TCN.

One of the most interesting findings is that some layers of convolutional filters appear to learn temporal shifts. There are certain actions in each dataset which are not easy to distinguish given the sensor data. By visualizing the activations for each layer, we found our model surmounts this issue by learning temporal offsets from activations in the previous layer. In addition, we find that despite the fact that we do not use a traditional temporal model, such as an RNN or CRF, our predictions do not suffer as heavily from issues like over-segmentation. This is highlighted by the large increase in edit score on most experiments.

Richard *et al.* [14] evaluated their model on the mid-level action granularity of 50 Salads which has 17 action classes. Their model achieved 54.2% accuracy, 44.8% edit, 0.379 mAP IoU overlap with a threshold of 0.1, and 0.229 mAP with a threshold of 0.5.² Our model achieves 59.7% accuracy, 47.3% edit, 0.579 mAP at 0.1, and 0.378 mAP at 0.5.

On GTEA, Singh *et al.* [16] reported 64.4% accuracy by performing cross validation on users 1 through 3. We achieve 62.5% using this setup. We found performance of our model has high variance between different trials on GTEA – even with the same hyper parameters – thus, the difference in accuracy is not likely to be statistically significant. Our approach could be used in tandem with features from Singh *et al.* to achieve superior performance.

Our model can be trained much faster than an RNN-LSTM. Using an Nvidia Titan X, it takes on the order of a minute to train a TCN for each split, whereas it takes on the order of an hour to train an RNN-LSTM. The speedup comes from the fact that we compute one set of convolutions for each layer, whereas RNN-LSTM effectively computes one set of convolutions for each time step.

²We computed our metrics using the predictions given by the authors.

50 Salads (“eval” setup)		
Sensor-based	Edit	Acc
[9] LC-SC-CRF	50.2	77.8
LSTM	54.5	73.3
TCN	65.6	82.0
Video-based	Edit	Acc
[8] VGG	7.6	38.3
[8] IDT	16.8	54.3
[8] Seg-ST-CNN	62.0	72.0
Spatial CNN	28.4	68.6
ST-CNN	55.5	74.2
TCN	61.1	74.4

GTEA		
Video-based	Edit	Acc
[3] Hand-crafted	-	47.7
[16] EgoNet	-	57.6
[16] TDD	-	59.5
[16] EgoNet+TDD	-	68.5
Spatial CNN	36.6	56.1
ST-CNN	53.4	64.5
TCN	58.8	66.1

JIGSAWS		
Sensor-based	Edit	Acc
[2] LSTM	75.3	80.5
[9] LC-SC-CRF	76.8	83.4
[2] Bidir LSTM	81.1	83.3
[17] SD-SDL	83.3	78.6
TCN	85.8	79.6
Vision-based	Edit	Acc
[19] MsM-CRF	-	71.7
[8] IDT	8.5	53.9
[8] VGG	24.3	45.9
[8] Seg-ST-CNN	66.6	74.7
Spatial CNN	37.7	74.0
ST-CNN	68.0	77.7
TCN	83.1	81.4

Table 1. Results on 50 Salads, Georgia Tech Egocentric Activities, and JHU-ISI Gesture and Skill Assessment Working Set. Notes: (1) Results using VGG and Improved Dense Trajectories (IDT) were intentionally computed without a temporal component for ablative analysis, hence their low edit scores. (2) We re-computed [9] using the author’s public code to be consistent with the setup of [14].

Conclusion: We introduced a model for action segmentation that learns a hierarchy of intermediate feature representations, which contrasts with the traditional low- versus high-level paradigm. This model achieves competitive or superior performance on several datasets and can be trained much more quickly than other models. A future version of this manuscript will include more comparisons and insights on the TCN.

References

- [1] V. Badrinarayanan, A. Handa, and R. Cipolla. Segnet: A deep convolutional encoder-decoder architecture for robust semantic pixel-wise labelling. *arXiv preprint arXiv:1505.07293*, 2015. 2
- [2] R. DiPietro, C. Lea, A. Malpani, N. Ahmidi, S. S. Vedula, G. I. Lee, M. R. Lee, and G. D. Hager. Recognizing surgical activities with recurrent neural networks. In *MICCAI*, 2016. 4
- [3] A. Fathi, A. Farhadi, and J. M. Rehg. Understanding egocentric activities. In *ICCV*, 2011. 3, 4
- [4] A. Fathi, R. Xiaofeng, and J. M. Rehg. Learning to recognize objects in egocentric activities. In *CVPR*, 2011. 3
- [5] Y. Gao, S. S. Vedula, C. E. Reiley, N. Ahmidi, B. Varadara-jan, H. C. Lin, L. Tao, L. Zappella, B. Béjar, D. D. Yuh, et al. JHU-ISI Gesture and Skill Assessment Working Set (JIGSAWS): A surgical activity dataset for human motion modeling. In *MICCAI Workshop: M2CAI*, 2014. 3
- [6] F. G. Hofmann, P. Heyer, and G. Hommel. Velocity profile based recognition of dynamic gestures with discrete hidden markov models. In *International Workshop on Gesture and Sign Language in Human-Computer Interaction*, 1998. 1
- [7] P. Krähenbühl and V. Koltun. Efficient inference in fully connected CRFs with gaussian edge potentials. In *NIPS*, 2011. 2
- [8] C. Lea, A. Reiter, R. Vidal, and G. D. Hager. Segmental spatio-temporal CNNs for fine-grained action segmentation. *ECCV*, 2016. 1, 2, 3, 4
- [9] C. Lea, R. Vidal, and G. D. Hager. Learning convolutional action primitives for fine-grained action recognition. In *ICRA*, 2016. 3, 4
- [10] Y. Q. Limin Wang and X. Tang. Action recognition with trajectory-pooled deep-convolutional descriptors. In *CVPR*, 2015. 2
- [11] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. *CVPR*, 2015. 2
- [12] J. Y. Ng, M. J. Hausknecht, S. Vijayanarasimhan, O. Vinyals, R. Monga, and G. Toderici. Beyond short snippets: Deep networks for video classification. In *CVPR*, 2015. 1
- [13] R. Pascanu, T. Mikolov, and Y. Bengio. On the difficulty of training recurrent neural networks. In *ICML*, 2013. 2
- [14] A. Richard and J. Gall. Temporal action detection using a statistical language model. In *CVPR*, 2016. 3, 4
- [15] B. Singh, T. K. Marks, M. Jones, O. Tuzel, and M. Shao. A multi-stream bi-directional recurrent neural network for fine-grained action detection. In *CVPR*, 2016. 2
- [16] S. Singh, C. Arora, and C. V. Jawahar. First person action recognition using deep learned descriptors. In *CVPR*, June 2016. 3, 4
- [17] S. Stefati, N. Cowan, and R. Vidal. Learning shared, discriminative dictionaries for surgical gesture segmentation and classification. In *MICCAI Workshop: M2CAI*, 2015. 4
- [18] S. Stein and S. J. McKenna. Combining embedded accelerometers with computer vision for recognizing food preparation activities. In *UbiComp*, 2013. 3
- [19] L. Tao, L. Zappella, G. D. Hager, and R. Vidal. Surgical gesture segmentation and recognition. In *MICCAI*, 2013. 4
- [20] M. Vrigkas, C. Nikou, and I. Kakadiaris. A review of human activity recognition methods. *Frontiers in Robotics and AI*, 2015. 2
- [21] H. Wang and C. Schmid. Action recognition with improved trajectories. In *ICCV*, 2013. 1

Convolutional Sequence to Sequence Learning

Jonas Gehring
 Michael Auli
 David Grangier
 Denis Yarats
 Yann N. Dauphin
 Facebook AI Research

Abstract

The prevalent approach to sequence to sequence learning maps an input sequence to a variable length output sequence via recurrent neural networks. We introduce an architecture based entirely on convolutional neural networks.¹ Compared to recurrent models, computations over all elements can be fully parallelized during training to better exploit the GPU hardware and optimization is easier since the number of non-linearities is fixed and independent of the input length. Our use of gated linear units eases gradient propagation and we equip each decoder layer with a separate attention module. We outperform the accuracy of the deep LSTM setup of Wu et al. (2016) on both WMT’14 English-German and WMT’14 English-French translation at an order of magnitude faster speed, both on GPU and CPU.

1. Introduction

Sequence to sequence learning has been successful in many tasks such as machine translation, speech recognition (Sutskever et al., 2014; Chorowski et al., 2015) and text summarization (Rush et al., 2015; Nallapati et al., 2016; Shen et al., 2016) amongst others. The dominant approach to date encodes the input sequence with a series of bi-directional recurrent neural networks (RNN) and generates a variable length output with another set of decoder RNNs, both of which interface via a soft-attention mechanism (Bahdanau et al., 2014; Luong et al., 2015). In machine translation, this architecture has been demonstrated to outperform traditional phrase-based models by large margins (Sennrich et al., 2016b; Zhou et al., 2016; Wu et al., 2016; §2).

¹The source code and models are available at <https://github.com/facebookresearch/fairseq>.

Convolutional neural networks are less common for sequence modeling, despite several advantages (Waibel et al., 1989; LeCun & Bengio, 1995). Compared to recurrent layers, convolutions create representations for fixed size contexts, however, the effective context size of the network can easily be made larger by stacking several layers on top of each other. This allows to precisely control the maximum length of dependencies to be modeled. Convolutional networks do not depend on the computations of the previous time step and therefore allow parallelization over every element in a sequence. This contrasts with RNNs which maintain a hidden state of the entire past that prevents parallel computation within a sequence.

Multi-layer convolutional neural networks create hierarchical representations over the input sequence in which nearby input elements interact at lower layers while distant elements interact at higher layers. Hierarchical structure provides a shorter path to capture long-range dependencies compared to the chain structure modeled by recurrent networks, e.g. we can obtain a feature representation capturing relationships within a window of n words by applying only $\mathcal{O}(\frac{n}{k})$ convolutional operations for kernels of width k , compared to a linear number $\mathcal{O}(n)$ for recurrent neural networks. Inputs to a convolutional network are fed through a constant number of kernels and non-linearities, whereas recurrent networks apply up to n operations and non-linearities to the first word and only a single set of operations to the last word. Fixing the number of non-linearities applied to the inputs also eases learning.

Recent work has applied convolutional neural networks to sequence modeling such as Bradbury et al. (2016) who introduce recurrent pooling between a succession of convolutional layers or Kalchbrenner et al. (2016) who tackle neural translation without attention. However, none of these approaches has been demonstrated improvements over state of the art results on large benchmark datasets. Gated convolutions have been previously explored for machine translation by Meng et al. (2015) but their evaluation was restricted to a small dataset and the model was used in tandem with a traditional count-based model. Architec-

tures which are partially convolutional have shown strong performance on larger tasks but their decoder is still recurrent (Gehring et al., 2016).

In this paper we propose an architecture for sequence to sequence modeling that is entirely convolutional. Our model is equipped with gated linear units (Dauphin et al., 2016) and residual connections (He et al., 2015a). We also use attention in every decoder layer and demonstrate that each attention layer only adds a negligible amount of overhead. The combination of these choices enables us to tackle large scale problems (§3).

We evaluate our approach on several large datasets for machine translation as well as summarization and compare to the current best architectures reported in the literature. On WMT’16 English-Romanian translation we achieve a new state of the art, outperforming the previous best result by 1.9 BLEU. On WMT’14 English-German we outperform the strong LSTM setup of Wu et al. (2016) by 0.5 BLEU and on WMT’14 English-French we outperform the likelihood trained system of Wu et al. (2016) by 1.6 BLEU. Furthermore, our model can translate unseen sentences at an order of magnitude faster speed than Wu et al. (2016) on GPU and CPU hardware (§4, §5).

2. Recurrent Sequence to Sequence Learning

Sequence to sequence modeling has been synonymous with recurrent neural network based encoder-decoder architectures (Sutskever et al., 2014; Bahdanau et al., 2014). The encoder RNN processes an input sequence $\mathbf{x} = (x_1, \dots, x_m)$ of m elements and returns state representations $\mathbf{z} = (z_1, \dots, z_m)$. The decoder RNN takes $\mathbf{z}$ and generates the output sequence $\mathbf{y} = (y_1, \dots, y_n)$ left to right, one element at a time. To generate output y_{i+1} , the decoder computes a new hidden state h_{i+1} based on the previous state h_i , an embedding g_i of the previous target language word y_i , as well as a conditional input c_i derived from the encoder output $\mathbf{z}$. Based on this generic formulation, various encoder-decoder architectures have been proposed, which differ mainly in the conditional input and the type of RNN.

Models without attention consider only the final encoder state z_m by setting $c_i = z_m$ for all i (Cho et al., 2014), or simply initialize the first decoder state with z_m (Sutskever et al., 2014), in which case c_i is not used. Architectures with attention (Bahdanau et al., 2014; Luong et al., 2015) compute c_i as a weighted sum of $(z_1, \dots, z_m)$ at each time step. The weights of the sum are referred to as attention scores and allow the network to focus on different parts of the input sequence as it generates the output sequences. Attention scores are computed by essentially comparing each encoder state z_j to a combination of the previous decoder

state h_i and the last prediction y_i ; the result is normalized to be a distribution over input elements.

Popular choices for recurrent networks in encoder-decoder models are long short term memory networks (LSTM; Hochreiter & Schmidhuber, 1997) and gated recurrent units (GRU; Cho et al., 2014). Both extend Elman RNNs (Elman, 1990) with a gating mechanism that allows the memorization of information from previous time steps in order to model long-term dependencies. Most recent approaches also rely on bi-directional encoders to build representations of both past and future contexts (Bahdanau et al., 2014; Zhou et al., 2016; Wu et al., 2016). Models with many layers often rely on shortcut or residual connections (He et al., 2015a; Zhou et al., 2016; Wu et al., 2016).

3. A Convolutional Architecture

Next we introduce a fully convolutional architecture for sequence to sequence modeling. Instead of relying on RNNs to compute intermediate encoder states $\mathbf{z}$ and decoder states $\mathbf{h}$ we use convolutional neural networks (CNN).

3.1. Position Embeddings

First, we embed input elements $\mathbf{x} = (x_1, \dots, x_m)$ in distributional space as $\mathbf{w} = (w_1, \dots, w_m)$, where $w_j \in \mathbb{R}^f$ is a column in an embedding matrix $\mathcal{D} \in \mathbb{R}^{V \times f}$. We also equip our model with a sense of order by embedding the absolute position of input elements $\mathbf{p} = (p_1, \dots, p_m)$ where $p_j \in \mathbb{R}^f$. Both are combined to obtain input element representations $\mathbf{e} = (w_1 + p_1, \dots, w_m + p_m)$. We proceed similarly for output elements that were already generated by the decoder network to yield output element representations that are being fed back into the decoder network $\mathbf{g} = (g_1, \dots, g_n)$. Position embeddings are useful in our architecture since they give our model a sense of which portion of the sequence in the input or output it is currently dealing with (§5.4).

3.2. Convolutional Block Structure

Both encoder and decoder networks share a simple block structure that computes intermediate states based on a fixed number of input elements. We denote the output of the l -th block as $\mathbf{h}^l = (h_1^l, \dots, h_n^l)$ for the decoder network, and $\mathbf{z}^l = (z_1^l, \dots, z_m^l)$ for the encoder network; we refer to blocks and layers interchangeably. Each block contains a one dimensional convolution followed by a non-linearity. For a decoder network with a single block and kernel width k , each resulting state h_i^1 contains information over k input elements. Stacking several blocks on top of each other increases the number of input elements represented in a state. For instance, stacking 6 blocks with $k = 5$ results in an input field of 25 elements, i.e. each output depends on 25

inputs. Non-linearities allow the networks to exploit the full input field, or to focus on fewer elements if needed.

Each convolution kernel is parameterized as $W \in \mathbb{R}^{2d \times kd}$, $b_w \in \mathbb{R}^{2d}$ and takes as input $X \in \mathbb{R}^{k \times d}$ which is a concatenation of k input elements embedded in d dimensions and maps them to a single output element $Y \in \mathbb{R}^{2d}$ that has twice the dimensionality of the input elements; subsequent layers operate over the k output elements of the previous layer. We choose gated linear units (GLU; Dauphin et al., 2016) as non-linearity which implement a simple gating mechanism over the output of the convolution $Y = [A \ B] \in \mathbb{R}^{2d}$:

$$v([A \ B]) = A \otimes \sigma(B)$$

where $A, B \in \mathbb{R}^d$ are the inputs to the non-linearity, $\otimes$ is the point-wise multiplication and the output $v([A \ B]) \in \mathbb{R}^d$ is half the size of Y . The gates $\sigma(B)$ control which inputs A of the current context are relevant. A similar non-linearity has been introduced in Oord et al. (2016b) who apply $\tanh$ to A but Dauphin et al. (2016) shows that GLUs perform better in the context of language modelling.

To enable deep convolutional networks, we add residual connections from the input of each convolution to the output of the block (He et al., 2015a).

$$h_i^l = v(W^l[h_{i-k/2}^{l-1}, \dots, h_{i+k/2}^{l-1}] + b_w^l) + h_i^{l-1}$$

For encoder networks we ensure that the output of the convolutional layers matches the input length by padding the input at each layer. However, for decoder networks we have to take care that no future information is available to the decoder (Oord et al., 2016a). Specifically, we pad the input by $k - 1$ elements on both the left and right side by zero vectors, and then remove k elements from the end of the convolution output.

We also add linear mappings to project between the embedding size f and the convolution outputs that are of size $2d$. We apply such a transform to w when feeding embeddings to the encoder network, to the encoder output z_j^u , to the final layer of the decoder just before the softmax $\mathbf{h}^L$, and to all decoder layers $\mathbf{h}^1$ before computing attention scores (1).

Finally, we compute a distribution over the T possible next target elements y_{i+1} by transforming the top decoder output h_i^L via a linear layer with weights W_o and bias b_o :

$$p(y_{i+1}|y_1, \dots, y_i, \mathbf{x}) = \text{softmax}(W_o h_i^L + b_o) \in \mathbb{R}^T$$

3.3. Multi-step Attention

We introduce a separate attention mechanism for each decoder layer. To compute the attention, we combine the current decoder state h_i^l with an embedding of the previous

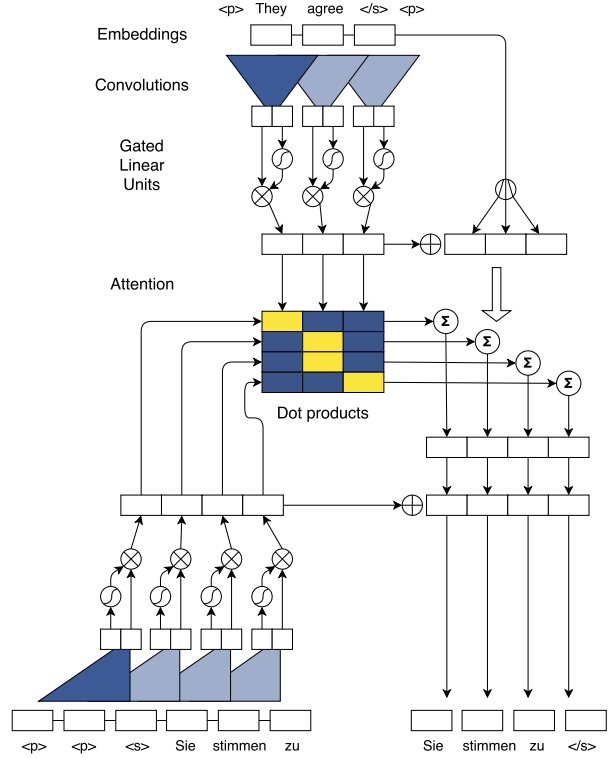


Figure 1. Illustration of batching during training. The English source sentence is encoded (top) and we compute all attention values for the four German target words (center) simultaneously. Our attentions are just dot products between decoder context representations (bottom left) and encoder representations. We add the conditional inputs computed by the attention (center right) to the decoder states which then predict the target words (bottom right). The sigmoid and multiplicative boxes illustrate Gated Linear Units.

target element g_i :

$$d_i^l = W_d^l h_i^l + b_d^l + g_i \quad (1)$$

For decoder layer l the attention a_{ij}^l of state i and source element j is computed as a dot-product between the decoder state summary d_i^l and each output z_j^u of the last encoder block u :

$$a_{ij}^l = \frac{\exp(d_i^l \cdot z_j^u)}{\sum_{t=1}^m \exp(d_i^l \cdot z_t^u)}$$

The conditional input c_i^l to the current decoder layer is a weighted sum of the encoder outputs as well as the input element embeddings e_j (Figure 1, center right):

$$c_i^l = \sum_{j=1}^m a_{ij}^l (z_j^u + e_j) \quad (2)$$

This is slightly different to recurrent approaches which compute both the attention and the weighted sum over z_j^u

only. We found adding e_j to be beneficial and it resembles key-value memory networks where the keys are the z_j^u and the values are the $z_j^u + e_j$ (Miller et al., 2016). Encoder outputs z_j^u represent potentially large input contexts and e_j provides point information about a specific input element that is useful when making a prediction. Once c_i^l has been computed, it is simply added to the output of the corresponding decoder layer h_i^l .

This can be seen as attention with multiple ‘hops’ (Sukhbaatar et al., 2015) compared to single step attention (Bahdanau et al., 2014; Luong et al., 2015; Zhou et al., 2016; Wu et al., 2016). In particular, the attention of the first layer determines a useful source context which is then fed to the second layer that takes this information into account when computing attention etc. The decoder also has immediate access to the attention history of the $k - 1$ previous time steps because the conditional inputs $c_{i-k}^{l-1}, \dots, c_i^{l-1}$ are part of $h_{i-k}^{l-1}, \dots, h_i^{l-1}$ which are input to h_i^l . This makes it easier for the model to take into account which previous inputs have been attended to already compared to recurrent nets where this information is in the recurrent state and needs to survive several non-linearities. Overall, our attention mechanism considers which words we previously attended to (Yang et al., 2016) and performs multiple attention ‘hops’ per time step. In Appendix §C, we plot attention scores for a deep decoder and show that at different layers, different portions of the source are attended to.

Our convolutional architecture also allows to batch the attention computation across all elements of a sequence compared to RNNs (Figure 1, middle). We batch the computations of each decoder layer individually.

3.4. Normalization Strategy

We stabilize learning through careful weight initialization (§3.5) and by scaling parts of the network to ensure that the variance throughout the network does not change dramatically. In particular, we scale the output of residual blocks as well as the attention to preserve the variance of activations. We multiply the sum of the input and output of a residual block by $\sqrt{0.5}$ to halve the variance of the sum. This assumes that both summands have the same variance which is not always true but effective in practice.

The conditional input c_i^l generated by the attention is a weighted sum of m vectors (2) and we counteract a change in variance through scaling by $m\sqrt{1/m}$; we multiply by m to scale up the inputs to their original size, assuming the attention scores are uniformly distributed. This is generally not the case but we found it to work well in practice.

For convolutional decoders with multiple attention, we scale the gradients for the encoder layers by the number

of attention mechanisms we use; we exclude source word embeddings. We found this to stabilize learning since the encoder received too much gradient otherwise.

3.5. Initialization

Normalizing activations when adding the output of different layers, e.g. residual connections, requires careful weight initialization. The motivation for our initialization is the same as for the normalization: maintain the variance of activations throughout the forward and backward passes. All embeddings are initialized from a normal distribution with mean 0 and standard deviation 0.1. For layers whose output is not directly fed to a gated linear unit, we initialize weights from $\mathcal{N}(0, \sqrt{1/n_l})$ where n_l is the number of input connections to each neuron. This ensures that the variance of a normally distributed input is retained.

For layers which are followed by a GLU activation, we propose a weight initialization scheme by adapting the derivations in (He et al., 2015b; Glorot & Bengio, 2010; Appendix A). If the GLU inputs are distributed with mean 0 and have sufficiently small variance, then we can approximate the output variance with $1/4$ of the input variance (Appendix A.1). Hence, we initialize the weights so that the input to the GLU activations have 4 times the variance of the layer input. This is achieved by drawing their initial values from $\mathcal{N}(0, \sqrt{4/n_l})$. Biases are uniformly set to zero when the network is constructed.

We apply dropout to the input of some layers so that inputs are retained with a probability of p . This can be seen as multiplication with a Bernoulli random variable taking value $1/p$ with probability p and 0 otherwise (Srivastava et al., 2014). The application of dropout will then cause the variance to be scaled by $1/p$. We aim to restore the incoming variance by initializing the respective layers with larger weights. Specifically, we use $\mathcal{N}(0, \sqrt{4p/n_l})$ for layers whose output is subject to a GLU and $\mathcal{N}(0, \sqrt{p/n_l})$ otherwise (Appendix A.3).

4. Experimental Setup

4.1. Datasets

We consider three major WMT translation tasks as well as a text summarization task.

WMT’16 English-Romanian. We use the same data and pre-processing as Sennrich et al. (2016b) but remove sentences with more than 175 words. This results in 2.8M sentence pairs for training and we evaluate on newstest2016.²

²We followed the pre-processing of <https://github.com/rsennrich/wmt16-scripts/blob/80e21e5/sample/preprocess.sh> and added the back-translated data from http://data.statmt.org/rsennrich/wmt16_

We experiment with word-based models using a source vocabulary of 200K types and a target vocabulary of 80K types. We also consider a joint source and target byte-pair encoding (BPE) with 40K types (Sennrich et al., 2016a;b).

WMT’14 English-German. We use the same setup as Luong et al. (2015) which comprises 4.5M sentence pairs for training and we test on newstest2014.³ As vocabulary we use 40K sub-word types based on BPE.

WMT’14 English-French. We use the full training set of 36M sentence pairs, and remove sentences longer than 175 words as well as pairs with a source/target length ratio exceeding 1.5. This results in 35.5M sentence-pairs for training. Results are reported on newstest2014. We use a source and target vocabulary with 40K BPE types.

In all setups a small subset of the training data serves as validation set (about 0.5-1% for each dataset) for early stopping and learning rate annealing.

Abstractive summarization. We train on the Gigaword corpus (Graff et al., 2003) and pre-process it identically to Rush et al. (2015) resulting in 3.8M training examples and 190K for validation. We evaluate on the DUC-2004 test data comprising 500 article-title pairs (Over et al., 2007) and report three variants of recall-based ROUGE (Lin, 2004), namely, ROUGE-1 (unigrams), ROUGE-2 (bigrams), and ROUGE-L (longest-common substring). We also evaluate on a Gigaword test set of 2000 pairs which is identical to the one used by Rush et al. (2015) and we report F1 ROUGE similar to prior work. Similar to Shen et al. (2016) we use a source and target vocabulary of 30K words and require outputs to be at least 14 words long.

4.2. Model Parameters and Optimization

We use 512 hidden units for both encoders and decoders, unless otherwise stated. All embeddings, including the output produced by the decoder before the final linear layer, have dimensionality 512; we use the same dimensionalities for linear layers mapping between the hidden and embedding sizes (§3.2).

We train our convolutional models with Nesterov’s accelerated gradient method (Sutskever et al., 2013) using a momentum value of 0.99 and renormalize gradients if their norm exceeds 0.1 (Pascanu et al., 2013). We use a learning rate of 0.25 and once the validation perplexity stops improving, we reduce the learning rate by an order of magnitude after each epoch until it falls below 10^{-4} .

Unless otherwise stated, we use mini-batches of 64 sentences. We restrict the maximum number of words in a mini-batch to make sure that batches with long sentences

backtranslations/en-ro.

³<http://nlp.stanford.edu/projects/nmt>

still fit in GPU memory. If the threshold is exceeded, we simply split the batch until the threshold is met and process the parts separately. Gradients are normalized by the number of non-padding tokens per mini-batch. We also use weight normalization for all layers except for lookup tables (Salimans & Kingma, 2016).

Besides dropout on the embeddings and the decoder output, we also apply dropout to the input of the convolutional blocks (Srivastava et al., 2014). All models are implemented in Torch (Collobert et al., 2011) and trained on a single Nvidia M40 GPU except for WMT’14 English-French for which we use a multi-GPU setup on a single machine. We train on up to eight GPUs synchronously by maintaining copies of the model on each card and split the batch so that each worker computes 1/8-th of the gradients; at the end we sum the gradients via Nvidia NCCL.

4.3. Evaluation

We report average results over three runs of each model, where each differs only in the initial random seed. Translations are generated by a beam search and we normalize log-likelihood scores by sentence length. We use a beam of width 5. We divide the log-likelihoods of the final hypothesis in beam search by their length $|y|$. For WMT’14 English-German we tune a length normalization constant on a separate development set (newstest2015) and we normalize log-likelihoods by $|y|^\alpha$ (Wu et al., 2016). On other datasets we did not find any benefit with length normalization.

For word-based models, we perform unknown word replacement based on attention scores after generation (Jean et al., 2015). Unknown words are replaced by looking up the source word with the maximum attention score in a pre-computed dictionary. If the dictionary contains no translation, then we simply copy the source word. Dictionaries were extracted from the word aligned training data that we obtained with fast_align (Dyer et al., 2013). Each source word is mapped to the target word it is most frequently aligned to. In our multi-step attention (§3.3) we simply average the attention scores over all layers. Finally, we compute case-sensitive tokenized BLEU, except for WMT’16 English-Romanian where we use detokenized BLEU to be comparable with Sennrich et al. (2016b).⁴

⁴<https://github.com/moses-smt/mosesdecoder/blob/617e8c8/scripts/generic/{multi-bleu.perl,mteval-v13a.pl}>

5. Results

5.1. Recurrent vs. Convolutional Models

We first evaluate our convolutional model on three translation tasks. On WMT’16 English-Romanian translation we compare to [Sennrich et al. \(2016b\)](#) which is the winning entry on this language pair at WMT’16 ([Bojar et al., 2016](#)). Their model implements the attention-based sequence to sequence architecture of [Bahdanau et al. \(2014\)](#) and uses GRU cells both in the encoder and decoder. We test both word-based and BPE vocabularies (§4).

Table 1 shows that our fully convolutional sequence to sequence model (ConvS2S) outperforms the WMT’16 winning entry for English-Romanian by 1.9 BLEU with a BPE encoding and by 1.3 BLEU with a word factored vocabulary. This instance of our architecture has 20 layers in the encoder and 20 layers in the decoder, both using kernels of width 3 and hidden size 512 throughout. Training took between 6 and 7.5 days on a single GPU.

On WMT’14 English to German translation we compare to the following prior work: [Luong et al. \(2015\)](#) is based on a four layer LSTM attention model, ByteNet ([Kalchbrenner et al., 2016](#)) propose a convolutional model based on characters without attention, with 30 layers in the encoder and 30 layers in the decoder, GNMT ([Wu et al., 2016](#)) represents the state of the art on this dataset and they use eight encoder LSTMs as well as eight decoder LSTMs, we quote their result for a word-based model, such as ours, as well as a word-piece model ([Schuster & Nakajima, 2012](#)).⁵

The results (Table 1) show that our convolutional model outperforms GNMT by 0.5 BLEU. Our encoder has 15 layers and the decoder has 15 layers, both with 512 hidden units in the first ten layers and 768 units in the subsequent three layers, all using kernel width 3. The final two layers have 2048 units which are just linear mappings with a single input. We trained this model on a single GPU over a period of 18.5 days with a batch size of 48. LSTM sparse mixtures have shown strong accuracy at 26.03 BLEU for a single run ([Shazeer et al., 2016](#)) which compares to 25.39 BLEU for our best run. This mixture sums the output of four experts, not unlike an ensemble which sums the output of multiple networks. ConvS2S also benefits from ensembling (§5.2), therefore mixtures are a promising direction.

Finally, we train on the much larger WMT’14 English-French task where we compare to the state of the art result of GNMT ([Wu et al., 2016](#)). Our model is trained with a simple token-level likelihood objective and we improve over GNMT in the same setting by 1.6 BLEU on average. We also outperform their reinforcement (RL) models by 0.5

⁵We did not use the exact same vocabulary size because word pieces and BPE estimate the vocabulary differently.

WMT’16 English-Romanian	BLEU
Sennrich et al. (2016b) GRU (BPE 90K)	28.1
ConvS2S (Word 80K)	29.45
ConvS2S (BPE 40K)	30.02

WMT’14 English-German	BLEU
Luong et al. (2015) LSTM (Word 50K)	20.9
Kalchbrenner et al. (2016) ByteNet (Char)	23.75
Wu et al. (2016) GNMT (Word 80K)	23.12
Wu et al. (2016) GNMT (Word pieces)	24.61
ConvS2S (BPE 40K)	25.16

WMT’14 English-French	BLEU
Wu et al. (2016) GNMT (Word 80K)	37.90
Wu et al. (2016) GNMT (Word pieces)	38.95
Wu et al. (2016) GNMT (Word pieces) + RL	39.92
ConvS2S (BPE 40K)	40.51

Table 1. Accuracy on WMT tasks compared to previous work. ConvS2S and GNMT results are averaged over several runs.

BLEU. Reinforcement learning is equally applicable to our architecture and we believe that it would further improve our results.

The ConvS2S model for this experiment uses 15 layers in the encoder and 15 layers in the decoder, both with 512 hidden units in the first five layers, 768 units in the subsequent four layers, 1024 units in the next 3 layers, all using kernel width 3; the final two layers have 2048 units and 4096 units each but they are linear mappings with kernel width 1. This model has an effective context size of only 25 words, beyond which it cannot access any information on the target size. Our results are based on training with 8 GPUs for about 37 days and batch size 32 on each worker.⁶ The same configuration as for WMT’14 English-German achieves 39.41 BLEU in two weeks on this dataset in an eight GPU setup.

[Zhou et al. \(2016\)](#) report a non-averaged result of 39.2 BLEU. More recently, [Ha et al. \(2016\)](#) showed that one can generate weights with one LSTM for another LSTM. This approach achieves 40.03 BLEU but the result is not averaged. [Shazeer et al. \(2016\)](#) compares at 40.56 BLEU to our best single run of 40.70 BLEU.

⁶This is half of the GPU time consumed by a basic model of [Wu et al. \(2016\)](#) who use 96 GPUs for 6 days. We expect the time to train our model to decrease substantially in a multi-machine setup.

WMT'14 English-German	BLEU
Wu et al. (2016) GNMT	26.20
Wu et al. (2016) GNMT + RL	26.30
ConvS2S	26.43

WMT'14 English-French	BLEU
Zhou et al. (2016)	40.4
Wu et al. (2016) GNMT	40.35
Wu et al. (2016) GNMT + RL	41.16
ConvS2S	41.44
ConvS2S (10 models)	41.62

Table 2. Accuracy of ensembles with eight models. We show both likelihood and Reinforce (RL) results for GNMT; Zhou et al. (2016) and ConvS2S use simple likelihood training.

The translations produced by our models often match the length of the references, particularly for the large WMT'14 English-French task, or are very close for small to medium data sets such as WMT'14 English-German or WMT'16 English-Romanian.

5.2. Ensemble Results

Next, we ensemble eight likelihood-trained models for both WMT'14 English-German and WMT'14 English-French and compare to previous work which also reported ensemble results. For the former, we also show the result when ensembling 10 models. Table 2 shows that we outperform the best current ensembles on both datasets.

5.3. Generation Speed

Next, we evaluate the inference speed of our architecture on the development set of the WMT'14 English-French task which is the concatenation of newstest2012 and newstest2013; it comprises 6003 sentences. We measure generation speed both on GPU and CPU hardware. Specifically, we measure GPU speed on three generations of Nvidia cards: a GTX-1080ti, an M40 as well as an older K40 card. CPU timings are measured on one host with 48 hyper-threaded cores (Intel Xeon E5-2680 @ 2.50GHz) with 40 workers. In all settings, we batch up to 128 sentences, composing batches with sentences of equal length. Note that the majority of batches is smaller because of the small size of the development set. We experiment with beams of size 5 as well as greedy search, i.e beam of size 1. To make generation fast, we do not recompute convolution states that have not changed compared to the previous time step but rather copy (shift) these activations.

We compare to results reported in Wu et al. (2016) who

	BLEU	Time (s)
GNMT GPU (K80)	31.20	3,028
GNMT CPU 88 cores	31.20	1,322
GNMT TPU	31.21	384
ConvS2S GPU (K40) $b = 1$	33.45	327
ConvS2S GPU (M40) $b = 1$	33.45	221
ConvS2S GPU (GTX-1080ti) $b = 1$	33.45	142
ConvS2S CPU 48 cores $b = 1$	33.45	142
ConvS2S GPU (K40) $b = 5$	34.10	587
ConvS2S CPU 48 cores $b = 5$	34.10	482
ConvS2S GPU (M40) $b = 5$	34.10	406
ConvS2S GPU (GTX-1080ti) $b = 5$	34.10	256

Table 3. CPU and GPU generation speed in seconds on the development set of WMT'14 English-French. We show results for different beam sizes b . GNMT figures are taken from Wu et al. (2016). CPU speeds are not directly comparable because Wu et al. (2016) use a 88 core machine versus our 48 core setup.

use Nvidia K80 GPUs which are essentially two K40s. We did not have such a GPU available and therefore run experiments on an older K40 card which is inferior to a K80, in addition to the newer M40 and GTX-1080ti cards. The results (Table 3) show that our model can generate translations on a K40 GPU at 9.3 times the speed and 2.25 higher BLEU; on an M40 the speed-up is up to 13.7 times and on a GTX-1080ti card the speed is 21.3 times faster. A larger beam of size 5 decreases speed but gives better BLEU.

On CPU, our model is up to 9.3 times faster, however, the GNMT CPU results were obtained with an 88 core machine whereas our results were obtained with just over half the number of cores. On a per CPU core basis, our model is 17 times faster at a better BLEU. Finally, our CPU speed is 2.7 times higher than GNMT on a custom TPU chip which shows that high speed can be achieved on commodity hardware. We do not report TPU figures as we do not have access to this hardware.

5.4. Position Embeddings

In the following sections, we analyze the design choices in our architecture. The remaining results in this paper are based on the WMT'14 English-German task with 13 encoder layers at kernel size 3 and 5 decoder layers at kernel size 5. We use a target vocabulary of 160K words as well as vocabulary selection (Mi et al., 2016; L'Hostis et al., 2016) to decrease the size of the output layer which speeds up training and testing. The average vocabulary size for each training batch is about 20K target words. All figures are averaged over three runs (§4) and BLEU is reported on newstest2014 before unknown word replacement.

We start with an experiment that removes the position em-

	PPL	BLEU
ConvS2S	6.64	21.7
-source position	6.69	21.3
-target position	6.63	21.5
-source & target position	6.68	21.2

Table 4. Effect of removing position embeddings from our model in terms of validation perplexity (valid PPL) and BLEU.

beddings from the encoder and decoder (§3.1). These embeddings allow our model to identify which portion of the source and target sequence it is dealing with but also impose a restriction on the maximum sentence length. Table 4 shows that position embeddings are helpful but that our model still performs well without them. Removing the source position embeddings results in a larger accuracy decrease than target position embeddings. However, removing both source and target positions decreases accuracy only by 0.5 BLEU. We had assumed that the model would not be able to calibrate the length of the output sequences very well without explicit position information, however, the output lengths of models without position embeddings closely matches models with position information. This indicates that the models can learn relative position information within the contexts visible to the encoder and decoder networks which can observe up to 27 and 25 words respectively.

Recurrent models typically do not use explicit position embeddings since they can learn where they are in the sequence through the recurrent hidden state computation. In our setting, the use of position embeddings requires only a simple addition to the input word embeddings which is a negligible overhead.

5.5. Multi-step Attention

The multiple attention mechanism (§3.3) computes a separate source context vector for each decoder layer. The computation also takes into account contexts computed for preceding decoder layers of the current time step as well as previous time steps that are within the receptive field of the decoder. How does multiple attention compare to attention in fewer layers or even only in a single layer as is usual? Table 5 shows that attention in all decoder layers achieves the best validation perplexity (PPL). Furthermore, removing more and more attention layers decreases accuracy, both in terms of BLEU as well as PPL.

The computational overhead for attention is very small compared to the rest of the network. Training with attention in all five decoder layers processes 3624 target words per second on average on a single GPU, compared to 3772 words per second for attention in a single layer. This is only

Attn Layers	PPL	BLEU
1,2,3,4,5	6.65	21.63
1,2,3,4	6.70	21.54
1,2,3	6.95	21.36
1,2	6.92	21.47
1,3,5	6.97	21.10
1	7.15	21.26
2	7.09	21.30
3	7.11	21.19
4	7.19	21.31
5	7.66	20.24

Table 5. Multi-step attention in all five decoder layers or fewer layers in terms of validation perplexity (PPL) and test BLEU.

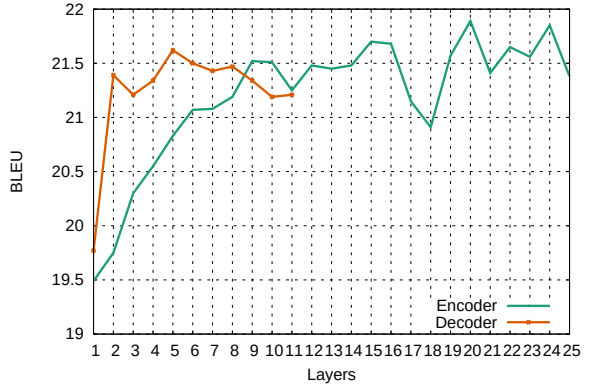


Figure 2. Encoder and decoder with different number of layers.

a 4% slow down when adding 4 attention modules. Most neural machine translation systems only use a single module. This demonstrates that attention is not the bottleneck in neural machine translation, even though it is quadratic in the sequence length (cf. Kalchbrenner et al., 2016). Part of the reason for the low impact on speed is that we batch the computation of an attention module over all target words, similar to Kalchbrenner et al. (2016). However, for RNNs batching of the attention may be less effective because of the dependence on the previous time step.

5.6. Kernel size and Depth

Figure 2 shows accuracy when we change the number of layers in the encoder or decoder. The kernel width for layers in the encoder is 3 and for the decoder it is 5. Deeper architectures are particularly beneficial for the encoder but less so for the decoder. Decoder setups with two layers already perform well whereas for the encoder accuracy keeps increasing steadily with more layers until up to 9 layers when accuracy starts to plateau.

	DUC-2004			Gigaword		
	RG-1 (R)	RG-2 (R)	RG-L (R)	RG-1 (F)	RG-2 (F)	RG-L (F)
RNN MLE (Shen et al., 2016)	24.92	8.60	22.25	32.67	15.23	30.56
RNN MRT (Shen et al., 2016)	30.41	10.87	26.79	36.54	16.59	33.44
WFE (Suzuki & Nagata, 2017)	32.28	10.54	27.80	36.30	17.31	33.88
ConvS2S	30.44	10.84	26.90	35.88	17.48	33.29

Table 6. Accuracy on two summarization tasks in terms of Rouge-1 (RG-1), Rouge-2 (RG-2), and Rouge-L (RG-L).

Kernel width	Encoder layers		
	5	9	13
3	20.61	21.17	21.63
5	20.80	21.02	21.42
7	20.81	21.30	21.09

Table 7. Encoder with different kernel width in terms of BLEU.

Kernel width	Decoder layers		
	3	5	7
3	21.10	21.71	21.62
5	21.09	21.63	21.24
7	21.40	21.31	21.33

Table 8. Decoder with different kernel width in terms of BLEU.

Aside from increasing the depth of the networks, we can also change the kernel width. Table 7 shows that encoders with narrow kernels and many layers perform better than wider kernels. These networks can also be faster since the amount of work to compute a kernel operating over 3 input elements is less than half compared to kernels over 7 elements. We see a similar picture for decoder networks with large kernel sizes (Table 8). Dauphin et al. (2016) shows that context sizes of 20 words are often sufficient to achieve very good accuracy on language modeling for English.

5.7. Summarization

Finally, we evaluate our model on abstractive sentence summarization which takes a long sentence as input and outputs a shortened version. The current best models on this task are recurrent neural networks which either optimize the evaluation metric (Shen et al., 2016) or address specific problems of summarization such as avoiding repeated generations (Suzuki & Nagata, 2017). We use standard likelihood training for our model and a simple model with six layers in the encoder and decoder each, hidden size 256, batch size 128, and we trained on a single GPU in one night. Table 6 shows that our likelihood trained model outperforms the likelihood trained model (RNN MLE) of Shen et al. (2016) and is not far behind the best models on this task which benefit from task-specific optimization and

model structure. We expect our model to benefit from these improvements as well.

6. Conclusion and Future Work

We introduce the first fully convolutional model for sequence to sequence learning that outperforms strong recurrent models on very large benchmark datasets at an order of magnitude faster speed. Compared to recurrent networks, our convolutional approach allows to discover compositional structure in the sequences more easily since representations are built hierarchically. Our model relies on gating and performs multiple attention steps.

We achieve a new state of the art on several public translation benchmark data sets. On the WMT’16 English-Romanian task we outperform the previous best result by 1.9 BLEU, on WMT’14 English-French translation we improve over the LSTM model of Wu et al. (2016) by 1.6 BLEU in a comparable setting, and on WMT’14 English-German translation we outperform the same model by 0.5 BLEU. In future work, we would like to apply convolutional architectures to other sequence to sequence learning problems which may benefit from learning hierarchical representations as well.

Acknowledgements

We thank Benjamin Graham for providing a fast 1-D convolution, and Ronan Collobert as well as Yann LeCun for helpful discussions related to this work.

References

- Ba, Jimmy Lei, Kiros, Jamie Ryan, and Hinton, Geoffrey E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Bahdanau, Dzmitry, Cho, Kyunghyun, and Bengio, Yoshua. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- Bojar, Ondrej, Chatterjee, Rajen, Federmann, Christian, Graham, Yvette, Haddow, Barry, Huck, Matthias,

A. Weight Initialization

We derive a weight initialization scheme tailored to the GLU activation function similar to [Glorot & Bengio \(2010\)](#); [He et al. \(2015b\)](#) by focusing on the variance of activations within the network for both forward and backward passes. We also detail how we modify the weight initialization for dropout.

A.1. Forward Pass

Assuming that the inputs $\mathbf{x}_l$ of a convolutional layer l and its weights W_l are independent and identically distributed (i.i.d.), the variance of its output, computed as $\mathbf{y}_l = W_l \mathbf{x}_l + \mathbf{b}_l$, is

$$Var[\mathbf{y}_l] = n_l Var[w_l x_l] \quad (3)$$

where n_l is the number inputs to the layer. For one-dimensional convolutional layers with kernel width k and input dimension c , this is kc . We adopt the notation in [\(He et al., 2015b\)](#), i.e. y_l , w_l and x_l represent the random variables in $\mathbf{y}_l$, W_l and $\mathbf{x}_l$. With w_l and x_l independent from each other and normally distributed with zero mean, this amounts to

$$Var[y_l] = n_l Var[w_l] Var[x_l]. \quad (4)$$

$\mathbf{x}_l$ is the result of the GLU activation function $\mathbf{y}_{l-1}^a \sigma(\mathbf{y}_{l-1}^b)$ with $\mathbf{y}_{l-1} = (\mathbf{y}_{l-1}^a, \mathbf{y}_{l-1}^b)$, and $\mathbf{y}_{l-1}^a, \mathbf{y}_{l-1}^b$ i.i.d. Next, we formulate upper and lower bounds in order to approximate $Var[x_l]$. If $\mathbf{y}_{l-1}$ follows a symmetric distribution with mean 0, then

$$Var[x_l] = Var[y_{l-1}^a \sigma(\mathbf{y}_{l-1}^b)] \quad (5)$$

$$= E[(y_{l-1}^a \sigma(\mathbf{y}_{l-1}^b))^2] - E^2[y_{l-1}^a \sigma(\mathbf{y}_{l-1}^b)] \quad (6)$$

$$= Var[y_{l-1}^a] E[\sigma(\mathbf{y}_{l-1}^b)^2]. \quad (7)$$

A lower bound is given by $(1/4)Var[y_{l-1}^a]$ when expanding (6) with $E^2[\sigma(\mathbf{y}_{l-1}^b)] = 1/4$:

$$Var[x_l] = Var[y_{l-1}^a \sigma(\mathbf{y}_{l-1}^b)] \quad (8)$$

$$= Var[y_{l-1}^a] E^2[\sigma(\mathbf{y}_{l-1}^b)] + \quad (9)$$

$$Var[y_{l-1}^a] Var[\sigma(\mathbf{y}_{l-1}^b)] \\ = \frac{1}{4} Var[y_{l-1}^a] + Var[y_{l-1}^a] Var[\sigma(\mathbf{y}_{l-1}^b)] \quad (10)$$

and $Var[y_{l-1}^a] Var[\sigma(\mathbf{y}_{l-1}^b)] > 0$. We utilize the relation $\sigma(x)^2 \leq (1/16)x^2 - 1/4 + \sigma(x)$ ([Appendix B](#)) to provide an upper bound on $E[\sigma(x)^2]$:

$$E[\sigma(x)^2] \leq E\left[\frac{1}{16}x^2 - \frac{1}{4} + \sigma(x)\right] \quad (11)$$

$$= \frac{1}{16} E[x^2] - \frac{1}{4} + E[\sigma(x)] \quad (12)$$

With $x \sim \mathcal{N}(0, std(x))$, this yields

$$E[\sigma(x)^2] \leq \frac{1}{16} E[x^2] - \frac{1}{4} + \frac{1}{2} \quad (13)$$

$$= \frac{1}{16} Var[x] + \frac{1}{4}. \quad (14)$$

With (7) and $Var[y_{l-1}^a] = Var[y_{l-1}^b] = Var[y_{l-1}]$, this results in

$$Var[x_l] \leq \frac{1}{16} Var[y_{l-1}]^2 + \frac{1}{4} Var[y_{l-1}]. \quad (15)$$

We initialize the embedding matrices in our network with small variances (around 0.01), which allows us to dismiss the quadratic term and approximate the GLU output variance with

$$Var[x_l] \approx \frac{1}{4} Var[y_{l-1}]. \quad (16)$$

If L network layers of equal size and with GLU activations are combined, the variance of the final output $\mathbf{y}_L$ is given by

$$Var[y_L] \approx Var[y_1] \left(\prod_{l=2}^L \frac{1}{4} n_l Var[w_l] \right). \quad (17)$$

Following [\(He et al., 2015b\)](#), we aim to satisfy the condition

$$\frac{1}{4} n_l Var[w_l] = 1, \forall l \quad (18)$$

so that the activations in a network are neither exponentially magnified nor reduced. This is achieved by initializing W_l from $\mathcal{N}(0, \sqrt{4/n_l})$.

A.2. Backward Pass

The gradient of a convolutional layer is computed via back-propagation as $\Delta \mathbf{x}_l = \hat{W}_l \mathbf{y}_l$. Considering separate gradients $\Delta \mathbf{y}_l^a$ and $\Delta \mathbf{y}_l^b$ for GLU, the gradient of $\mathbf{x}$ is given by

$$\Delta \mathbf{x}_l = \hat{W}_l^a \Delta \mathbf{y}_l^a + \hat{W}_l^b \Delta \mathbf{y}_l^b. \quad (19)$$

$\hat{W}$ corresponds to W with re-arranged weights to enable back-propagation. Analogously to the forward pass, Δx_l , $\hat{w}_l$ and Δy_l represent the random variables for the values in $\Delta \mathbf{x}_l$, $\hat{W}_l$ and $\Delta \mathbf{y}_l$, respectively. Note that W and $\hat{W}$ contain the same values, i.e. $\hat{w} = w$. Similar to (3), the variance of Δx_l is

$$Var[\Delta x_l] = \hat{n}_l \left(Var[w_l^a] Var[\Delta y_l^a] + Var[w_l^b] Var[\Delta y_l^b] \right). \quad (20)$$

Here, $\hat{n}_l$ is the number of inputs to layer $l+1$. The gradients for the GLU inputs are:

$$\Delta \mathbf{y}_l^a = \Delta \mathbf{x}_{l+1} \sigma(\mathbf{y}_l^b) \quad \text{and} \quad (21)$$

$$\Delta \mathbf{y}_l^b = \Delta \mathbf{x}_{l+1} \mathbf{y}_l^a \sigma'(\mathbf{y}_l^b). \quad (22)$$

The approximation for the forward pass can be used for $Var[\Delta y_l^a]$, and for estimating $Var[\Delta y_l^b]$ we assume an upper bound on $E[\sigma'(y_l^b)^2]$ of $1/16$ since $\sigma'(y_l^b) \in [0, \frac{1}{4}]$. Hence,

$$Var[\Delta y_l^a] - \frac{1}{4}Var[\Delta x_{l+1}] \leq \frac{1}{16}Var[\Delta x_{l+1}]Var[y_l^b] \quad (23)$$

$$Var[\Delta y_l^b] \leq \frac{1}{16}\Delta Var[\Delta x_{l+1}]Var[y_l^a] \quad (24)$$

We observe relatively small gradients in our network, typically around 0.001 at the start of training. Therefore, we approximate by discarding the quadratic terms above, i.e.

$$Var[\Delta y_l^a] \approx \frac{1}{4}Var[\Delta x_{l+1}] \quad (25)$$

$$Var[\Delta y_l^b] \approx 0 \quad (26)$$

$$Var[\Delta x_l] \approx \frac{1}{4}\hat{n}_l Var[w_l^a]Var[\Delta x_{l+1}] \quad (27)$$

As for the forward pass, the above result can be generalized to backpropagation through many successive layers, resulting in

$$Var[\Delta x_2] \approx Var[\Delta x_{L+1}] \left(\prod_{l=2}^L \frac{1}{4}\hat{n}_l Var[w_l^a] \right) \quad (28)$$

and a similar condition, i.e. $(1/4)\hat{n}_l Var[w_l^a] = 1$. In the networks we consider, successions of convolutional layers usually operate on the same number of inputs so that most cases $n_l = \hat{n}_l$. Note that W_l^b is discarded in the approximation; however, for the sake of consistency we use the same initialization for W_l^a and W_l^b .

For arbitrarily large variances of network inputs and activations, our approximations are invalid; in that case, the initial values for W_l^a and W_l^b would have to be balanced for the input distribution to be retained. Alternatively, methods that explicitly control the variance in the network, e.g. batch normalization (Ioffe & Szegedy, 2015) or layer normalization (Ba et al., 2016) could be employed.

A.3. Dropout

Dropout retains activations in a neural network with a probability p and sets them to zero otherwise (Srivastava et al., 2014). It is common practice to scale the retained activations by $1/p$ during training so that the weights of the network do not have to be modified at test time when p is set to 1. In this case, dropout amounts to multiplying activations x by a Bernoulli random variable r where $\Pr[r = 1/p] = p$ and $\Pr[r = 0] = 1 - p$ (Srivastava et al., 2014). It holds that $E[r] = 1$ and $Var[r] = (1 - p)/p$. If x is independent

of r and $E[x] = 0$, the variance after dropout is

$$Var[xr] = E[r]^2 Var[x] + Var[r]Var[x] \quad (29)$$

$$= \left(1 + \frac{1-p}{p}\right) Var[x] \quad (30)$$

$$= \frac{1}{p} Var[x] \quad (31)$$

Assuming that the *input* of a convolutional layer has been subject to dropout with a retain probability p , the variations of the forward and backward activations from §A.1 and §A.2 can now be approximated with

$$Var[x_{l+1}] \approx \frac{1}{4p}n_l Var[w_l]Var[x_l] \quad \text{and} \quad (32)$$

$$Var[\Delta x_l] \approx \frac{1}{4p}n_l Var[w_l^a]Var[\Delta x_{l+1}]. \quad (33)$$

This amounts to a modified initialization of W_l from a normal distribution with zero mean and a standard deviation of $\sqrt{4p/n}$. For layers without a succeeding GLU activation function, we initialize weights from $\mathcal{N}(0, \sqrt{p/n})$ to calibrate for any immediately preceding dropout application.

B. Upper Bound on Squared Sigmoid

The sigmoid function $\sigma(x)$ can be expressed as a hyperbolic tangent by using the identity $\tanh(x) = 2\sigma(2x) - 1$. The derivative of $\tanh$ is $\tanh'(x) = 1 - \tanh^2(x)$, and with $\tanh(x) \in [0, 1]$, $x \geq 0$ it holds that

$$\tanh'(x) \leq 1, x \geq 0 \quad (34)$$

$$\int_0^x \tanh'(x) dx \leq \int_0^x 1 dx \quad (35)$$

$$\tanh(x) \leq x, x \geq 0 \quad (36)$$

We can express this relation with $\sigma(x)$ as follows:

$$2\sigma(x) - 1 \leq \frac{1}{2}x, x \geq 0 \quad (37)$$

Both terms of this inequality have rotational symmetry w.r.t 0, and thus

$$(2\sigma(x) - 1)^2 \leq \left(\frac{1}{2}x\right)^2 \quad \forall x \quad (38)$$

$$\Leftrightarrow \sigma(x)^2 \leq \frac{1}{16}x^2 - \frac{1}{4} + \sigma(x). \quad (39)$$

C. Attention Visualization

Figure 3 shows attention scores for a generated sentence from the WMT'14 English-German task. The model used for this plot has 8 decoder layers and a 80K BPE vocabulary. The attention passes in different decoder layers capture different portions of the source sentence. Layer 1, 3

and 6 exhibit a linear alignment. The first layer shows the clearest alignment, although it is slightly off and frequently attends to the corresponding source word of the previously generated target word. Layer 2 and 8 lack a clear structure and are presumably collecting information about the whole source sentence. The fourth layer shows high alignment scores on nouns such as “festival”, “way” and “work” for both the generated target nouns as well as their preceding words. Note that in German, those preceding words depend on gender and object relationship of the respective noun. Finally, the attention scores in layer 5 and 7 focus on “built”, which is reordered in the German translation and is moved from the beginning to the very end of the sentence. One interpretation for this is that as generation progresses, the model repeatedly tries to perform the re-ordering. “aufgebaut” can be generated after a noun or pronoun only, which is reflected in the higher scores at positions 2, 5, 8, 11 and 13.

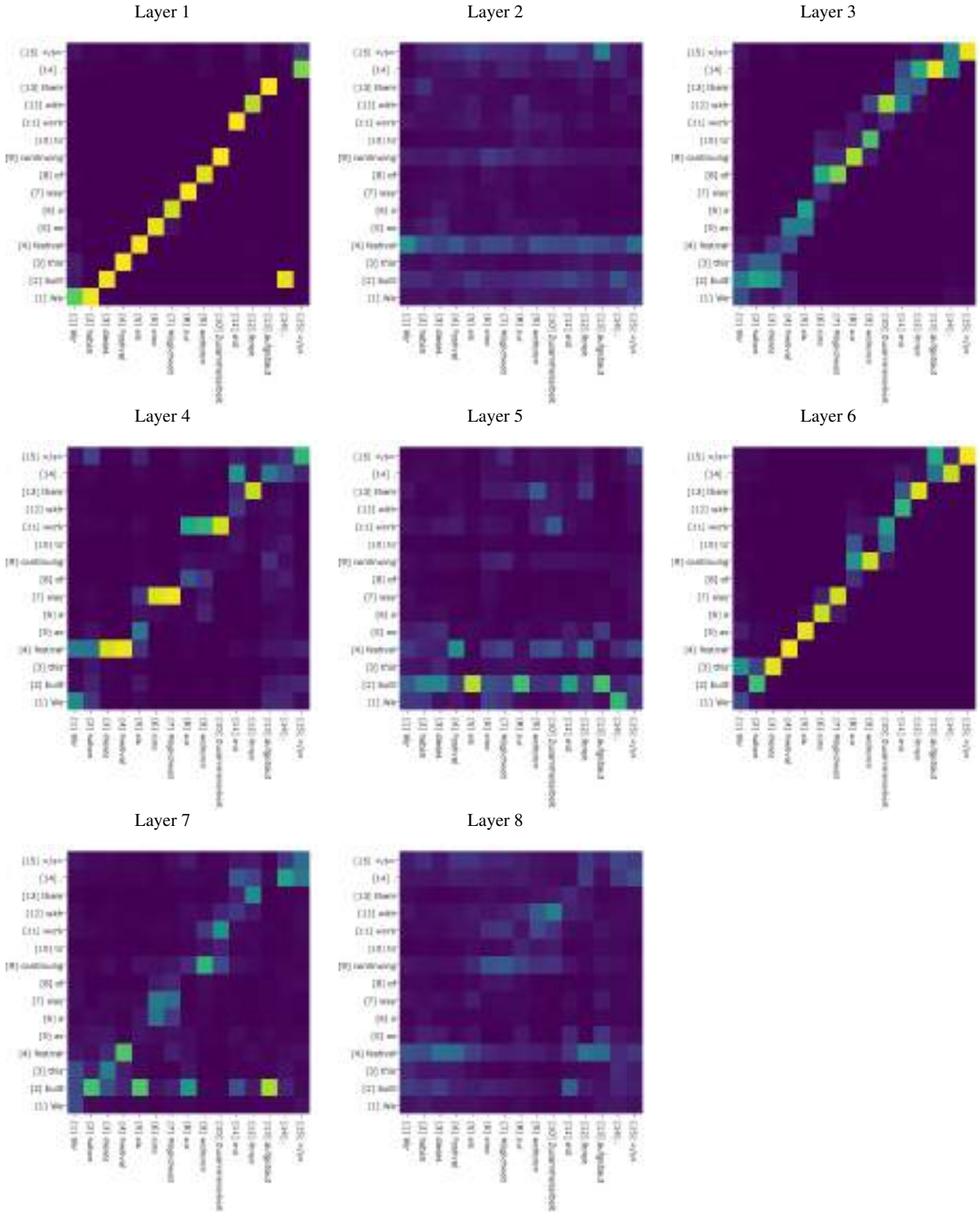


Figure 3. Attention scores for different decoder layers for a sentence translated from English (y-axis) to German (x-axis). This model uses 8 decoder layers and a 80k BPE vocabulary.

WAVENET: A GENERATIVE MODEL FOR RAW AUDIO

Aäron van den Oord

Sander Dieleman

Heiga Zen[†]

Karen Simonyan

Oriol Vinyals

Alex Graves

Nal Kalchbrenner

Andrew Senior

Koray Kavukcuoglu

{avdnoord, sedielem, heigazen, simonyan, vinyals, graves, nalk, andrewsenior, korayk}@google.com
Google DeepMind, London, UK

[†] Google, London, UK

ABSTRACT

This paper introduces WaveNet, a deep neural network for generating raw audio waveforms. The model is fully probabilistic and autoregressive, with the predictive distribution for each audio sample conditioned on all previous ones; nonetheless we show that it can be efficiently trained on data with tens of thousands of samples per second of audio. When applied to text-to-speech, it yields state-of-the-art performance, with human listeners rating it as significantly more natural sounding than the best parametric and concatenative systems for both English and Mandarin. A single WaveNet can capture the characteristics of many different speakers with equal fidelity, and can switch between them by conditioning on the speaker identity. When trained to model music, we find that it generates novel and often highly realistic musical fragments. We also show that it can be employed as a discriminative model, returning promising results for phoneme recognition.

1 INTRODUCTION

This work explores raw audio generation techniques, inspired by recent advances in neural autoregressive generative models that model complex distributions such as images (van den Oord et al., 2016a,b) and text (Jósefowicz et al., 2016). Modeling joint probabilities over pixels or words using neural architectures as products of conditional distributions yields state-of-the-art generation

Remarkably, these architectures are able to model distributions over thousands of random variables (e.g. 64×64 pixels as in PixelRNN (van den Oord et al., 2016a)). The question this paper addresses is whether similar approaches can succeed in generating wideband raw audio waveforms, which are signals with very high temporal resolution, at least 16,000 samples per second (see Fig. 1).



Figure 1: A second of generated speech.

This paper introduces *WaveNet*, an audio generative model based on the PixelCNN (van den Oord et al., 2016a,b) architecture. The main contributions of this work are as follows:

- We show that WaveNets can generate raw speech signals with subjective naturalness never before reported in the field of text-to-speech (TTS), as assessed by human raters.

- In order to deal with long-range temporal dependencies needed for raw audio generation, we develop new architectures based on dilated causal convolutions, which exhibit very large receptive fields.
- We show that when conditioned on a speaker identity, a single model can be used to generate different voices.
- The same architecture shows strong results when tested on a small speech recognition dataset, and is promising when used to generate other audio modalities such as music.

We believe that WaveNets provide a generic and flexible framework for tackling many applications that rely on audio generation (e.g. TTS, music, speech enhancement, voice conversion, source separation).

2 WAVE.NET

In this paper we introduce a new generative model operating directly on the raw audio waveform. The joint probability of a waveform $\mathbf{x} = \{x_1, \dots, x_T\}$ is factorised as a product of conditional probabilities as follows:

$$p(\mathbf{x}) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}) \quad (1)$$

Each audio sample x_t is therefore conditioned on the samples at all previous timesteps.

Similarly to PixelCNNs (van den Oord et al., 2016a,b), the conditional probability distribution is modelled by a stack of convolutional layers. There are no pooling layers in the network, and the output of the model has the same time dimensionality as the input. The model outputs a categorical distribution over the next value x_t with a softmax layer and it is optimized to maximize the log-likelihood of the data w.r.t. the parameters. Because log-likelihoods are tractable, we tune hyper-parameters on a validation set and can easily measure if the model is overfitting or underfitting.

2.1 DILATED CAUSAL CONVOLUTIONS

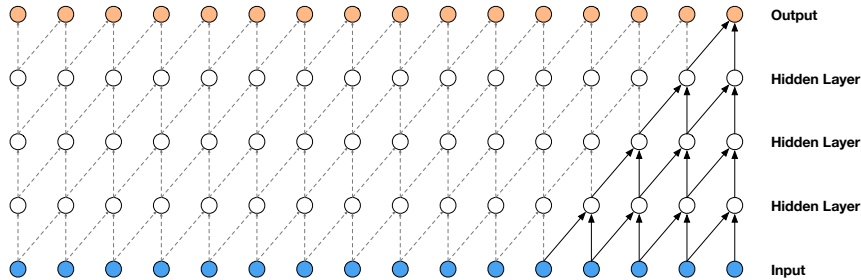


Figure 2: Visualization of a stack of causal convolutional layers.

The main ingredient of WaveNet are causal convolutions. By using causal convolutions, we make sure the model cannot violate the ordering in which we model the data: the prediction $p(x_{t+1} | x_1, \dots, x_t)$ emitted by the model at timestep t cannot depend on any of the future timesteps $x_{t+1}, x_{t+2}, \dots, x_T$ as shown in Fig. 2. For images, the equivalent of a causal convolution is a masked convolution (van den Oord et al., 2016a) which can be implemented by constructing a mask tensor and doing an elementwise multiplication of this mask with the convolution kernel before applying it. For 1-D data such as audio one can more easily implement this by shifting the output of a normal convolution by a few timesteps.

At training time, the conditional predictions for all timesteps can be made in parallel because all timesteps of ground truth $\mathbf{x}$ are known. When generating with the model, the predictions are sequential: after each sample is predicted, it is fed back into the network to predict the next sample.

Because models with causal convolutions do not have recurrent connections, they are typically faster to train than RNNs, especially when applied to very long sequences. One of the problems of causal convolutions is that they require many layers, or large filters to increase the receptive field. For example, in Fig. 2 the receptive field is only 5 ($= \text{\#layers} + \text{filter length} - 1$). In this paper we use dilated convolutions to increase the receptive field by orders of magnitude, without greatly increasing computational cost.

A dilated convolution (also called *à trous*, or convolution with holes) is a convolution where the filter is applied over an area larger than its length by skipping input values with a certain step. It is equivalent to a convolution with a larger filter derived from the original filter by dilating it with zeros, but is significantly more efficient. A dilated convolution effectively allows the network to operate on a coarser scale than with a normal convolution. This is similar to pooling or strided convolutions, but here the output has the same size as the input. As a special case, dilated convolution with dilation 1 yields the standard convolution. Fig. 3 depicts dilated causal convolutions for dilations 1, 2, 4, and 8. Dilated convolutions have previously been used in various contexts, e.g. signal processing (Holschneider et al., 1989; Dutilleul, 1989), and image segmentation (Chen et al., 2015; Yu & Koltun, 2016).

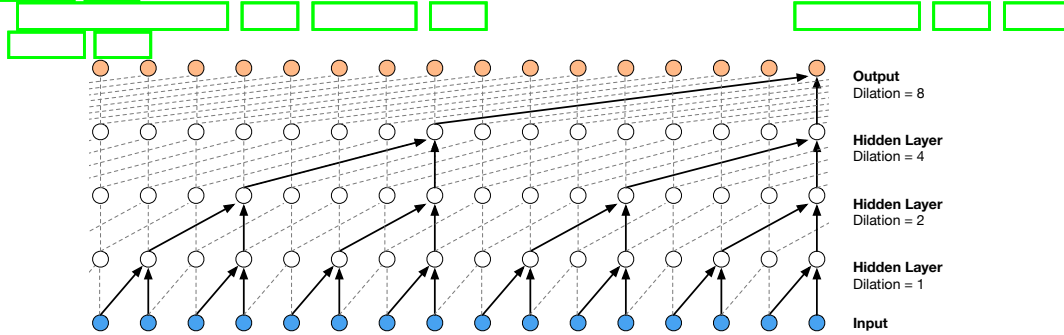


Figure 3: Visualization of a stack of *dilated* causal convolutional layers.

Stacked dilated convolutions enable networks to have very large receptive fields with just a few layers, while preserving the input resolution throughout the network as well as computational efficiency. In this paper, the dilation is doubled for every layer up to a limit and then repeated: e.g.

$$1, 2, 4, \dots, 512, 1, 2, 4, \dots, 512, 1, 2, 4, \dots, 512.$$

The intuition behind this configuration is two-fold. First, exponentially increasing the dilation factor results in exponential receptive field growth with depth (Yu & Koltun, 2016). For example each $1, 2, 4, \dots, 512$ block has receptive field of size 1024, and can be seen as a more efficient and discriminative (non-linear) counterpart of a 1×1024 convolution. Second, stacking these blocks further increases the model capacity and the receptive field size.

2.2 SOFTMAX DISTRIBUTIONS

One approach to modeling the conditional distributions $p(x_t | x_1, \dots, x_{t-1})$ over the individual audio samples would be to use a mixture model such as a mixture density network (Bishop, 1994) or mixture of conditional Gaussian scale mixtures (MCGSM) (Theis & Bethge, 2015). However, van den Oord et al. (2016a) showed that a softmax distribution tends to work better, even when the data is implicitly continuous (as is the case for image pixel intensities or audio sample values). One of the reasons is that a categorical distribution is more flexible and can more easily model arbitrary distributions because it makes no assumptions about their shape.

Because raw audio is typically stored as a sequence of 16-bit integer values (one per timestep), a softmax layer would need to output 65,536 probabilities per timestep to model all possible values. To make this more tractable, we first apply a μ -law companding transformation (ITU-T, 1988) to the data, and then quantize it to 256 possible values:

$$f(x_t) = \text{sign}(x_t) \frac{\ln(1 + \mu |x_t|)}{\ln(1 + \mu)},$$

where $-1 < x_t < 1$ and $\mu = 255$. This non-linear quantization produces a significantly better reconstruction than a simple linear quantization scheme. Especially for speech, we found that the reconstructed signal after quantization sounded very similar to the original.

2.3 GATED ACTIVATION UNITS

We use the same gated activation unit as used in the gated PixelCNN (van den Oord et al., 2016b):

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x}) \odot \sigma(W_{g,k} * \mathbf{x}), \quad (2)$$

where $*$ denotes a convolution operator, $\odot$ denotes an element-wise multiplication operator, $\sigma(\cdot)$ is a sigmoid function, k is the layer index, f and g denote filter and gate, respectively, and W is a learnable convolution filter. In our initial experiments, we observed that this non-linearity worked significantly better than the rectified linear activation function (Nair & Hinton, 2010) for modeling audio signals.

2.4 RESIDUAL AND SKIP CONNECTIONS

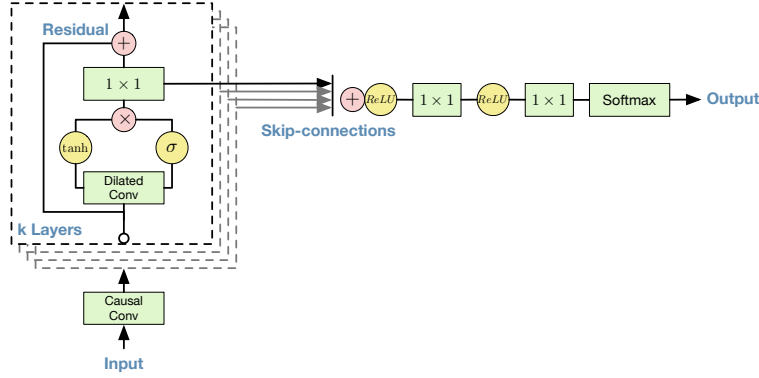


Figure 4: Overview of the residual block and the entire architecture.

Both residual (He et al., 2015) and parameterised skip connections are used throughout the network, to speed up convergence and enable training of much deeper models. In Fig. 4 we show a residual block of our model, which is stacked many times in the network.

2.5 CONDITIONAL WAVENETS

Given an additional input $\mathbf{h}$, WaveNets can model the conditional distribution $p(\mathbf{x} | \mathbf{h})$ of the audio given this input. Eq. (1) now becomes

$$p(\mathbf{x} | \mathbf{h}) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}, \mathbf{h}). \quad (3)$$

By conditioning the model on other input variables, we can guide WaveNet’s generation to produce audio with the required characteristics. For example, in a multi-speaker setting we can choose the speaker by feeding the speaker identity to the model as an extra input. Similarly, for TTS we need to feed information about the text as an extra input.

We condition the model on other inputs in two different ways: global conditioning and local conditioning. Global conditioning is characterised by a single latent representation $\mathbf{h}$ that influences the output distribution across all timesteps, e.g. a speaker embedding in a TTS model. The activation function from Eq. (2) now becomes:

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x} + V_{f,k}^T \mathbf{h}) \odot \sigma(W_{g,k} * \mathbf{x} + V_{g,k}^T \mathbf{h}).$$

where $V_{*,k}$ is a learnable linear projection, and the vector $V_{*,k}^T \mathbf{h}$ is broadcast over the time dimension.

For local conditioning we have a second timeseries h_t , possibly with a lower sampling frequency than the audio signal, e.g. linguistic features in a TTS model. We first transform this time series using a transposed convolutional network (learned upsampling) that maps it to a new time series $\mathbf{y} = f(\mathbf{h})$ with the same resolution as the audio signal, which is then used in the activation unit as follows:

$$\mathbf{z} = \tanh(W_{f,k} * \mathbf{x} + V_{f,k} * \mathbf{y}) \odot \sigma(W_{g,k} * \mathbf{x} + V_{g,k} * \mathbf{y}),$$

where $V_{f,k} * \mathbf{y}$ is now a 1×1 convolution. As an alternative to the transposed convolutional network, it is also possible to use $V_{f,k} * \mathbf{h}$ and repeat these values across time. We saw that this worked slightly worse in our experiments.

2.6 CONTEXT STACKS

We have already mentioned several different ways to increase the receptive field size of a WaveNet: increasing the number of dilation stages, using more layers, larger filters, greater dilation factors, or a combination thereof. A complementary approach is to use a separate, smaller *context* stack that processes a long part of the audio signal and locally conditions a larger WaveNet that processes only a smaller part of the audio signal (cropped at the end). One can use multiple context stacks with varying lengths and numbers of hidden units. Stacks with larger receptive fields have fewer units per layer. Context stacks can also have pooling layers to run at a lower frequency. This keeps the computational requirements at a reasonable level and is consistent with the intuition that less capacity is required to model temporal correlations at longer timescales.

3 EXPERIMENTS

To measure WaveNet’s audio modelling performance, we evaluate it on three different tasks: multi-speaker speech generation (not conditioned on text), TTS, and music audio modelling. We provide samples drawn from WaveNet for these experiments on the accompanying webpage:

<https://www.deepmind.com/blog/wavenet-generative-model-raw-audio/>.

3.1 MULTI-SPEAKER SPEECH GENERATION

For the first experiment we looked at free-form speech generation (not conditioned on text). We used the English multi-speaker corpus from CSTR voice cloning toolkit (VCTK) (Yamagishi, 2012) and conditioned WaveNet only on the speaker. The conditioning was applied by feeding the speaker ID to the model in the form of a one-hot vector. The dataset consisted of 44 hours of data from 109 different speakers.

Because the model is not conditioned on text, it generates non-existent but human language-like words in a smooth way with realistic sounding intonations. This is similar to generative models of language or images, where samples look realistic at first glance, but are clearly unnatural upon closer inspection. The lack of long range coherence is partly due to the limited size of the model’s receptive field (about 300 milliseconds), which means it can only remember the last 2–3 phonemes it produced.

A single WaveNet was able to model speech from any of the speakers by conditioning it on a one-hot encoding of a speaker. This confirms that it is powerful enough to capture the characteristics of all 109 speakers from the dataset in a single model. We observed that adding speakers resulted in better validation set performance compared to training solely on a single speaker. This suggests that WaveNet’s internal representation was shared among multiple speakers.

Finally, we observed that the model also picked up on other characteristics in the audio apart from the voice itself. For instance, it also mimicked the acoustics and recording quality, as well as the breathing and mouth movements of the speakers.

3.2 TEXT-TO-SPEECH

For the second experiment we looked at TTS. We used the same single-speaker speech databases from which Google’s North American English and Mandarin Chinese TTS systems are built. The North American English dataset contains 24.6 hours of speech data, and the Mandarin Chinese dataset contains 34.8 hours; both were spoken by professional female speakers.

WaveNets for the TTS task were locally conditioned on *linguistic features* which were derived from input texts. We also trained WaveNets conditioned on the logarithmic fundamental frequency ($\log F_0$) values in addition to the linguistic features. External models predicting $\log F_0$ values and phone durations from linguistic features were also trained for each language. The receptive field size of the WaveNets was 240 milliseconds. As example-based and model-based speech synthesis baselines, hidden Markov model (HMM)-driven unit selection concatenative (Gonzalvo et al., 2016) and long short-term memory recurrent neural network (LSTM-RNN)-based statistical parametric (Zen et al., 2016) speech synthesizers were built. Since the same datasets and linguistic features were used to train both the baselines and WaveNets, these speech synthesizers could be fairly compared.

To evaluate the performance of WaveNets for the TTS task, subjective paired comparison tests and mean opinion score (MOS) tests were conducted. In the paired comparison tests, after listening to each pair of samples, the subjects were asked to choose which they preferred, though they could choose “neutral” if they did not have any preference. In the MOS tests, after listening to each stimulus, the subjects were asked to rate the naturalness of the stimulus in a five-point Likert scale score (1: Bad, 2: Poor, 3: Fair, 4: Good, 5: Excellent). Please refer to Appendix B for details.

Fig. 5 shows a selection of the subjective paired comparison test results (see Appendix B for the complete table). It can be seen from the results that WaveNet outperformed the baseline statistical parametric and concatenative speech synthesizers in both languages. We found that WaveNet conditioned on linguistic features could synthesize speech samples with natural segmental quality but sometimes it had unnatural prosody by stressing wrong words in a sentence. This could be due to the long-term dependency of F_0 contours: the size of the receptive field of the WaveNet, 240 milliseconds, was not long enough to capture such long-term dependency. WaveNet conditioned on both linguistic features and F_0 values did not have this problem: the external F_0 prediction model runs at a lower frequency (200 Hz) so it can learn long-range dependencies that exist in F_0 contours.

Table 1 show the MOS test results. It can be seen from the table that WaveNets achieved 5-scale MOSs in naturalness above 4.0, which were significantly better than those from the baseline systems. They were the highest ever reported MOS values with these training datasets and test sentences. The gap in the MOSs from the best synthetic speech to the natural ones decreased from 0.69 to 0.34 (51%) in US English and 0.42 to 0.13 (69%) in Mandarin Chinese.

Speech samples	Subjective 5-scale MOS in naturalness	
	North American English	Mandarin Chinese
LSTM-RNN parametric	3.67 ± 0.098	3.79 ± 0.084
HMM-driven concatenative	3.86 ± 0.137	3.47 ± 0.108
WaveNet (L+F)	4.21 ± 0.081	4.08 ± 0.085
Natural (8-bit μ -law)	4.46 ± 0.067	4.25 ± 0.082
Natural (16-bit linear PCM)	4.55 ± 0.075	4.21 ± 0.071

Table 1: Subjective 5-scale mean opinion scores of speech samples from LSTM-RNN-based statistical parametric, HMM-driven unit selection concatenative, and proposed WaveNet-based speech synthesizers, 8-bit μ -law encoded natural speech, and 16-bit linear pulse-code modulation (PCM) natural speech. WaveNet improved the previous state of the art significantly, reducing the gap between natural speech and best previous model by more than 50%.

3.3 MUSIC

For our third set of experiments we trained WaveNets to model two music datasets:

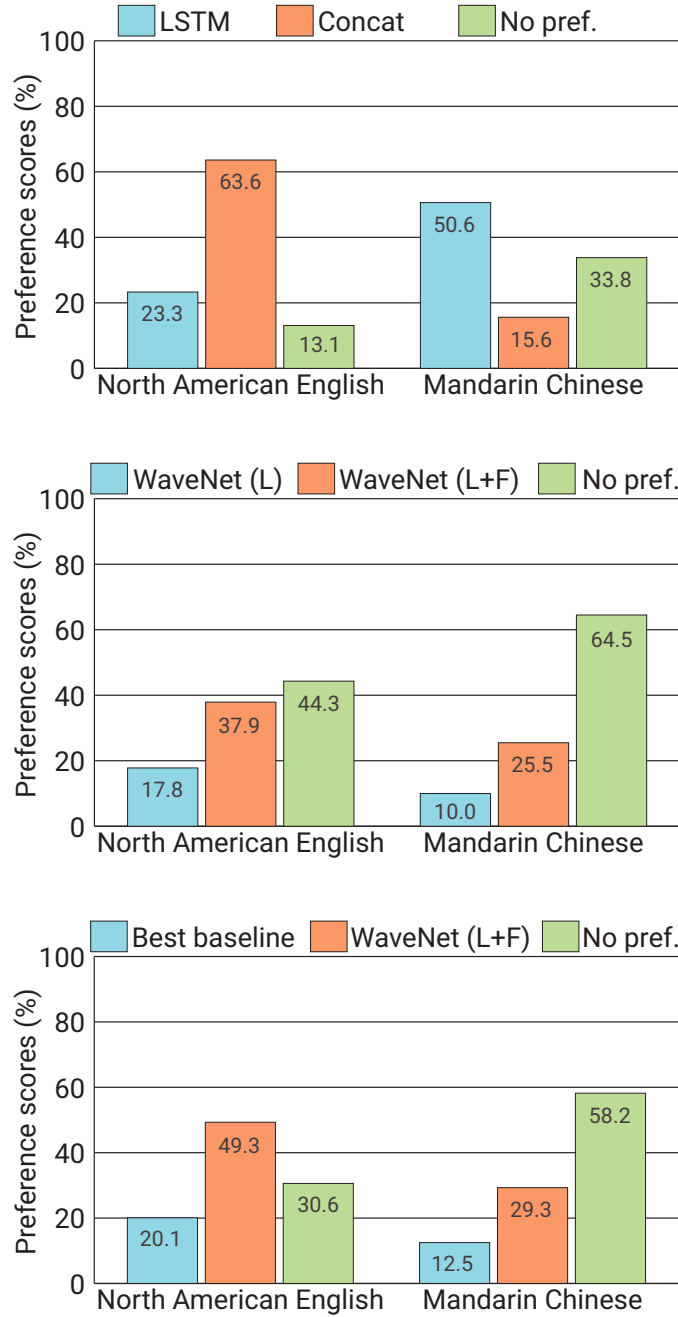


Figure 5: Subjective preference scores (%) of speech samples between (top) two baselines, (middle) two WaveNets, and (bottom) the best baseline and WaveNet. Note that LSTM and Concat correspond to LSTM-RNN-based statistical parametric and HMM-driven unit selection concatenative baseline synthesizers, and WaveNet (L) and WaveNet (L+F) correspond to the WaveNet conditioned on linguistic features only and that conditioned on both linguistic features and $\log F_0$ values.

- the MagnaTagATune dataset (Law & Von Ahn, 2009), which consists of about 200 hours of music audio. Each 29-second clip is annotated with tags from a set of 188, which describe the genre, instrumentation, tempo, volume and mood of the music.
- the YouTube piano dataset, which consists of about 60 hours of solo piano music obtained from YouTube videos. Because it is constrained to a single instrument, it is considerably easier to model.

Although it is difficult to quantitatively evaluate these models, a subjective evaluation is possible by listening to the samples they produce. We found that enlarging the receptive field was crucial to obtain samples that sounded musical. Even with a receptive field of several seconds, the models did not enforce long-range consistency which resulted in second-to-second variations in genre, instrumentation, volume and sound quality. Nevertheless, the samples were often harmonic and aesthetically pleasing, even when produced by unconditional models.

Of particular interest are conditional music models, which can generate music given a set of tags specifying e.g. genre or instruments. Similarly to conditional speech models, we insert biases that depend on a binary vector representation of the tags associated with each training clip. This makes it possible to control various aspects of the output of the model when sampling, by feeding in a binary vector that encodes the desired properties of the samples. We have trained such models on the MagnaTagATune dataset; although the tag data bundled with the dataset was relatively noisy and had many omissions, after cleaning it up by merging similar tags and removing those with too few associated clips, we found this approach to work reasonably well.

3.4 SPEECH RECOGNITION

Although WaveNet was designed as a generative model, it can straightforwardly be adapted to discriminative audio tasks such as speech recognition.

Traditionally, speech recognition research has largely focused on using log mel-filterbank energies or mel-frequency cepstral coefficients (MFCCs), but has been moving to raw audio recently (Palaz et al., 2013; Tüske et al., 2014; Hoshen et al., 2015; Sainath et al., 2015). Recurrent neural networks such as LSTM-RNNs (Hochreiter & Schmidhuber, 1997) have been a key component in these new speech classification pipelines, because they allow for building models with long range contexts. With WaveNets we have shown that layers of dilated convolutions allow the receptive field to grow longer in a much cheaper way than using LSTM units.

As a last experiment we looked at speech recognition with WaveNets on the TIMIT (Garofolo et al., 1993) dataset. For this task we added a mean-pooling layer after the dilated convolutions that aggregated the activations to coarser frames spanning 10 milliseconds ($160\times$ downsampling). The pooling layer was followed by a few non-causal convolutions. We trained WaveNet with two loss terms, one to predict the next sample and one to classify the frame, the model generalized better than with a single loss and achieved 18.8 PER on the test set, which is to our knowledge the best score obtained from a model trained directly on raw audio on TIMIT.

4 CONCLUSION

This paper has presented WaveNet, a deep generative model of audio data that operates directly at the waveform level. WaveNets are autoregressive and combine causal filters with dilated convolutions to allow their receptive fields to grow exponentially with depth, which is important to model the long-range temporal dependencies in audio signals. We have shown how WaveNets can be conditioned on other inputs in a global (e.g. speaker identity) or local way (e.g. linguistic features). When applied to TTS, WaveNets produced samples that outperform the current best TTS systems in subjective naturalness. Finally, WaveNets showed very promising results when applied to music audio modeling and speech recognition.

ACKNOWLEDGEMENTS

The authors would like to thank Lasse Espeholt, Jeffrey De Fauw and Grzegorz Swirszcz for their inputs, Adam Cain, Max Cant and Adrian Bolton for their help with artwork, Helen King, Steven

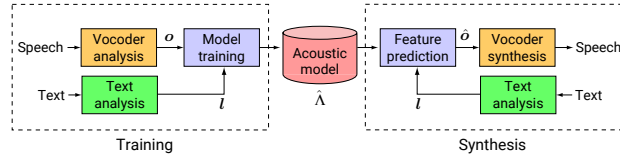


Figure 6: Outline of statistical parametric speech synthesis.

Zen, Heiga, Tokuda, Keiichi, and Kitamura, Tadashi. Reformulating the HMM as a trajectory model by imposing explicit relationships between static and dynamic features. *Comput. Speech Lang.*, 21(1):153–173, 2007.

Zen, Heiga, Tokuda, Keiichi, and Black, Alan W. Statistical parametric speech synthesis. *Speech Commn.*, 51(11):1039–1064, 2009.

Zen, Heiga, Senior, Andrew, and Schuster, Mike. Statistical parametric speech synthesis using deep neural networks. In *Proc. ICASSP*, pp. 7962–7966, 2013.

Zen, Heiga, Agiomyrghiannakis, Yannis, Egberts, Niels, Henderson, Fergus, and Szczepaniak, Przemysław. Fast, compact, and high quality LSTM-RNN based statistical parametric speech synthesizers for mobile devices. In *Interspeech*, 2016. URL <https://arxiv.org/abs/1606.06061>.

A TEXT-TO-SPEECH BACKGROUND

The goal of TTS synthesis is to render naturally sounding speech signals given a text to be synthesized. Human speech production process first translates a text (or concept) into movements of muscles associated with articulators and speech production-related organs. Then using air-flow from lung, vocal source excitation signals, which contain both periodic (by vocal cord vibration) and aperiodic (by turbulent noise) components, are generated. By filtering the vocal source excitation signals by time-varying vocal tract transfer functions controlled by the articulators, their frequency characteristics are modulated. Finally, the generated speech signals are emitted. The aim of TTS is to mimic this process by computers in some way.

TTS can be viewed as a sequence-to-sequence mapping problem; from a sequence of discrete symbols (text) to a real-valued time series (speech signals). A typical TTS pipeline has two parts; 1) text analysis and 2) speech synthesis. The text analysis part typically includes a number of natural language processing (NLP) steps, such as sentence segmentation, word segmentation, text normalization, part-of-speech (POS) tagging, and grapheme-to-phoneme (G2P) conversion. It takes a word sequence as input and outputs a phoneme sequence with a variety of linguistic contexts. The speech synthesis part takes the context-dependent phoneme sequence as its input and outputs a synthesized speech waveform. This part typically includes prosody prediction and speech waveform generation.

There are two main approaches to realize the speech synthesis part; non-parametric, example-based approach known as concatenative speech synthesis (Moulines & Charpentier, 1990; Sagisaka et al., 1992; Hunt & Black, 1996), and parametric, model-based approach known as statistical parametric speech synthesis (Yoshimura, 2002; Zen et al., 2009). The concatenative approach builds up the utterance from units of recorded speech, whereas the statistical parametric approach uses a generative model to synthesize the speech. The statistical parametric approach first extracts a sequence of vocoder parameters (Dudley, 1939) $\mathbf{o} = \{\mathbf{o}_1, \dots, \mathbf{o}_N\}$ from speech signals $\mathbf{x} = \{x_1, \dots, x_T\}$ and linguistic features $\mathbf{l}$ from the text W , where N and T correspond to the numbers of vocoder parameter vectors and speech signals. Typically a vocoder parameter vector $\mathbf{o}_n$ is extracted at every 5 milliseconds. It often includes cepstra (Imai & Furuichi, 1988) or line spectral pairs (Itakura, 1975), which represent vocal tract transfer function, and fundamental frequency (F_0) and aperiodicity (Kawahara et al., 2001), which represent characteristics of vocal source excitation signals. Then a set of generative models, such as hidden Markov models (HMMs) (Yoshimura, 2002), feed-forward neural networks (Zen et al., 2013), and recurrent neural networks (Tuerk & Robinson, 1993; Karaali et al., 1997; Fan et al., 2014), is trained from the extracted vocoder parameters and linguistic features

as

$$\hat{\Lambda} = \arg \max_{\Lambda} p(\mathbf{o} | \mathbf{l}, \Lambda), \quad (4)$$

where Λ denotes the set of parameters of the generative model. At the synthesis stage, the most probable vocoder parameters are generated given linguistic features extracted from a text to be synthesized as

$$\hat{\mathbf{o}} = \arg \max_{\mathbf{o}} p(\mathbf{o} | \mathbf{l}, \hat{\Lambda}). \quad (5)$$

Then a speech waveform is reconstructed from $\hat{\mathbf{o}}$ using a vocoder. The statistical parametric approach offers various advantages over the concatenative one such as small footprint and flexibility to change its voice characteristics. However, its subjective naturalness is often significantly worse than that of the concatenative approach; synthesized speech often sounds muffled and has artifacts. Zen et al. (2009) reported three major factors that can degrade the subjective naturalness; quality of vocoders, accuracy of generative models, and effect of oversmoothing. The first factor causes the artifacts and the second and third factors lead to the muffleness in the synthesized speech. There have been a number of attempts to address these issues individually, such as developing high-quality vocoders (Kawahara et al., 1999; Agiomyrgiannakis, 2015; Morise et al., 2016), improving the accuracy of generative models (Zen et al., 2007, 2013; Fan et al., 2014; Uria et al., 2015), and compensating the oversmoothing effect (Toda & Tokuda, 2007; Takamichi et al., 2016). Zen et al. (2016) showed that state-of-the-art statistical parametric speech synthesizers matched state-of-the-art concatenative ones in some languages. However, its vocoded sound quality is still a major issue.

Extracting vocoder parameters can be viewed as estimation of a generative model parameters given speech signals (Itakura & Saito, 1970; Imai & Furuichi, 1988). For example, linear predictive analysis (Itakura & Saito, 1970), which has been used in speech coding, assumes that the generative model of speech signals is a linear auto-regressive (AR) zero-mean Gaussian process;

$$x_t = \sum_{p=1}^P a_p x_{t-p} + \epsilon_t \quad (6)$$

$$\epsilon_t \sim \mathcal{N}(0, G^2) \quad (7)$$

where a_p is a p -th order linear predictive coefficient (LPC) and G^2 is a variance of modeling error. These parameters are estimated based on the maximum likelihood (ML) criterion. In this sense, the training part of the statistical parametric approach can be viewed as a two-step optimization and sub-optimal: extract vocoder parameters by fitting a generative model of speech signals then model trajectories of the extracted vocoder parameters by a separate generative model for time series (Tokuda, 2011). There have been attempts to integrate these two steps into a single one (Toda & Tokuda, 2008; Wu & Tokuda, 2008; Maia et al., 2010; Nakamura et al., 2014; Muthukumar & Black, 2014; Tokuda & Zen, 2015, 2016; Takaki & Yamagishi, 2016). For example, Tokuda & Zen (2015) integrated non-stationary, nonzero-mean Gaussian process generative model of speech signals and LSTM-RNN-based sequence generative model to a single one and jointly optimized them by back-propagation. Although they showed that this model could approximate natural speech signals, its segmental naturalness was significantly worse than the non-integrated model due to over-generalization and over-estimation of noise components in speech signals.

The conventional generative models of raw audio signals have a number of assumptions which are inspired from the speech production, such as

- Use of fixed-length analysis window; They are typically based on a stationary stochastic process (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Kameoka et al., 2010). To model time-varying speech signals by a stationary stochastic process, parameters of these generative models are estimated within a fixed-length, overlapping and shifting analysis window (typically its length is 20 to 30 milliseconds, and shift is 5 to 10 milliseconds). However, some phones such as stops are time-limited by less than 20 milliseconds (Rabiner & Juang, 1993). Therefore, using such fixed-size analysis window has limitations.
- Linear filter; These generative models are typically realized as a linear time-invariant filter (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Kameoka et al., 2010) within a windowed frame. However, the relationship between successive audio samples can be highly non-linear.

- Gaussian process assumption; The conventional generative models are based on Gaussian process (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Kameoka et al., 2010; Tokuda & Zen, 2015, 2016). From the source-filter model of speech production (Chiba & Kajiyama, 1942; Fant, 1970) point of view, this is equivalent to assuming that a vocal source excitation signal is a sample from a Gaussian distribution (Itakura & Saito, 1970; Imai & Furuichi, 1988; Poritz, 1982; Juang & Rabiner, 1985; Tokuda & Zen, 2015; Kameoka et al., 2010; Tokuda & Zen, 2016). Together with the linear assumption above, it results in assuming that speech signals are normally distributed. However, distributions of real speech signals can be significantly different from Gaussian.

Although these assumptions are convenient, samples from these generative models tend to be noisy and lose important details to make these audio signals sounding natural.

WaveNet, which was described in Section 2, has none of the above-mentioned assumptions. It incorporates almost no prior knowledge about audio signals, except the choice of the receptive field and μ -law encoding of the signal. It can also be viewed as a non-linear causal filter for quantized signals. Although such non-linear filter can represent complicated signals while preserving the details, designing such filters is usually difficult (Peltonen et al., 2001). WaveNets give a way to train them from data.

B DETAILS OF TTS EXPERIMENT

The HMM-driven unit selection and WaveNet TTS systems were built from speech at 16 kHz sampling. Although LSTM-RNNs were trained from speech at 22.05 kHz sampling, speech at 16 kHz sampling was synthesized at runtime using a resampling functionality in the Vocode vocoder (Agiomyriannakis, 2015). Both the LSTM-RNN-based statistical parametric and HMM-driven unit selection speech synthesizers were built from the speech datasets in the 16-bit linear PCM, whereas the WaveNet-based ones were trained from the same speech datasets in the 8-bit μ -law encoding.

The linguistic features include phone, syllable, word, phrase, and utterance-level features (Zen, 2006) (e.g. phone identities, syllable stress, the number of syllables in a word, and position of the current syllable in a phrase) with additional frame position and phone duration features (Zen et al., 2013). These features were derived and associated with speech every 5 milliseconds by phone-level forced alignment at the training stage. We used LSTM-RNN-based phone duration and autoregressive GNN-based $\log F_0$ prediction models. They were trained so as to minimize the mean squared errors (MSE). It is important to note that no post-processing was applied to the audio signals generated from the WaveNets.

The subjective listening tests were blind and crowdsourced. 100 sentences not included in the training data were used for evaluation. Each subject could evaluate up to 8 and 63 stimuli for North American English and Mandarin Chinese, respectively. Test stimuli were randomly chosen and presented for each subject. In the paired comparison test, each pair of speech samples was the same text synthesized by the different models. In the MOS test, each stimulus was presented to subjects in isolation. Each pair was evaluated by eight subjects in the paired comparison test, and each stimulus was evaluated by eight subjects in the MOS test. The subjects were paid and native speakers performing the task. Those ratings (about 40%) where headphones were not used were excluded when computing the preference and mean opinion scores. Table 2 shows the full details of the paired comparison test shown in Fig. 5.

Language	Subjective preference (%) in naturalness					p value
	LSTM	Concat	WaveNet (L)	WaveNet (L+F)	No preference	
North American English	23.3	63.6			13.1	$\ll 10^{-9}$
	18.7		69.3		12.0	$\ll 10^{-9}$
	7.6			82.0	10.4	$\ll 10^{-9}$
		32.4	41.2		26.4	0.003
		20.1		49.3	30.6	$\ll 10^{-9}$
			17.8	37.9	44.3	$\ll 10^{-9}$
Mandarin Chinese	50.6	15.6			33.8	$\ll 10^{-9}$
	25.0		23.3		51.8	0.476
	12.5			29.3	58.2	$\ll 10^{-9}$
		17.6	43.1		39.3	$\ll 10^{-9}$
		7.6		55.9	36.5	$\ll 10^{-9}$
			10.0	25.5	64.5	$\ll 10^{-9}$

Table 2: Subjective preference scores of speech samples between LSTM-RNN-based statistical parametric (**LSTM**), HMM-driven unit selection concatenative (**Concat**), and proposed WaveNet-based speech synthesizers. Each row of the table denotes scores of a paired comparison test between two synthesizers. Scores of the synthesizers which were significantly better than their competing ones at $p < 0.01$ level were shown in the bold type. Note that **WaveNet (L)** and **WaveNet (L+F)** correspond to WaveNet conditioned on linguistic features only and that conditioned on both linguistic features and F_0 values.



LSTM-Based Architecture for Sequential Quantum Circuit Inputs

Proven Use of LSTMs in Sequential RL Tasks

LSTM networks (Long Short-Term Memory) are a **well-established choice** for RL tasks involving sequential, variable-length observations. In fact, DeepMind's **A3C** (Asynchronous Advantage Actor-Critic) paper showed that adding a 256-unit LSTM layer on top of a feedforward policy significantly improved performance on partially observed games ¹. Similarly, OpenAI's **Five** Dota2 agent relied on a single-layer **4096-unit LSTM** as the core of each bot's policy network, processing an extremely large state input (on the order of 20,000 features) and outputting a distribution over actions ². These successes demonstrate that LSTMs can effectively handle long 1D sequences with repeating patterns in an RL setting.

The advantage of using LSTMs is their ability to **capture long-term dependencies** in a sequence via an internal memory state. This makes them natural for encoding your quantum circuit (a sequence of gate instructions) into a fixed-size representation for decision-making. Even in supervised sequence tasks (e.g. language modeling or text-based games), LSTMs have proven robust. For instance, an LSTM-based agent was used to play **text adventure games** by encoding textual state descriptions into a policy network, yielding good results in those sequential decision problems ³ ⁴. Overall, these cases give confidence that an LSTM policy/value network can learn the repeating patterns in your instruction sequences.

Recommended LSTM Network Design for the Policy/Value Network

Given the structure of your input (a sequence of instructions, each represented by a 150-dimensional feature vector), a **stacked LSTM architecture** is recommended. Below are specific design choices, backed by practical research and implementations:

- **Stacked LSTM Layers with Residual Connections:** Using more than one LSTM layer (e.g. 2–3 layers deep) can help the network learn hierarchical features from the sequence. To ensure stable training of a deep LSTM, we can employ **residual connections** between layers. In NLP, a *stacked residual LSTM* (where the input to each LSTM layer is added to its output) was shown to train effectively and outperform shallow LSTMs ⁵. You can implement this by feeding the output of layer k into layer $k+1$ and also adding it (possibly after a linear projection) to the output of layer $k+1$. This technique was used in a paraphrase generation model to successfully train LSTMs 5–7 layers deep ⁵. In practice, 2 layers might be sufficient for your case, but the residual connection is a useful addition if you go deeper.
- **Hidden State Size:** The LSTM hidden dimension defines the size of the encoded state representation. Prior work suggests using a **few hundred units** is a good starting point. For example, the A3C agents used a 256-dimensional LSTM state ¹, and many open-source RL implementations use 128–256 units for LSTM policies. If your problem is very complex with high-

dimensional observations, you might scale this up (OpenAI Five went up to 4096 units for extreme complexity ²). A reasonable choice would be **256 or 512 units** for the LSTM hidden size, given your 150-dimensional input features and the complexity of quantum circuits. This size has been “**battle-tested**” in Atari and continuous control benchmarks to provide a good balance between capacity and training stability.

- **Input Encoding/Embedding:** Each time-step input to the LSTM is a 150-length vector encoding one gate instruction. You can feed this directly into the LSTM. However, you might consider adding a **learned linear embedding layer** beforehand that projects the 150 features to the LSTM’s hidden size (or another convenient size). This would be analogous to word embeddings in NLP (mapping one-hot words to dense vectors). For example, if your LSTM has 256 units, a linear layer from $\mathbb{R}^{150} \rightarrow \mathbb{R}^{256}$ could be applied to each instruction vector before the LSTM. This could help the network scale/normalize the different components (gate type, qubit flags, parameters) more flexibly. Many implementations simply let the first LSTM layer itself do this projection (since the input-to-hidden weights can learn an embedding), so this layer is optional. If the range of the numeric gate parameters varies widely, you may want to **normalize those features** or at least initialize the network in a way to handle unscaled values.
- **Actor-Critic Structure:** For an advantage actor-critic (A2C/A3C-style) network, you will have **two output heads** from the LSTM encoder: one for the policy (action probability distribution π) and one for the value function V . There are two common design patterns here:
 - **Shared LSTM encoder:** Use one LSTM stack to encode the input sequence into a final hidden state. Then apply two separate feedforward layers to that state – one outputs the policy logits and another outputs the scalar value. This approach is simpler and has fewer parameters. It was used, for example, in the recurrent PPO reference implementation: a single LSTM processes observations, then its output is passed through one hidden fully-connected layer and split into value and policy streams ⁶. In your case, you could take the final LSTM hidden state h_T (after processing the last instruction in the circuit) and feed it into (a) a fully-connected layer to N action logits and (b) a fully-connected layer to 1 value output. If needed, you can include an intermediate FC layer with nonlinearity for each head (e.g. DeepMind’s IMPALA agent and Stable-Baselines use a small MLP on top of the LSTM) ⁷.
 - **Separate LSTM for actor and critic:** To avoid any interference between policy and value learning, some implementations give each head its *own* LSTM module. For instance, Stable Baselines3’s recurrent PPO defines two LSTMs with identical architecture – one feeds into the policy MLP, the other feeds into the value MLP ⁸. This *Actor-Recurrent-Critic (ARC)* design doubles the recurrent parameters but can improve stability if the value function updates were destabilizing the shared representation. Both approaches have been used successfully in practice. If your training is stable, the shared approach is perfectly fine (and more efficient). If you observe training difficulties, the separate LSTM approach (as in SB3 or in some research on LSTM-augmented DDPG/PPO ⁹ ¹⁰) is a fallback.
- **Activation Functions and Normalization:** Inside the LSTM, the gating uses sigmoids/tanh by design, so you typically don’t need to add extra activation there. For the output head layers, **ReLU or LeakyReLU** are common choices (many RL implementations use ReLU in the policy/value MLP). In the example recurrent PPO architecture, all FC layers after the LSTM used ReLU ¹¹, whereas Stable

Baselines3 used Tanh in a small two-layer MLP ⁷ – either can work, with ReLU being a safe default. You can also consider applying **layer normalization** to the LSTM (there are variants of LSTM that include layer norm on the gates, which can stabilize training on noisy RL data). If using PyTorch, `nn.LSTM` doesn't have built-in layernorm, but you could switch to `nn.LSTMCell` and manually add `nn.LayerNorm` on the cell/hidden states. This is an optional tweak – many baseline implementations did fine *without* layernorm or weight norm on the LSTM. (Weight normalization is more common in conv nets like your TCN ⁶, but for LSTM a better analogue is layer norm or recurrent dropout.)

- **Sequence Processing Details:** Because each quantum circuit can have a different length of instructions, you'll need to handle variable-length sequences. The LSTM can process them by either **padding** to a max length or by looping over time steps until the sequence ends. A typical RL implementation would pad sequences in a batch to equal length and use a mask (to ignore padded timesteps in loss computations). If you roll out one environment at a time, you can also feed the sequence step-by-step into the LSTM and take the final state. In either case, ensure you **reset the LSTM's hidden state** at the start of each new sequence (episode) so that each circuit is encoded independently (unless you have a reason to carry state across episodes). The open-source truncated BPTT PPO implementation provides examples of how to manage hidden states and sequence resets in an RL training loop ¹² ¹³.

Optional Enhancements: Attention and Hybrid Models

While a standard stacked LSTM should work well, there are a couple of extensions you might consider if you need extra performance:

- **Attention Mechanism on LSTM Outputs:** *Attention* isn't limited to Transformers – one can attach an attention layer on top of an LSTM to help it focus on important parts of the sequence. For example, in NLP classification tasks, adding a **soft attention pooling** over the LSTM's outputs often improved accuracy by letting the model weight which words (timesteps) were most relevant to the prediction ¹⁴. In your context, an attention module could learn to assign higher weight to certain critical gate instructions in the circuit when computing the policy or value. Implementationally, you would take the sequence of LSTM hidden states $(h_1, \dots, h_T)$ and compute attention weights over them (via a small feedforward attention network, or even a simple dot-product query if you have a fixed context query). Then compute a context vector as the weighted sum of hidden states, and feed that to the policy/value heads. This adds complexity, but *if* you suspect that not all instructions are equally important (perhaps some pattern in the middle of the circuit heavily influences the reward), attention could be beneficial. There are code examples of adding attention to LSTM in open source – e.g., a bi-LSTM+attention for sentiment analysis (with code on GitHub) reported better performance than LSTM alone ¹⁴. Since you prefer to avoid full Transformers, this lightweight attention is a reasonable compromise if needed.
- **Hybrid CNN/TCN + LSTM Architectures:** If pure LSTM or pure TCN struggle to capture the sequence structure alone, you can combine them. A **hybrid TCN-LSTM** model could, for instance, use convolutional layers (TCN) to extract local patterns from the instruction sequence, then an LSTM to capture longer-term dependencies. Some research in time-series forecasting found that such hybrid models outperformed either component alone (one study reported a TCN-LSTM achieved ~33% higher accuracy than a standalone LSTM on a particulate matter forecast task) ¹⁵. For your case,

one imaginable hybrid is to apply a 1-D CNN or TCN over the sequence of instructions to get an initial set of features (capturing any short-range repeating patterns of gates), then feed the sequence of CNN outputs into an LSTM for global order processing. This approach is more complex and computationally heavy; given that you already have a well-performing TCN design, it might be more straightforward to keep the designs separate (pure TCN vs pure LSTM) for your benchmarking. But it's good to know that if a single-model approach underperforms, combining the **temporal convolution** (for pattern extraction) with **LSTM** (for order/memory) is a viable path (and indeed has open-source examples in other domains).

Implementation Resources

Crucially, you requested architectures with available implementations. Here are a few relevant resources that you can consult or draw inspiration from:

- **Stable Baselines3 – Recurrent PPO:** The *sb3-contrib* repository contains `RecurrentPPO` with an `MlpLstmPolicy`. It uses two LSTM layers (actor & critic) of 256 units each by default, and wraps the PPO algorithm to train with truncated BPTT. The architecture (as printed in an issue discussion) shows a Flatten → LSTM(...,256) → two-stream MLP design ¹⁶ ⁸. This is a robust, industry-tested implementation; you can find the code in the SB3-Contrib GitHub and documentation ⁷ ⁸. Adapting that architecture to libtorch C++ would be straightforward (libtorch has an LSTM module; you'd just need to handle the hidden state logic manually).
- **MarcoMeter's Recurrent PPO (PyTorch)** – This open-source project implements PPO with truncated BPTT and supports LSTM or GRU policies ¹⁷ ¹⁸. Their README and code illustrate how to organize sequences, reset hidden states, etc., and the included example uses (for visual input) a CNN encoder followed by an LSTM ⁶. For purely vector input, their design would feed the observation directly into the LSTM. This repo can serve as a **reference for training procedure** and verification that the architecture works in practice. It's also easy to modify the network definition there (`model.py`) to add residual connections or attention if you desire.
- **Residual LSTM Seq2Seq (Paraphrase Generation):** The GitHub repo `iamaaditya/neural-paraphrase-generation` provides an implementation of a stacked residual LSTM network for sequence-to-sequence tasks. By running it with the `--use_residual_lstm` flag, it enables skip connections across LSTM layers ⁵ ¹⁹. This implementation is in TensorFlow, but it concretely demonstrates the training of deeper LSTMs. Since your use-case is not seq-to-seq but rather sequence-to-policy, you wouldn't need the decoder part; however, the encoder (which is essentially multiple LSTMs in stack) is directly applicable. It's a good example if you want to see how residual connections were added and verify that it indeed improved training as reported in their paper.
- **Attention Layer Examples:** There are plenty of code snippets for adding an attention layer on top of an LSTM. One simple reference is the PyTorch forum discussion on *"Attention for sequence classification using LSTM"*, which provides code for an attention mechanism on the LSTM outputs ²⁰. Additionally, the W&B report by Aman Arora (linked above) contains a clear PyTorch implementation of a Bi-LSTM with attention (and a comparison of performance) ¹⁴. These can guide you in implementing the attention module if you choose to include it.

In summary, a **1D LSTM-based architecture is a strong candidate** for your problem. A suitable design is to use an LSTM (or stacked LSTMs) to encode the variable-length circuit sequence into a fixed-size hidden state, then use feedforward layers to output the action probabilities and value. This design is conceptually simple and has been proven in both the research literature and real-world RL implementations:

- It handles sequences of varying length inherently (via hidden state carry-over and masking).
- It can capture repeating patterns through its gating mechanisms and long memory.
- It has known successful examples in similar settings (from text-based RL to partially observable physics tasks).

By leveraging implementations like SB3's recurrent policy or the referenced GitHub projects, you can jump-start integrating a **deep LSTM policy network** in your C++ (LibTorch) pipeline. The combination of **residual connections** (to go deep), possibly **attention** (to highlight key instructions), and well-chosen hyperparameters (layers, units as noted above) should give you a robust model to compare against your TCN. Since you're generating random circuits every episode (no fixed dataset), overfitting isn't a concern – so feel free to make the network reasonably large if needed. The primary concern will be training stability and convergence speed, and the design choices above (shared vs separate LSTM, use of layer norm, etc.) are aimed at ensuring the LSTM learns effectively.

References:

- DeepMind A3C paper – used a 256-cell LSTM to improve agent performance on partially observed environments ¹.
- OpenAI Five – each agent was a single-layer 4096-unit LSTM network driving a complex policy (proved an LSTM can scale to very large sequential inputs) ².
- Residual LSTM for Paraphrase Generation – demonstrated that adding residual connections between LSTM layers allows training of deeper (3+ layer) LSTM networks successfully ⁵.
- Stable Baselines3 Recurrent PPO – open-source implementation with separate LSTM policy and value networks (256 units), followed by feedforward layers for policy/value outputs ⁷ ⁸.
- Recurrent PPO (Marco et al. 2023) – uses a shared LSTM core and splits into policy/value heads, with ReLU activations ⁶. Good reference for sequence handling in RL.
- Attention on LSTM – e.g. adding a learned attention pool over LSTM outputs improved text classification accuracy in a bi-LSTM model ¹⁴. This concept can be adapted to help the policy network focus on crucial parts of the input sequence.
- Hybrid TCN-LSTM – studies in other domains report that combining convolutional sequence modeling with LSTM memory can outperform either alone on certain structured sequence tasks ¹⁵. This is an avenue to explore if pure LSTM doesn't meet expectations.

¹ Asynchronous Methods for Deep Reinforcement Learning

<http://proceedings.mlr.press/v48/mnih16.pdf>

² OpenAI Five - Wikipedia

https://en.wikipedia.org/wiki/OpenAI_Five

³ Reinforcement Learning for Text Games - Leon Overweel

<https://leonoverweel.com/projects/2019/mlp-textworld-lstm-dqn/>

4 matthewsparr/Deep-Zork: Using NLP and reinforcement learning to ...

<https://github.com/matthewsparr/Deep-Zork>

5 19 GitHub - iamaaditya/neural-paraphrase-generation: Neural Paraphrase Generation

<https://github.com/iamaaditya/neural-paraphrase-generation>

6 11 12 13 17 18 GitHub - MarcoMeter/recurrent-ppo-truncated-bptt: Baseline implementation of recurrent PPO using truncated BPTT

<https://github.com/MarcoMeter/recurrent-ppo-truncated-bptt>

7 8 16 How to use LSTM ? RecurrentPPO from sb3-contrib · Issue #209 · Stable-Baselines-Team/stable-baselines3-contrib · GitHub

<https://github.com/Stable-Baselines-Team/stable-baselines3-contrib/issues/209>

9 10 Actor -Recurrent -Critic(ARC) networks architecture. The LSTM network... | Download Scientific Diagram

https://www.researchgate.net/figure/Actor-Recurrent-CriticARC-networks-architecture-The-LSTM-network-simulate-the-state_fig4_339059515

14 Sentiment Classification using Bi-LSTM with Attention | Written-Reports – Weights & Biases

<https://wandb.ai/amanarora/Written-Reports/reports/Sentiment-Classification-using-Bi-LSTM-with-Attention--Vmldzo0NzUxNjA5>

15 Deep learning coupled model based on TCN-LSTM for particulate ...

<https://www.sciencedirect.com/science/article/abs/pii/S1309104223000570>

20 Attention for sequence classification using a LSTM - PyTorch Forums

<https://discuss.pytorch.org/t/attention-for-sequence-classification-using-a-lstm/26044>

Language Understanding for Text-based Games using Deep Reinforcement Learning

Karthik Narasimhan*

CSAIL, MIT

karthikn@csail.mit.edu

Tejas D Kulkarni*

CSAIL, BCS, MIT

tejask@mit.edu

Regina Barzilay

CSAIL, MIT

regina@csail.mit.edu

Abstract

In this paper, we consider the task of learning control policies for text-based games. In these games, all interactions in the virtual world are through text and the underlying state is not observed. The resulting language barrier makes such environments challenging for automatic game players. We employ a deep reinforcement learning framework to jointly learn state representations and action policies using game rewards as feedback. This framework enables us to map text descriptions into vector representations that capture the semantics of the game states. ^[1] We evaluate our approach on two game worlds, comparing against baselines using bag-of-words and bag-of-bigrams for state representations. Our algorithm outperforms the baselines on both worlds demonstrating the importance of learning expressive representations. ^[1]

1 Introduction

In this paper, we address the task of learning control policies for text-based strategy games. These games, predecessors to modern graphical ones, still enjoy a large following worldwide.^[2] They often involve complex worlds with rich interactions and elaborate textual descriptions of the underlying states (see Figure ^[1]). Players read descriptions of the current world state and respond with natural language commands to take actions. Since the underlying state is not directly observable, the player has to understand the text in order to act, making it

State 1: The old bridge

You are standing very close to the bridge's eastern foundation. If you go east you will be back on solid ground ... The bridge sways in the wind.

Command: Go east

State 2: Ruined gatehouse

The old gatehouse is near collapse. Part of its northern wall has already fallen down ... East of the gatehouse leads out to a small open area surrounded by the remains of the castle. There is also a standing archway offering passage to a path along the old southern inner wall.

Exits: Standing archway, castle corner, Bridge over the abyss

Figure 1: Sample gameplay from a Fantasy World. The player with the quest of finding a secret tomb, is currently located on an *old bridge*. She then chooses an action to *go east* that brings her to a *ruined gatehouse* (State 2).

challenging for existing AI programs to play these games (DePristo and Zubek, 2001).

In designing an autonomous game player, we have considerable latitude when selecting an adequate state representation to use. The simplest method is to use a bag-of-words representation derived from the text description. However, this scheme disregards the ordering of words and the finer nuances of meaning that evolve from composing words into sentences and paragraphs. For instance, in State 2 in Figure ^[1], the agent has to understand that going *east* will lead it to the castle whereas moving *south* will take it to the standing archway. An alternative approach is to convert text descriptions to pre-specified representations using annotated training data, commonly used in language grounding tasks (Matuszek et al., 2013;

*Both authors contributed equally to this work.

¹Code is available at <http://people.csail.mit.edu/karthikn/mud-play>.

²<http://mudstats.com/>

Kushman et al., 2014).

In contrast, our goal is to learn useful representations in conjunction with control policies. We adopt a reinforcement learning framework and formulate game sequences as Markov Decision Processes. An agent playing the game aims to maximize rewards that it obtains from the game engine upon the occurrence of certain events. The agent learns a policy in the form of an action-value function $Q(s, a)$ which denotes the long-term merit of an action a in state s .

The action-value function is parametrized using a deep recurrent neural network, trained using the game feedback. The network contains two modules. The first one converts textual descriptions into vector representations that act as proxies for states. This component is implemented using Long Short-Term Memory (LSTM) networks (Hochreiter and Schmidhuber, 1997). The second module of the network scores the actions given the vector representation computed by the first.

We evaluate our model using two Multi-User Dungeon (MUD) games (Curtis, 1992; Amir and Doyle, 2002). The first game is designed to provide a controlled setup for the task, while the second is a publicly available one and contains human generated text descriptions with significant language variability. We compare our algorithm against baselines of a random player and models that use bag-of-words or bag-of-bigrams representations for a state. We demonstrate that our model LSTM-DQN significantly outperforms the baselines in terms of number of completed quests and accumulated rewards. For instance, on a fantasy MUD game, our model learns to complete 96% of the quests, while the bag-of-words model and a random baseline solve only 82% and 5% of the quests, respectively. Moreover, we show that the acquired representation can be reused across games, speeding up learning and leading to faster convergence of Q-values.

2 Related Work

Learning control policies from text is gaining increasing interest in the NLP community. Example applications include interpreting help documentation for software (Branavan et al., 2010), navigating with directions (Vogel and Jurafsky, 2010; Kollar et al., 2010; Artzi and Zettlemoyer, 2013; Matuszek et al., 2013; Andreas and Klein, 2015) and playing computer games (Eisenstein et al.,

2009; Branavan et al., 2011a).

Games provide a rich domain for grounded language analysis. Prior work has assumed perfect knowledge of the underlying state of the game to learn policies. Gorniak and Roy (2005) developed a game character that can be controlled by spoken instructions adaptable to the game situation. The grounding of commands to actions is learned from a transcript manually annotated with actions and state attributes. Eisenstein et al. (2009) learn game rules by analyzing a collection of game-related documents and precompiled traces of the game. In contrast to the above work, our model combines text interpretation and strategy learning in a single framework. As a result, textual analysis is guided by the received control feedback, and the learned strategy directly builds on the text interpretation.

Our work closely relates to an automatic game player that utilizes text manuals to learn strategies for Civilization (Branavan et al., 2011a). Similar to our approach, text analysis and control strategies are learned jointly using feedback provided by the game simulation. In their setup, states are fully observable, and the model learns a strategy by combining state/action features and features extracted from text. However, in our application, the state representation is not provided, but has to be inferred from a textual description. Therefore, it is not sufficient to extract features from text to supplement a simulation-based player.

Another related line of work consists of automatic video game players that infer state representations directly from raw pixels (Koutník et al., 2013; Mnih et al., 2015). For instance, Mnih et al. (2015) learn control strategies using convolutional neural networks, trained with a variant of Q-learning (Watkins and Dayan, 1992). While both approaches use deep reinforcement learning for training, our work has important differences. In order to handle the sequential nature of text, we use Long Short-Term Memory networks to automatically learn useful representations for arbitrary text descriptions. Additionally, we show that decomposing the network into a representation layer and an action selector is useful for transferring the learnt representations to new game scenarios.

3 Background

Game Representation We represent a game by the tuple $\langle H, A, T, R, \Psi \rangle$, where H is the set of all possible game states, $A = \{(a, o)\}$ is the set of

all commands (action-object pairs), $T(h' | h, a, o)$ is the stochastic transition function between states and $R(h, a, o)$ is the reward function. The game state H is *hidden* from the player, who only receives a varying textual description, produced by a stochastic function $\Psi : H \rightarrow S$. Specifically, the underlying state h in the game engine keeps track of attributes such as the player’s location, her health points, time of day, etc. The function Ψ (also part of the game framework) then converts this state into a textual description of the location the player is at or a message indicating low health. We do not assume access to either H or Ψ for our agent during both training and testing phases of our experiments. We denote the space of all possible text descriptions s to be S . Rewards are generated using R and are only given to the player upon completion of in-game quests.

Q-Learning Reinforcement Learning is a commonly used framework for learning control policies in game environments (Silver et al., 2007; Amato and Shani, 2010; Branavan et al., 2011b; Szita, 2012). The game environment can be formulated as a sequence of state transitions (s, a, r, s') of a Markov Decision Process (MDP). The agent takes an action a in state s by consulting a state-action value function $Q(s, a)$, which is a measure of the action’s expected long-term reward. Q-Learning (Watkins and Dayan, 1992) is a model-free technique which is used to learn an optimal $Q(s, a)$ for the agent. Starting from a random Q-function, the agent continuously updates its Q-values by playing the game and obtaining rewards. The iterative updates are derived from the Bellman equation (Sutton and Barto, 1998):

$$Q_{i+1}(s, a) = E[r + \gamma \max_{a'} Q_i(s', a') | s, a] \quad (1)$$

where γ is a discount factor for future rewards and the expectation is over all game transitions that involved the agent taking action a in state s .

Using these evolving Q-values, the agent chooses the action with the highest $Q(s, a)$ to maximize its expected future rewards. In practice, the trade-off between exploration and exploitation can be achieved following an ϵ -greedy policy (Sutton and Barto, 1998), where the agent performs a random action with probability ϵ .

Deep Q-Network In large games, it is often impractical to maintain the Q-value for all possible state-action pairs. One solution to this problem

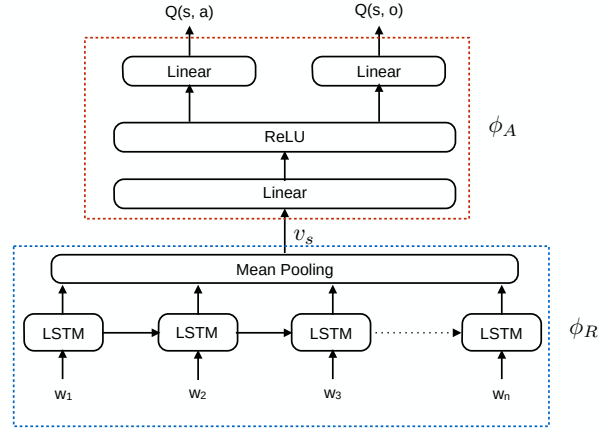


Figure 2: Architecture of LSTM-DQN: The Representation Generator (ϕ_R) (bottom) takes as input a stream of words observed in state s and produces a vector representation v_s , which is fed into the action scorer (ϕ_A) (top) to produce scores for all actions and argument objects.

is to approximate $Q(s, a)$ using a parametrized function $Q(s, a; \theta)$, which can generalize over states and actions by considering higher-level attributes (Sutton and Barto, 1998; Branavan et al., 2011a). However, creating a good parametrization requires knowledge of the state and action spaces. One way to bypass this feature engineering is to use a Deep Q-Network (DQN) (Mnih et al., 2015). The DQN approximates the Q-value function with a deep neural network to predict $Q(s, a)$ for all possible actions a simultaneously given the current state s . The non-linear function layers of the DQN also enable it to learn better value functions than linear approximators.

4 Learning Representations and Control Policies

In this section, we describe our model (DQN) and describe its use in learning good Q-value approximations for games with stochastic textual descriptions. We divide our model into two parts. The first module is a *representation generator* that converts the textual description of the current state into a vector. This vector is then input into the second module which is an *action scorer*. Figure 2 shows the overall architecture of our model. We learn the parameters of both the representation generator and the action scorer jointly, using the in-game reward feedback.

Representation Generator (ϕ_R) The representation generator reads raw text displayed to the

agent and converts it to a vector representation v_s . A bag-of-words (BOW) representation is not sufficient to capture higher-order structures of sentences and paragraphs. The need for a better semantic representation of the text is evident from the average performance of this representation in playing MUD-games (as we show in Section 6).

In order to assimilate better representations, we utilize a Long Short-Term Memory network (LSTM) (Hochreiter and Schmidhuber, 1997) as a representation generator. LSTMs are recurrent neural networks with the ability to connect and recognize long-range patterns between words in text. They are more robust than BOW to small variations in word usage and are able to capture underlying semantics of sentences to some extent. In recent work, LSTMs have been used successfully in NLP tasks such as machine translation (Sutskever et al., 2014) and sentiment analysis (Tai et al., 2015) to compose vector representations of sentences from word-level embeddings (Mikolov et al., 2013; Pennington et al., 2014). In our setup, the LSTM network takes in word embeddings w_k from the words in a description s and produces output vectors x_k at each step.

To get the final state representation v_s , we add a *mean pooling* layer which computes the element-wise mean over the output vectors x_k ³

$$v_s = \frac{1}{n} \sum_{k=1}^n x_k \quad (2)$$

Action Scorer (ϕ_A) The action scorer module produces scores for the set of possible actions given the current state representation. We use a multi-layered neural network for this purpose (see Figure 2). The input to this module is the vector from the representation generator, $v_s = \phi_R(s)$ and the outputs are scores for actions $a \in A$. Scores for all actions are predicted simultaneously, which is computationally more efficient than scoring each state-action pair separately. Thus, by combining the representation generator and action scorer, we can obtain the approximation for the Q-function as $Q(s, a) \approx \phi_A(\phi_R(s))[a]$.

An additional complexity in playing MUD-games is that the actions taken by the player are multi-word natural language *commands* such as

eat apple or *go east*. Due to computational constraints, in this work we limit ourselves to consider commands to consist of one action (e.g. *eat*) and one argument object (e.g. *apple*). This assumption holds for the majority of the commands in our worlds, with the exception of one class of commands that require two arguments (e.g. *move red-root right*, *move blue-root up*). We consider all possible actions and objects available in the game and predict both for each state using the same network (Figure 2). We consider the Q-value of the entire command (a, o) to be the average of the Q-values of the action a and the object o . For the rest of this section, we only show equations for $Q(s, a)$ but similar ones hold for $Q(s, o)$.

Parameter Learning We learn the parameters θ_R of the representation generator and θ_A of the action scorer using stochastic gradient descent with *RMSprop* (Tieleman and Hinton, 2012). The complete training procedure is shown in Algorithm 1. In each iteration i , we update the parameters to reduce the discrepancy between the predicted value of the current state $Q(s_t, a_t; \theta_i)$ (where $\theta_i = [\theta_R; \theta_A]_i$) and the expected Q-value given the reward r_t and the value of the next state $\max_a Q(s_{t+1}, a; \theta_{i-1})$.

We keep track of the agent’s previous experiences in a *memory* $\mathcal{D}$ ⁴. Instead of performing updates to the Q-value using transitions from the current episode, we sample a random transition $(\hat{s}, \hat{a}, s', r)$ from $\mathcal{D}$. Updating the parameters in this way avoids issues due to strong correlation when using transitions of the same episode (Mnih et al., 2015). Using the sampled transition and (II), we obtain the following loss function to minimize:

$$\mathcal{L}_i(\theta_i) = \mathbb{E}_{\hat{s}, \hat{a}} [(y_i - Q(\hat{s}, \hat{a}; \theta_i))^2] \quad (3)$$

where $y_i = \mathbb{E}_{\hat{s}, \hat{a}} [r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) \mid \hat{s}, \hat{a}]$ is the target Q-value with parameters θ_{i-1} fixed from the previous iteration.

The updates on the parameters θ can be performed using the following gradient of $\mathcal{L}_i(\theta_i)$:

$$\nabla_{\theta_i} \mathcal{L}_i(\theta_i) = \mathbb{E}_{\hat{s}, \hat{a}} [2(y_i - Q(\hat{s}, \hat{a}; \theta_i)) \nabla_{\theta_i} Q(\hat{s}, \hat{a}; \theta_i)]$$

For each epoch of training, the agent plays several episodes of the game, which is restarted after every terminal state.

³We also experimented with considering just the output vector of the LSTM after processing the last word. Empirically, we find that mean pooling leads to faster learning, so we use it in all our experiments.

⁴The memory is limited and rewritten in a first-in-first-out (FIFO) fashion.

Algorithm 1 Training Procedure for DQN with prioritized sampling

```
1: Initialize experience memory  $\mathcal{D}$ 
2: Initialize parameters of representation generator ( $\phi_R$ ) and action scorer ( $\phi_A$ ) randomly
3: for  $episode = 1, M$  do
4:   Initialize game and get start state description  $s_1$ 
5:   for  $t = 1, T$  do
6:     Convert  $s_t$  (text) to representation  $v_{s_t}$  using  $\phi_R$ 
7:     if  $random() < \epsilon$  then
8:       Select a random action  $a_t$ 
9:     else
10:      Compute  $Q(s_t, a)$  for all actions using  $\phi_A(v_{s_t})$ 
11:      Select  $a_t = \operatorname{argmax} Q(s_t, a)$ 
12:    Execute action  $a_t$  and observe reward  $r_t$  and new state  $s_{t+1}$ 
13:    Set priority  $p_t = 1$  if  $r_t > 0$ , else  $p_t = 0$ 
14:    Store transition  $(s_t, a_t, r_t, s_{t+1}, p_t)$  in  $\mathcal{D}$ 
15:    Sample random mini batch of transitions  $(s_j, a_j, r_j, s_{j+1}, p_j)$  from  $\mathcal{D}$ ,
      with fraction  $\rho$  having  $p_j = 1$ 
16:    Set  $y_j = \begin{cases} r_j & \text{if } s_{j+1} \text{ is terminal} \\ r_j + \gamma \max_{a'} Q(s_{j+1}, a'; \theta) & \text{if } s_{j+1} \text{ is non-terminal} \end{cases}$ 
17:    Perform gradient descent step on the loss  $\mathcal{L}(\theta) = (y_j - Q(s_j, a_j; \theta))^2$ 
```

Mini-batch Sampling In practice, online updates to the parameters θ are performed over a mini batch of state transitions, instead of a single transition. This increases the number of experiences used per step and is also more efficient due to optimized matrix operations.

The simplest method to create these mini-batches from the experience memory $\mathcal{D}$ is to sample uniformly at random. However, certain experiences are more valuable than others for the agent to learn from. For instance, rare transitions that provide positive rewards can be used more often to learn optimal Q-values faster. In our experiments, we consider such positive-reward transitions to have higher *priority* and keep track of them in $\mathcal{D}$. We use *prioritized sampling* (inspired by Moore and Atkeson (1993)) to sample a fraction ρ of transitions from the higher priority pool and a fraction $1 - \rho$ from the rest.

5 Experimental Setup

Game Environment For our game environment, we modify Evennia,⁵ an open-source library for building online textual MUD games. Evennia is a Python-based framework that allows one to easily create new games by writing a batch file describing the environment with details of rooms,

Stats	Home World	Fantasy World
Vocabulary size	84	1340
Avg. words / description	10.5	65.21
Max descriptions / room	3	100
# diff. quest descriptions	12	-
State transitions	Deterministic	Stochastic
# states (underlying)	16	≥ 56
Branching factor		
(# commands / state)	40	222

Table 1: Various statistics of the two game worlds

objects and actions. The game engine keeps track of the game state internally, presenting textual descriptions to the player and receiving text commands from the player. We conduct experiments on two worlds - a smaller *Home world* we created ourselves, and a larger, more complex *Fantasy world* created by Evennia’s developers. The motivation behind Home world is to abstract away high-level planning and focus on the language understanding requirements of the game.

Table 1 provides statistics of the game worlds. We observe that the Fantasy world is moderately sized with a vocabulary of 1340 words and up to 100 different descriptions for a room. These descriptions were created manually by the game developers. These diverse, engaging descriptions are designed to make it interesting and exciting for human players. Several rooms have many alternative descriptions, invoked randomly on each visit by

⁵<http://www.evennia.com/>

the player.

Comparatively, the Home world is smaller: it has a very restricted vocabulary of 84 words and the room descriptions are relatively structured. However, both the room descriptions (which are also varied and randomly provided to the agent) and the quest descriptions were adversarially created with negation and conjunction of facts to force an agent to actually understand the state in order to play well. Therefore, this domain provides an interesting challenge for language understanding.

In both worlds, the agent receives a positive reward on completing a quest, and negative rewards for getting into bad situations like falling off a bridge, or losing a battle. We also add small deterministic negative rewards for each non-terminating step. This incentivizes the agent to learn policies that solve quests in fewer steps. The supplementary material has details on the reward structure.

Home World We created *Home world* to mimic the environment of a typical house.⁶ The world consists of four rooms - a living room, a bedroom, a kitchen and a garden with connecting pathways. Every room is reachable from every other room. Each room contains a representative object that the agent can interact with. For instance, the kitchen has an *apple* that the player can *eat*. Transitions between the rooms are deterministic. At the start of each game episode, the player is placed in a random room and provided with a randomly selected quest. The text provided to the player contains both the description of her current state and that of the quest. Thus, the player can begin in one of 16 different states ($4 \text{ rooms} \times 4 \text{ quests}$), which adds to the world’s complexity.

An example of a quest given to the player in text is *Not you are sleepy now but you are hungry now*. To complete this quest and obtain a reward, the player has to navigate through the house to reach the kitchen and eat the apple (i.e type in the command *eat apple*). More importantly, the player should interpret that the quest does not require her to take a nap in the bedroom. We created such misguiding quests to make it hard for agents to succeed without having an adequate level of language understanding.

⁶An illustration is provided in the supplementary material.

Fantasy World The Fantasy world is considerably more complex and involves quests such as navigating through a broken bridge or finding the secret tomb of an ancient hero. This game also has stochastic transitions in addition to varying state descriptions provided to the player. For instance, there is a possibility of the player falling from the bridge if she lingers too long on it.

Due to the large command space in this game,⁷ we make use of cues provided by the game itself to narrow down the set of possible objects to consider in each state. For instance, in the MUD example in Figure 1, the game provides a list of possible exits. If the game does not provide such clues for the current state, we consider all objects in the game.

Evaluation We use two metrics for measuring an agent’s performance: (1) the cumulative reward obtained per episode averaged over the episodes and (2) the fraction of quests completed by the agent. The evaluation procedure is as follows. In each epoch, we first train the agent on M episodes of T steps each. At the end of this training, we have a testing phase of running M episodes of the game for T steps. We use $M = 50, T = 20$ for the Home world and $M = 20, T = 250$ for the Fantasy world. For all evaluation episodes, we run the agent following an ϵ -greedy policy with $\epsilon = 0.05$, which makes the agent choose the best action according to its Q-values 95% of the time. We report the agent’s performance at each epoch.

Baselines We compare our LSTM-DQN model with three baselines. The first is a *Random* agent that chooses both actions and objects uniformly at random from all available choices.⁸ The other two are BOW-DQN and BI-DQN, which use a bag-of-words and a bag-of-bigrams representation of the text, respectively, as input to the DQN action scorer. These baselines serve to illustrate the importance of having a good representation layer for the task.

Settings For our DQN models, we used $\mathcal{D} = 100000, \gamma = 0.5$. We use a learning rate of 0.0005 for RMSprop. We anneal the ϵ for ϵ -greedy from 1 to 0.2 over 100000 transitions. A mini-batch gradient update is performed every 4 steps of the gameplay. We roll out the LSTM (over words) for

⁷We consider 222 possible command combinations of 6 actions and 37 object arguments.

⁸In the case of the Fantasy world, the object choices are narrowed down using game clues as described earlier.

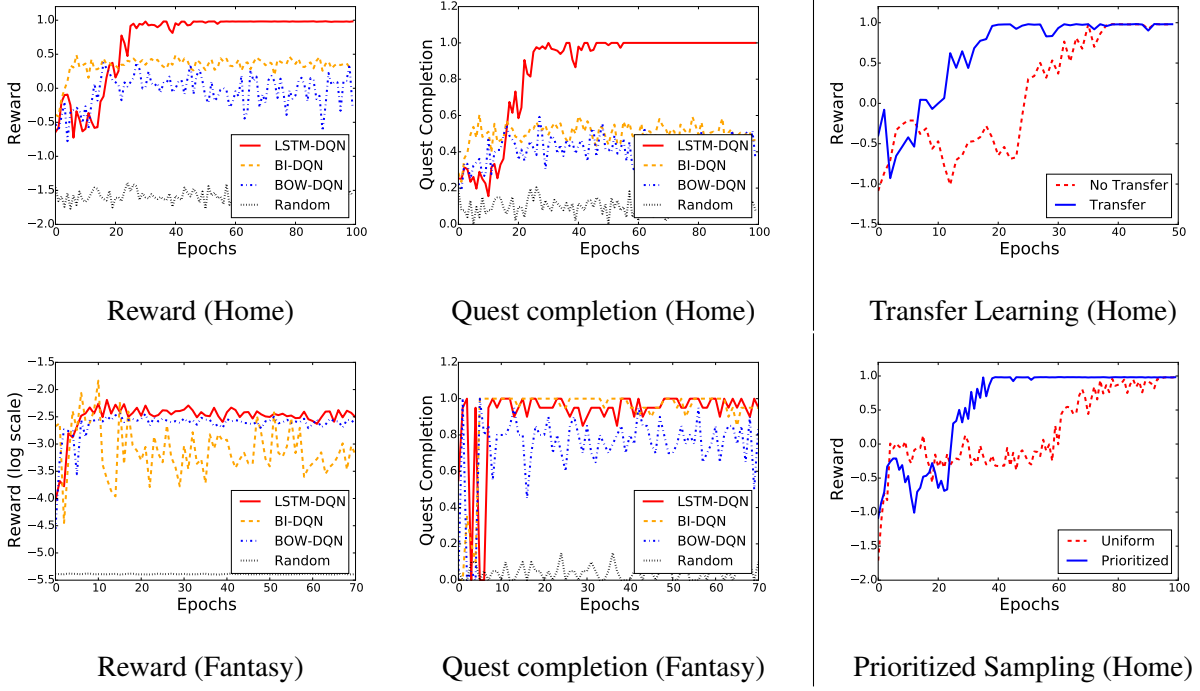


Figure 3: **Left:** Graphs showing the evolution of average reward and quest completion rate for BOW-DQN, LSTM-DQN and a Random baseline on the Home world (**top**) and Fantasy world (**bottom**). Note that the reward is shown in log scale for the Fantasy world. **Right:** Graphs showing effects of transfer learning and prioritized sampling on the Home world.

a maximum of 30 steps on the Home world and for 100 steps on the Fantasy world. For the prioritized sampling, we used $\rho = 0.25$ for both worlds. We employed a mini-batch size of 64 and word embedding size $d = 20$ in all experiments.

6 Results

Home World Figure 3 illustrates the performance of LSTM-DQN compared to the baselines. We can observe that the Random baseline performs quite poorly, completing only around 10% of quests on average⁹ obtaining a low reward of around -1.58 . The BOW-DQN model performs significantly better and is able to complete around 46% of the quests, with an average reward of 0.20. The improvement in reward is due to both greater quest success rate and a lower rate of issuing invalid commands (e.g. *eat apple* would be invalid in the bedroom since there is no apple). We notice that both the reward and quest completion graphs of this model are volatile. This is because the model fails to pick out differences between quests like *Not you are hungry now but you are sleepy now* and *Not you are sleepy now but you*

⁹Averaged over the last 10 epochs.

are hungry now. The BI-DQN model suffers from the same issue although it performs slightly better than BOW-DQN by completing 48% of quests. In contrast, the LSTM-DQN model does not suffer from this issue and is able to complete 100% of the quests after around 50 epochs of training, achieving close to the optimal reward possible.¹⁰ This demonstrates that having an expressive representation for text is crucial to understanding the game states and choosing intelligent actions.

In addition, we also investigated the impact of using a deep neural network for modeling the action scorer ϕ_A . Figure 4 illustrates the performance of the BOW-DQN and BI-DQN models along with their simpler versions BOW-LIN and BI-LIN, which use a single linear layer for ϕ_A . It can be seen that the DQN models clearly achieve better performance than their linear counterparts, which points to them modeling the control policy better.

Fantasy World We evaluate all the models on the Fantasy world in the same manner as before and report reward, quest completion rates and Q-

¹⁰Note that since each step incurs a penalty of -0.01 , the best reward (on average) a player can get is around 0.98.

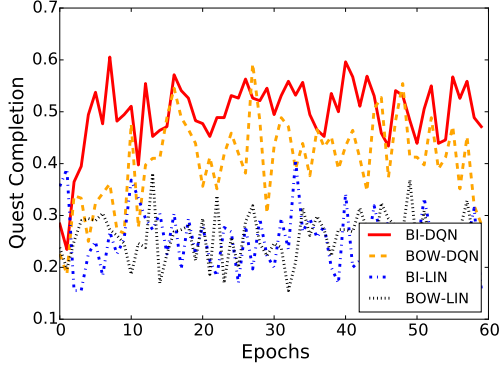


Figure 4: Quest completion rates of DQN vs. Linear models on Home world.

values. The quest we evaluate on involves crossing the broken bridge (which takes a minimum of five steps), with the possibility of falling off at random (a 5% chance) when the player is on the bridge. The game has an additional quest of reaching a secret tomb. However, this is a complex quest that requires the player to memorize game events and perform high-level planning which are beyond the scope of this current work. Therefore, we focus only on the first quest.

From Figure 3 (bottom), we can see that the Random baseline does poorly in terms of both average per-episode reward¹¹ and quest completion rates. BOW-DQN converges to a much higher average reward of -12.68 and achieves around 82% quest completion. Again, the BOW-DQN is often confused by varying (10 different) descriptions of the portions of the bridge, which reflects in its erratic performance on the quest. The BI-DQN performs very well on quest completion by finishing 97% of quests. However, this model tends to find sub-optimal solutions and gets an average reward of -26.68 , even worse than BOW-DQN. One reason for this is the negative rewards the agent obtains after falling off the bridge. The LSTM-DQN model again performs best, achieving an average reward of -11.33 and completing 96% of quests on average. Though this world does not contain descriptions adversarial to BOW-DQN or BI-DQN, the LSTM-DQN obtains higher average reward by completing the quest in fewer steps and showing more resilience to variations in the state descriptions.

Transfer Learning We would like the representations learnt by ϕ_R to be generic enough and

¹¹Note that the rewards graph is in log scale.

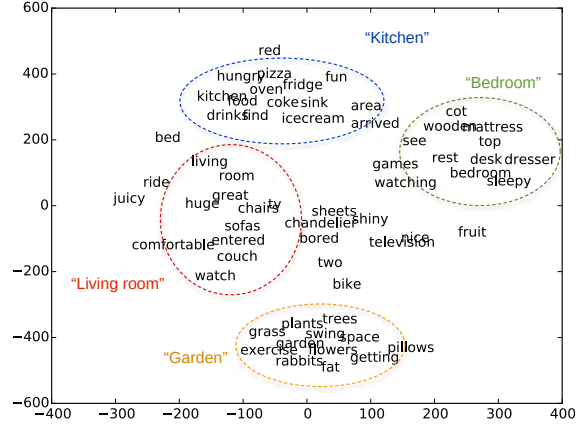


Figure 5: t-SNE visualization of the word embeddings (except stopwords) after training on Home world. The embedding values are initialized randomly.

transferable to new game worlds. To test this, we created a second Home world with the same rooms, but a completely different map, changing the locations of the rooms and the pathways between them. The main differentiating factor of this world from the original home world lies in the high-level planning required to complete quests.

We initialized the LSTM part of an LSTM-DQN agent with parameters θ_R learnt from the original home world and trained it on the new world¹². Figure 3 (top right) demonstrates that the agent with transferred parameters is able to learn quicker than an agent starting from scratch initialized with random parameters (*No Transfer*), reaching the optimal policy almost 20 epochs earlier. This indicates that these simulated worlds can be used to learn good representations for language that transfer across worlds.

Prioritized sampling We also investigate the effects of different minibatch sampling procedures on the parameter learning. From Figure 3 (bottom right), we observe that using prioritized sampling significantly speeds up learning, with the agent achieving the optimal policy around 50 epochs faster than using uniform sampling. This shows promise for further research into different schemes of assigning priority to transitions.

Representation Analysis We analyzed the representations learnt by the LSTM-DQN model on the Home world. Figure 5 shows a visualization

¹²The parameters for the Action Scorer (θ_A) are initialized randomly.

Description	Nearest neighbor
You are halfway out on the unstable bridge. From the castle you hear a distant howling sound, like that of a large dog or other beast.	The bridge slopes precariously where it extends westwards towards the lowest point - the center point of the hang bridge. You clasp the ropes firmly as the bridge sways and creaks under you.
The ruins opens up to the sky in a small open area, lined by columns. ... To the west is the gatehouse and entrance to the castle, whereas southwards the columns make way for a wide open courtyard.	The old gatehouse is near collapse. East the gatehouse leads out to a small open area surrounded by the remains of the castle. There is also a standing archway offering passage to a path along the old southern inner wall.

Table 2: Sample descriptions from the Fantasy world and their nearest neighbors (NN) according to their vector representations from the LSTM representation generator. The NNs are often descriptions of the same or similar (nearby) states in the game.

of learnt word embeddings, reduced to two dimensions using t-SNE (Van der Maaten and Hin-ton, 2008). All the vectors were initialized randomly before training. We can see that semantically similar words appear close together to form coherent subspaces. In fact, we observe four different subspaces, each for one type of room along with its corresponding object(s) and quest words. For instance, food items like *pizza* and rooms like *kitchen* are very close to the word *hungry* which appears in a quest description. This shows that the agent learns to form meaningful associations between the semantics of the quest and the environment. Table 2 shows some examples of descriptions from Fantasy world and their nearest neighbors using cosine similarity between their corresponding vector representations produced by LSTM-DQN. The model is able to correlate descriptions of the same (or similar) underlying states and project them onto nearby points in the representation subspace.

7 Conclusions

We address the task of end-to-end learning of control policies for text-based games. In these games, all interactions in the virtual world are through text and the underlying state is not observed. The resulting language variability makes such environments challenging for automatic game players. We employ a deep reinforcement learning framework to jointly learn state representations and action policies using game rewards as feedback. This framework enables us to map text descriptions into vector representations that capture the semantics of the game states. Our experiments demonstrate the importance of learning good representations of text in order to play these games well. Future directions include tackling high-level

planning and strategy learning to improve the performance of intelligent agents.

Acknowledgements

We are grateful to the developers of Evennia, the game framework upon which this work is based. We also thank Nate Kushman, Clement Gehring, Gustavo Goretkin, members of MIT’s NLP group and the anonymous EMNLP reviewers for insightful comments and feedback. T. Kulkarni was graciously supported by the Leventhal Fellowship. We would also like to acknowledge MIT’s Center for Brains, Minds and Machines (CBMM) for support.

References

- [Amato and Shani2010] Christopher Amato and Guy Shani. 2010. High-level reinforcement learning in strategy games. In *Proceedings of the 9th International Conference on Autonomous Agents and Multi-agent Systems: Volume 1*, pages 75–82. International Foundation for Autonomous Agents and Multiagent Systems.
- [Amir and Doyle2002] Eyal Amir and Patrick Doyle. 2002. Adventure games: A challenge for cognitive robotics. In *Proc. Int. Cognitive Robotics Workshop*, pages 148–155.
- [Andreas and Klein2015] Jacob Andreas and Dan Klein. 2015. Alignment-based compositional semantics for instruction following. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*.
- [Artzi and Zettlemoyer2013] Yoav Artzi and Luke Zettlemoyer. 2013. Weakly supervised learning of semantic parsers for mapping instructions to actions. *Transactions of the Association for Computational Linguistics*, 1(1):49–62.
- [Branavan et al.2010] SRK Branavan, Luke S Zettlemoyer, and Regina Barzilay. 2010. Reading be-

Counting to Explore and Generalize in Text-based Games

Xingdi Yuan^{*1} Marc-Alexandre Côté^{*1} Alessandro Sordoni¹ Romain Laroché¹ Remi Tachet des Combes¹
Matthew Hausknecht¹ Adam Trischler¹

Abstract

We propose a recurrent RL agent with an episodic exploration mechanism that helps discovering good policies in text-based game environments. We show promising results on a set of generated text-based games of varying difficulty where the goal is to collect a coin located at the end of a chain of rooms. In contrast to previous text-based RL approaches, we observe that our agent learns policies that generalize to unseen games of greater difficulty.

1. Introduction

Text-based games like Zork (Infocom, 1980) are complex, interactive simulations. They use natural language to describe the state of the world, to accept actions from the player, and to report subsequent changes in the environment. The player works toward goals which are seldom specified explicitly and must be discovered through exploration. The observation and action spaces in text games are both combinatorial and compositional, and players must contend with partial observability, since descriptive text does not communicate complete, unambiguous information about the underlying game state.

In this paper, we study several methods of exploration in text-based games. Our basic task is a deterministic text-based version of the *chain experiment* (Osband et al., 2016; Plappert et al., 2017) with distractor nodes that are off-chain: the agent must navigate a path composed of discrete locations (rooms) to the goal, ideally without revisiting dead ends. We propose a DQN-based recurrent model for solving text-based games, where the recurrence gives the model the capacity to condition its policy on historical state information. To encourage exploration, we extend count-based exploration approaches (Ostrovski et al., 2017; Tang et al.,

2017), which assign an intrinsic reward derived from the count of state visitations during learning, across episodes. Specifically, we propose an episodic count-based exploration scheme, where state counts are reset at the beginning of each episode. This reward plays the role of an *episodic memory* (Gershman & Daw, 2017) that pushes the agent to visit states not previously encountered within an episode. Although the recurrent policy architecture has the capacity to solve the task by remembering and avoiding previously visited locations, we hypothesize that exploration rewards will help the agent learn to utilize its memory.

We generate a set of games of varying difficulty (measured with respect to the path length and the number of off-chain rooms) with a text-based game generator (Côté et al., 2018). We observe that, in contrast to a baseline model and standard count-based exploration methods, the recurrent model with episodic bonus learns policies that not only complete multiple training games at same time successfully but also generalize to *unseen* games of *greater difficulty*.

2. Text-based Games as POMDPs

Text-based games are sequential decision-making problems that can be described naturally by the Reinforcement Learning (RL) setting. Fundamentally, text-based games are partially observable Markov decision processes (POMDP) (Kaelbling et al., 1998) where the environment state is never observed directly. To act optimally, an agent must keep track of all observations. Formally, a text-based game is a discrete-time POMDP defined by $(S, T, A, \Omega, O, R, \gamma)$, where $\gamma \in [0, 1]$ is the discount factor.

Environment States (S): The environment state at turn t in the game is $s_t \in S$. It contains the complete internal information of the game, much of which is hidden from the agent. When an agent issues a command c_t (defined next), the environment transitions to state s_{t+1} with probability $T(s_{t+1} | s_t, c_t)$.

Actions (A): At each turn t , the agent issues a text command c_t . The interpreter can accept any sequence of characters but will only recognize a tiny subset thereof. Furthermore, only a fraction of recognized commands will actually change the state of the world. The resulting action space

^{*}Equal contribution ¹Microsoft Research. Correspondence to: Eric Yuan <eric.yuan@microsoft.com>, Marc-Alexandre Côté <macote@microsoft.com>.

is enormous and intractable for existing RL algorithms. In this work, we make the following two simplifying assumptions. (1) **Word-level** Each command is a two-word sequence where the words are taken from a fixed vocabulary V . (2) **Command syntax** Each command is a (verb, object) pair (direction words are considered objects).

Observations (Ω): The text information perceived by the agent at a given turn t in the game is the agent’s observation, $o_t \in \Omega$, which depends on the environment state and the previous command with probability $O(o_t | s_t, c_{t-1})$. Thus, the function O selects from the environment state what information to show to the agent given the last command.

Reward Function (R): Based on its actions, the agent receives reward signals $r_t = R(s_t, a_t)$. The goal is to maximize the expected discounted sum of rewards $E[\sum_t \gamma^t r_t]$.

3. Method

3.1. Model Architecture

In this work, we adopt the LSTM-DQN (Narasimhan et al., 2015) model as baseline. It has two modules: a representation generator Φ_R , and an action scorer Φ_A . Φ_R takes observation strings o as input, after a stacked embedding layer and LSTM (Hochreiter & Schmidhuber, 1997) encoder, a mean-pooling layer produces a vector representation of the observation. This feeds into Φ_A , in which two MLPs, sharing a lower layer, predict the Q-values over all verbs w_v and object words w_o independently. The average of the two resulting scores gives the Q-values for the composed actions. The LSTM-DQN does not condition on previous actions or observations, so it cannot deal with partial observability. We concatenate the previous command c_{t-1} to the current observation o_t to lessen this limitation.

To further enhance the agent’s capacity to remember previous states, we replace the shared MLP in Φ_A by an LSTM cell. This model is inspired by (Hausknecht & Stone, 2015; Lample & Chaplot, 2016) and we call it LSTM-DRQN. The LSTM cell in Φ_A takes the representation generated by Φ_R together with history information h_{t-1} from the previous game step as input. It generates the state information at the current game step, which is then fed into the two MLPs as well as passed forward to next game step. Figure 1 shows the LSTM-DRQN architecture.

3.2. Discovery Bonus

To promote exploration we use an intrinsic reward by counting state visits (Kolter & Ng, 2009; Tang et al., 2017; Martin et al., 2017; Ostrovski et al., 2017). We investigate two approaches to counting rewards. The first is inspired by (Kolter & Ng, 2009), where we define the **cumulative counting bonus** as $r^+(o_t) = \beta \cdot n(o_t)^{-1/3}$, where $n(o_t)$ is the num-

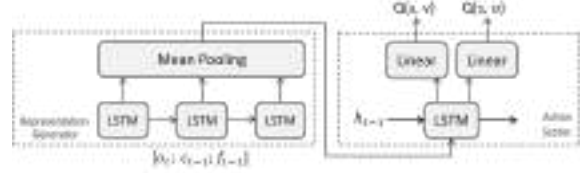


Figure 1. LSTM-DRQN processes textual observations word-by-word to generate a fixed-length vector representation. This representation is used by the recurrent policy to estimate Q-values for all verbs $Q(s, v)$ and objects $Q(s, o)$.

ber of times the agent has observed o_t since the beginning of training (across episodes), and β is the bonus coefficient. During training, as the agent observes new states more and more, the cumulative counting bonus gradually converges to 0.

The second approach is the **episodic discovery bonus**, which encourages the agent to discover unseen states by assigning a positive reward whenever it sees a new state.

It is defined as: $r^+(o_t) = \begin{cases} \beta & \text{if } n(o_t) = 1 \\ 0.0 & \text{otherwise} \end{cases}$, where $n(\cdot)$ is reset to zero at the beginning of each episode. Taking inspiration from (Gershman & Daw, 2017), we hope this behavior pushes the agent to visit states not previously encountered in the current episode and teaches the agent how to use its memory for this purpose so it may generalize to unseen environments.

4. Related Work

RL Applied to Text-based Games: Narasimhan et al. (2015) test their LSTM-DQN in two text-based environments: Home World and Fantasy World. They report the quest completion ratio over multiple runs but not how many steps it takes to complete them. He et al. (2015) introduce the Deep Reinforcement Relevance Network (DRRN) for tackling choice-based (as opposed to parser-based) text games, evaluating the DRRN on one deterministic game and one larger-scale stochastic game. The DRRN model converges on both games; however, this model must know in advance the valid commands at each state. Fulda et al. (2017) propose a method to reduce the action space for parser-based games by training word embeddings to be aware of verb-noun affordances. One drawback of this approach is it requires pre-trained embeddings.

Count-based Exploration: The Model Based Interval Estimation-Exploration Bonus (MBIE-EB) (Strehl & Littman, 2008) derives an intrinsic reward by counting state-action pairs with a table $n(s, a)$. Their exploration bonus has the form $\beta/\sqrt{n(s, a)}$ to encourage exploring less-visited pairs. In this work, we use $n(s)$ rather than $n(s, a)$, since the majority of actions leave the agent in the same state

(i.e., unrecognized commands). Using the latter would reward the agent for trying invalid commands, which is not sensible in our setting.

Tang et al. (2017) propose a hashing function for count-based exploration in order to discretize high-dimensional, continuous state spaces. Their exploration bonus $r^+ = \beta / \sqrt{n(\phi(s))}$, where $\phi(\cdot)$ is a hashing function that can either be static or learned. This is similar to the *cumulative counting bonus* defined above.

Deep Recurrent Q-Learning: Hausknecht & Stone (2015) propose the Deep Recurrent Q-Networks (DRQN), adding a recurrent neural network (such as an LSTM (Hochreiter & Schmidhuber, 1997)) on top of the standard DQN model. DRQN estimates $Q(o_t, h_{t-1}, a_t)$ instead of $Q(o_t, a_t)$, so it has the capacity to memorize the state history. Lample & Chaplot (2016) use a model built on the DRQN architecture to learn to play FPS games.

A major difference between the work presented in this paper and the related work is that we test on unseen games and train on a set of similar (but not identical) games rather than training and testing on the same game.

5. Experiments

5.1. Coin Collector Game Setup

To evaluate the two models described above and the proposed discovery bonus, we designed a set of simple text-based games inspired by the chain experiment (Osband et al., 2016; Plappert et al., 2017). Each game contains a given number of rooms that are randomly connected to each other to form a chain (see figures in Appendix C). The goal is to find and collect a “coin” placed in one of the rooms. The player’s initial position is at one end of the chain and the coin is at the other. These games have deterministic state transitions.

Games stop after a set number of steps or after the player has collected the coin. The game interpreter understands only five commands (**go north**, **go east**, **go south**, **go west** and **take coin**), while the action space is twice as large: **go**, **take** $\times$ **north**, **south**, **east**, **west**, **coin**. See Figure 12, Appendix C for an example of what the agent observes in-game.

Our games have 3 **modes**: *easy* (mode 0), there are no distractor rooms (dead ends) along the path; *medium* (mode 1), each room along the optimal trajectory has one distractor room randomly connected to it; *hard* (mode 2), each room on the path has two distractor rooms, i.e., within a room on the optimal trajectory, all 4 directions lead to a connected room. We use difficulty **levels** to indicate the optimal trajectory’s length of a game.

To solve easy games, the agent must learn to recall its previous directional action and to issue the command that does not reverse it (e.g., if the agent entered the current room by going **east**, do not now **go west**). Conversely, to solve medium and hard games, the agent must reverse its previous action when it enters distractor rooms to return to the chain, and also recall farther into the past to track which exits it has already passed through. Alternatively, since there are no cycles, it can learn a less memory intensive “wall-following” strategy by, e.g., taking exits in a clockwise order from where it enters a room.

We refer to models with the cumulative counting bonus as MODEL+, and models with episodic discovery bonus as MODEL++, where MODEL = DQN, DRQN¹ (implementation details in Appendix A). In this section we cover part of the experiment results, the full extent of our experiment results are provided in Appendix B.

5.2. Solving Training Games

We first investigate whether the variant models can learn to solve single games with different difficulty modes (easy, medium, hard) and levels $L5, L10, L15, L20, L25, L30$ ². As shown in Figure 2 (top row), when the games are simple, vanilla DQN and DRQN already fail to learn. Adding the cumulative bonus helps somewhat and models perform similarly with and without recurrence. When the games become harder, the cumulative bonus helps less, while episodic bonus remains very helpful and recurrence in the model becomes very helpful.

Next, we are interested to see whether models can learn to solve a distribution of games. Note that each game has its own counting memory, i.e., the states visited in one game do not affect the counters for other games. Here, we fix the game difficulty level to 10, and randomly generate training sets that contain 2, 5, 10, 30, 50, 100 games in each mode. As shown in Figure 2 (bottom row), when the game mode becomes harder, the episodic bonus has an advantage over the cumulative bonus, and recurrence becomes more crucial for memorizing the game distribution. It is also clear that the episodic bonus and recurrence help significantly when more training games are provided.

5.3. Zero-shot Evaluation

Finally, we want to see if a pre-trained model can generalize to unseen games. The generated training set contains 1, 2, 5, 10, 30, 50, 100, 500 $\times$ L10 games for each mode. Then, for each corresponding mode the test set contains 10 *unseen* $L5, L10, L15, L20, L30$ games. There is no

¹Since all models use the LSTM representation generator, we omit “LSTM” for abbreviation.

²We use Lk to indicate level k game.

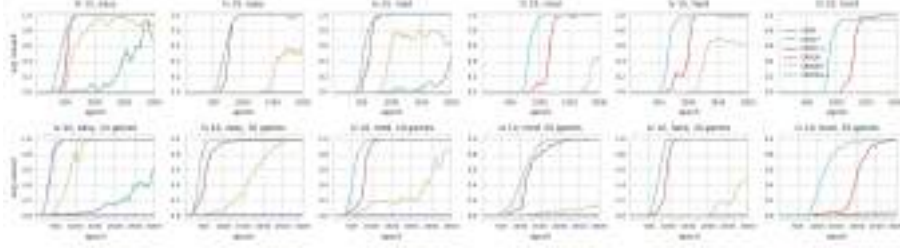


Figure 2. Model performance on single games (top row) and multiple games (bottom row).

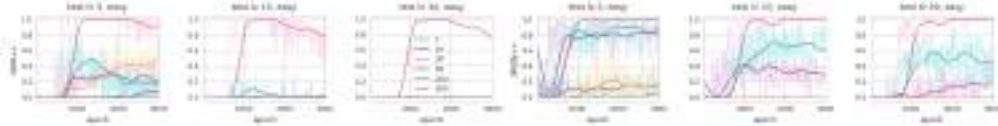


Figure 3. Zero-shot evaluation: Average rewards of DQN++ (left) and DRQN++ (right) as a function of the number of games in the training set.



Figure 4. Average rewards and steps used corresponding to best validation performance in hard games.

overlap between training and test games in either text descriptions or optimal trajectories. At test time, the counting modules are disabled, the agent is not updated, and its generates verb and noun actions based on the *argmax* of their Q-values.

As shown in Figure 3, when the game mode is easy, both models with and without recurrence can generalize well on unseen games by training on a large training set. It is worth noting that by training on 500 L10 easy games, both models can almost perfectly solve level 30 unseen easy games. We also observe that models with recurrence are able to generalize better when trained on fewer games.

When testing on hard mode games, we observe that both models suffer from overfitting (after a certain number of episodes, average test reward starts to decrease while training reward increases). Therefore, we further generated a validation set that contains 10 L10 hard games, and report test results corresponding to best validation performance. In

addition, we investigated what happens when concatenating the previous 4 steps’ history observation into the input. In Figure 4, we add *H* to model names to indicate this variant.

As shown in Figure 4, all models can memorize the 500 training games, while DQN++ and DRQN++H are able to generalize better on unseen games. In particular, the former performs near perfectly on test games. To investigate this, we looked into all the bi-grams of generated commands (*i.e.*, two commands from adjacent game steps) from DQN++ model. Surprisingly, except for moving back from dead end rooms, the agent always explores exits in anti-clockwise order. This means the agent has learned a general strategy that does not require history information beyond the previous command. This strategy generalizes perfectly to all possible hard games because there are no cycles in the maps.

6. Final Remarks

We propose an RL model with a recurrent component, together with an episodic count-based exploration scheme that promotes the agent’s discovery of the game environment. We show promising results on a set of generated text-based games of varying difficulty. In contrast to baselines, our approach learns policies that generalize to unseen games of greater difficulty.

In future work, we plan to experiment on games with more complex topology, such as cycles (where the “wall-following” strategy will not work). We would like to explore games that require multi-word commands (*e.g.*, unlock red door with red key), necessitating a model that generates sequences of words. Other interesting directions include agents that learn to map or to deal with stochastic transitions in text-based games.

A. Implementation Details

Implementation details of our neural baseline agent are as follows³. In all experiments, the word embeddings are initialized with 20-dimensional random matrices; the number of hidden units of the encoder LSTM is 100. In the non-recurrent action scorer we use a 1-layer MLP which has 64 hidden units, with *ReLU* as non-linear activation function, in the recurrent action scorer, we use an LSTM cell which hidden size is 64.

In replay memory, we used a memory with capacity of 500000, a mini-batch gradient update is performed every 4 steps in the gameplay, the mini-batch size is 32. We apply prioritized sampling in all experiments, in which, we used $\rho = 0.25$. In LSTM-DQN and LSTM-DRQN model, we used discount factor $\gamma = 0.9$, in all models with discovery bonus, we used $\gamma = 0.5$.

When updating models with recurrent components, we follow the update strategy in (Lample & Chaplot, 2016), *i.e.*, we randomly sample sequences of length 8 from the replay memory, zero initialize hidden state and cell state, use the first 4 states to bootstrap a reliable hidden state and cell state, and then update on rest of the sequence.

We anneal the ϵ for ϵ -greedy from 1 to 0.2 over 1000 epochs, it remains at 0.2 afterwards. In both cumulative and episodic discovery bonus, we use coefficient β of 1.0.

When zero-shot evaluating hard games, we use $max_train_step = 100$, in all other experiments we use $max_train_step = 50$; during test, we always use $max_test_step = 200$.

We use *adam* (Kingma & Ba, 2014) as the step rule for optimization. The learning rate is $1e^{-3}$. The model is implemented using *PyTorch* (Paszke et al., 2017).

All games are generated using TextWorld framework (Côté et al., 2018), we used the house grammar.

³Our implementation is publicly available at <https://github.com/xingdi-eric-yuan/TextWorld-Coin-Collector>.

B. More Results

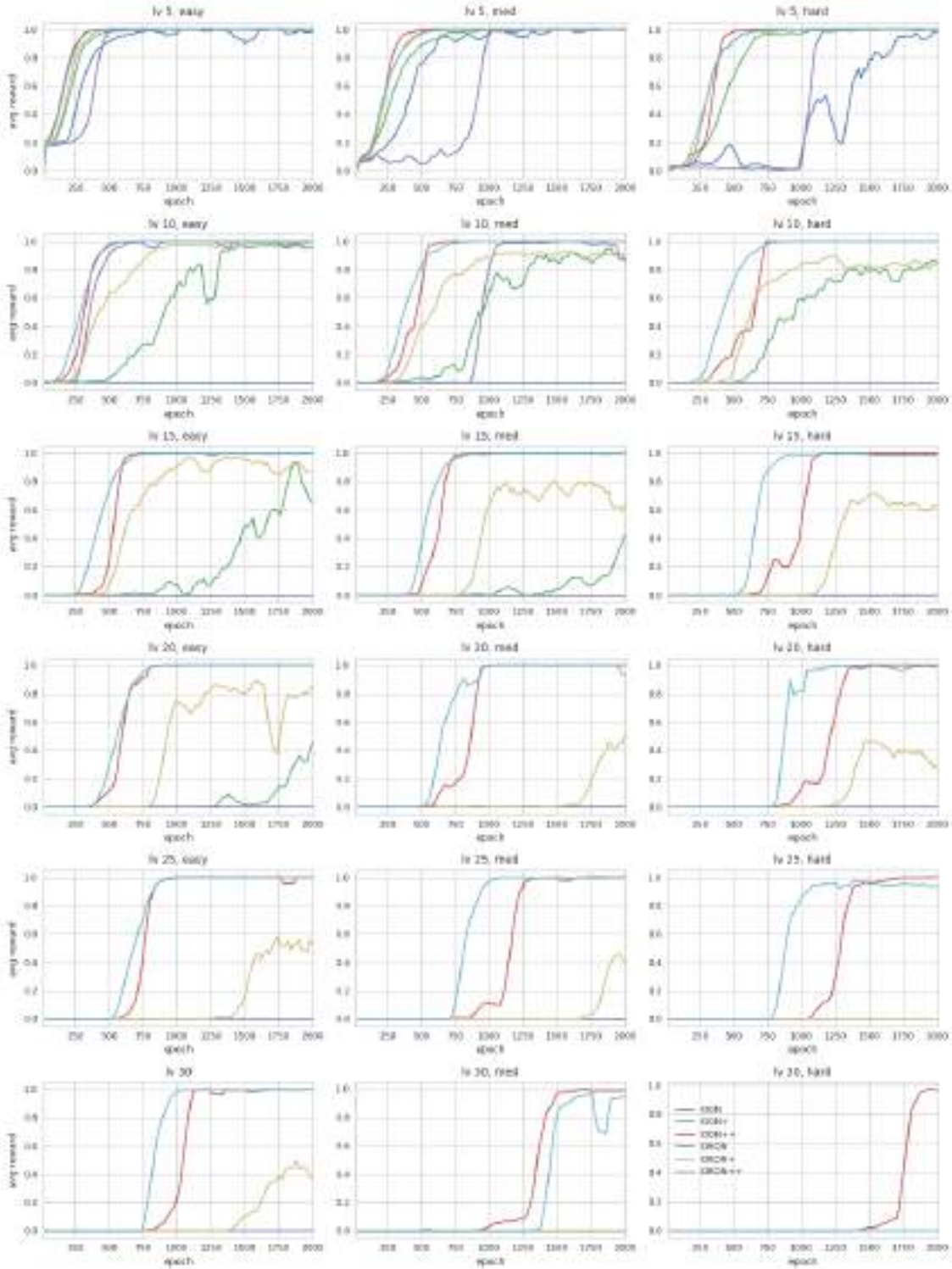


Figure 5. Model performance on single games.

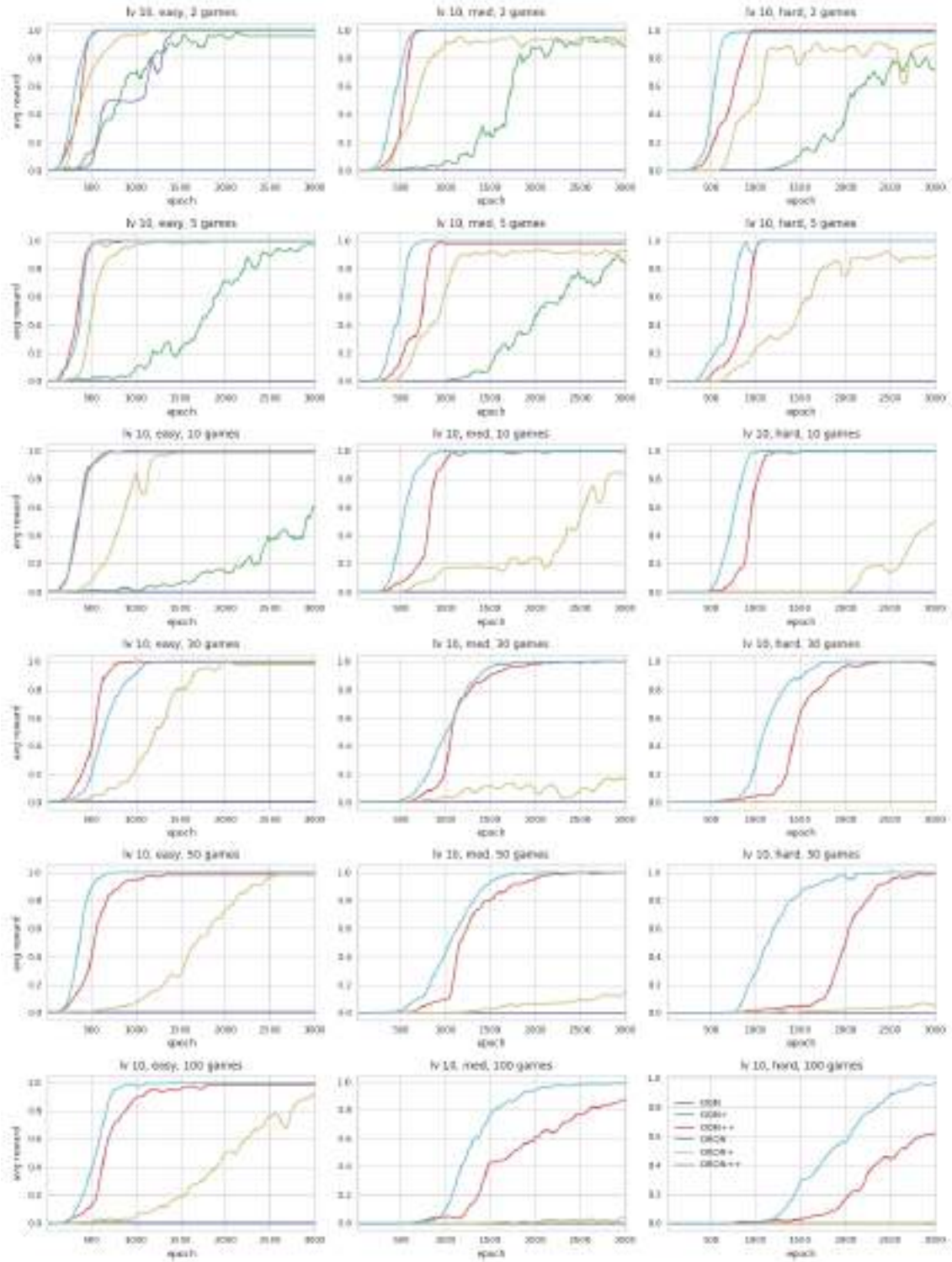


Figure 6. Model performance on multiple games.

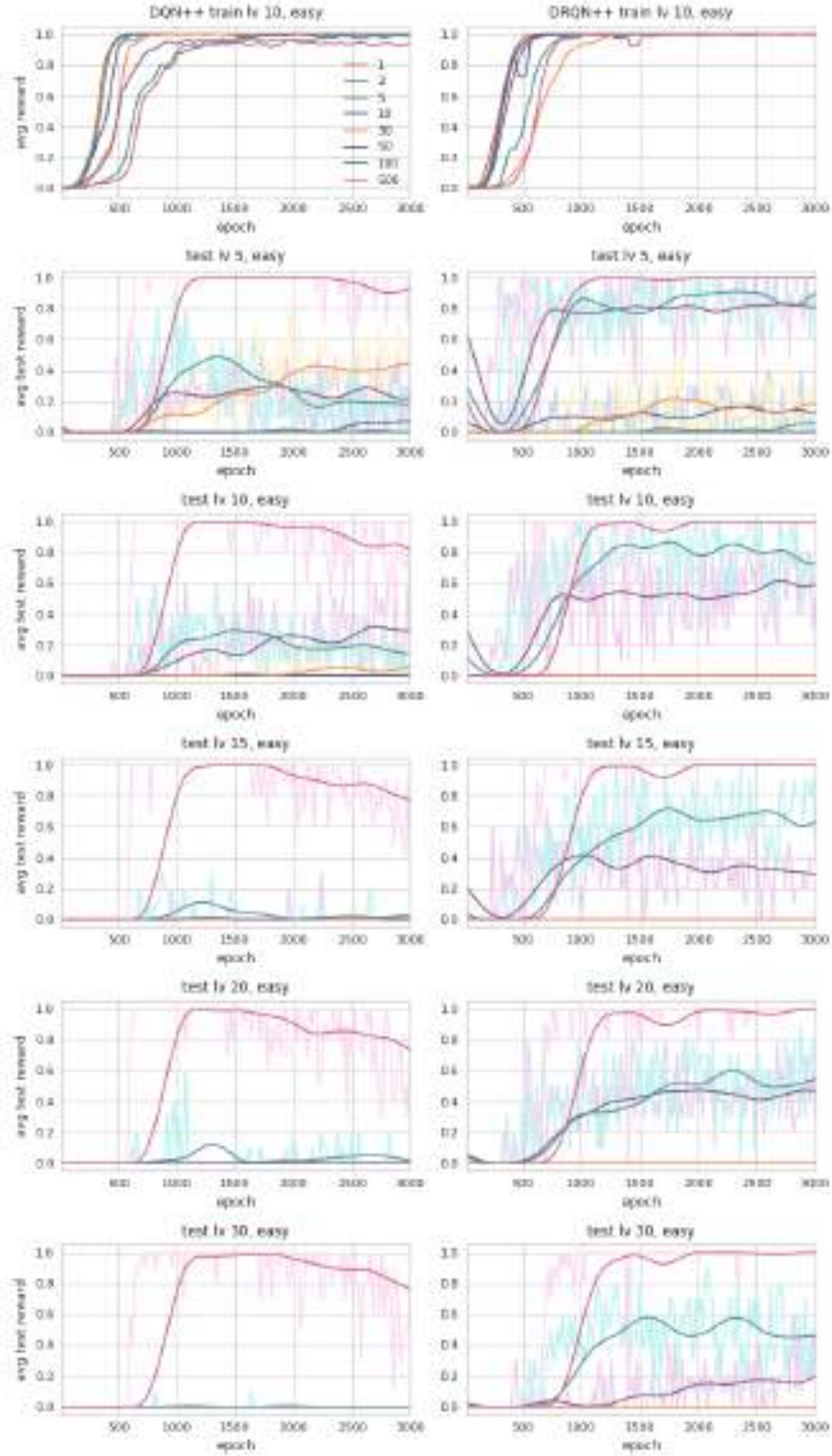


Figure 7. Model performance on unseen easy test games when pre-trained on easy games.

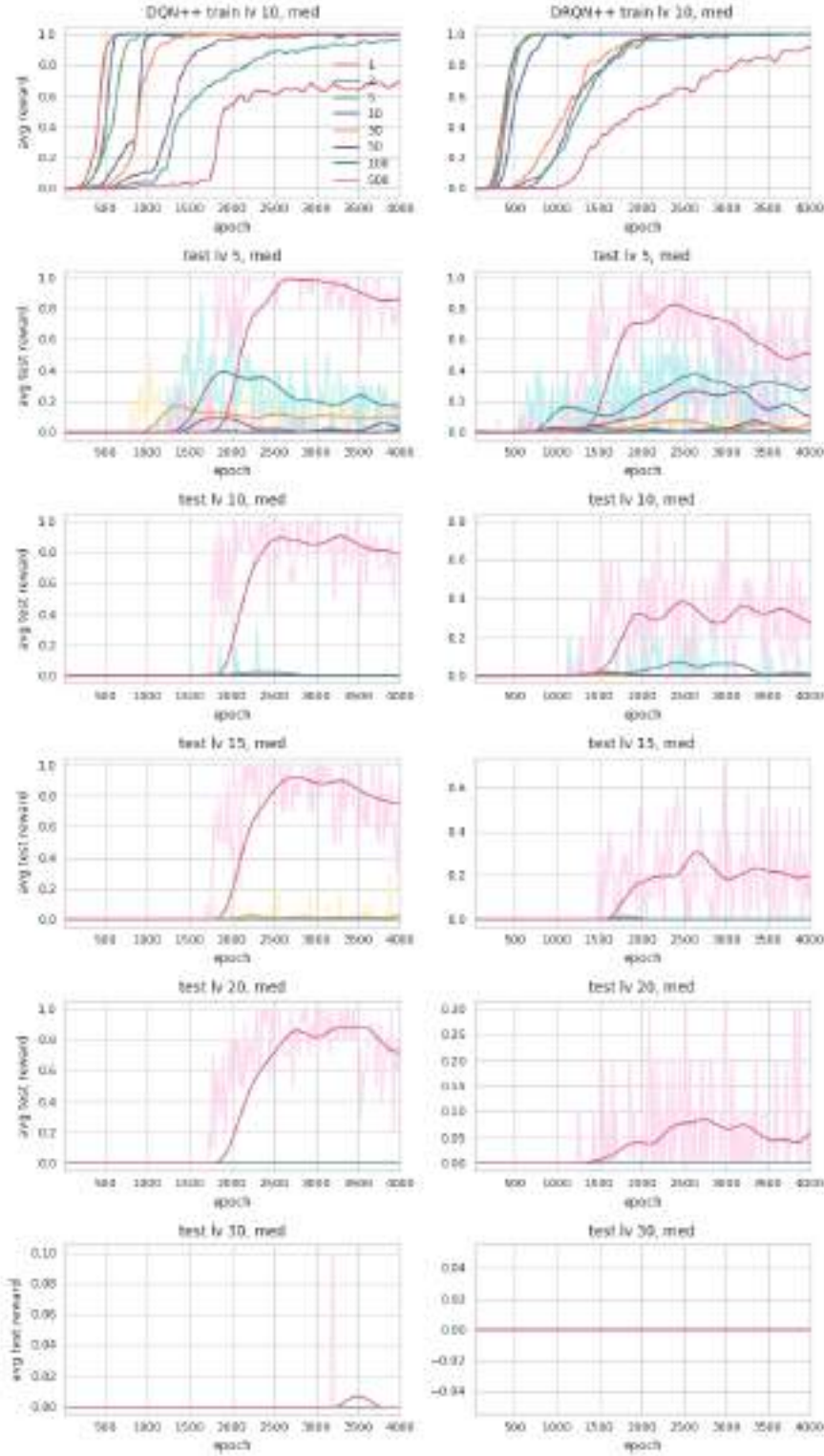


Figure 8. Model performance on unseen medium test games when pre-trained on medium games.

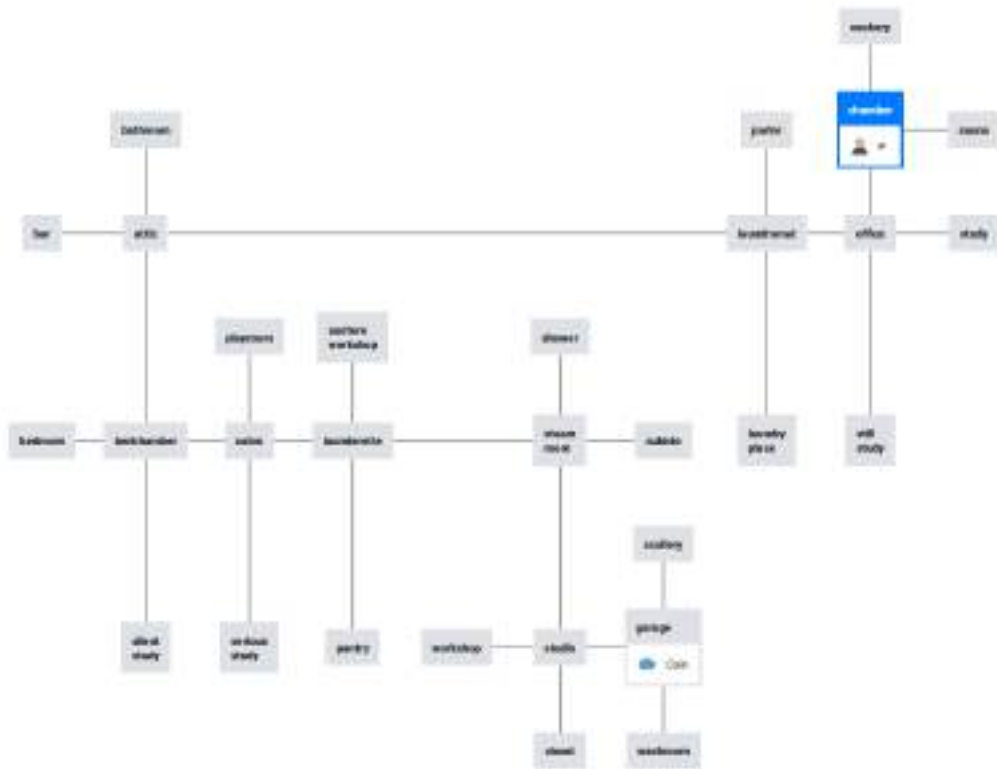


Figure 11. Examples of the games used in the experiments: level 10, hard

```
I hope your ready to go into rooms and interact with objects, because you've
just entered TextWorld! take the coin that's in the launderette.

== Cookhouse ==
Fancy seeing you here. Here, by the way, being the cookhouse.

There is an unguarded exit to the south.
> go south

== Study ==
You've entered a study. The room seems oddly familiar, as though it were only
superficially different from the other rooms in the building.

You need an unguarded exit? You should try going west. You need an unguarded
exit? You should try going north.
>
```

Figure 12. Text the agent gets to observe for one of the level 10 easy games.

Sentiment Classification using Bi-LSTM with Attention

This article post provides a guide to using Bi-LSTM and attention mechanisms for sentiment classification. It includes a hands-on implementation with code examples.

[Aman Arora](#)

Created on June 28 | Last edited on July 7

Introduction

This article post provides a comprehensive guide to using Bi-LSTM and attention mechanisms for sentiment classification. It includes a hands-on implementation of a Bi-LSTM model for tweet classification and demonstrates how to enhance the model with a soft attention mechanism.

The post concludes with a comparison of the two models, highlighting the improved performance achieved by incorporating attention. The complete code is available in the linked GitHub repository.

As part of this blog post, we are performing tweet classification using Bi-LSTM. We will be using [Cyberbullying Classification dataset](#) from Kaggle.

Here is what will be covered in this article:

Table of Contents

[Introduction](#)

[Table of Contents](#)

[Introduction to RNNs, LSTMs & Bi-LSTMs](#)

[Recurrent Neural Networks \(RNNs\)](#)

[LSTM:](#)

[Bi-LSTM:](#)

[Introduction to Attention Mechanisms](#)

↑ Share

💬 Comment

☆ 1 star

[What is Attention?](#)

[How does Attention work?](#)

[Types of Attention](#)

[Bi-LSTM for Tweet Classification with complete training & evaluation code + W&B for experiment tracking](#)

[Data Preparation](#)

[Sentiment Classifier](#)

[Model Training](#)

[Model Evaluation](#)

[Adding Soft Attention to Bi-LSTM for Tweet Classification](#)

[Sentiment Classifier with Attention](#)

[Conclusion](#)

Let's get going!

Introduction to RNNs, LSTMs & Bi-LSTMs

Recurrent Neural Networks (RNNs)

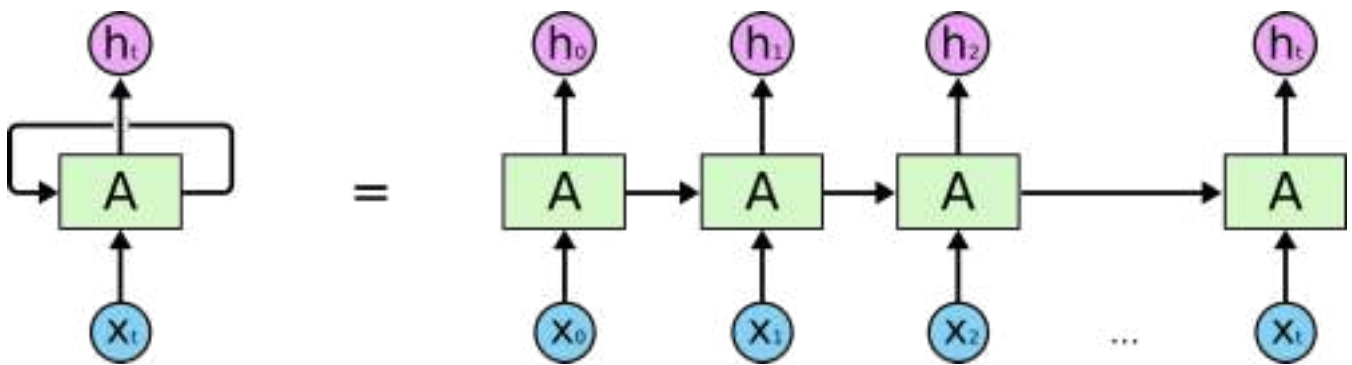
[Recurrent Neural Networks \(RNNs\)](#) are a type of neural network that have been successful in modeling sequential data. They have the unique feature of maintaining an internal state that can remember information about previous inputs in the sequence, making them particularly suited for tasks such as language modeling and time-series prediction.

RNNs were the go-to-network choice for many tasks until LSTMs & [Transformers](#) came along. Check out this blog by none other than Andrej Karpathy, titled “[The Unreasonable Effectiveness of Recurrent Neural Networks](#)”.

However, RNNs suffer from the vanishing gradient problem, which makes it difficult for them to learn long-range dependencies in the data.

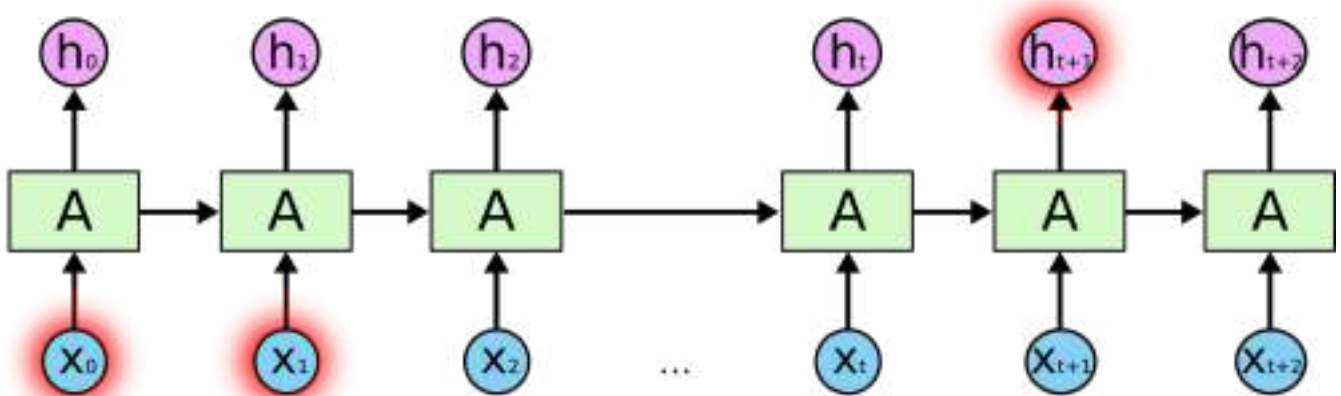
To further understand why, let's borrow a representation of RNN from one of the best blogs on RNNs & LSTMs, one called [***Understanding LSTM Networks](#), one of





Here x_0, x_1, x_2 are the first, second & third words respectively. x_t represents word at position t , so, as we can see in the image above, RNNs consume the sentence word-by-word while updating the hidden-state h_t at every step.

But, as we go further in the sequence, *as that gap grows, RNNs become unable to learn to connect the information.*



From Christopher Olah's blog:

In theory, RNNs are a good way to learn long-term dependencies. A human could carefully study toy problems of this form. Sadly, in practice, RNNs don't seem to be able to learn them.

Share

Comment

1 star

Suppose we have a sentence: "*I am feeling very good today because the weather is nice.*" We want to predict the sentiment of this sentence.

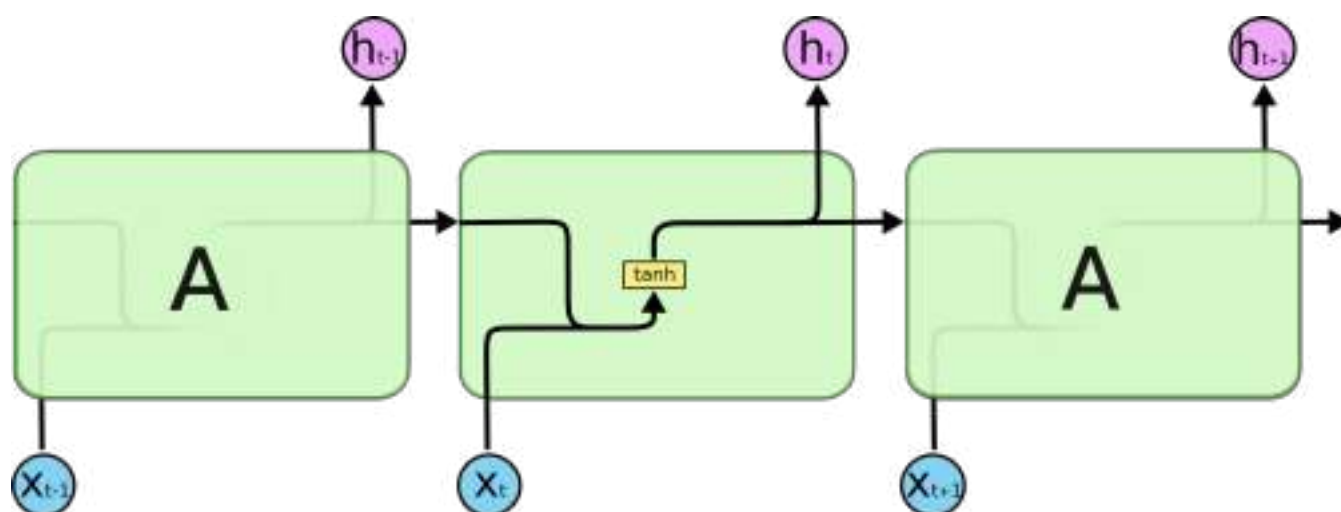
A standard RNN would process this sentence one word at a time, maintaining an internal state that captures the information about the words it has seen so far. For example, when it gets to the word "good", the RNN's state should contain information about the preceding word "I am feeling very". However, due to the vanishing gradient problem, RNNs tend to forget information about earlier words as the sentence gets longer. So, by the time it gets to "nice", it might have forgotten much of the context, which could lead to less accurate predictions.

LSTM:

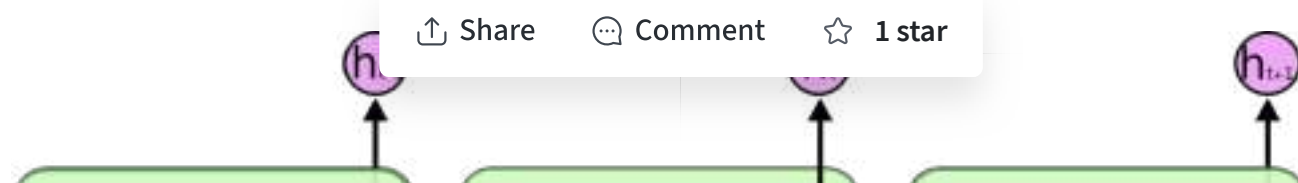
Long Short-Term Memory (LSTM) networks, a type of RNN, were designed to overcome this vanishing gradient problem. They introduce a memory cell that can maintain its state over time and gating units that regulate the flow of information into and out of the cell. This architecture allows LSTMs to learn longer sequences than traditional RNNs.

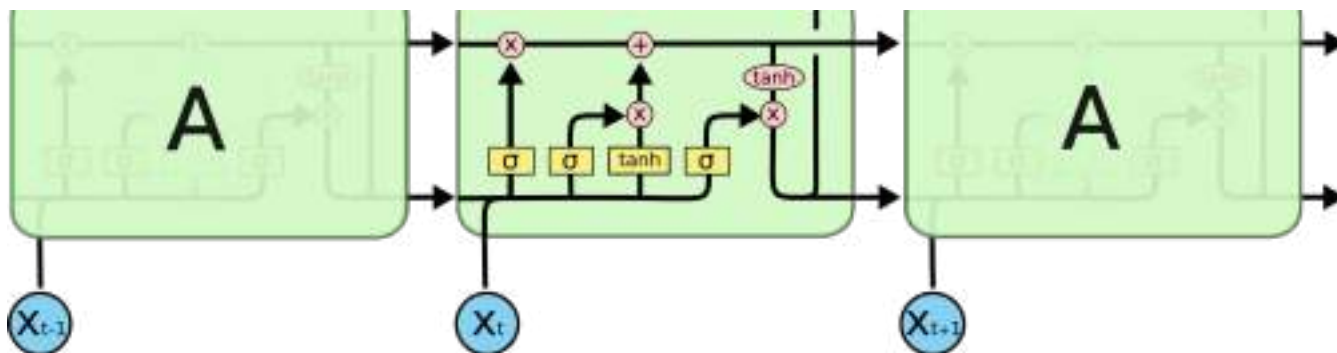
Again, borrowing from Christopher Olah's blog:

All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.



LSTMs also have this chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four, interacting in a very special way.





As part of this blog post, we won't be going through the inner workings of LSTMs, but I would refer the readers to [Christopher Olah's blog](#).

However, the main difference between RNNs and LSTMs is in this inner structure of repeating chains. The main idea is that LSTMs have logic gates that allow the network to keep or forget information as needed. Thus, they are able to maintain longer-term dependencies.

For the same example sentence, "*I am feeling very good today because the weather is nice.*"

When the LSTM processes the word "good," it not only updates its internal state but also updates a separate memory cell. *As mentioned before, the information in this cell is regulated by gating units, which decide what information to keep or forget.* This allows the LSTM to maintain a longer context, so when it gets to "nice", it still remembers the earlier parts of the sentence, leading to a more accurate prediction.

Bi-LSTM:

Bi-directional LSTMs (Bi-LSTMs) extend the idea of LSTMs by having two LSTMs in each layer. One LSTM processes the sequence from left to right (forward), and the other from right to left (backward). The outputs of both LSTMs are then concatenated. This allows the network to have access to past (from the forward LSTM) and future (from the backward LSTM) contexts at the same time, which can be very useful in many tasks.

For the same example sentence, "*I am feeling very good today because the weather is nice*" -one LSTM goes from "I" to "nice", and the other goes from "nice" to "I". The final representation for each word is then the concatenation of the two LSTM outputs. For example, the representation for "good" would contain information about both the preceding words "*I am feeling very*" and the following words "*today because the weather is nice*". Th

[Share](#)
[Comment](#)
[1 star](#)

understanding of the sentence, which can lead to even more accurate predictions.

Introduction to Attention Mechanisms

What is Attention?

"Attention" is inspired by the human visual attention mechanism, which allows us to focus on a part of a visual scene while perceiving the rest of it in lower resolution. Similarly, in neural networks, attention allows the model to focus on certain parts of the input when producing an output.

Let's take our example sentence: *"I am feeling very good today because the weather is nice"*. In a standard RNN or LSTM, each word in the sentence would contribute equally to the final prediction. However, with attention, the model can learn to focus more on the words "good" and "nice" when predicting the sentiment of the sentence, as these words are more indicative of the sentiment than the other words.

How does Attention work?

In the simplest form, an attention mechanism scores each item in the input sequence, and these scores are then used to weight the contribution of each item to the output. The scores are based on the current output and each item in the input sequence, allowing the model to focus on relevant inputs.

For instance, in our example sentence, the attention mechanism might assign higher scores to the words "good" and "nice", causing these words to have a greater impact on the final sentiment prediction.

Types of Attention

There are several types of attention mechanisms, including additive attention, multiplicative attention, and scaled dot-product attention (used in the Transformer model). Each of these mechanisms uses a different method to calculate the attention scores.

In Large Language Models (which are transformer based) such as ChatGPT, Claude, Bard, LLaMA, scaled-dot-product attention is used with some tips & tricks like [fast-attention](#), [AliBi](#) and more.

In this blog post, we will explore the concept of attention in the context of a Bi-LSTM model for sentiment analysis.

↑ Share

💬 Comment

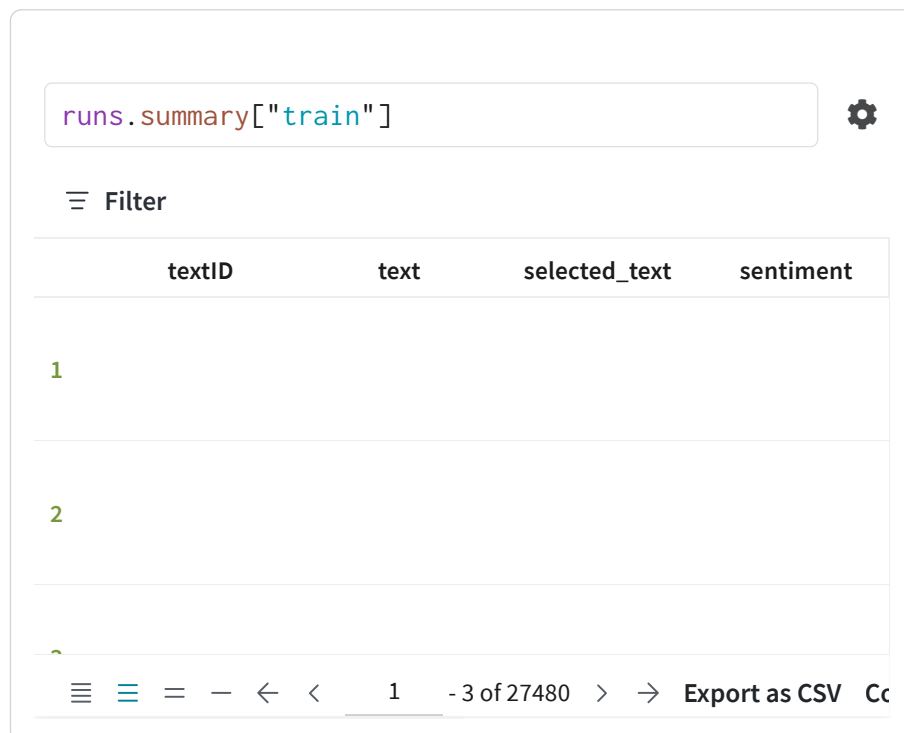
☆ 1 star

attention in the context

Bi-LSTM for Tweet Classification with complete training & evaluation code + W&B for experiment tracking

As part of this section, we will perform Tweet Classification using a Bi-LSTM in PyTorch. All code has been shared publicly for you to reproduce in [this repository](#).

But, before we can train our model, let's look at the dataset that we will be working with first. I have uploaded the dataset as a W&B table for you to experiment with.



textID	text	selected_text	sentiment
1			
2			

The table above is our training set, you can access the training and test set as W&B tables [here](#).

As part of this Tweet Classification task, we take the `text` column as input and `sentiment` column as output.

Data Preparation

Since we can't directly feed the data to Neural Networks, we will create a class called `TweetDataset`, that takes in the text as input, tokenizes it using `BertTokenizer` and returns a dictionary.

```
from transformers import BertTokenizer
from torch.utils.data import Dataset
import torch
```

↑ Share 💬 Comment ☆ 1 star

```
import pandas as pd
```

```
class TweetDataset(Dataset):
```

```
    def __init__(self, filename, maxlen):
```

```
        # Store the contents of the file in a pandas dataframe
```

```
        self.df = pd.read_csv(filename, delimiter=",")
```

```
        self.classes = ["negative", "neutral", "positive"]
```

```
        # Initialize the BERT tokenizer
```

```
        self.tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```
        self.maxlen = maxlen
```

```
    def __len__(self):
```

```
        return len(self.df)
```

```
    def __getitem__(self, index):
```

```
        # Selecting the sentence and label at the specified index in the data frame
```

```
        sentence = self.df.loc[index, "text"]
```

```
        label = torch.tensor(
```

```
            self.classes.index(self.df.loc[index, "sentiment"]), dtype=torch.long
```

```
        )
```

```
        # Preprocessing the text to be suitable for BERT
```

```
        input_ids = self.tokenizer(
```

```
            sentence,
```

```
            padding="max_length",
```

```
            truncation=True,
```

```
            return_tensors="pt",
```

```
            max_length=self.maxlen,
```

```
        )["input_ids"][0]
```

```
        attention_mask = self.tokenizer(
```

```
            sentence,
```

```
            padding="max_length",
```

```
            truncation=True,
```

```
            return_tensors="pt",
```

```
            max_length=self.maxlen,
```

```
        )["attention_mask"][0]
```

```
    return {
```

```
        "sentence": sentence,
```

```
        "input_ids": input_ids,
```

```
        "attention_mask": attention_mask,
```

↑ Share

💬 Comment

☆ 1 star

```
    "label": label,  
  }
```

This is what the output looks like for an example sentence “*Shanghai is also really exciting (precisely -- skyscrapers galore). Good tweeps in China: (SH) (BJ).*”:

```
{'sentence': 'Shanghai is also really exciting (precisely -- skyscrapers galore). G',  
  'input_ids': tensor([ 101, 8344, 2003, 2036, 2428, 10990, 1006, 10785, 1011,  
                        24581, 2015, 14891, 5686, 1007, 1012, 2204, 1056, 28394, 4523,  
                        1999, 2859, 1024, 1006, 14021, 1007, 1006, 1038, 3501, 1007,  
                        1012, 102, 0, 0, 0, 0, 0, 0, 0, 0,  
                        0, 0, 0, 0, 0, 0, 0, 0]),  
  'attention_mask': tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,  
                             1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]),  
  'label': tensor(1)}
```

Sentiment Classifier

Now, once we have the dataset ready, which outputs token IDs, we are ready to create our Bi-LSTM. PyTorch has already implemented the `nn.LSTM` class for us, to make it Bi-LSTM, all we need to do is to pass `bidirectional=True`.

```
class SentimentClassifier(nn.Module):
    def __init__(
        self,
        tokenizer,
        embedding_dim=128,
        hidden_dim=256,
        output_dim=3,
        n_layers=2,
        bidirectional=True,
        dropout=0.1,
    ):
        super().__init__()

        # Embedding layer
        self.embedding = nn.Embedding(len(tokenizer.get_vocab()), embedding_dim)

        # LSTM layer
        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim,
            num_layers=n_layers,
```

```

        bidirectional=bidirectional,
        dropout=dropout,
        batch_first=True,
    )

    # Dense layer
    self.fc = nn.Sequential(
        nn.Linear(hidden_dim * 2, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, output_dim),
    )

    # Activation function
    self.act = nn.Sigmoid()

def forward(self, text):
    # text = [batch size, sent_length]
    embedded = self.embedding(text)
    _, (hidden, _) = self.lstm(embedded)
    # hidden = [batch size, num layers * num directions, hid dim]

    # concat the final forward and backward hidden state
    hidden = torch.cat((hidden[-2, :, :], hidden[-1, :, :]), dim=1)

    # hidden = [batch size, hid dim * num directions]
    dense_outputs = self.fc(hidden)

    # Final activation function
    outputs = self.act(dense_outputs)

    return outputs

```

The model takes in outputs of the tokenizer, and converts them to embeddings using `self.embedding` which is a `nn.Embedding` class from vocab size to `embedding_dim`. Thus, each token gets converted to `embedding_dim` long vector.

Once, we have that, we pass the embedded tokens to `self.lstm`, and because `bidirectional=True`, therefore, this is a Bi-LSTM. Finally, we take the outputs and concatenate the forward and backward hidden states. (Can you think of why?)

We concatenate forward and backward hidden states of two lstms in each direction - forward and backward, as mentioned in the section



before, the outputs of both LSTMs are then concatenated. This allows the network to have access to past (from the forward LSTM) and future (from the backward LSTM) context at the

↑ Share

💬 Comment

☆ 1 star

same time.

Model Training

Next up, we need to define a training function that can take the data and model, and train our model. We will use `nn.CrossEntropyLoss` as our loss function, because we are performing text classification.

```
def train_one_epoch(model, data_loader, criterion, optimizer, device):
    model.train()
    epoch_loss = 0
    epoch_accuracy = 0

    for i, batch in tqdm(enumerate(data_loader), total=len(data_loader)):
        input_ids = batch["input_ids"]
        attention_mask = batch["attention_mask"]
        labels = batch["label"]

        # Move tensors to GPU
        input_ids = input_ids.to(device)
        attention_mask = attention_mask.to(device)
        labels = labels.to(device)

        # Forward pass
        outputs = model(input_ids)
        accuracy = (outputs.argmax(dim=1) == labels).float().mean()
        # Compute loss
        loss = criterion(outputs, labels)

        # Backward pass and optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_accuracy

    return epoch_loss
```

↑ Share

💬 Comment

☆ 1 star

The training function above takes the outputs from the tokenizer - `input_ids` and

passes them to the Sentiment Classifier `model` to get outputs of shape `batch_size, 3` because we have 3 labels - “positive”, “neutral,” & “negative”.

We keep track of loss and accuracy and return the mean of both metrics, which becomes our `epoch_loss` and `epoch_accuracy`.

Model Evaluation

Same as above, we define an evaluation function that takes in the model.

```
def evaluate_one_epoch(model, data_loader, criterion, device):
    model.eval()
    epoch_loss = 0
    epoch_accuracy = 0

    with torch.no_grad():
        for i, batch in tqdm(enumerate(data_loader), total=len(data_loader)):
            input_ids = batch["input_ids"]
            attention_mask = batch["attention_mask"]
            labels = batch["label"]

            # Move tensors to GPU
            input_ids = input_ids.to(device)
            attention_mask = attention_mask.to(device)
            labels = labels.to(device)

            # Forward pass
            outputs = model(input_ids)
            accuracy = (outputs.argmax(dim=1) == labels).float().mean()

            # Compute loss
            loss = criterion(outputs, labels)

            epoch_loss += loss.item()
            epoch_accuracy += accuracy.item()

    return epoch_loss / len(data_loader), epoch_accuracy / len(data_loader)
```

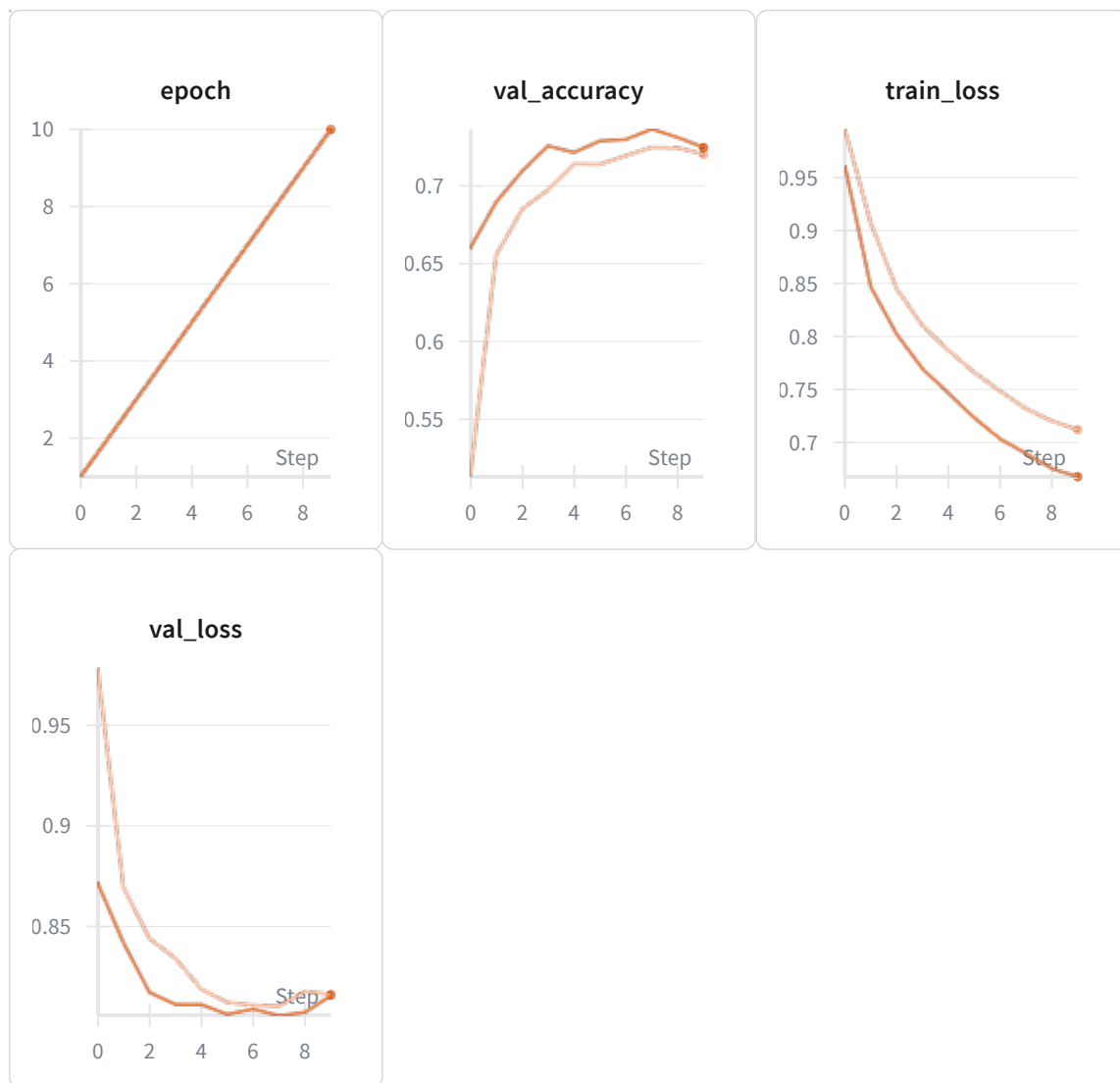
The function above takes in the test data loader and calculates epoch loss and accuracy, which are then returned.

[↑ Share](#) [💬 Comment](#) [★ 1 star](#)

To train and evaluate the model, run the following commands:

```
git clone https://github.com/amaarora/attention_lstm
cd attention_lstm/src/
python main.py
```

This will kickoff a training run on the Tweet Sentiment dataset and also create a Weights and Biases dashboard.



As can be seen above, we can track the validation loss and accuracy for our Sentiment Classifier which uses a Bi-LSTM. The model achieves 72% accuracy on the validation dataset.

Adding Soft Attention to Bi-LSTM for Tweet Classification

How can we further improve our model? Let's add attention. Remember, attention will help the model to focus on the right parts of the sentence.

For example, in the sentence “*I really like the song Love Story by Taylor Swift*”, the model would want to pay more attention to the word “*like*” when classifying the sentiment of this sentence.

```
{'sentence': 'I really really like the song Love Story by Taylor Swift',
 'input_ids': tensor([ 101, 1045, 2428, 2428, 2066, 1996, 2299, 2293, 2466, 2011, 4104,
                      102,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
                      0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,
                      0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0,   0]),
 'attention_mask': tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
                           0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]),
 'label': tensor(2)}
```

So, how can we achieve this? Simple - we must allow the model to add weights to each separate token. By doing so, the model can automatically give more weights to the tokens for the word “like,” and therefore, it should help increase the performance of the model.

Sentiment Classifier with Attention

Let’s now define our model that has the capability to attend to the right parts of the sentence through an attention mechanism.

First, let’s write a class to define the attention mechanism.

```
class Attention(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.attn = nn.Linear(hidden_dim, 1, bias=False)

    def forward(self, lstm_output):
        # lstm_output = [batch size, seq_len, hidden_dim]
        attention_scores = self.attn(lstm_output)
        # attention_scores = [batch size, seq_len, 1]
        attention_scores = attention_scores.squeeze(2)
        # attention_scores = [batch size, seq_len]
        return F.softmax(attention_scores, dim=1)
```

Above, we have a simple `nn.Linear` layer that has shape `[hidden_dim, 1]`, since each token gets represented by a single score. Thereby, [↑ Share](#) [💬 Comment](#) [☆ 1 star](#) value and convert it to an attention score. This will allow the model to give high scores to words such as “like”

when classifying “*I really like the song Love Story by Taylor Swift*”.

Let’s now update our Sentiment Classifier model from before to add an attention mechanism to it.

```
class SentimentClassifierWithSoftAttention(nn.Module):
    def __init__(
        self,
        tokenizer,
        embedding_dim=128,
        hidden_dim=256,
        output_dim=3,
        n_layers=2,
        bidirectional=True,
        dropout=0.1,
    ):
        super().__init__()

        # Embedding layer
        self.embedding = nn.Embedding(len(tokenizer.get_vocab()), embedding_dim)

        # attention
        self.attention = Attention(hidden_dim * 2 if bidirectional else hidden_dim)

        # LSTM layer
        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim,
            num_layers=n_layers,
            bidirectional=bidirectional,
            dropout=dropout,
            batch_first=True,
        )

        # Dense layer
        self.fc = nn.Sequential(
            nn.Linear(hidden_dim * 2, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, output_dim),
        )

        # Activation f
        self.act = nn.Sigmoid()

    def forward(self, text):
```

[↑ Share](#) [💬 Comment](#) [★ 1 star](#)

```

# text = [batch size, sent_length]
embedded = self.embedding(text)
lstm_output, (hidden, _) = self.lstm(embedded)
# lstm_output = [batch size, seq_len, hidden_dim*num_directions]
# concat the final forward and backward hidden state
hidden = torch.cat((hidden[-2, :, :], hidden[-1, :, :]), dim=1)
attention_weights = self.attention(lstm_output)
# attention_weights = [batch size, seq_len]
attention_weights = attention_weights.unsqueeze(2)
weighted = lstm_output * attention_weights
# weighted = [batch size, seq_len, hidden_dim]

weighted_sum = weighted.sum(dim=1)
# weighted_sum = [batch size, hidden_dim]

dense_outputs = self.fc(weighted_sum)
# dense_outputs = [batch size, output_dim]

# Final activation function
outputs = self.act(dense_outputs)

return outputs

```

Most of the code is the same as before, except that we now take the `lstm_output` and create a score for each token in the sentence using attention - `attention_weights = self.attention(lstm_output)`. This way, each token gets a score. Finally, we multiply the `lstm_output` with `attention_weights` to get weighted outputs - allowing the model to give higher weights to more relevant tokens and lower weights to less relevant tokens.

We can train and evaluate this model using the training and evaluation script as before.

To train and evaluate on your own machine, run the following commands:

```

git clone https://github.com/amaarora/attention_lstm
cd attention_lstm/src/
python main.py

```

But, in this case, make :

SentimentClassifierWi

↑ Share

💬 Comment

☆ 1 star

Finally, we can compare the two models since we used Weights & Biases for

experiment tracking. From the dashboard we saw before, our Sentiment Classifier model with attention achieves a higher validation accuracy and a much lower training loss. Thus, adding attention to the model helped the model fit better to the dataset, thereby improving the performance of the model.

Remember, this improvement for the model came by allowing the model to be able to give more weights to relevant tokens when classifying tweets. From “How does attention work”



section of our blog post:

”In the simplest form, an attention mechanism scores each item in the input sequence, and these scores are then used to weight the contribution of each item to the output.”

Conclusion

In this blog post, we have covered the basics of RNNs, LSTMs, and Bi-LSTMs, and introduced the concept of attention mechanisms. We have implemented a Bi-LSTM model for tweet classification and enhanced it with soft attention. We have also provided complete training and evaluation code and used the WandB library for experiment tracking, which can be found in [this GitHub repository](#).

We hope that this post has provided you with a good understanding of how to use LSTMs and attention mechanisms for text classification tasks. As always, we encourage you to experiment with the code and try out different architectures and attention mechanisms to see what works best for your specific task. Happy coding!



Tags: Sentiment Analysis, NLP, Articles, Kaggle, Intermediate

New course:
Build production-grade AI agents

ENROLL NOW →

Created with on Weights & Biases.

<https://wandb.ai/amanarora/Written-Reports/reports/Sentiment-Classification-using-Bi-LSTM-with-Attention--VmllDzo0NzUxNjA5>



Share



Comment



1 star



Contents lists available at ScienceDirect

Atmospheric Pollution Research

journal homepage: www.elsevier.com/locate/apr

Deep learning coupled model based on TCN-LSTM for particulate matter concentration prediction

Ying Ren, Siyuan Wang, Bisheng Xia^{*}

School of Mathematics and Computer Science, Yan'an University, Yan'an, 716000, China

ARTICLE INFO

Keywords:

Air pollution

Particulate matter prediction

Temporal convolutional network

Long short-term memory network model

Sensitivity analysis

ABSTRACT

In this study, we combined the Temporal Convolutional Network (TCN) model with the Long Short-Term Memory (LSTM) network model and applied it to prediction of atmospheric particulate matter (PM) concentrations. The study area is Xi'an City, Shaanxi Province, and the study period is from January 2015 to July 2022. During this period, Xi'an exceeded China's National Ambient Air Quality Grade I standard for PM for up to 70% of the days. The prediction results of the TCN-LSTM model were compared with those of deep learning models (Convolutional Neural Network-LSTM, TCN, and LSTM) and machine learning models (Support Vector Regression and Random Forest). The R^2 values of the TCN-LSTM model were all >0.88 , indicating better performance than that of the other five models, and the errors of the TCN-LSTM model were all lower than those of the other five models. The results showed that high-accuracy PM predictions using deep learning models can improve air quality monitoring by compensating for problems in the environmental monitoring process such as pollutant monitoring errors caused by instrument failures. Additionally, sensitivity analysis helps to identify the key factors influencing the behavior of PM. A sensitivity analysis of PM for different periods of COVID-19 found that $PM_{2.5}$ is more sensitive to O_3 , while PM_{10} is mainly influenced by $PM_{2.5}$. The sensitivity analysis for the whole period showed that PM was closely related to CO. Removing variables that do not contribute to the model output based on the sensitivity analysis results improves modeling efficiency while reducing operating costs and improving environmental monitoring activities and management strategies.

1. Introduction

Atmospheric particulate matter (PM), one of the main factors in the occurrence of haze weather, is receiving increasing attention globally. The PM in the atmosphere mainly includes $PM_{2.5}$ (PM with an aerodynamic diameter of $<2.5 \mu m$) and PM_{10} (PM with an aerodynamic diameter of $<10 \mu m$) (Samet and Medicine, 2022), which comprises mainly water-soluble ions, PM, and organic matter. The sources of PM can be divided into two main categories: natural sources and artificial sources (Yang et al., 2013). Natural sources mainly include the weathering of rocks and soils, forest fires, volcanic eruptions, and sea salt particles, while artificial sources mainly include vehicle exhaust emissions, industrial emissions, construction site dust, and waste incineration. Increase in the concentration of atmospheric PM can lead to increased probability of respiratory disease (Wang et al., 2021; Xing et al., 2016) and cardiovascular disease (Gutierrez-Avila et al., 2018; Lin et al., 2017; Santibañez et al., 2013) in humans, and also drive change in

the global climate (Nguyen et al., 2019; Tai et al., 2010; Westervelt et al., 2016). Accurate and efficient prediction of PM has a nonnegligible role in both haze weather management and sustainable human development.

The main methods currently used for prediction of atmospheric PM comprise physically based mechanistic models and data-driven statistical models. Mechanistic models mainly include the Community Multi-scale Air Quality model (Foley et al., 2010; Otte et al., 2005), Operational Street Pollution Model (Wang and Xie, 2009), and Nested Air Quality Prediction Model (Wang et al., 2014b). The regional atmospheric dynamics Weather Research and Forecasting model coupled with chemistry (Chuang et al., 2011; Grell et al., 2005) is used widely to simulate the formation and dispersion of PM. However, the use of such mechanistic models is based on assumptions of specific conditions and the representation of reality is often variable. The default parameter settings limit the performance of mechanistic models, resulting in lower accuracy than that of site-specific empirical air quality prediction

Peer review under responsibility of Turkish National Committee for Air Pollution Research and Control.

^{*} Corresponding author.E-mail addresses: 1404917692@qq.com (Y. Ren), 1054265529@qq.com (S. Wang), bishengxia@163.com (B. Xia).<https://doi.org/10.1016/j.apr.2023.101703>

Received 12 November 2022; Received in revised form 7 February 2023; Accepted 3 March 2023

Available online 4 March 2023

1309-1042/© 2023 Turkish National Committee for Air Pollution Research and Control. Production and hosting by Elsevier B.V. All rights reserved.

models.

Traditional statistical models do not use complex physical parameters, but predict air pollutant concentrations based on historical data. For example, multiple linear regression (Ausati and Amanollahi, 2016; Gupta and Christopher, 2009), autoregressive integrated moving average (Díaz-Robles et al., 2008; Jian et al., 2012), and geographically and temporally weighted regression models (He and Huang, 2018; Xiao et al., 2018) treat the relationship between variables as linear, contrary to most real-world situations. To address this problem, many researchers have predicted concentrations of atmospheric pollutants using nonlinear machine learning techniques such as the Artificial Neural Network (Feng et al., 2015; Perez and Reyes, 2006; Zhou et al., 2014), Random Forest (RF) (Liu and Sun, 2019; Shamsoddini et al., 2017) and Support Vector Regression (SVR) (Ahmad et al., 2020; Bui et al., 2019; Sun and Sun, 2017) methods. However, the feature extraction capability and learning ability of such models approaches decrease as the amount of data and model training time increase, thereby leading to reduction in model prediction accuracy. More importantly, such models do not have storage units and cannot account for the temporal characteristics of contaminant data (Shen et al., 2020). Some scholars have applied machine learning models in combination with other models for the study of PM, such as a hybrid model combining XGBoost, four GARCH models and MLP models (XGBoost GARCH MLP) (Dai et al., 2022b), and a combination of land use regression and light gradient enhancement (LUR-GBM) (Dai et al., 2022a). The idea of these combined models provides a new idea for the study of atmospheric PM.

In recent years, with the rise of deep learning, many studies have applied such techniques to the environmental field (Lamba et al., 2019; Lee et al., 2017), among which the most widely used deep learning approaches are Recurrent Neural Network (RNN) models and Convolutional Neural Network (CNN) models. Owing to their unique self-looping structure, which is suitable for processing sequential data, RNNs have been applied to predict PM concentrations (Feng et al., 2019; Ong et al., 2016). However, RNN models have long-term dependency and vanishing gradient problems, which are well resolved by the derived Long Short-Term Memory (LSTM) network model. Application of the LSTM approach in environmental prediction is often combined with a CNN model, which can process information in parallel and has powerful feature extraction capabilities. Thus, a CNN-LSTM model is suitable for prediction of air quality indexes (Yan et al., 2021; Zhang and Li, 2022), O₃ (Pak et al., 2018; Yafouz et al., 2021), and other atmospheric substances. However, the CNN structure is complex and might lead to information loss when the dimensions of the input and output information differ. To address this problem, Bai et al. (2018) proposed a Temporal Convolutional Network (TCN). The TCN has both the parallelism feature of CNNs and the memory function of RNNs. The TCN can perform rapid feature extraction for complex sequence data and ensures that the dimensions of the input and output information are the same. Its excellent feature extraction capability is exploited widely in fields such as severe weather forecasting (Yan et al., 2020), short-term traffic prediction (Zhao et al., 2019a), multivariate time series prediction (Fan et al., 2021), and computer vision (Hou et al., 2018), and all such applications have reported satisfactory results.

The performance of a deep learning model is affected by the inputs (Zhao et al., 2019b), and identifying the critical input variables of such models is beneficial to improving prediction efficiency and reducing modeling costs (Carta et al., 2020). More importantly, the formation of atmospheric pollutants is complex, and deep learning models cannot identify the dominant factors of pollutants. Sensitivity analysis methods are used in the environmental domain to identify key variables that affect model output, reduce model inputs, and avoid parameter redundancy (Gan et al., 2014), while analyzing the uncertain relationship between model input variables and the target output to increase model applicability (Makler-Pick et al., 2011; Zhan et al., 2013). Sensitivity analysis combined with deep learning models can not only help the model remove unnecessary inputs but also interpret the target output

(Samek et al., 2017) of the model to reveal the effects of different influencing factors on urban air pollutants.

Based on the above analysis, this study combines the TCN and LSTM models to design a new particle concentration prediction model TCN-LSTM model. The main works of this paper are:

- (1) A hybrid TCN-LSTM model was constructed, and low-cost, readily available meteorological factors and pollutant concentration data were selected for modeling. The TCN model extracted features from the influence factors of PM. The LSTM model learned the extracted features to predict the PM concentration in the atmosphere of Xi'an city. The prediction results compared with classical deep learning models (CNN-LSTM, LSTM, TCN) and machine learning (SVR and RF) were compared and validated by RMSE, MSE, MAE, and R² metrics, and the results showed that TCN-LSTM performed better. In addition, the TCN-LSTM model was also used to predict the PM concentrations in Beijing, Shanghai, Chengdu, and Shenzhen, and the model still performed well. In the cross-validated experiments, the TCN-LSTM model still performs stably;
- (2) The effects of each influencing factor on PM concentrations in Xi'an before (2015–2019) and after the outbreak (2020–2022) were studied using sensitivity analysis and compared with the results of sensitivity analysis for the whole period (2015–2022);
- (3) Using the sensitivity analysis methods to assess the magnitude of the contribution of the model's input variables to the target output. The parameters that contribute more to the model output are used to build a fast prediction model for PM concentration and reduce the modeling cost.

The rest of the paper is organized as follows. Section 2 describes the study area, data sources, and methods used in this study. Section 3 presents the concentration trends and model prediction results for PM, as well as the sensitivity analysis results. Section 4 discusses the model prediction performance as well as the results of the sensitivity analysis, and Section 5 summarizes the study.

2. Materials and methods

2.1. Study area and data sources

2.1.1. Geographical location of Xi'an

Xi'an is the capital city of Shaanxi Province in China (Fig. 1). Xi'an lies in the central region of the Weihe Basin with the Loess Plateau to the north and the Qinling Mountains to the south. In Xi'an, northeasterly winds prevail, and a gap in the northeastern part of the Weihe Basin allows these northeasterly winds to transport pollutants from North China into the Weihe Basin (Meng and Sun, 2021), which cause poor air quality in Xi'an. As a new first-tier city with national priorities, the increased population, industrial development, and road and subway construction in the region all contribute to higher concentrations of atmospheric PM that have adverse impact on the urban environment.

2.1.2. Variables

The pollutant data were obtained from the National Real-Time Urban Air Quality Release Platform of the China General Environmental Monitoring Station (<https://air.cnemc.cn:18014/>). The platform currently contains data of pollutants that include PM_{2.5}, PM₁₀, SO₂, NO₂, O₃, and CO for 367 cities in China, which have been acquired since December 2013. Fig. 2 shows the correlation of the six pollutants, with the highest correlation between PM_{2.5} and PM₁₀, with a Pearson coefficient of 0.86. Both PM_{2.5} and PM₁₀ are PM in the atmosphere and are similar. Therefore, it is sufficient to use one model for PM studies.

The meteorological data, obtained from NASA's MERRA-2 Native Resolution Daily Data, access the platform's data through the link <https://power.larc.nasa.gov/data-access-viewer/>, included wind speed (WS),

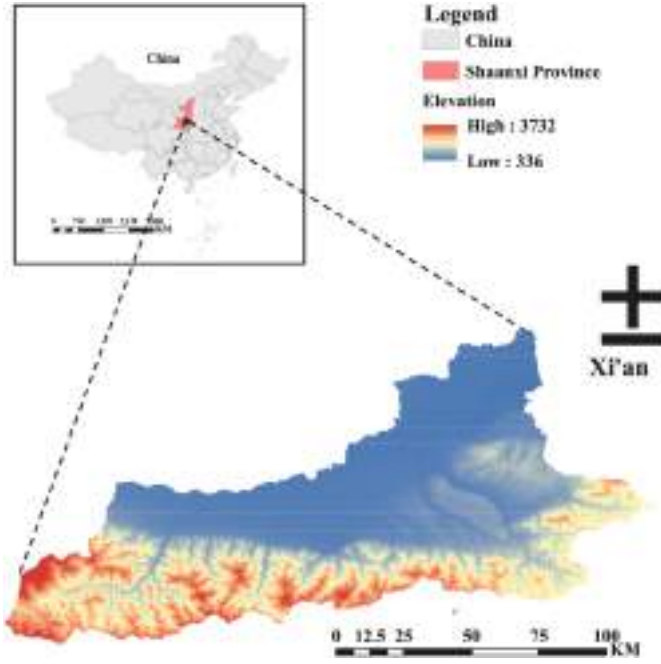


Fig. 1. Study area.

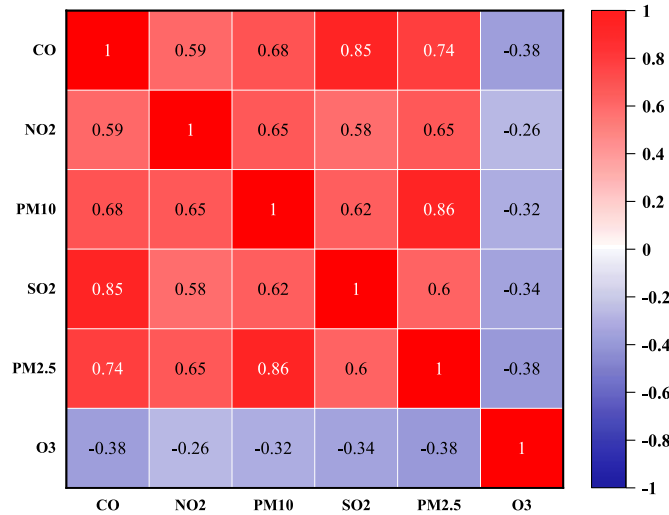


Fig. 2. Pollutant correlation analysis.

wind direction (WD), relative humidity (RH), precipitation (PR), pressure (P) temperature (T), dew point (DP), and solar irradiance (SI).

In this study, the PM indicators (PM_{2.5} and PM₁₀) were used as output variables and other indicators were used as input variables. Data from January 1, 2015, to July 9, 2022, were selected for the study. Errors attributable to dimensional differences were eliminated by normalizing each variable (Except for wind direction, wind direction data to one-hot code) using the following normalization formula:

$$v_i = \frac{V_i - \min(V_i)}{\max(V_i) - \min(V_i)} \quad (1)$$

where V_i is the normalized variable value, and $\max(V_i)$ and $\min(V_i)$ are the maximum and minimum values of the variable, respectively.

2.2. Methods

2.2.1. TCN model

The TCN model is a simple and general CNN architecture for solving time series problems, which consists primarily of a stack of multiple residual units. The residual module consists of two convolutional units and one nonlinear mapping unit, as shown in Fig. 3(a). Each convolutional unit includes a one-dimensional dilated causal convolution, weight normalization, ReLU activation function, and Dropout operation. The dilated causal convolution, shown in Fig. 3(b), is where the network's output is obtained by convolving the inputs with causal relations. In sequential modeling, causal convolution is expressed as the current moment t being related only to the input at moment t and before moment t . It is described in mathematical form as follows:

$$y_t = f(x_1, x_2, \dots, x_t) \quad (2)$$

where x_t is a one-dimensional vector containing n features, and y_t is the variable to be predicted. There is some connection between x_t and y_t , and f is the function that establishes the relationship between x_t and y_t . The causal convolution is a one-way structure that guarantees the sequential nature of the processing data by processing the values at moment t and using only the data before moment t for processing. For a one-dimensional sequence input, the dilation convolution operation $F_{(s)}$ on its elements and filter $f: \{0, \dots, k-1\}$ is defined as follows:

$$F_{(s)} = (x * df)(s) = \sum_{i=0}^{k-1} f(i) \bullet x_{s-di} \quad (3)$$

where s is the input sequence information, d is the dilation coefficient, k is the filter size, and $s-di$ is the localization of certain information in the history. The dilation coefficient $d = (1, 2, 4)$, as shown in Fig. 3(b), and as d increases, the perceptual field of the TCN increases and the amount of received information increases.

Once the dilated causal convolution is completed, the weights of each layer are normalized using WeightNorm, the activation function ReLU is used for nonlinear computation, and the Dropout operation controls the probability of dropping neurons to deactivate some neurons to avoid overfitting. If the input and output vectors of the residual blocks are of different shapes, the dimensionality of the input information is changed using the one-dimensional convolution, which drives the network to produce output with the same dimensions as that of the input.

2.2.2. LSTM model

The LSTM model is a modification of the traditional RNN model. The structure of the LSTM model consists primarily of a forget gate f_t , input gate i_t , output gate O_t , and unit state C (Fig. 3(c)). The forget gate retains useful information and forgets useless historical information, the input gate controls the degree of retention of the current cell state information, and the output gate determines the output information of the network. When the input of the model is x_t , the LSTM calculates the output h_t of the network using Eqs. (4)–(9):

$$F_{(s)} = (x * df)(s) = \sum_{i=0}^{k-1} f(i) \bullet x_{s-di} \quad (4)$$

$$i_t = \sigma(W_i \bullet [h_{t-1}, x_t] + b_i) \quad (5)$$

$$\tilde{C}_t = \tanh(W_c \bullet [h_{t-1}, x_t] + b_c) \quad (6)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (7)$$

$$O_t = \sigma(W_o \bullet [h_{t-1}, x_t] + b_o) \quad (8)$$

$$h_t = O_t * \tanh(C_t) \quad (9)$$

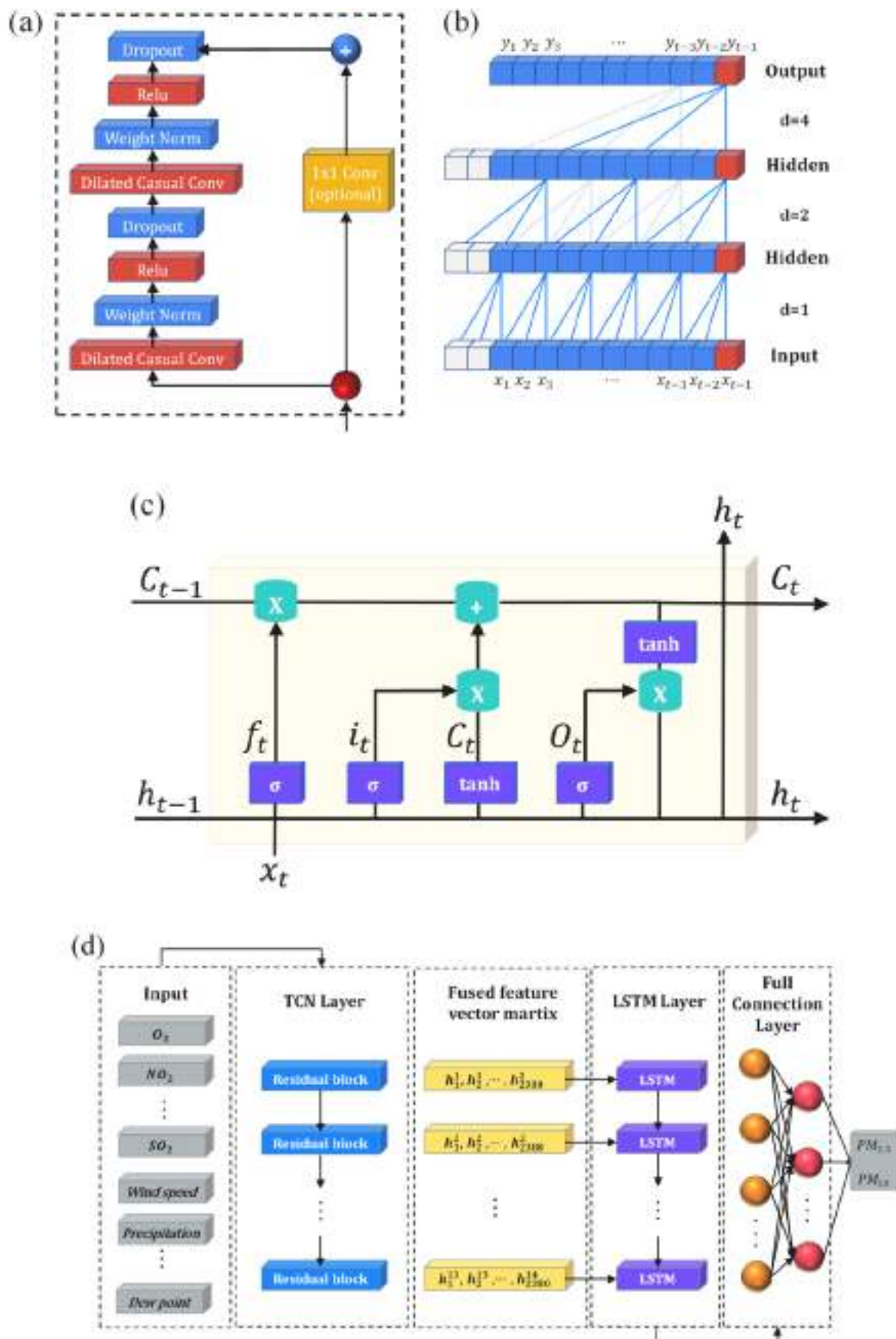


Fig. 3. Diagram of the model structure: (a) TCN model, (b) dilated causal convolution, (c) LSTM, and (d) TCN-LSTM.

where f_t , i_t , and O_t denote the forget gate, input gate, and output gate, respectively; $\tilde{C}_t$ is the state of the cell being updated; C_t is the state of the cell at the current moment; σ and $\tanh$ represent the Sigmoid and hyperbolic tangent activation functions, respectively; W_f , W_i , W_o , and W_c are the weight matrices of the LSTM model; b_f , b_i , b_o , and b_c are the bias matrices of the LSTM model; $[\]$ represents the connection of the two matrices; and $*$ represents the product of the matrices.

2.2.3. Model for PM concentration prediction based on TCN-LSTM

The unique one-dimensional causal convolution structure of TCN ensures the time series characteristics of the data, the residual connection unit accelerates the convergence speed of the network, and the expansion convolution ensures the extraction of all data features. The LSTM model, as a variant of the classical RNN, has a nonlinear solid fitting capability that makes it suitable for dealing with sequential modeling problems. Prediction of PM concentration is a time series prediction problem with nonlinear characteristics. The factors affecting the concentration of PM are the increase/decrease of the concentration of particles themselves, the content of other pollutants in the atmosphere, and meteorological factors. This study combined the TCN model with the LSTM model to build a TCN-LSTM model for predicting the concentration of atmospheric PM considering the formation of multiple factors and the nonlinear characteristics of particles. The architecture of the TCN-LSTM model is shown in Fig. 3(d), and the primary process of predicting PM concentration is as described in the following.

- (1) The acquired meteorological and pollutant data are processed and divided into a test set and a training set according to a specific ratio. The training set is used to tune the hyperparameters of the model. The training set is used as input for the TCN model, and the TCN model performs feature extraction on the input information. After the TCN model has extracted the data, the impurities of the data will be substantially reduced and the features will be exposed more obviously, which is conducive for the learning of the LSTM model.
- (2) The features extracted by the TCN model are input to the LSTM model, which controls the output of the network through the forget gates, input gates, and output gates, remembering critical information that requires long-term memory and forgetting unimportant information, which enables the model to process long-term serial data and accurately predict the concentration of PM at the next moment.
- (3) The prediction target of the training set is compared with the prediction result, the loss of the training set is calculated, the loss is determined to be minimized, and the model parameters are determined to ensure that the model is optimally tuned. The test set is used as input for the model to verify that the loss is minimized and to complete the prediction task.

The program used in this study was written in Python language and based on the TensorFlow 2.0 framework. The entire model is optimized using the Adam optimizer, the mean square error (MSE) function is used as the loss function of the model (Nhu et al., 2020), and each model parameter is continuously adjusted through training. The entire process can be expressed as follows:

$$Feature_1 = TCN(a(x_t)) \quad (10)$$

$$Feature_2 = LSTM(TCN(a(x_t))) \quad (11)$$

$$F(x_t) = Linear(LSTM(TCN(a(x_t)))) \quad (12)$$

where $Feature_1$ represents the result of the TCN layer learning, $TCN(\cdot)$ represents the operation related to the TCN layer and TCN layer, $Feature_2$ represents the result of the LSTM layer learning, $LSTM(\cdot)$ represents the operation related to the LSTM layer and LSTM layer, $a(x_t)$

represents the related sequence information after normalization, $F(x_t)$ represents the output of the model, and $Linear(\cdot)$ represents the linear mapping of fully connected layers.

Data from January 1, 2015 to July 8, 2021 were used as the training set to train each model entry, and data from July 9, 2021 to July 9, 2022 were used as the test set to test the model using the trained model. The root mean square error (RMSE), mean absolute error (MAE), MSE, and correlation coefficient (R^2) were selected as metrics with which to evaluate the predictive capability of the model (Akhter et al., 2022).

2.3. Sobol sensitivity analysis method

Sobol is a global sensitivity analysis method based on variance decomposition, which can be used to analyze the degree of influence of multiple parameters on the model output results. The method decomposes the total variance of the model output results into the sum of the variances determined by the input parameters, and quantitatively evaluates the degree of the contribution of each parameter of the input model to the output results. Assuming a model $Y = f(X)$, where $X = [x_1, x_2, \dots, x_n]$, $i = 1, 2, \dots, n$, and x_i obeys the $[0, 1]$ distribution, then Y can be decomposed into the following form:

$$f(X) = f_0 + \sum_i f_i(x_i) + \sum_{i < j} f_{i,j}(x_i, x_j) + f_{1,2,\dots,n}(x_1, x_2, \dots, x_n) \quad (13)$$

Sobol proves that the decomposition is unique and that multiple integrations can determine each decomposition term. The total variance of $f(x)$ is as follows:

$$D = \int_0^1 \int_0^1 (x) dx - \int_0^1 \int_0^1 (x) dx \quad (14)$$

The bias variance is as follows:

$$D_{i_1, i_2, \dots, i_s} = \int_0^1 \dots \int_0^1 f_{i_1, i_2, \dots, i_s}^2(x_{i_1}, x_{i_2}, \dots, x_{i_s}) dx_{i_1} \dots dx_{i_s} \quad (15)$$

where $1 \leq i_t < \dots < i_s \leq n$, and $s = 1, 2, \dots, n$.

Then, the sensitivity coefficient $S_{i_1, i_2, \dots, i_s}$ can be expressed as follows:

$$S_{i_1, i_2, \dots, i_s} = \frac{D_{i_1, i_2, \dots, i_s}}{D} \quad (16)$$

where S_i is the first-order sensitivity index of x_i , which indicates the effect of the parameters alone on the model output results, which is also known as the main effect, and $S_{i_1, i_2, \dots, i_s}$ is the s -order sensitivity index of factors $x_{i_1}, x_{i_2}, \dots, x_{i_s}$. This study investigated the relationship between the input and output variables of the TCN-LSTM model using the Sobol first-order sensitivity indexes

3. Results

3.1. Trends of atmospheric PM in Xi'an

To improve environmental quality, predictions of atmospheric PM ($PM_{2.5}$ and PM_{10}) must reflect the current concentrations of PM quantitatively or qualitatively. Table 1 shows the descriptive statistics of PM concentrations in Xi'an from 2015 to 2021. The concentration of PM_{10} is significantly higher than $PM_{2.5}$, and the gap between the highest and lowest concentrations of PM is extensive and fluctuates significantly. In general, the concentration of PM remains high, and the environmental pollution problem is serious.

PM concentrations from 2015 to 2019 showed an overall decreasing trend, with $PM_{2.5}$ concentrations decreasing by 12% and PM_{10} concentrations decreasing by 30%, with PM_{10} decreasing more than $PM_{2.5}$. The outbreak of novel coronavirus pneumonia (COVID-19) in late 2019 resulted in human control measures implemented to prevent the spread of the virus (e.g., plant closures and home quarantine) (Xu et al., 2020). The overall decreasing trend in PM concentrations from 2019 to 2021,

Table 1

Descriptive statistics of PM_{2.5} and PM₁₀ concentrations in Xi'an, 2015–2021 (mean, standard deviation, median, minimum and maximum values).

Overall	PM _{2.5} Concentration (μg/m ³)			PM ₁₀ Concentration (μg/m ³)		
	Mean(SD)	Md	Min-Max	Mean(SD)	Md	Min-Max
2015	54.67 (34.11)	49	13–199	121.41 (73.92)	103	24–395
2016	76.35 (50.90)	60	15–254	140.64 (83.17)	112	28–366
2017	62.54 (39.42)	55	9–233	113.31 (56.84)	102	16–307
2018	43.23 (34.14)	32	0–192	82.67 (51.72)	73	0–350
2019	48.32 (34.78)	38	6–181	84.48 (50.01)	78	11–233
2020	46.92 (30.22)	37	10–146	93.58 (53.28)	83	20–255
2021	33.44 (28.23)	26	4–157	70.71 (51.48)	57	8–232
Spring	46.55 (26.25)	42	0–183	114.61 (68.91)	104.5	0–658
Summer	28.60 (12.31)	27	6–76	63.15 (27.81)	61	9–288
Autumn	52.21 (38.85)	42	0–254	100.97 (65.46)	86	0–395
Winter	105.56 (70.71)	88	0–493	164.22 (86.86)	144	0–589

with a 31% decrease in PM_{2.5} concentrations and a 13.77% decrease in PM₁₀ concentrations, is higher than the decrease in PM₁₀ compared to the period before the COVID-19 outbreak. It shows that the control measures during COVID-19 had an impact on the change in the concentration of PM in the atmosphere (Ding et al., 2021).

Generally, concentrations of PM in the study area are higher in spring and winter than in summer and autumn. In winter, the stable dry and cold climatic conditions with low precipitation, together with fossil fuel/biomass burning, lead to higher concentrations of atmospheric PM (Wang et al., 2014a).

3.2. Performance of TCN-LSTM-based prediction of PM concentration

This study developed a multi-input TCN-LSTM-based model for predicting PM concentration, and the prediction results are presented in Fig. 4. It can be seen that the predicted and observed values are in reasonable agreement and that the results are generally satisfactory. The R² value for PM_{2.5} and PM₁₀ was 0.948 and 0.876, respectively. The lowest RMSE, MAE, and MSE values were 12.014, 7.458, and 144.341, respectively, indicating that the proposed TCN-LSTM model represents a suitable and effective model for predicting atmospheric PM

concentrations. Moreover, the prediction results showed that the fit between the predicted and the observed values of PM_{2.5} was notably higher than that of PM₁₀, the R² value of the PM_{2.5} prediction results was 7.6% higher than that of the PM₁₀ results, and the PM₁₀ scatter plot was more discrete than that of PM_{2.5}, indicating that the difference between the predicted and observed values of PM₁₀ was greater. Thus, the level of agreement between the observed PM_{2.5} values and those predicted by the TCN-LSTM model was higher than that for PM₁₀.

3.3. Comparison and evaluation with other models

The predictions of PM concentration by the TCN-LSTM model were compared with the results of the deep learning models (i.e., the CNN-LSTM model, TCN model, and LSTM model). Additionally, the results from two typical machine learning algorithms (i.e., SVR and RF) were also considered for comparison. The degree of fit between the observed values and those predicted by each of the TCN-LSTM, CNN-LSTM, LSTM, TCN, SVR, and RF models is shown in Fig. 5. For both PM_{2.5} and PM₁₀ concentrations, the TCN-LSTM model produced the highest R² values of all the tested models, indicating that the TCN-LSTM model achieved the closest agreement between the predicted and observed values, R² is shown in Fig. 6(d). For PM_{2.5} and PM₁₀, the R² values of the TCN-LSTM model were 2% and 3% higher, respectively, than those of the CNN-LSTM model, 7% and 5% higher, respectively, than those of the LSTM model, and 6% and 9% higher, respectively, than those of the TCN model. These results indicate that the TCN-LSTM model had the highest prediction accuracy among the four deep learning models considered (i.e., TCN-LSTM, CNN-LSTM, LSTM, and TCN). For PM_{2.5} and PM₁₀, the R² values of the TCN-LSTM model were 24% and 16% higher, respectively, than those of the SVR model, and 17% and 14% higher, respectively, than those of the RF model. These performance results indicate that the TCN-LSTM model also has advantages in comparison with the machine learning models.

The R² values of the CNN-LSTM model for PM_{2.5} and PM₁₀ concentrations were 22% and 13% higher, respectively, than those of the SVR model, and 15% and 12% higher, respectively, than those of the RF model. The R² values of the LSTM model for PM_{2.5} and PM₁₀ concentrations were 18% and 11% higher, respectively, than those of the SVR model, and 10% and 9% higher, respectively, than those of the RF model. The R² values of the TCN model for PM_{2.5} and PM₁₀ concentrations were 9% and 7% higher, respectively, than those of the SVR model, and 12% and 6% higher, respectively, than those of the RF model. Thus, in this study, the performance of the deep learning models was better than that of the machine learning models.

Additionally, the low fit of the six models for extreme values in the prediction results of PM_{2.5} and PM₁₀ concentrations might reflect the complex formation process of PM, which is related not only to pollutants

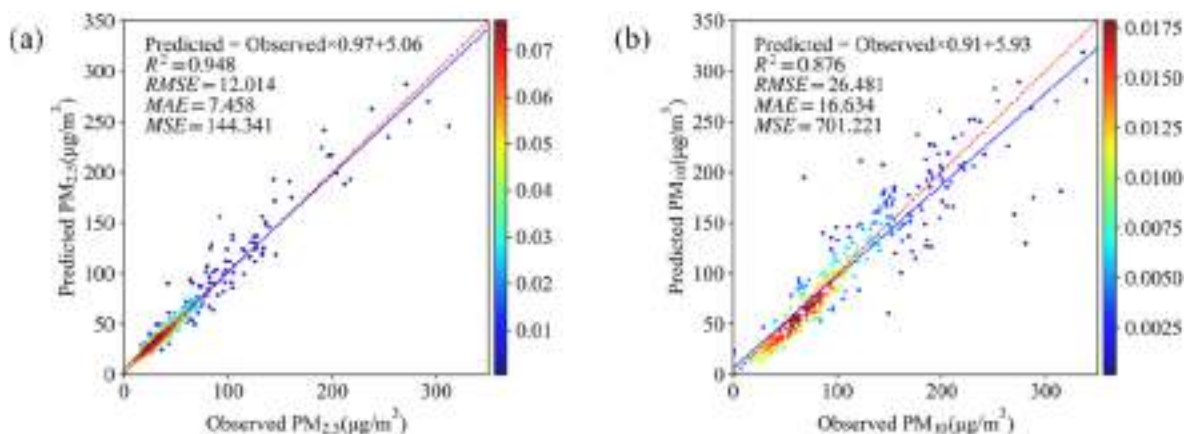


Fig. 4. Prediction results of the TCN-LSTM model: (a) PM_{2.5} and (b) PM₁₀.

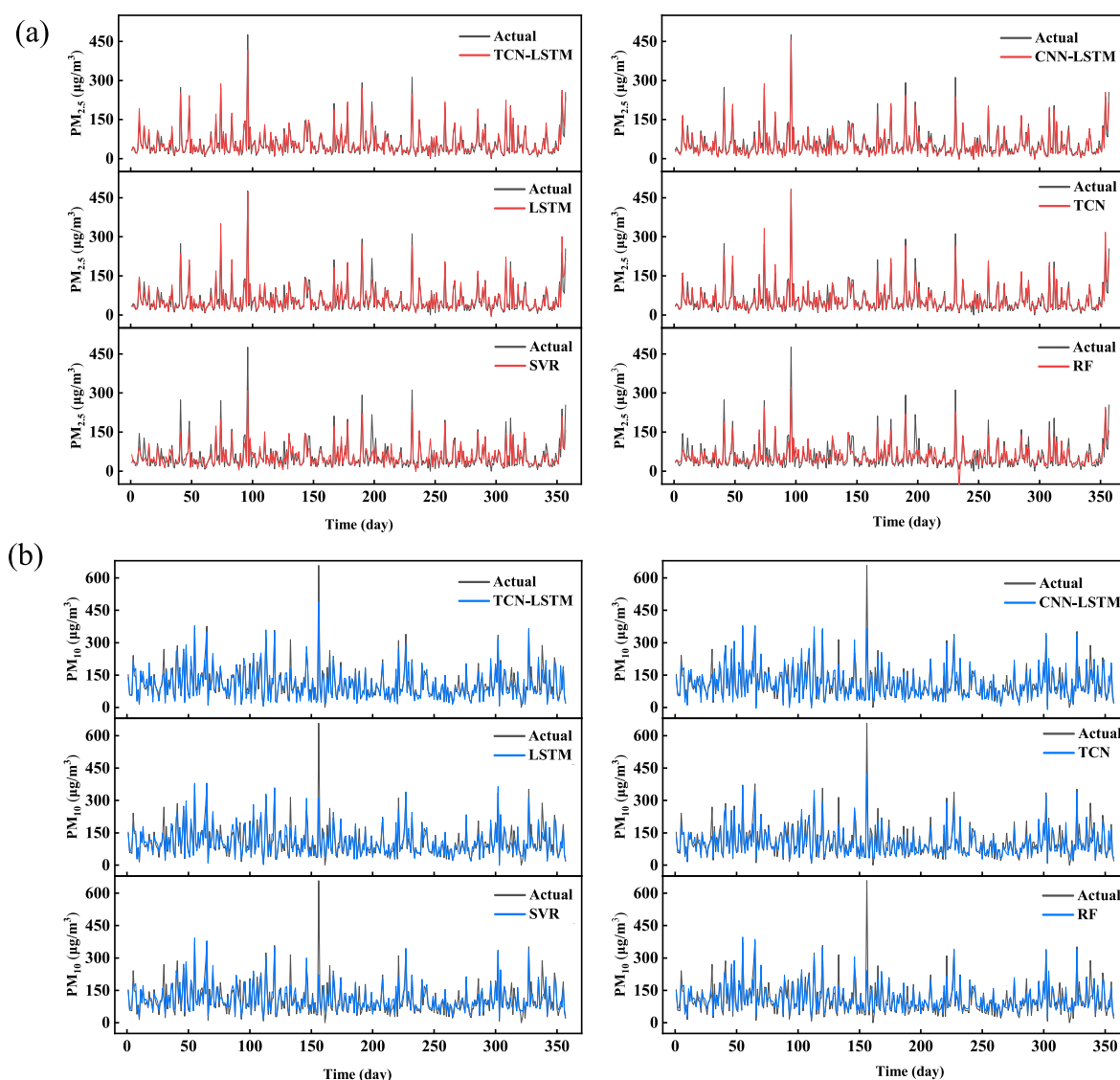


Fig. 5. Comparison of the prediction results of each model: (a) $PM_{2.5}$ and (b) PM_{10} .

and meteorological factors but also to sudden weather, population movements, and the degree of industrialization.

The results of quantitative analysis of the six models for the two particulate concentrations, shown in Fig. 6, explicitly present the trends of RMSE (Fig. 6(a)), MAE (Fig. 6(b)), MSE (Fig. 6(c)), and R^2 (Fig. 6(d)). The results are consistent with Fig. 5 and confirm that the errors of the deep learning models were smaller than those of the machine learning models. The errors of the predicted and observed values of the TCN-LSTM model were smallest, with RMSE, MAE, and MSE values of 26.48, 16.63, and 701.22, respectively, for prediction of $PM_{2.5}$, and RMSE, MAE, and MSE values of 12.01, 7.46, and 144.34, respectively, for prediction of PM_{10} . With the increase of the RMSE, MAE, and MSE values for each of the other models, the R^2 values showed trends of decline, indicating that the error between the predicted and observed data increased.

The robustness of the TCN-LSTM model was examined using Monte Carlo cross-validation of time series (Monte CarloCV). Table 2 shows the cross-validation results, and it can be seen that the TCN-LSTM model captures most of the PM variation in predicting PM ($PM_{2.5}$: $R^2 = 0.88$, PM_{10} : $R^2 = 0.87$), indicating that the TCN-LSTM model performs stably.

In order to evaluate the generalization ability of the TCN-LSTM model, the TCN-LSTM model was used to predict the PM concentrations in Beijing, Shanghai, Shenzhen, and Chengdu, which are typical

mega-cities in China with dense populations and developed economies. The same characteristic variables as those of Xi'an city were selected, and input into the TCN-LSTM model, and the model prediction results are shown in Table 3. The differences between the predicted and true values of PM concentrations were the smallest in Chengdu and Shenzhen. The largest in Beijing and Shanghai, probably due to the differences in geographic location and industrial development in different regions, resulting in different climatic conditions and pollutant contents in the atmosphere (Zhang et al., 2021), which produced uneven capture ability of the model for PM. Overall, the TCN-LSTM model predicted $PM_{2.5}$ and PM_{10} concentrations in the four cities satisfactorily, with R^2 up to 0.96 and down to 0.74. This indicates that the model has some generalization ability and still performs well predicting PM concentrations for different cities.

3.4. Understanding changes in PM through sensitivity analysis

The TCN-LSTM model proposed in this study has high accuracy for PM concentration prediction but cannot explore the relationship between the variables of the input model and the target output, which can be remedied by combining sensitivity analysis methods. In addition, the COVID-19 outbreak in December 2019 and the government implemented a series of control measures to stop the spread of the virus, which

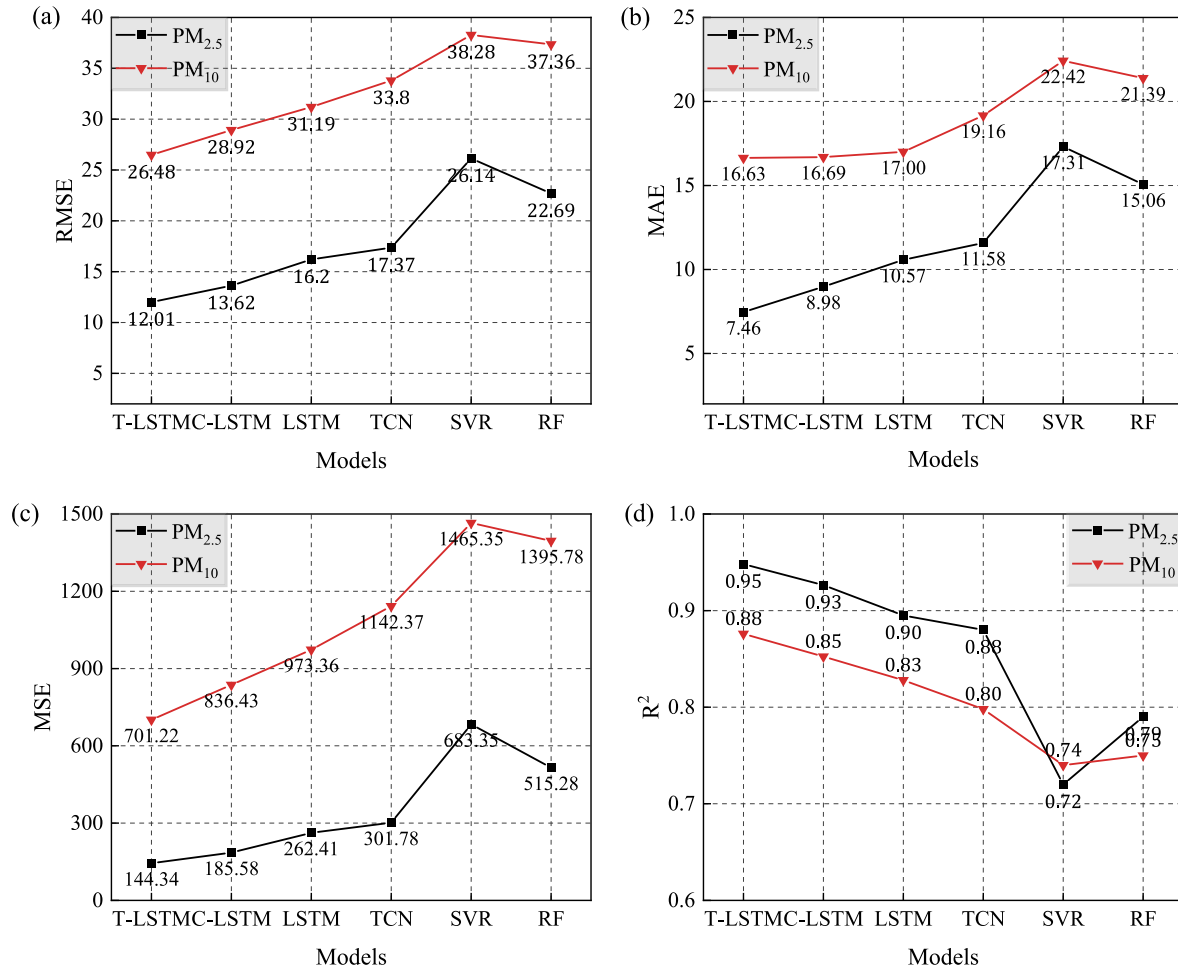


Fig. 6. Comparison of evaluation indicators of each model: (a) RMSE, (b) MAE, (c) MSE, and (d) R².

Table 2

Monte Carlo cross-validation of the TCN-LSTM model using time series. Taking Xi'an city as an example.

PM _{2.5}	RMSE	MAE	MSE	R ²	PM ₁₀	RMSE	MAE	MSE	R ²
1	12.24	7.81	149.87	0.93	1	33.05	18.68	1092.30	0.81
2	14.17	7.81	173.19	0.89	2	22.73	14.49	516.78	0.92
3	11.34	7.36	128.60	0.86	3	24.89	15.12	619.58	0.88
4	19.80	13.08	392.11	0.85	4	24.28	14.11	589.52	0.89
5	19.61	13.09	384.67	0.87	5	28.01	16.34	784.60	0.87
Average	15.43	9.83	245.69	0.88	Average	26.59	15.75	720.56	0.87

Table 3

TCN-LSTM model predicts the results of other cities.

PM _{2.5}	RMSE	MAE	MSE	R ²	PM ₁₀	RMSE	MAE	MSE	R ²
Chengdu	7.481	4.94	55.96	0.96	Chengdu	13.03	10.37	169.85	0.95
Shanghai	8.48	6.05	71.94	0.89	Shanghai	11.71	8.33	137.2	0.82
Beijing	15.96	10.26	254.78	0.90	Beijing	31.31	17.78	980.35	0.74
Shenzhen	4.87	3.36	23.73	0.95	Shenzhen	6.36	5.00	40.42	0.91

made the pattern of particle concentration changes in 2015–2019 and 2020–2022 different and to explore this change. The Sobol-based sensitivity analysis method was used to predict the COVID-19 outbreak before (2015–2019), after (2020–2022) years), post-outbreak (2020–2022), and throughout the period (2015–2022) for sensitivity analysis of the PM. Sobol's first-order sensitivity index was used to represent the contribution of the variables entered into the TCN-LSTM model to the model target output, with a larger sensitivity index

representing a greater sensitivity of the model output to that variable. Table 4 shows the results of the sensitivity analysis of PM concentrations before, after, and throughout the COVID-19 outbreak.

Before the COVID-19 outbreak, O₃, CO, and dew point had the greatest effect on PM_{2.5} concentrations, and PM_{2.5}, NO₂, and dew point had the greatest effect on PM₁₀ concentrations. after the COVID-19 outbreak, SO₂, O₃ and CO had the greatest effect on PM_{2.5} concentrations, and PM_{2.5}, CO and dew point had the greatest effect on PM₁₀

Table 4

Sensitivity analysis of PM_{2.5} and PM₁₀ before, after, and throughout the COVID-19 outbreak using the Sobol sensitivity analysis method. (SI: Solar Irradiance, PR: Precipitation, WS: Wind Speed, WD: Wind Direction, RH: Relative Humidity, DP: Dew Point, T: Temperature, P: Pressure).

Target	Periods	Pollutant Data						Meteorological data							
		PM _{2.5}	PM ₁₀	SO ₂	NO ₂	O ₃	CO	SI	PR	WS	WD	RH	DP	T	P
PM _{2.5}	Before	–	0.044	0.035	0.060	0.215	0.071	0.030	0.004	0.052	0.026	0.041	0.110	0.020	0.019
	After	–	0.016	0.037	0.029	0.091	0.136	0.002	0.014	0.018	0.009	0.007	0.014	0.014	0.018
	whole	–	0.049	0.105	0.043	0.025	0.227	0.009	0.003	0.003	0.002	0.133	0.048	0.111	0.038
PM ₁₀	Before	0.589	–	0.067	0.121	0.076	0.019	0.028	0.010	0.019	0.017	0.024	0.235	0.044	0.062
	After	0.807	–	0.105	0.060	0.028	0.651	0.019	0.013	0.091	0.072	0.130	0.256	0.094	0.014
	whole	0.735	–	0.123	0.021	0.108	0.134	0.005	0.009	0.018	0.014	0.039	0.025	0.100	0.049

concentrations.

PM_{2.5} concentrations before the COVID-19 outbreak were also related to the meteorological factor dew point. After the COVID-19 outbreak, PM_{2.5} concentrations were mainly influenced by pollutants, indicating that PM_{2.5} concentration changes are more sensitive to pollutant emission sources (Shao et al., 2021). PM₁₀ concentrations before and after the COVID-19 outbreak were related to PM_{2.5} concentrations and dew point and dew point, in addition to PM_{2.5} and dew point, NO₂ was the key factor affecting PM₁₀ before the COVID-19 outbreak, and CO was the key factor affecting PM₁₀ after the COVID-19 outbreak.

The sensitivity analysis for the entire period showed that PM_{2.5} concentrations were mainly influenced by CO, relative humidity, and temperature, and PM_{2.5}, SO₂, and CO mainly influenced PM₁₀ concentrations. The sensitivity analysis results indicated that CO always influenced the concentration of PM in the atmosphere, and the COVID-19 outbreak did not disrupt this influence. In contrast, other key factors affecting PM_{2.5} and PM₁₀ were changed at different times of COVID-19.

3.5. Decreasing experiment

To further determine the influence of the input variables on the output results of the TCN-LSTM model, a sequential decreasing experiment was conducted to predict the PM concentrations based on the results of the Sobol analysis. The results of the decreasing experiments are shown in Fig. 7. The blue lines in the figure are the evaluated index values of the model at the original 13 inputs, with the sensitivity of the horizontal coordinate variables from low to high and the inflection points marked (mainly R²). For PM_{2.5} prediction, the model prediction performance did not change notably when the four variables of WS, WD, PR, and SI were removed, with a value of R² of 0.93 and RMSE, MAE, and MSE values of 13.03, 8.88, and 169.69, respectively. In comparison with the input of 13 variables, the R² value decreased by 2%, the RMSE and MSE values increased by 8% and 15%, respectively, and the MAE value decreased by 27%. For PM₁₀, when the six input variables of SI, PR, WD, WS, NO₂, and DP were removed, the R² value was 0.87, and the RMSE, MAE, and MSE values were 27.62, 18.9, and 762.68, respectively, i.e., the R² value decreased by 1%, the RMSE and MSE values

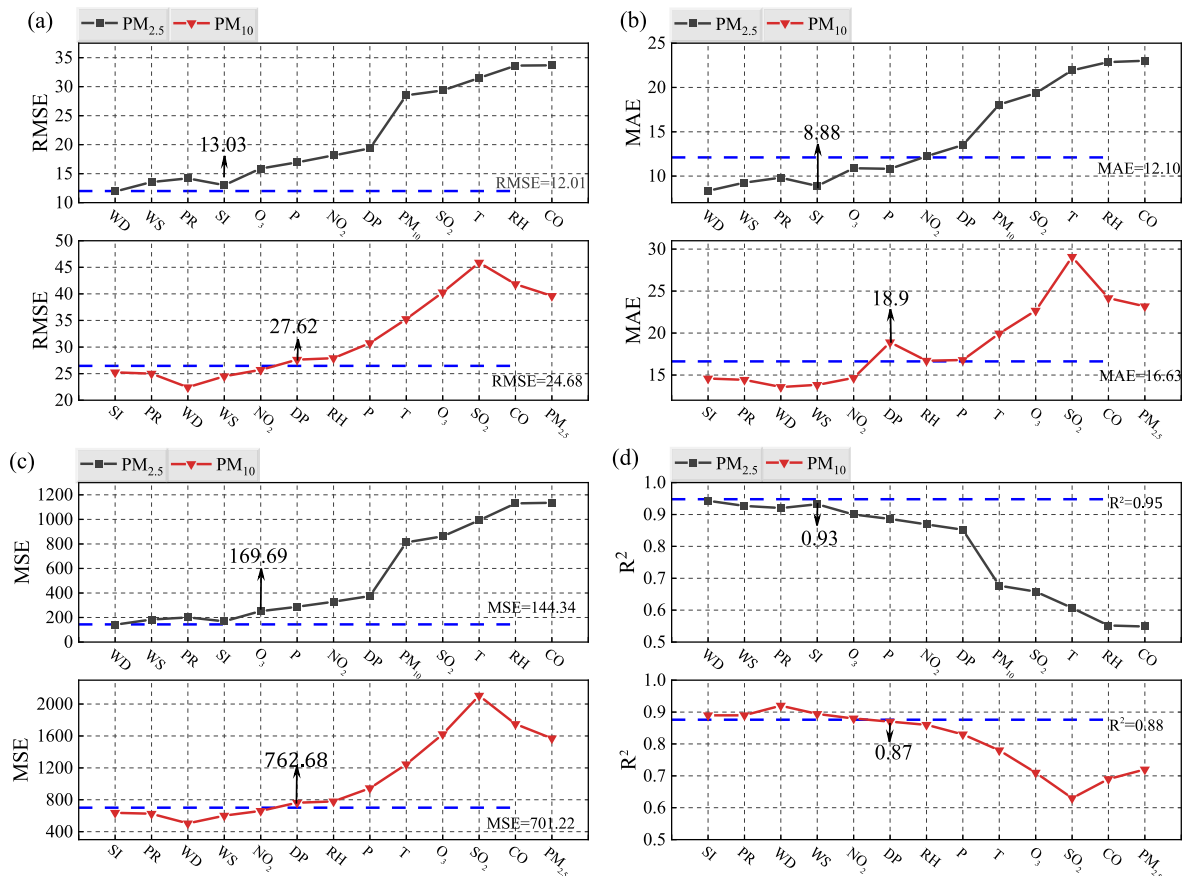


Fig. 7. Results of Sobol-based regressive experiments: (a) RMSE, (b) MAE, (c) MSE, and (d) R².

increased by 11% and 8%, respectively, and the MAE value decreased by 12% in comparison with the input of 13 variables. After the inflection point, the R^2 value decreased as the input variables decreased, and the RMSE, MAE, and MSE values increased, indicating that the error between the predicted and observed values of the TCN-LSTM model increased, which implies that the removal of the more sensitive variables reduces the performance of the TCN-LSTM model.

Overall, by removing the input variables that contribute less to the model output, no notable drop in model performance occurred, and all metrics varied within an acceptable range. The variables of CO, RH, T, SO₂, PM₁₀, DP, NO₂, P, and O₃ contributed most to the prediction of PM_{2.5} by the TCN-LSTM, while PM_{2.5}, CO, SO₂, O₃, T, P, and RH contributed most to the prediction of PM₁₀ by the TCN-LSTM model. Thus, these key variables directly affect the performance of the model.

4. Discussion

4.1. Ability of TCN-LSTM model to predict atmospheric PM_{2.5} and PM₁₀

The deep learning model developed in this study constructs a nonlinear mapping relationship between multisource data (meteorological factors and pollutant concentrations) and PM (PM_{2.5}, PM₁₀) concentrations, thereby achieving accurate prediction of atmospheric PM concentrations. The predicted and observed values of atmospheric PM concentrations in Xi'an were consistent, with R^2 values of 0.88. More importantly, the data used in this study comprised easily available meteorological data and pollutant concentration data. Real-time prediction models constructed using these variables avoid the time constraints involved in collecting, storing, and delivering PM concentration data, and can forecast PM concentrations, even when conventional monitoring equipment fails.

For the prediction of atmospheric PM concentration, the fit between the observed values and the values predicted by the single TCN model and by the single LSTM model was lower than that of the combined model. The results from the CNN-LSTM model and the TCN-LSTM model indicate that the single models have certain limitation. Adding a feature extraction function is helpful for improving the learning capability of the model. The degree of fit to the observations by the values predicted by the TCN-LSTM model was higher than that of the values predicted by the CNN-LSTM model, indicating that the feature extraction capability of the TCN model is stronger than that of the CNN model.

The fit between the observed values and the values predicted by the TCN-LSTM, CNN-LSTM, TCN, and LSTM models was higher than that of the other models. The results of the SVR and RF models indicate that the prediction accuracy of deep learning models is higher than that of machine learning models owing to the large amount of data used in this study. The ability of the SVR and RF models to extract features is poor when faced with large-scale data (Zhang et al., 2020b), and model learning efficiency is reduced. For deep learning models, increase in the volume of data means that model learning can be improved (LeCun et al., 2015; Mahmud et al., 2021), and the pattern of variation between data explored more accurately, which helps improve the prediction accuracy of the model and makes it more suitable for the prediction of atmospheric PM concentration. Furthermore, the results of cross-validation show that the stability of the TCN-LSTM model is good. The PM prediction results for several cities show that the TCN-LSTM model has some generalization ability.

However, the difference between the predicted and observed peak values reduces the overall prediction accuracy of the model, i.e., the peaks are too high and the troughs are too low in the sample data. Exploring this pattern of change is challenging (McCormick et al., 2016; Roe and Baker, 2007). It can lead to specific errors in the observations and calculations, resulting in deviations between the original sample data and the actual situation, thereby affecting the accuracy of the model prediction results.

4.2. Low-cost and fast modeling capabilities

This study constructed a TCN-LSTM model to predict atmospheric PM concentrations. The variables used are easily accessible, reducing costs in comparison with traditional PM concentration monitoring. However, for deep learning models, the choice of input variables affects the model output, and incorporation of irrelevant variables reduces model efficiency (Kim et al., 2020; Ly et al., 2021; Optis and Perr-Sauer, 2019). The removal of unnecessary parameters helps model performance and reduces costs. Data monitored in real-time require expensive machine and human supervision, which can raise costs. Thus, removal of variables that offer minimal contribution to the accuracy of the model output can reduce the amount of raw sample data needed to build the model, and achieve the goal of low-cost and rapid modeling.

Sensitivity analysis reveals key factors affecting the variation of PM concentrations. The Sobol-based sensitivity analysis identified nine variables (i.e., CO, RH, T, SO₂, PM₁₀, DP, NO₂, P, and O₃) that contribute positively to the prediction of atmospheric PM_{2.5} concentration by the TCN-LSTM model, and seven variables (i.e., PM_{2.5}, CO, SO₂, O₃, T, PR, and RH) that contribute positively to the prediction of atmospheric PM₁₀ concentration by the TCN-LSTM model. Pollutants such as CO, SO₂, PM₁₀, NO₂, and O₃ are common in the atmosphere, and they are closely related to atmospheric PM owing to their common source, e.g., fossil fuel combustion (Lei et al., 2020; Maciejczyk et al., 2021), and their prevalence in a high temperature and high humidity environment that is not conducive to the dispersion of atmospheric pollutants (Shuangchen et al., 2017) and favors the increase in PM concentrations. The results of the sensitivity analysis also indicate that the control measures during the COVID-19 outbreak have a long-term impact on the change in pollutants and that the key factors for the change in PM concentrations are not the same for 2015–2019, 2020–2022, and 2015–2022.

The decreasing experiments based on sensitivity analysis showed that four input variables (i.e., WS, WD, PR, and SI) could be removed in predicting PM_{2.5} concentration, and six input variables (i.e., SI, PR, WD, WS, NO₂, and DP) could be removed in predicting PM₁₀ concentration. Following the removal of such variables, the prediction performance of the TCN-LSTM model did not show precipitous decrease, and the R^2 value was within an acceptable range of variation. The variables of SI, PR, WD, and WS contributed little to the model output, which can be explained by consideration of the geographical location of Xi'an. The area receives little PR throughout the year, the region is surrounded by mountains, which means that WS and WD do not play major roles in the variation of atmospheric pollutant concentrations, and SI generally has little impact on PM concentrations (Zhang et al., 2020a).

5. Conclusions

This study developed a TCN-LSTM-based method for prediction of atmospheric PM concentration to address the problematic pollution situation in Xi'an (Shaanxi Province, China), and to consider the time series and nonlinear characteristics of the main influencing factors of atmospheric PM. The TCN model can extract the long-term temporal relationships and features of historical data, which combined with the nonlinear fitting capability and profound learning ability of the LSTM model, can be used to predict atmospheric PM concentrations. Thirteen metrics of meteorological factors and pollutant data were selected to predict PM concentrations. The TCN-LSTM model predicted PM_{2.5} and PM₁₀ concentrations with satisfactory R^2 values of 0.95 and 0.88, respectively, i.e., higher than those achieved by any other model considered in this study. The Monte Carlo cross-validation of the time series tests the robustness of the model, and the results showed the high stability of the TCN-LSTM model. The TCN-LSTM model performed stably in predicting the PM concentrations in Beijing, Shanghai, Shenzhen, and Chengdu with R^2 in the range of 0.74–0.95, indicating the strong generalization ability of the model. The Sobol-based sensitivity analysis showed that CO was the key factor affecting PM_{2.5}

concentration throughout the period, and PM_{2.5} was the key factor for the change of PM₁₀. The sensitivity analysis of COVID-19 in different periods showed that the key factors for the change of PM concentration were different before and after the COVID-19 outbreak, and the COVID-19 outbreak changed the changing pattern of PM. The decreasing experiment removed the variables that contributed less to the model output, and the prediction performance of the model did not change significantly, with R² of 0.93 and 0.87 for predicting PM_{2.5} and PM₁₀, respectively, which only decreased by 2% and 1%, indicating that the fit between the predicted and observed values did not regress significantly. In conclusion, the model designed in this study has the potential to become a new tool in the field of atmospheric prediction.

Credit author statement

Ying Ren: Conceptualization, Methodology, data analysis, data collection, Writing - original draft. **Siyuan Wang:** Formal analysis, Investigation, Validation. **Bisheng Xia:** Supervision, Validation, Writing-review & editing. Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Yan 'an City Science and Technology Development Program (No.203010096) and Yan 'an University Doctoral Program (No.20504306).

References

- Ahmad, M.S., Adnan, S.M., Zaidi, S., Bhargava, P., 2020. A novel support vector regression (SVR) model for the prediction of splice strength of the unconfined beam specimens. *Construct. Build. Mater.* 248, 118475.
- Akhter, M.N., Mekhilef, S., Mokhlis, H., Ali, R., Usama, M., Muhammad, M.A., Khairuddin, A.S., 2022. A hybrid deep learning method for an hour ahead power output forecasting of three different photovoltaic systems. *M.J.A.E.* 307, 118185.
- Ausati, S., Amanollahi, J., 2016. Assessing the accuracy of ANFIS, EEMD-GRNN, PCR, and MLR models in predicting PM_{2.5}. *Atmos. Environ.* 142, 465–474.
- Bai, S., Kolter, J.Z., Koltun, V., 2018. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling arXiv preprint arXiv:1803.01271.
- Bui, X.-N., Lee, C.W., Nguyen, H., Bui, H.-B., Long, N.Q., Le, Q.-T., Nguyen, V.-D., Nguyen, N.-B., Moayedi, H., 2019. Estimating PM₁₀ concentration from drilling operations in open-pit mines using an assembly of SVR and PSO. *Appl. Sci.* 9, 2806.
- Carta, J.A., Díaz, S., Castañeda, A., 2020. A global sensitivity analysis method applied to wind farm power output estimation models. *Appl. Energy* 280, 115968.
- Chuang, M.-T., Zhang, Y., Kang, D., 2011. Application of WRF/Chem-MADRID for real-time air quality forecasting over the Southeastern United States. *Atmos. Environ.* 45, 6241–6250.
- Dai, H., Huang, G., Wang, J., Zeng, H., Zhou, F., 2022a. Spatio-temporal characteristics of PM_{2.5} concentrations in China based on multiple sources of data and LUR-GBM during 2016–2021. *Int. J. Environ. Res. Publ. Health* 19, 6292.
- Dai, H., Huang, G., Zeng, H., Zhou, F., 2022b. PM_{2.5} volatility prediction by XGBoost-MLP based on GARCH models. *J. Clean. Prod.* 356, 131898.
- Díaz-Robles, L.A., Ortega, J.C., Fu, J.S., Reed, G.D., Chow, J.C., Watson, J.G., Moncada-Herrera, J.A., 2008. A hybrid ARIMA and artificial neural networks model to forecast particulate matter in urban areas: the case of Temuco, Chile. *Atmos. Environ.* 42, 8331–8340.
- Ding, J., Dai, Q., Li, Y., Han, S., Zhang, Y., Feng, Y., 2021. Impact of meteorological condition changes on air quality and particulate chemical composition during the COVID-19 lockdown. *J.j.o.e.s.* 109, 45–56.
- Fan, J., Zhang, K., Huang, Y., Zhu, Y., Chen, B., 2021. Parallel spatio-temporal attention-based TCN for multivariate time series prediction. *Neural Comput. Appl.* 1–10.
- Feng, R., Zheng, H.-j., Gao, H., Zhang, A.-r., Huang, C., Zhang, J.-x., Luo, K., Fan, J.-r., 2019. Recurrent Neural Network and random forest for analysis and accurate forecast of atmospheric pollutants: a case study in Hangzhou, China. *J. Clean. Prod.* 231, 1005–1015.
- Feng, X., Li, Q., Zhu, Y., Hou, J., Jin, L., Wang, J., 2015. Artificial neural networks forecasting of PM_{2.5} pollution using air mass trajectory based geographic model and wavelet transformation. *Atmos. Environ.* 107, 118–128.
- Foley, K., Roselle, S., Appel, K., Bhawe, P., Pleim, J., Otte, T., Mathur, R., Sarwar, G., Young, J., Gilliam, R., 2010. Incremental testing of the community multiscale air quality (CMAQ) modeling system version 4.7. *Geosci. Model Dev. (GMD)* 3, 205–226.
- Gan, Y., Duan, Q., Gong, W., Tong, C., Sun, Y., Chu, W., Ye, A., Miao, C., Di, Z., 2014. A comprehensive evaluation of various sensitivity analysis methods: a case study with a hydrological model. *Environ. Model. Software* 51, 269–285.
- Grell, G.A., Peckham, S.E., Schmitz, R., McKeen, S.A., Frost, G., Skamarock, W.C., Eder, B., 2005. Fully coupled “online” chemistry within the WRF model. *Atmos. Environ.* 39, 6957–6975.
- Gupta, P., Christopher, S.A., 2009. Particulate matter air quality assessment using integrated surface, satellite, and meteorological products: multiple regression approach. *J. Geophys. Res. Atmos.* 114.
- Gutierrez-Avila, I., Rojas-Bracho, L., Riojas-Rodriguez, H., Kloog, I., Just, A.C., Rothenberg, S.J., 2018. Cardiovascular and cerebrovascular mortality associated with acute exposure to PM_{2.5} in Mexico city. *Stroke* 49, 1734–1736.
- He, Q., Huang, B., 2018. Satellite-based mapping of daily high-resolution ground PM_{2.5} in China via space-time regression modeling. *Rem. Sens. Environ.* 206, 72–83.
- Hou, J., Wang, G., Chen, X., Xue, J.-H., Zhu, R., Yang, H., 2018. Spatial-temporal attention res-TCN for skeleton-based dynamic hand gesture recognition. *Proc. Eur. Conf. Comput. vision (ECCV) workshops*, 0–0.
- Jian, L., Zhao, Y., Zhu, Y.-P., Zhang, M.-B., Bertolatti, D., 2012. An application of ARIMA model to predict submicron particle concentrations from meteorological factors at a busy roadside in Hangzhou, China. *Sci. Total Environ.* 426, 336–345.
- Kim, W., Han, Y., Kim, K.J., Song, K.-W., 2020. Electricity load forecasting using advanced feature selection and optimal deep learning model for the variable refrigerant flow systems. *Energy Rep.* 6, 2604–2618.
- Lamba, A., Cassey, P., Segaran, R.R., Koh, L.P., 2019. Deep learning for environmental conservation. *Curr. Biol.* 29, R977–R982.
- LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. *Nature* 521, 436–444.
- Lee, D., Kang, S., Shin, J., 2017. Using deep learning techniques to forecast environmental consumption level. *Sustainability* 9, 1894.
- Lei, R., Feng, S., Lauvaux, T., 2020. Country-scale trends in air pollution and fossil fuel CO₂ emissions during 2001–2018: confronting the roles of national policies and economic growth. *Environ. Res. Lett.* 16, 014006.
- Lin, H., Liu, T., Xiao, J., Zeng, W., Guo, L., Li, X., Xu, Y., Zhang, Y., Chang, J.J., Vaughn, M.G., Qian, Z.M., Ma, W., 2017. Hourly peak PM_{2.5} concentration associated with increased cardiovascular mortality in Guangzhou, China. *J. Expo. Sci. Environ. Epidemiol.* 27, 333–338.
- Liu, D., Sun, K., 2019. Short-term PM_{2.5} forecasting based on CEEMD-RF in five cities of China. *Environ. Sci. Pollut. Res. Int.* 26, 32790–32803.
- Ly, H.-B., Nguyen, T.-A., Tran, V.Q., 2021. Development of deep neural network model to predict the compressive strength of rubber concrete. *Construct. Build. Mater.* 301, 124081.
- Maciejczyk, P., Chen, L.-C., Thurston, G., 2021. The role of fossil fuel combustion metals in PM_{2.5} air pollution health associations. *Atmosphere* 12, 1086.
- Mahmud, M., Kaiser, M.S., McGinnity, T.M., Hussain, A., 2021. Deep learning in mining biological data. *Cognit. Comput.* 13, 1–33.
- Makler-Pick, V., Gal, G., Gorfine, M., Hipsey, M.R., Carmel, Y., 2011. Sensitivity analysis for complex ecological models—a new approach. *Environ. Model. Software* 26, 124–134.
- McCormick, M.C., Schultz, F.A., Baik, M.-H., 2016. Glassy carbon electrodes deliver unpredictable reduction potentials for platinum (IV) antitumor prodrugs. *Polyhedron* 103, 28–34.
- Meng, Y., Sun, W., 2021. Relationship between the formation of PM_{2.5} and meteorological factors in northern China: the periodic characteristics of wavelet analysis. *Adv. Meteorol.* 2021.
- Nguyen, G.T.H., Shimadera, H., Uranishi, K., Matsuo, T., Kondo, A., 2019. Numerical assessment of PM_{2.5} and O₃ air quality in Continental Southeast Asia: impacts of potential future climate change. *Atmos. Environ.* 215, 116901.
- Nhu, V.-H., Hoang, N.-D., Nguyen, H., Ngo, P.T.T., Bui, T.T., Hoa, P.V., Samui, P., Bui, D. T., 2020. Effectiveness assessment of Keras based deep learning with different robust optimization algorithms for shallow landslide susceptibility mapping at tropical area. *J.C.* 188, 104458.
- Ong, B.T., Sugiura, K., Zettsu, K., 2016. Dynamically pre-trained deep recurrent neural networks using environmental monitoring data for predicting PM_{2.5}. *Neural Comput. Appl.* 27, 1553–1566.
- Optis, M., Perr-Sauer, J., 2019. The importance of atmospheric turbulence and stability in machine-learning models of wind farm power production. *Renew. Sustain. Energy Rev.* 112, 27–41.
- Otte, T.L., Pouliot, G., Pleim, J.E., Young, J.O., Schere, K.L., Wong, D.C., Lee, P.C., Tsidulko, M., McQueen, J.T., Davidson, P., 2005. Linking the Eta model with the Community Multiscale Air Quality (CMAQ) modeling system to build a national air quality forecasting system. *Weather Forecast.* 20, 367–384.
- Pak, U., Kim, C., Ryu, U., Sok, K., Pak, S., 2018. A hybrid model based on convolutional neural networks and long short-term memory for ozone concentration prediction. *Air Quality, Atmosphere & Health* 11, 883–895.
- Perez, P., Reyes, J., 2006. An integrated neural network model for PM₁₀ forecasting. *Atmos. Environ.* 40, 2845–2851.
- Roe, G.H., Baker, M.B., 2007. Why is climate sensitivity so unpredictable? *Science* 318, 629–632.
- Samek, W., Wiegand, T., Müller, K.-R., 2017. Explainable Artificial Intelligence: Understanding, Visualizing and Interpreting Deep Learning Models arXiv preprint arXiv:1708.08296.
- Samet, J.M., 2022. Do coarse mass particles increase daily mortality? New findings from a multi-country, multi-city study. *J.A.J.o.R., Medicine, C.C.* 932–933. American Thoracic Society.